

Oeste de savoir

Le langage C

samedi 04 octobre 2025

Table des matières

Introduction	15
I. Remerciements	17
II. Premiers pas	19
II.1. Présentation du langage C	20
Introduction	20
II.1.1. La programmation, qu'est-ce que c'est ?	20
II.1.1.1. Qu'est-ce qu'un programme ?	20
II.1.1.2. Le langage machine	21
II.1.1.3. Un peu d'histoire	21
II.1.2. Le langage C	22
II.1.2.1. L'histoire du C	23
II.1.2.2. Pourquoi apprendre le C ?	23
II.1.2.3. La norme	24
II.1.3. Notre cible	24
II.2. Les outils nécessaires	27
Introduction	27
II.2.1. Windows	27
II.2.1.1. Le compilateur	27
II.2.1.2. L'éditeur de texte	29
II.2.1.3. Introduction à la ligne de commande	29
II.2.2. GNU/Linux et *BSD	30
II.2.2.1. Le compilateur	30
II.2.2.2. L'éditeur de texte	30
II.2.2.3. Introduction à la ligne de commande	30
II.2.3. Mac OS X	31
II.2.3.1. Le compilateur	31
II.2.3.2. L'éditeur de texte	32
II.2.3.3. Introduction à la ligne de commande	32
II.3. Notre premier programme C	33
Introduction	33
II.3.1. Premier programme	33
II.3.2. Erreur lors de la compilation	34
II.3.3. Les commentaires	36

Conclusion	36
III. Les bases du langage C	37
III.1. Les variables	38
Introduction	38
III.1.1. Qu'est-ce qu'une variable ?	38
III.1.1.1. Codage des informations	38
III.1.1.2. La mémoire	40
III.1.1.3. Les variables	42
III.1.2. Déclarer une variable	42
III.1.2.1. Les types	43
III.1.2.2. Les identificateurs	45
III.1.2.3. Déclaration	47
III.1.3. Initialiser une variable	47
III.1.3.1. Rendre une variable constante	49
III.1.4. Affecter une valeur à une variable	49
III.1.5. Les représentations octale et hexadécimale	50
III.1.5.1. Constantes octales et hexadécimales	51
Conclusion	52
III.2. Manipulations basiques des entrées/sorties	53
Introduction	53
III.2.1. Les sorties	53
III.2.1.1. Première approche	53
III.2.1.2. Les formats	54
III.2.1.3. Précision des nombres flottants	57
III.2.1.4. Les caractères spéciaux	58
III.2.1.5. Sur plusieurs lignes	59
III.2.2. Interagir avec l'utilisateur	59
Conclusion	63
III.3. Les opérations mathématiques	64
Introduction	64
III.3.1. Les opérations mathématiques de base	64
III.3.1.1. Division et modulo	64
III.3.1.2. Utilisation	65
III.3.1.3. La priorité des opérateurs	67
III.3.2. Raccourcis	68
III.3.2.1. Les opérateurs combinés	68
III.3.2.2. L'incrément et la décrémentation	69
III.3.3. Le type d'une constante	71
III.3.4. Le type d'une opération	73
III.3.5. Les conversions	75
III.3.6. Exercices	77
Conclusion	77
Contenu masqué	78

III.4. Tests et conditions	79
Introduction	79
III.4.1. Les booléens	79
III.4.2. Les opérateurs de comparaison	80
III.4.2.1. Comparaisons	80
III.4.2.2. Résultat d'une comparaison	81
III.4.3. Les opérateurs logiques	82
III.4.3.1. Les opérateurs logiques de base	82
III.4.3.2. Évaluation en court-circuit	83
III.4.3.3. Combinaisons	84
III.4.4. Priorité des opérations	84
Conclusion	85
III.5. Les sélections	86
Introduction	86
III.5.1. La structure if	86
III.5.1.1. L'instruction if	86
III.5.1.2. La clause else	88
III.5.1.3. if, else if	90
III.5.2. L'instruction switch	94
III.5.2.1. Plusieurs entrées pour une même action	96
III.5.2.2. Plusieurs entrées sans instruction break	97
III.5.3. L'opérateur conditionnel	99
III.5.3.1. Exercice	100
Conclusion	101
Contenu masqué	101
III.6. TP : déterminer le jour de la semaine	103
Introduction	103
III.6.1. Objectif	103
III.6.2. Première étape	103
III.6.2.1. Déterminer le jour de la semaine	103
III.6.3. Correction	104
III.6.4. Deuxième étape	105
III.6.5. Correction	105
III.6.6. Troisième et dernière étape	106
III.6.6.1. Les calendriers Julien et Grégorien	106
III.6.6.2. Mode de calcul	107
III.6.7. Correction	107
Conclusion	108
Contenu masqué	108
III.7. Les boucles	115
Introduction	115
III.7.1. La boucle while	115
III.7.1.1. Syntaxe	116
III.7.1.2. Exercice	117
III.7.2. La boucle do-while	117
III.7.2.1. Syntaxe	118

III.7.3.	La boucle for	120
III.7.3.1.	Syntaxe	120
III.7.3.2.	Plusieurs compteurs	123
III.7.4.	Imbrications	123
III.7.5.	Boucles infinies	124
III.7.6.	Exercices	125
III.7.6.1.	Calcul du PGCD de deux nombres	125
III.7.6.2.	Une overdose de lapins	125
III.7.6.3.	Des pieds et des mains pour convertir mille miles	126
III.7.6.4.	Puissances de trois	127
III.7.6.5.	La disparition : le retour	127
	Conclusion	128
	Contenu masqué	129
III.8.	Les sauts	135
	Introduction	135
III.8.1.	L’instruction break	136
III.8.1.1.	Exemple	136
III.8.2.	L’instruction continue	137
III.8.3.	Boucles imbriquées	138
III.8.4.	L’instruction goto	139
III.8.4.1.	Exemple	139
III.8.4.2.	Le dessous des boucles	140
III.8.4.3.	Goto Hell ?	141
	Conclusion	141
III.9.	Les fonctions	142
	Introduction	142
III.9.1.	Qu’est-ce qu’une fonction ?	142
III.9.2.	Définir et utiliser une fonction	144
III.9.2.1.	Le type de retour	145
III.9.2.2.	Les paramètres	145
III.9.2.3.	Les arguments et les paramètres	147
III.9.2.4.	Les étiquettes et l’instruction	148
III.9.3.	Les prototypes	148
III.9.4.	Variables globales et classes de stockage	150
III.9.4.1.	Les variables globales	150
III.9.4.2.	Les classes de stockage	151
III.9.5.	Exercices	153
III.9.5.1.	Afficher un rectangle	153
III.9.5.2.	Afficher un triangle	154
III.9.5.3.	En petites coupures ?	154
	Conclusion	155
	Contenu masqué	156
III.10.	TP : une calculatrice basique	159
	Introduction	159
III.10.1.	Objectif	159

III.10.2. Préparation	160
III.10.2.1. Précisions concernant scanf	160
III.10.2.2. Les puissances	160
III.10.2.3. La factorielle	161
III.10.2.4. Le PGCD	161
III.10.2.5. Le PPCD	161
III.10.2.6. Exemple d'utilisation	162
III.10.2.7. Derniers conseils	162
III.10.3. Correction	162
Conclusion	165
III.11. Découper son projet	166
Introduction	166
III.11.1. Portée et masquage	166
III.11.1.1. La notion de portée	166
III.11.1.2. La notion de masquage	167
III.11.2. Partager des fonctions et variables	169
III.11.2.1. Les fonctions	169
III.11.2.2. Les variables	170
III.11.3. Fonctions et variables exclusives	172
III.11.3.1. Les fonctions	172
III.11.3.2. Les variables	175
III.11.4. Les fichiers d'en-têtes	177
Conclusion	179
III.12. La gestion d'erreurs (1)	180
Introduction	180
III.12.1. Détection d'erreurs	180
III.12.1.1. Valeurs de retour	180
III.12.1.2. Variable globale errno	182
III.12.2. Prévenir l'utilisateur	183
III.12.3. Un exemple d'utilisation des valeurs de retour	184
Conclusion	185
IV. Agrégats, mémoire et fichiers	186
IV.1. Les pointeurs	187
Introduction	187
IV.1.1. Présentation	187
IV.1.1.1. Les pointeurs	187
IV.1.1.2. Utilité des pointeurs	188
IV.1.2. Déclaration et initialisation	189
IV.1.2.1. Initialisation	189
IV.1.2.2. Pointeur nul	190
IV.1.3. Utilisation	191
IV.1.3.1. Indirection (ou déréférencement)	191
IV.1.3.2. Passage comme argument	192
IV.1.3.3. Retour de fonction	193

IV.1.3.4. Pointeur de pointeur	194
IV.1.4. Pointeurs génériques et affichage	194
IV.1.4.1. Le type	194
IV.1.4.2. Afficher une adresse	195
IV.1.5. Exercice	196
IV.1.5.1. Correction	196
Conclusion	196
Contenu masqué	196
IV.2. Les structures	198
Introduction	198
IV.2.1. Définition, initialisation et utilisation	198
IV.2.1.1. Définition d'une structure	198
IV.2.1.2. Définition d'une variable de type structure	199
IV.2.1.3. Initialisation	200
IV.2.1.4. Accès à un membre	202
IV.2.1.5. Exercice	202
IV.2.2. Structures et pointeurs	203
IV.2.2.1. Accès via un pointeur	203
IV.2.2.2. Adressage	204
IV.2.2.3. Pointeurs sur structures et fonctions	204
IV.2.3. Portée et déclarations	204
IV.2.3.1. Portée	204
IV.2.3.2. Déclarations	206
IV.2.4. Les structures littérales	208
IV.2.5. Un peu de mémoire	210
IV.2.5.1. L'opérateur sizeof	211
IV.2.5.2. Alignement en mémoire	213
IV.2.5.3. L'opérateur __Alignof	217
Conclusion	218
Contenu masqué	218
IV.3. Les tableaux	220
Introduction	220
IV.3.1. Les tableaux simples (à une dimension)	220
IV.3.1.1. Définition	220
IV.3.1.2. Initialisation	221
IV.3.1.3. Accès aux éléments d'un tableau	222
IV.3.1.4. Parcours et débordement	227
IV.3.1.5. Tableaux et fonctions	228
IV.3.2. La vérité sur les tableaux	230
IV.3.2.1. Un peu d'histoire	230
IV.3.2.2. Conséquences de l'absence d'un pointeur	232
IV.3.3. Les tableaux multidimensionnels	232
IV.3.3.1. Définition	232
IV.3.3.2. Initialisation	233
IV.3.3.3. Utilisation	233
IV.3.3.4. Parcours	235

IV.3.3.5. Tableaux multidimensionnels et fonctions	235
IV.3.4. Les tableaux littéraux	237
IV.3.5. Exercices	238
IV.3.5.1. Somme des éléments	238
IV.3.5.2. Maximum et minimum	238
IV.3.5.3. Recherche d'un élément	238
IV.3.5.4. Inverser un tableau	239
IV.3.5.5. Produit des lignes	239
IV.3.5.6. Triangle de Pascal	239
Conclusion	240
Contenu masqué	241
IV.4. Les chaînes de caractères	245
Introduction	245
IV.4.1. Qu'est-ce qu'une chaîne de caractères ?	245
IV.4.1.1. Représentation en mémoire	245
IV.4.2. Définition, initialisation et utilisation	246
IV.4.2.1. Définition	246
IV.4.2.2. Initialisation	246
IV.4.2.3. Utilisation	249
IV.4.3. Afficher et récupérer une chaîne de caractères	249
IV.4.3.1. Printf	250
IV.4.3.2. Scanf	251
IV.4.4. Lire et écrire depuis et dans une chaîne de caractères	253
IV.4.4.1. La fonction snprintf	253
IV.4.4.2. La fonction sscanf	255
IV.4.5. Les classes de caractères	256
IV.4.6. Exercices	259
IV.4.6.1. Palindrome	259
IV.4.6.2. Compter les parenthèses	259
Conclusion	259
Contenu masqué	260
IV.5. TP: l'en-tête <string.h>	263
Introduction	263
IV.5.1. Préparation	263
IV.5.1.1. strlen	263
IV.5.1.2. strcpy	264
IV.5.1.3. strcat	265
IV.5.1.4. strcmp	265
IV.5.1.5. strchr	266
IV.5.1.6. strpbrk	267
IV.5.1.7. strstr	268
IV.5.2. Correction	269
IV.5.2.1. strlen	269
IV.5.2.2. strcpy	269
IV.5.2.3. strcat	269
IV.5.2.4. strcmp	269

IV.5.2.5. strchr	269
IV.5.2.6. strpbrk	269
IV.5.2.7. strstr	270
IV.5.3. Pour aller plus loin : strtok	270
IV.5.3.1. Correction	271
Contenu masqué	271
IV.6. L'allocation dynamique	276
Introduction	276
IV.6.1. La notion d'objet	276
IV.6.2. Malloc et consœurs	277
IV.6.2.1. malloc	278
IV.6.2.2. free	280
IV.6.2.3. calloc	281
IV.6.2.4. realloc	282
IV.6.3. Les tableaux multidimensionnels	284
IV.6.3.1. Allocation en un bloc	284
IV.6.3.2. Allocation de plusieurs tableaux	285
IV.6.4. Les tableaux de longueur variable	286
IV.6.4.1. Définition	287
IV.6.4.2. Utilisation	288
IV.6.4.3. Application en dehors des tableaux de longueur variable	290
IV.6.4.4. Limitations	291
Conclusion	293
IV.7. Les fichiers (1)	295
Introduction	295
IV.7.1. Les fichiers	295
IV.7.1.1. Extension de fichier	295
IV.7.1.2. Système de fichiers	295
IV.7.2. Les flux : un peu de théorie	296
IV.7.2.1. Pourquoi utiliser des flux ?	297
IV.7.2.2. stdin, stdout et stderr	298
IV.7.3. Ouverture et fermeture d'un flux	298
IV.7.3.1. La fonction fopen	298
IV.7.3.2. La fonction fclose	301
IV.7.4. Écriture vers un flux de texte	302
IV.7.4.1. Écrire un caractère	302
IV.7.4.2. Écrire une ligne	303
IV.7.4.3. La fonction fprintf	304
IV.7.5. Lecture depuis un flux de texte	304
IV.7.5.1. Récupérer un caractère	304
IV.7.5.2. Récupérer une ligne	306
IV.7.5.3. La fonction fscanf	307
IV.7.6. Écriture vers un flux binaire	307
IV.7.6.1. Écrire un multiplet	307
IV.7.6.2. Écrire une suite de multiplets	308

IV.7.7. Lecture depuis un flux binaire	309
IV.7.7.1. Lire un multiplet	309
IV.7.7.2. Lire une suite de multiplets	309
Conclusion	310
IV.8. Les fichiers (2)	311
Introduction	311
IV.8.1. Détection d'erreurs et fin de fichier	311
IV.8.1.1. La fonction feof	311
IV.8.1.2. La fonction ferror	311
IV.8.1.3. Exemple	311
IV.8.2. Position au sein d'un flux	312
IV.8.2.1. La fonction ftell	313
IV.8.2.2. La fonction fseek	313
IV.8.2.3. La fonction fgetpos	314
IV.8.2.4. La fonction fsetpos	314
IV.8.2.5. La fonction rewind	314
IV.8.3. La temporisation	315
IV.8.3.1. Introduction	315
IV.8.3.2. Interagir avec la temporisation	317
IV.8.4. Flux ouverts en lecture et écriture	321
Conclusion	323
IV.9. Le préprocesseur	324
Introduction	324
IV.9.1. Les inclusions	324
IV.9.2. Les macroconstantes	326
IV.9.2.1. Substitutions de constantes	326
IV.9.2.2. Substitutions d'instructions	327
IV.9.2.3. Macros dans d'autres macros	329
IV.9.2.4. Définition sur plusieurs lignes	330
IV.9.2.5. Annuler une définition	332
IV.9.3. Les macrofonctions	333
IV.9.3.1. Priorité des opérations	333
IV.9.3.2. Les effets de bords	334
IV.9.4. Les directives conditionnelles	335
IV.9.4.1. Les directives #if, #elif et #else	335
IV.9.4.2. Les directives #ifdef et #ifndef	337
Conclusion	337
Contenu masqué	338
IV.10. TP : un Puissance 4	340
Introduction	340
IV.10.1. Première étape : le jeu	340
IV.10.2. Correction	341
IV.10.3. Deuxième étape : une petite IA	342
IV.10.3.1. Introduction	342
IV.10.3.2. Tirage au sort	344
IV.10.4. Correction	347

IV.10.5. Troisième et dernière étape : un système de sauvegarde/restauration	347
IV.10.6. Correction	347
Contenu masqué	348
IV.11. La gestion d'erreurs (2)	380
Introduction	380
IV.11.1. Gestion de ressources	380
IV.11.1.1. Allocation de ressources	380
IV.11.1.2. Utilisation de ressources	382
IV.11.2. Fin d'un programme	384
IV.11.2.1. Terminaison normale	385
IV.11.2.2. Terminaison anormale	386
IV.11.3. Les assertions	386
IV.11.3.1. La macrofonction assert	386
IV.11.3.2. Suppression des assertions	388
IV.11.4. Les fonctions strerror et perror	388
Conclusion	390
 V. Notions avancées	 391
V.1. La représentation des types	392
Introduction	392
V.1.1. La représentation des entiers	392
V.1.1.1. Les entiers non signés	392
V.1.1.2. Les entiers signés	394
V.1.1.3. Les bits de bourrage	398
V.1.1.4. Les entiers de taille fixe	398
V.1.2. La représentations des flottants	401
V.1.2.1. Première approche	401
V.1.2.2. Une histoire de virgule flottante	402
V.1.2.3. Formalisation	404
V.1.3. La représentation des pointeurs	404
V.1.4. Ordre des multipléts et des bits	404
V.1.4.1. Ordre des multipléts	404
V.1.4.2. Ordre des bits	406
V.1.4.3. Applications	407
V.1.5. Les fonctions memset, memcpy, memmove et memcmp	409
V.1.5.1. La fonction memset	409
V.1.5.2. Les fonctions memcpy et memmove	410
V.1.5.3. La fonction memcmp	411
Conclusion	412
V.2. Les limites des types	413
Introduction	413
V.2.1. Les limites des types	413
V.2.1.1. L'en-tête <limits.h>	414
V.2.1.2. L'en-tête <float.h>	415

V.2.2.	Les dépassements de capacité	416
V.2.2.1.	Dépassement lors d'une opération	417
V.2.2.2.	Dépassement lors d'une conversion	418
V.2.3.	Gérer les dépassements entiers	419
V.2.3.1.	Préparation	419
V.2.3.2.	L'addition	419
V.2.3.3.	La soustraction	421
V.2.3.4.	La négation	421
V.2.3.5.	La multiplication	421
V.2.3.6.	La division	423
V.2.3.7.	Le modulo	423
V.2.4.	Gérer les dépassements flottants	423
V.2.4.1.	L'addition	424
V.2.4.2.	La soustraction	426
V.2.4.3.	La multiplication	426
V.2.4.4.	La division	428
	Conclusion	429
	Contenu masqué	429
V.3.	Manipulation des bits	431
	Introduction	431
V.3.1.	Les opérateurs de manipulation des bits	431
V.3.1.1.	Présentation	431
V.3.1.2.	Les opérateurs « et », « ou inclusif » et « ou exclusif »	431
V.3.1.3.	L'opérateur de négation ou de complément	433
V.3.1.4.	Les opérateurs de décalage	434
V.3.1.5.	Opérateurs combinés	436
V.3.2.	Masques et champs de bits	437
V.3.2.1.	Les masques	437
V.3.2.2.	Les champs de bits	439
V.3.3.	Les drapeaux	444
V.3.4.	Exercices	446
V.3.4.1.	Obtenir la valeur maximale d'un type non signé	446
V.3.4.2.	Afficher la représentation en base deux d'un entier	447
V.3.4.3.	Déterminer si un nombre est une puissance de deux	447
	Conclusion	447
	Contenu masqué	448
V.4.	Internationalisation et localisation	451
	Introduction	451
V.4.1.	Définitions	451
V.4.1.1.	L'internationalisation	451
V.4.1.2.	La localisation	451
V.4.2.	La fonction setlocale	451
V.4.3.	La catégorie LC_NUMERIC	453
V.4.4.	La catégorie LC_TIME	454
	Conclusion	456

V.5. La représentation des chaînes de caractères	457
Introduction	457
V.5.1. Les séquences d'échappement	457
V.5.2. Les tables de correspondances	458
V.5.2.1. La table ASCII	458
V.5.2.2. La multiplication des tables	459
V.5.2.3. La table Unicode	459
V.5.3. Des chaînes, des encodages	461
V.5.3.1. Vers une utilisation globale de l'UTF-8	463
V.5.3.2. Choisir la table et l'encodage utilisés par le compilateur	464
Conclusion	465
V.6. Les caractères larges	466
Introduction	466
V.6.1. Introduction	466
V.6.2. Traduction en chaîne de caractères larges et vice versa	466
V.6.2.1. Les chaînes de caractères littérales	466
V.6.2.2. Les entrées récupérées depuis le terminal	468
V.6.2.3. Les données provenant d'un fichier	471
V.6.3. L'en-tête <wchar.h>	473
V.6.4. <wchar.h> : les fonctions de lecture/écriture	473
V.6.4.1. L'orientation des flux	475
V.6.4.2. La fonction fwide	477
V.6.4.3. L'indicateur de conversion ls	478
V.6.5. <wchar.h> : les fonctions de manipulation des chaînes de caractères	481
V.6.6. L'en-tête <wctype.h>	482
Conclusion	483
V.7. Les énumérations	484
Introduction	484
V.7.1. Définition	484
V.7.2. Utilisation	485
Conclusion	487
V.8. Les unions	488
Introduction	488
V.8.1. Définition	488
V.8.2. Utilisation	489
Conclusion	492
V.9. Les définitions de type	493
Introduction	493
V.9.1. Définition et utilisation	493
Conclusion	495
V.10. Les pointeurs de fonction	496
Introduction	496
V.10.1. Déclaration et initialisation	496
V.10.1.1. Initialisation	496

V.10.2. Utilisation	497
V.10.2.1. Déréférencement	497
V.10.2.2. Passage en argument	498
V.10.2.3. Retour de fonction	499
V.10.3. Pointeurs de fonction et pointeurs génériques	500
V.10.3.1. La promotion des arguments	501
V.10.3.2. Les pointeurs nuls	502
Conclusion	503
V.11. Les fonctions et macrofonctions à nombre variable d'arguments	504
Introduction	504
V.11.1. Présentation	504
V.11.2. L'en-tête <stdarg.h>	505
V.11.3. Méthodes pour déterminer le nombre et le type des arguments	507
V.11.3.1. Les chaînes de formats	507
V.11.3.2. Les suites d'arguments de même type	507
V.11.3.3. Emploi d'un pivot	508
V.11.4. Les macrofonctions à nombre variable d'arguments	509
Conclusion	509
V.12. Les sélections génériques	511
Introduction	511
V.12.1. Définition et utilisation	511
Conclusion	514
V.13. T.P.: un allocateur statique de mémoire	515
Introduction	515
V.13.1. Objectif	515
V.13.2. Première étape : allouer de la mémoire	515
V.13.2.1. Mémoire statique	515
V.13.3. Correction	517
V.13.4. Deuxième étape : libérer de la mémoire	517
V.13.4.1. Ajout d'un en-tête	517
V.13.4.2. Les listes chaînées	517
V.13.5. Correction	519
V.13.6. Troisième étape : fragmentation et défragmentation	520
V.13.6.1. Fusion de blocs	520
V.13.7. Correction	520
Contenu masqué	521
VI. Annexes	530
VI.1. Index	531
Introduction	531
VI.1.1. Index	531
VI.1.1.1. *	531
VI.1.1.2. A	532
VI.1.1.3. B	532

VI.1.1.4. C	532
VI.1.1.5. D	533
VI.1.1.6. E	533
VI.1.1.7. F	534
VI.1.1.8. G	534
VI.1.1.9. H	534
VI.1.1.10. I	535
VI.1.1.11. J	535
VI.1.1.12. L	536
VI.1.1.13. M	536
VI.1.1.14. N	536
VI.1.1.15. O	537
VI.1.1.16. P	537
VI.1.1.17. R	537
VI.1.1.18. S	538
VI.1.1.19. T	538
VI.1.1.20. U	539
VI.1.1.21. V	539
VI.1.1.22. W	539

Conclusion	540
------------	-----

Introduction

Introduction

Vous souhaitez apprendre à programmer, mais vous ne savez pas comment vous y prendre ? Vous connaissez déjà le C, mais vous avez besoin de revoir un certain nombre de points ? Ou encore, vous êtes curieux de découvrir un nouveau langage de programmation ? Si oui, alors permettez-nous de vous souhaiter la bienvenue dans ce cours de programmation consacré au langage C.

Pour pouvoir suivre ce cours, **aucun prérequis n'est nécessaire** : tout sera détaillé de la manière la plus complète possible, accompagné d'exemples, d'exercices et de travaux pratiques.

Première partie

Remerciements

I. Remerciements

Avant de commencer, nous souhaitons remercier plusieurs personnes :

- [Mewtow](#)  pour sa participation à la rédaction et à l'évolution de ce cours ainsi que pour ses nombreux conseils ;
- @Arius, @Saroupille, @amael et @Glordim pour la validation de ce cours ;
- @Pouet_forever, [SofEvans](#)  , @paraze et Mathuin pour leur soutien lors des débuts de la rédaction ;
- @**Dominus Carnufex** et @patapouf pour leur suivi minutieux et leur bienveillance face à nos (nombreuses) fautes de français ;
- @Karnaj, @Ifrit et @AScriabine pour leurs suggestions et corrections ;
- @**Maëlan** pour sa relecture attentive du chapitre sur les encodages ;
- toute l'équipe de Progdupeupl et de Zeste de Savoir ;
- tous ceux qui, au fil du temps et de la rédaction, nous ont apporté leurs avis, leurs conseils, leurs points de vue et qui nous ont aidés à faire de ce cours ce qu'il est aujourd'hui ;
- et surtout vous, lecteurs, pour avoir choisi ce cours.

Deuxième partie

Premiers pas

II.1. Présentation du langage C

Introduction

Malgré tous ces langages de programmation disponibles nous allons, dans ce tutoriel, nous concentrer sur un seul d'entre eux : le C. Avant de parler des caractéristiques de ce langage et des choix qui nous amènent à l'étudier dans ce cours, faisons un peu d'histoire.

II.1.1. La programmation, qu'est-ce que c'est ?

i

Dans cette section, nous nous contenterons d'une présentation succincte qui est suffisante pour vous permettre de poursuivre la lecture de ce cours. Toutefois, si vous souhaitez un propos plus étayé, nous vous conseillons la lecture du [cours d'introduction à la programmation](#)  présent sur ce site.

La programmation est une branche de l'informatique qui crée des **programmes**. Tout ce que vous possédez sur votre ordinateur est un programme : votre navigateur (Edge, Firefox, Chrome, etc.), votre lecteur MP3, votre logiciel de discussion instantanée, vos jeux vidéos et même votre système d'exploitation (Windows, GNU/Linux, Mac OS X, etc.), bien qu'il s'agisse plutôt d'un ensemble de programmes.

II.1.1.1. Qu'est-ce qu'un programme ?

Un programme est une séquence d'**instructions**, d'ordres, donnés à notre ordinateur afin qu'il exécute des actions. Ces instructions sont assez basiques. On trouve ainsi des instructions d'addition, de soustraction, de multiplication, ou d'autres opérations mathématiques de base, qui font que notre ordinateur est techniquement une vraie machine à calculer. D'autres instructions existent, comme des opérations permettant de comparer des valeurs ou de se rendre à un point précis du programme. Créer un programme, c'est tout simplement utiliser ces instructions basiques afin de réaliser ce que l'on veut.

Ces instructions sont exécutées par un composant précis de l'ordinateur : le **processeur**.

/opt/zds/data/contents-public/le-langage-c-1__

FIGURE II.1.1. – Arrière d'un processeur Intel 80486DX ([source ↗](#))

i

Notez qu'il n'est pas possible de créer des instructions. Ces dernières sont imprimées dans les circuits du processeur ce qui fait qu'il ne peut en gérer qu'un nombre précis et prédéfini.

II.1.1.2. Le langage machine

?

D'accord, mais comment fait-on pour demander au processeur d'exécuter une instruction ? Par exemple de calculer $5 + 6$?

Vous vous en doutez, ce n'est pas aussi simple que de lui demander en français. 🍊

Le processeur ne connaît qu'un seul langage : le **langage machine**. De manière simplifiée, dans ce dernier chaque instruction est représentée par un code (basiquement un nombre) éventuellement suivi d'un ou plusieurs autres nombres (par exemple les opérandes d'une opération mathématique). Ainsi, dans un langage machine fictif où l'addition aurait pour code 57, additionner 5 et 6 pourrait s'écrire comme suit : « 57 5 6 ».

II.1.1.2.1. Le système binaire

Cependant, s'agissant d'un composant électronique, le processeur manipule et ne sait manipuler que des données **binaires**, c'est-à-dire composées d'un ou plusieurs chiffres binaires (appelés **bits** en anglais) ayant chacun deux valeurs possibles : 0 ou 1.

Dans les faits, il serait possible de construire des circuits différents, mais il est plus facile et plus fiable de se contenter de deux valeurs possibles car elles représentent la présence ou l'absence de tension électrique.

En binaire, notre exemple précédent d'instruction fictive pourrait s'écrire comme suit : « 00111001 00000101 00000110 » (nous reviendrons plus tard sur la représentation binaire, gardez juste en mémoire que c'est cette représentation qui est utilisée).

II.1.1.3. Un peu d'histoire

Originellement, la programmation se faisait directement en langage machine, au début à l'aide de cartes ou bandes perforées¹, puis à l'aide de consoles.



FIGURE II.1.2. – Une longue bande perforée et annotée de l'ordinateur IBM Mark I ([source ↗](#))



/opt/zds/data/contents-public/le-langage-c-1__

FIGURE II.1.3. – Console d’un ordinateur IBM 701 ([source ↗](#))

Inutile de vous dire que la programmation à l’époque était quelque peu... fastidieuse. 🍊

II.1.1.3.1. L’avènement des langages de programmation

Avec l’amélioration progressive et rapide des ordinateurs, vers la fin des années 1950, il devient possible d’envisager des solutions plus efficaces. Un des problèmes était celui du stockage : il n’était pas possible de stocker un programme et de l’exécuter, il devait être fourni à la machine via un support externe, typiquement des bandes perforées.

Une fois cette contrainte disparue, d’autres approches deviennent envisageables, notamment de programmer dans un autre langage qui, lui, sera traduit en langage machine. Plus précisément, un premier programme de « traduction » est écrit en langage machine et est ensuite utilisé pour traduire un autre langage vers le langage machine. Un de ces premiers langages est **l’Assembleur**.

Ce dernier est très proche du langage machine, mais utilise de courts symboles pour représenter les instructions au lieu de leur code et ne nécessite pas de travailler en binaire (ce qui est déjà beaucoup !). Ainsi, notre exemple d’instruction fictive précédent pourrait s’écrire comme suit en Assembleur : « add 5, 6 ».

Toutefois, même s’il est plus simple à utiliser que le langage machine, écrire un programme en Assembleur reste (très) fastidieux. Aussi, au fur et à mesure des progrès techniques, d’autres langages ont vu le jour en vue de simplifier la programmation. Et, vous vous en doutez, l’un d’eux est le C. 🍊

II.1.2. Le langage C

Malgré tous ces langages de programmation disponibles nous allons, dans ce tutoriel, nous concentrer sur un seul d’entre eux : le C. Avant de parler des caractéristiques de ce langage et des choix qui nous amènent à l’étudier dans ce cours, faisons un peu d’histoire.

1. Ces bandes ou cartes étaient organisées en une suite de colonne et de ligne représentant une donnée en binaire (par exemple une instruction). Sur l’image de l’IBM Mark I, on peut voir que chaque colonne est composée de 24 lignes (24 trous potentiels si vous préférez) soit de 24 *bits*. La présence d’une perforation (donc d’un trou) représente un 1 et l’absence de perforation un 0. Les instructions étaient donc encodées en langage machine colonne après colonne.

II.1.2.1. L'histoire du C

Le langage C est né au début des années 1970 dans les laboratoires de la société **AT&T** aux États-Unis. Son concepteur, **Dennis MacAlistair Ritchie** [↗](#), souhaitait améliorer un langage existant, le B, afin de lui adjoindre des nouveautés. En 1973, le C était pratiquement au point et il commença à être distribué l'année suivante. Son succès fut tel auprès des informaticiens qu'en 1989, l' **ANSI**, puis en 1990, l' **ISO**, décidèrent de le normaliser, c'est-à-dire d'établir des règles internationales et officielles pour ce langage. À l'heure actuelle, il existe quatre normes : la norme **ANSI** C89 ou **ISO** C90, la norme **ISO** C99, la norme **ISO** C11 et la norme **ISO** C18.

i

Si vous voulez en savoir plus sur l'histoire du C, lisez donc [ce tutoriel](#) [↗](#).

i

Peut-être avez-vous entendu parler du **C++** ? Il s'agit d'un langage de programmation qui a été inventé dans les années 1980 par **Bjarne Stroustrup** [↗](#), un collègue de Dennis Ritchie, qui souhaitait rajouter des éléments au C. Bien qu'il fût très proche du C lors de sa création, le C++ est aujourd'hui un langage très différent du C et n'a pour ainsi dire plus de rapport avec lui (si ce n'est une certaine proximité au niveau d'une partie de sa syntaxe). Ceci est encore plus vrai en ce qui concerne la manière de programmer et de raisonner qui sont *radicalement* différentes.

Ne croyez toutefois pas, comme peut le laisser penser leur nom ou leur date de création, qu'il y a un langage meilleur que l'autre, ils sont simplement *différents*. Si d'ailleurs votre but est d'apprendre le C++, nous vous encourageons à le faire. En effet, contrairement à ce qui est souvent dit ou lu, *il n'y a pas besoin de connaître le C pour apprendre le C++*. Si tel est votre souhait, sachez qu'il existe un [cours C++](#) [↗](#) sur ce site.

II.1.2.2. Pourquoi apprendre le C ?

C'est une très bonne question. 🍊

Ne nous mentons pas d'entrée de jeu : le C est un vieux langage (il a soufflé ses 50 bougies) et il est justifié de se demander pourquoi il devrait être encore étudié ou même utilisé de nos jours. À cette question, nous voyons principalement deux réponses. Nous pourrions en citer d'autres, comme les performances du langage, mais elles ne sont pas nécessairement propres au C.

1. La première est la base de code existante. Le C a été massivement utilisé par le passé ce qui fait que de *très* nombreux programmes ou bibliothèques que nous utilisons et desquelles nous dépendons ont été écrites et sont toujours écrites en C. Si vous souhaitez contribuer ou étudier ces programmes ou bibliothèques, l'apprentissage du C est indispensable.
2. La seconde est le caractère « proche du langage machine » du C. Bien qu'imprécise et manquant de nuances, cette affirmation traduit le fait qu'il est globalement facile d'entrevoir à quoi ressemblera une portion de code C une fois traduit en langage machine. Cette « proximité » permet de disposer d'un important contrôle sur le comportement du programme, ce qui est crucial dans certains domaines, comme le développement de système d'exploitation ou la programmation sur des systèmes embarqués qui ont longtemps été deux chasses gardées du C.

II. Premiers pas

Ces deux éléments sont à notre goût suffisant pour justifier l'apprentissage de ce langage. 🍊

II.1.2.3. La norme

Comme précisé plus haut, le C est un langage qui a été normalisé à trois reprises. Ces normes sont avant tout destinées aux développeurs de **compilateurs** (il s'agit du nom donné au programme chargé de traduire un code C en langage machine) et sont un peu l'équivalent de nos règles d'orthographe, de grammaire et de conjugaison. Elles fixent un cadre et des règles de sorte que tous les compilateurs utilisent la même syntaxe et qu'un même code compilé avec différents compilateurs ait le même comportement.

Toutefois, ces normes servent également de référence à tous les programmeurs et peuvent les aider chaque fois qu'ils ont un doute ou une question en rapport avec le langage. Elles ne sont pas parfaites et ne répondent pas à toutes les questions, mais elles restent *la* référence pour tout programmeur.

Dans ce cours, nous avons décidé de nous reposer sur la norme C11. Il ne s'agit pas de la dernière en date, toutefois la norme C18 n'apporte dans les faits rien de nouveaux et vise uniquement à corriger ou préciser certains points de la norme C11. Aussi, ce choix n'aura aucun impact sur votre apprentissage du C.

i

Pour les curieux, voici [un lien](#) vers le brouillon de cette norme. Cela signifie qu'il ne s'agit pas de la version définitive et officielle, cependant il est largement suffisant pour notre niveau et, surtout, il est gratuit (la norme officielle coûtant *très* cher 🍊). Notez que celui-ci est rédigé en anglais.

II.1.3. Notre cible

Avant de commencer à programmer, il nous faut aussi définir ce que nous allons programmer, autrement dit le type de programme que nous allons réaliser. Il existe en effet deux grands types de programmes : les programmes **graphiques** et les programmes **en console**.

Les programmes graphiques sont les plus courants et les plus connus puisqu'il n'y a pratiquement qu'eux sous Windows ou Mac OS X par exemple. Vous en connaissez sans doute énormément tel que les lecteurs de musique, les navigateurs, les logiciels de discussions instantanées, les suites bureautiques, les jeux vidéos, etc. Ce sont tous des programmes dit « graphiques » car disposant d'une interface graphique (ou **GUI**).

II. Premiers pas

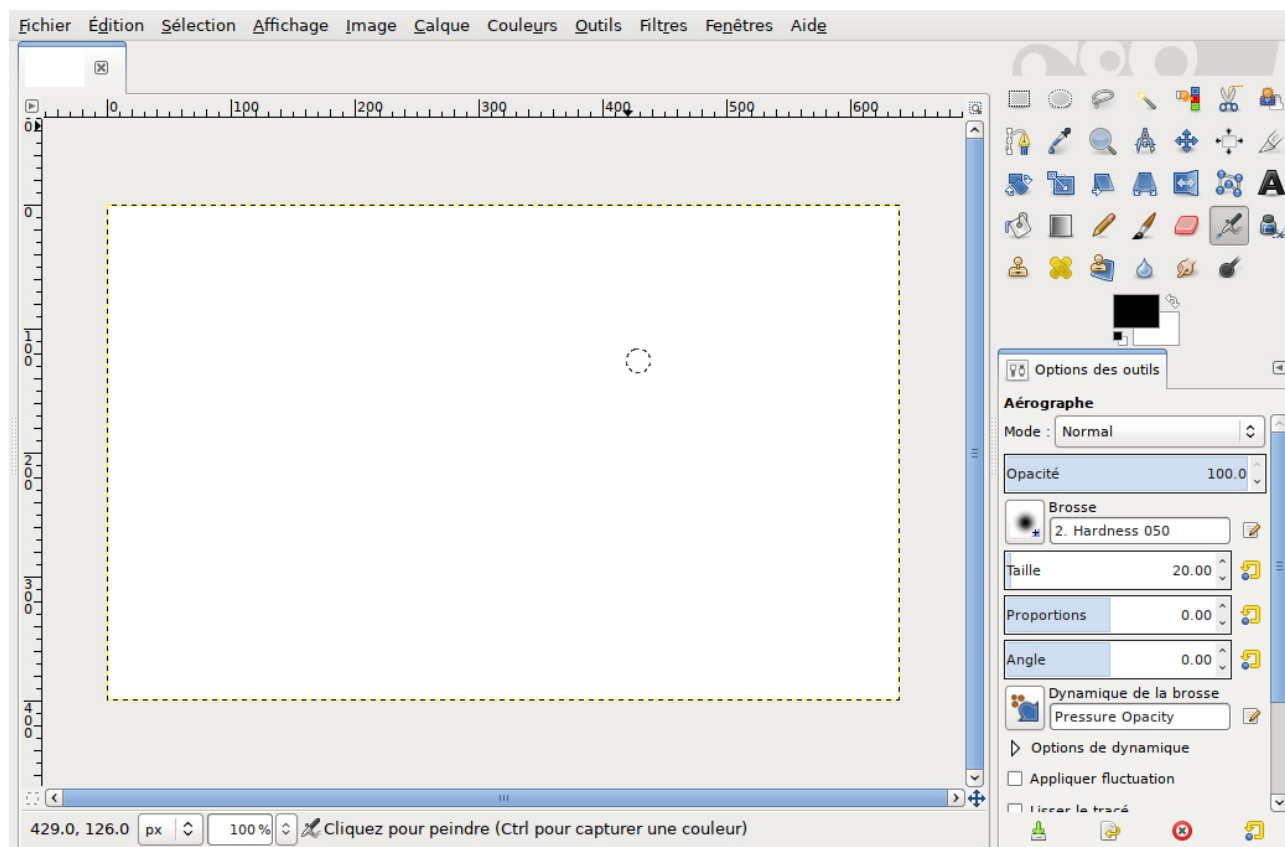


FIGURE II.1.4. – L’éditeur d’images Gimp est un programme graphique

Cependant, écrire ce genre de programmes demande beaucoup de connaissances, ce qui nous manque justement pour l’instant. Aussi, nous allons devoir nous rabattre sur le deuxième type de programme : les programmes en console.

Les programmes console sont apparus en même temps que l’écran. Ils étaient très utilisés dans les années 1970/1980 (certains d’entre vous se souviennent peut-être de MS-DOS), mais ont fini par être remplacés par une interface graphique avec la sortie de Mac OS et de Windows. Cependant, ils existent toujours et redeviennent quelque peu populaires chez les personnes utilisant GNU/Linux ou *BSD.

Voici un exemple de programme en console (il s’agit de [GNU Chess](#), un jeu d’échecs performant entièrement en ligne de commande).

```
1 White (1) : d4
2 1. d4
3
4 black KQkq d3
5 r n b q k b n r
6 p p p p p p p p
7 . . . . . . . .
8 . . . . . . . .
9 . . . P . . . .
10 . . . . . . . .
11 P P P . P P P P
```

II. Premiers pas

```
12 R N B Q K B N R
13
14 Thinking...
15
16 white KQkq
17 r n b q k b . r
18 p p p p p p p p
19 . . . . . n . .
20 . . . . . . . .
21 . . . P . . . .
22 . . . . . . . .
23 P P P . P P P P
24 R N B Q K B N R
25
26
27 My move is : Nf6
28 White (2) :
```

Ce sera le type de programme que nous allons apprendre à créer. Rassurez-vous, quand vous aurez fini le cours, vous aurez les bases pour apprendre à créer des programmes graphiques.



II.2. Les outils nécessaires

Introduction

Maintenant que les présentations sont faites, il est temps de découvrir les outils nécessaires pour programmer en C. Le strict minimum pour programmer se résume en deux points :

- un **éditeur de texte** (à ne pas confondre avec un **traitement de texte** comme *Microsoft Word* ou *LibreOffice Writer*) : ce logiciel va servir à l'écriture du code source. Techniquement, n'importe quel éditeur de texte suffit, mais il est souvent plus agréable d'en choisir un qui n'est pas trop minimaliste ;
- un **compilateur** : c'est le logiciel le plus important puisqu'il va nous permettre de transformer le code écrit en langage C en un fichier exécutable.

À partir de là, il existe deux solutions : utiliser ces deux logiciels séparément ou bien les utiliser au sein d'un **environnement intégré de développement** (abrégé EDI). Dans le cadre de ce cours, nous avons choisi la première option, majoritairement dans un souci de transparence et de simplicité. En effet, si les EDI peuvent être des compagnons de choix, ceux-ci sont avant tout destinés à des programmeurs expérimentés et non à de parfaits débutants.

II.2.1. Windows

II.2.1.1. Le compilateur

Nous vous proposons d'installer le compilateur **GCC** à l'aide de **MSys2** [↗](#) qui est une suite logicielle permettant de disposer d'un environnement compatible **POSIX** [↗](#) sous Windows.

Rendez-vous sur le [site de MSys2](#) [↗](#) et téléchargez la version correspondant à votre système (32 ou 64 *bits*). Exécutez ensuite le programme et suivez la procédure d'installation. Une fois l'installation terminée, vous serez face au terminal de MSys2.

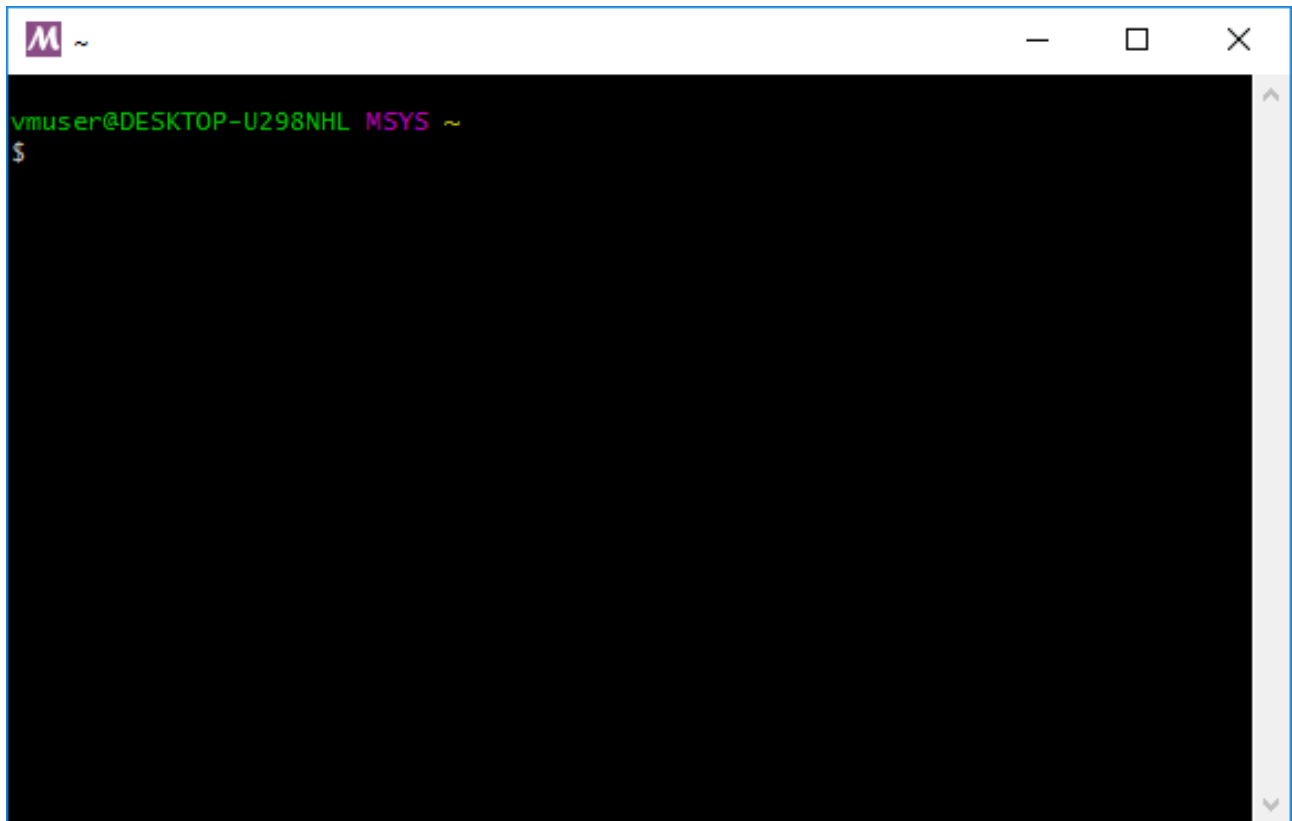
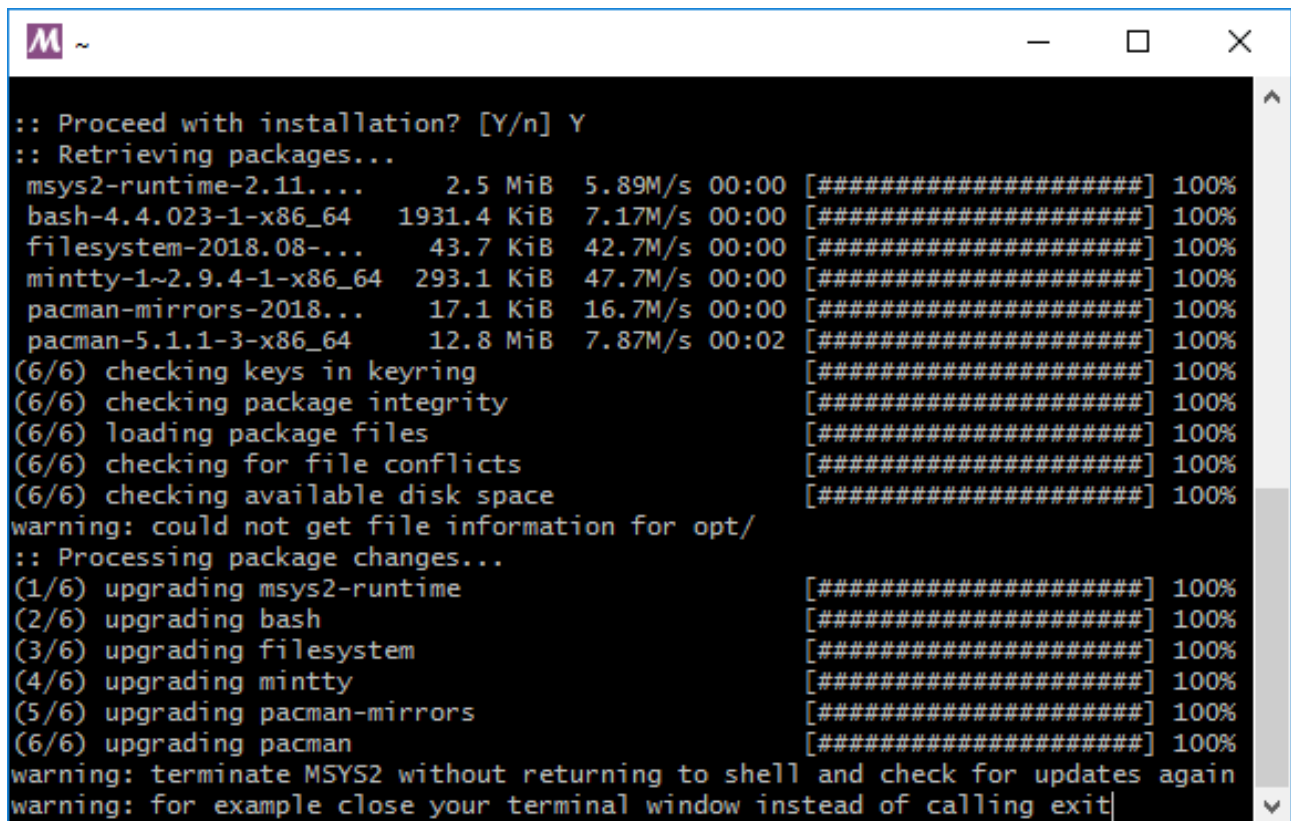


FIGURE II.2.1. – Terminal de MSys2

Entrez alors la commande `pacman -Syu` et validez ce qui est proposé. Une fois cette commande exécutée, il devrait vous être demandé de fermer le terminal.



```
:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
msys2-runtime-2.11...      2.5 MiB   5.89M/s  00:00 [#####] 100%
bash-4.4.023-1-x86_64     1931.4 KiB 7.17M/s  00:00 [#####] 100%
filesystem-2018.08-...    43.7 KiB  42.7M/s  00:00 [#####] 100%
mintty-1~2.9.4-1-x86_64   293.1 KiB 47.7M/s  00:00 [#####] 100%
pacman-mirrors-2018...    17.1 KiB  16.7M/s  00:00 [#####] 100%
pacman-5.1.1-3-x86_64     12.8 MiB  7.87M/s  00:02 [#####] 100%
(6/6) checking keys in keyring [#####] 100%
(6/6) checking package integrity [#####] 100%
(6/6) loading package files [#####] 100%
(6/6) checking for file conflicts [#####] 100%
(6/6) checking available disk space [#####] 100%
warning: could not get file information for opt/
:: Processing package changes...
(1/6) upgrading msys2-runtime [#####] 100%
(2/6) upgrading bash [#####] 100%
(3/6) upgrading filesystem [#####] 100%
(4/6) upgrading mintty [#####] 100%
(5/6) upgrading pacman-mirrors [#####] 100%
(6/6) upgrading pacman [#####] 100%
warning: terminate MSYS2 without returning to shell and check for updates again
warning: for example close your terminal window instead of calling exit|
```

FIGURE II.2.2. – Il est demandé de fermer le terminal

Fermez alors cette fenêtre et ouvrez en une nouvelle. Dans celle-ci, exécutez la commande `pacman -Su` puis `pacman -S gcc` et validez à chaque fois les actions proposées. Le compilateur est à présent installé. 🍊

II.2.1.2. L'éditeur de texte

L'éditeur de texte va nous permettre d'écrire notre code source et de l'enregistrer. L'idéal est d'avoir un éditeur de texte facile à utiliser et pas trop minimaliste. Si jamais vous avez déjà un éditeur de texte et que vous l'appréciez, n'en changez pas, il fera sûrement l'affaire.

Si vous n'avez pas d'idée, nous vous conseillons [Notepad++](#) qui est simple, pratique et efficace. Pour le télécharger, rendez-vous simplement dans la rubrique **Téléchargements** du menu principal.



Veillez-bien à ce que l'encodage de votre fichier soit **UTF-8 (sans BOM)** (voyez le menu éponyme à cet effet).

II.2.1.3. Introduction à la ligne de commande

Référez-vous à l'introduction dédiée à GNU/Linux et *BSD.



Le dossier `/home/utilisateur` (où « utilisateur » correspond à votre nom d'utilisateur) correspond au dossier `C:\msysxx\home\utilisateur` (où « xx » est 32 ou 64 suivant la version de MSys2 que vous avez installée).

II.2.2. GNU/Linux et *BSD

II.2.2.1. Le compilateur

Suivant le système que vous choisirez, vous aurez ou non le choix entre différents compilateurs. Si vous n'avez pas d'idée, nous vous conseillons d'opter pour **GCC** en installant le paquet éponyme à l'aide de votre gestionnaire de paquet.

II.2.2.2. L'éditeur de texte

Ce serait un euphémisme de dire que vous avez l'embarras du choix. Il existe une pléthore d'éditeurs de texte fonctionnant en ligne de commande ou avec une interface graphique, voire les deux.

Pour n'en citer que quelques-uns, en ligne de commande vous trouverez par exemple : Vim et Emacs (les deux monstres de l'édition), nano ou encore, pour les plus fous ou courageux, [ed](#) [↗](#). Côté graphique, vous avez entre autres : Gedit, Mousepad et Kate.

II.2.2.3. Introduction à la ligne de commande

La ligne de commande, derrière son aspect archaïque, n'est en fait qu'une autre manière de réaliser des tâches sur un ordinateur. La différence majeure avec une interface graphique étant que les instructions sont données non pas à l'aide de boutons et de cliques de souris, mais exclusivement à l'aide de texte. Ainsi, pour réaliser une tâche donnée, il sera nécessaire d'invoquer un programme (on parle souvent de **commandes**) en tapant son nom.

La première chose que vous devez garder à l'esprit, c'est le dossier dans lequel vous vous situez. Suivant le terminal que vous employez, celui-ci est parfois indiqué en début de ligne et terminé par le symbole `$` ou `%`. Ce dossier est celui dans lequel les actions (par exemple la création d'un répertoire) que vous demanderez seront exécutées. Normalement, par défaut, vous devriez vous situer dans le répertoire : `/home/utilisateur` (où `utilisateur` correspond à votre nom d'utilisateur). Ceci étant posé, voyons quelques commandes basiques.

La commande `pwd` (pour *print working directory*) vous permet de connaître le répertoire dans lequel vous êtes.

```
1 $ pwd
2 /home/utilisateur
```


II. Premiers pas

La commande `mkdir` (pour *make directory*) vous permet de créer un nouveau dossier. Pour ce faire, tapez `mkdir` suivi d'un espace et du nom du nouveau répertoire. Par exemple, vous pouvez créer un dossier `programmation` comme suit :

```
1 $ mkdir programmation
```

La commande `ls` (pour *list*) vous permet de lister le contenu d'un dossier. Vous pouvez ainsi vérifier qu'un nouveau répertoire a bien été créé.

```
1 $ ls
2 programmation
```

i

Le résultat ne sera pas forcément le même que ci-dessus, cela dépend du contenu de votre dossier. L'essentiel est que vous retrouviez bien le dossier que vous venez de créer.

Enfin, la commande `cd` (pour *change directory*) vous permet de vous déplacer d'un dossier à l'autre. Pour ce faire, spécifiez simplement le nom du dossier de destination.

```
1 $ cd programmation
2 $ pwd
3 /home/utilisateur/programmation
```

i

Le dossier spécial `..` représente le répertoire parent. Il vous permet donc de revenir en arrière dans la hiérarchie des dossiers. Le dossier spécial `.` représente quant à lui le dossier courant.

Voilà, avec ceci, vous êtes fin prêt pour compiler votre premier programme. Vous pouvez vous rendre à la deuxième partie de ce chapitre.

II.2.3. Mac OS X

II.2.3.1. Le compilateur

Allez dans le dossier `/Applications/Utilitaires` et lancez l'application `terminal.app`. Une fois ceci fait, entrez la commande suivante :

```
1 xcode-select --install
```

II. Premiers pas

et cliquez sur **Installer** dans la fenêtre qui apparaît. Si vous rencontrez le message d'erreur ci-dessous, cela signifie que vous disposez déjà des logiciels requis.

1	Impossible d'installer ce logiciel car il n'est pas disponible actuellement depuis le serveur de mise à jour de logiciels.
---	--

Si vous ne disposez pas de la commande indiquée, alors rendez-vous sur le site de développeur d'Apple : [Apple developer connection](#) . Il faudra ensuite vous rendre sur le **Mac Dev Center** puis, dans **Additional download**, cliquez sur **View all downloads**. Quand vous aurez la liste, il vous suffit de chercher la version 3 de Xcode (pour *Leopard* et *Snow Leopard*) ou 2 pour les versions antérieures (*Tiger*). Vous pouvez aussi utiliser votre CD d'installation pour installer Xcode (sauf pour *Lion*).

II.2.3.2. L'éditeur de texte

Comme pour GNU/Linux et *BSD, vous trouverez un bon nombres d'éditeurs de texte. Si toutefois vous êtes perdu, nous vous conseillons [TextWrangler](#) ou [Smultron](#) .

II.2.3.3. Introduction à la ligne de commande

Référez-vous à l'introduction dédiée à GNU/Linux et *BSD.

II.3. Notre premier programme C

Introduction

Maintenant que nous avons introduit le langage C et installé les outils nécessaires, il est temps de plonger dans le vif du sujet et de compiler notre premier programme. 🍊

II.3.1. Premier programme

Bien, il est à présent temps d'écrire et de compiler notre premier programme ! Pour ce faire, ouvrez votre éditeur de texte et entrez les lignes suivantes.

```
1 int main(void)
2 {
3     return 0;
4 }
```

Ensuite, enregistrez ce fichier dans un dossier de votre choix et nommez-le « main.c ».

i

Rappelez-vous, sous Windows, le dossier `/home/utilisateur` (où « utilisateur » correspond à votre nom d'utilisateur) correspond au dossier `C:\msysxx\home\utilisateur` (où « xx » est 32 ou 64 suivant la version de MSys2 que vous avez installée).

Une fois ceci fait, rendez-vous dans le dossier contenant le fichier à l'aide d'un terminal et exécutez la commande ci-dessous.

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin
   main.c
```

?

À quoi correspond tout ce texte entre `gcc` et `main.c` ?

Il s'agit d'options qui modifient le comportement de GCC. Nous n'allons pas toutes les détailler, mais basiquement nous demandons à GCC de nous avertir si certains points de notre code sont potentiellement problématiques ou s'ils ne respectent pas la norme C11.

II. Premiers pas

Si tout se passe bien, vous devriez obtenir un fichier « a.exe » sous Windows et un fichier « a.out » sous Linux. Vous pouvez exécuter ce programme en tapant `./a.exe` ou `./a.out`.

i

Si vous rencontrez des erreurs lors de la compilation, référez-vous à la section « Erreur lors de la compilation ».

?

Je viens de le faire, mais il ne se passe rien...

Cela tombe bien, c'est exactement ce que fait ce programme : rien. 🍌

Voyons cela plus en détails.

Ce bout de code est appelé une **fonction** qui est la brique de base de tout programme écrit en C. Une fonction se compose d'une ou plusieurs **instructions** placées entre deux accolades (`{` et `}`). Chaque instruction est terminée par un point-virgule (`;`). Ici, notre fonction se nomme **main**. Il s'agit d'une fonction *obligatoire* dans tout programme C, car il s'agit de la fonction par laquelle l'exécution commence et se termine.

Notre fonction `main()` ne comporte qu'une seule instruction : `return 0;`, qui met fin à son exécution (et, par conséquent, à celle du programme) et permet d'indiquer à notre système d'exploitation que l'exécution s'est correctement déroulée (une valeur différente de zéro indiquerait une erreur).

Le nom de la fonction est précédé du mot-clé `int` (pour *integer*) qui est un nom de type indiquant que la fonction retourne une valeur entière. À l'intérieur des parenthèses, il y a le mot `void` qui signifie que la fonction ne reçoit pas de paramètres, nous reviendrons sur tout cela en temps voulu.

II.3.2. Erreur lors de la compilation

Si cela ne vous est pas arrivé lors de la compilation du premier code d'exemple, il vous arrivera de rencontrer des erreurs lors de la compilation. Le plus souvent, ces dernières se produisent à cause d'erreurs de syntaxe.

Par exemple, si vous essayez de compiler le code suivant, vous obtiendrez une erreur.

```
1 intxxx main(void)
2 {
3     return 0;
4 }
```

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin
   main.c
```

II. Premiers pas

```
2 main.c:1:1: error: unknown type name 'intxxx'
3     1 | intxxx main(void)
4       | ^~~~~~
```

Suivant votre système, le message peut être en anglais ou en français. Dans les deux cas, le compilateur vous indique le fichier où l'erreur s'est produite (`main.c`) et une indication de la ligne et du caractère (1:1) où se situe l'erreur. Ici le compilateur nous précise que `intxxx` n'est pas un type connu (nous présenterons la notion de type d'ici peu). En effet, le type correct est `int`.

Essayons avec un autre exemple incorrect.

```
1 int main(void)
2 {
3     return 0
4 }
```

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin
   main.c
2 main.c: In function 'main':
3 main.c:3:13: error: expected ';' before '}' token
4     3 |         return 0
5       |                 ^
6       |                 ;
7     4 |     }
8       |     ~
```

De nouveau, le compilateur nous affiche un message d'erreur et nous précise cette fois qu'il manque visiblement un point-virgule à la fin de la ligne `return 0`.

Toutefois, en fonction des erreurs, le message fourni par le compilateur n'est pas forcément explicite ou n'indique pas nécessairement la ligne incriminée. Ainsi, si nous oublions d'écrire l'accolade ouvrante, nous obtenons un message un peu plus ésotérique.

```
1 int main(void)
2     return 0;
3 }
```

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin
   main.c
2 main.c: In function 'main':
3 main.c:2:5: error: expected declaration specifiers before 'return'
4     2 |     return 0;
```

II. Premiers pas

```
5      |      ^~~~~~
6 main.c:3:1: error: expected declaration specifiers before '}' token
7     3 | }
8      | ^
9 main.c:4: error: expected '{' at end of input
```

Cependant, ne paniquez pas si vous obtenez un tel message, relisez calmement votre code et vous tomberez assez rapidement sur la ligne qui pose problème. 🍊

II.3.3. Les commentaires

Il est possible d'ajouter des commentaires dans un code source, par exemple pour décrire des passages un peu moins lisibles ou tout simplement pour offrir quelques compléments d'information au lecteur du code. Nous en utiliserons souvent dans la suite de ce cours pour rendre certains exemples plus parlant.

Un commentaire est ignoré par le compilateur, il n'est pas présent dans l'exécutable final. Il ne sert qu'au programmeur et aux lecteurs du code.

Un commentaire en C est écrit soit en étant précédé de deux barres obliques `//`, soit en étant placé entre les signes `/*` et `*/`. Dans le dernier cas, le commentaire peut alors s'étendre sur plusieurs lignes.

```
1 // Ceci est un commentaire.
```

```
1 /* Ceci est un autre commentaire. */
```

```
1 /* Ceci est un commentaire qui
2    prends plusieurs lignes. */
```

Conclusion

En résumé

- Il est possible de compiler un programme écrit en C à l'aide de la commande `gcc`.
- Il est possible que la compilation échoue, auquel cas un message d'erreur est fourni par le compilateur. Ce message n'est pas toujours limpide, mais une relecture attentive vous permettra de trouver la source de l'erreur.
- Il est possible d'ajouter des commentaires dans un code source, soit en précédant la ligne de deux barres obliques (`//`), soit en plaçant une ou plusieurs lignes entre `/*` et `*/`.

Troisième partie

Les bases du langage C

III.1. Les variables

Introduction

Programmer, c'est avant tout donner des ordres à notre ordinateur afin qu'il réalise ce que l'on souhaite. Ces ordres vont permettre à notre ordinateur de manipuler de l'information sous différentes formes (nombres, textes, vidéos, etc). À ce stade, nous savons que ces ordres, ces instructions sont exécutées par notre processeur. Cependant, nous ne savons toujours pas comment donner des ordres, ni comment manipuler de l'information.

Ce chapitre vous expliquera comment manipuler les types de données les plus simples du langage C, les nombres et les lettres (ou caractères), grâce à ce qu'on appelle des **variables**. Après celui-ci, vous pourrez ainsi profiter de votre ordinateur comme s'il s'agissait d'une grosse calculatrice. Néanmoins, rassurez-vous, le niveau en maths de ce chapitre sera assez faible : si vous savez compter, vous pourrez le comprendre facilement !

Cela peut paraître un peu bête et pas très intéressant, mais il faut bien commencer par les bases. Manipuler du texte ou de la vidéo est complexe et nécessite en plus de savoir comment manipuler des nombres. *Eh oui !* Comme vous allez le voir, tout est nombre pour notre ordinateur, même le texte et la vidéo.

III.1.1. Qu'est-ce qu'une variable ?

Pour comprendre ce qu'est une variable et comment manipuler celles-ci, il faut commencer par comprendre comment notre ordinateur fait pour stocker des données. En théorie, un ordinateur est capable de stocker tout type d'information. Mais comment est-il possible de réaliser un tel miracle alors qu'il ne s'agit finalement que d'un amas de circuits électriques ?

III.1.1.1. Codage des informations

Peut-être avez-vous déjà entendu le proverbe suivant : « si le seul outil que vous avez est un marteau, vous verrez tout problème comme un clou » (Abraham Maslow). *Hé* bien, l'idée est un peu la même pour un ordinateur : ce dernier ne sachant utiliser que des nombres, il voit toute information comme une suite de nombres.

L'astuce consiste donc à transformer une information en nombre pour que l'ordinateur puisse la traiter, autrement dit la **numériser**. Différentes techniques sont possibles pour atteindre cet objectif, une des plus simples étant une table de correspondance, par exemple entre un nombre et un caractère.

III. Les bases du langage C

Carac- tère	Nombre
A	1
B	2
C	3

III.1.1.1.1. Binaire

Cependant, comme si cela ne suffisait pas, un ordinateur ne compte pas comme nous : il compte en base deux (l'andouille!).



En base deux ?

La base correspond au nombre de chiffres disponibles pour représenter un nombre. En base 10, nous disposons de dix chiffres : zéro, un, deux, trois, quatre, cinq, six, sept, huit et neuf. En base deux, nous en avons donc... deux : zéro et un. Pour ce qui est de compter, c'est du pareil au même : nous commençons par épuiser les unités : 0, 1 ; puis nous passons aux dizaines : 10, 11 ; puis aux centaines : 100, 101, 110, 111 ; et ainsi de suite. Ci-dessous un petit tableau de correspondance entre la base deux et la base dix.

Base deux	Base dix
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10

Un chiffre binaire (un zéro ou un un) est appelé un *bit* en anglais. Il s'agit de la contraction de l'expression « *binary digit* ». Nous l'emploierons assez souvent dans la suite de ce cours par souci d'économie.



Mais pourquoi utiliser la base deux et non la base dix ?

Parce que les données circulent sous forme de courants électriques. Or, la tension de ceux-ci n'étant pas toujours stable, il est difficile de réaliser un système fiable sachant détecter dix valeurs différentes. Par contre, c'est parfaitement possible avec deux valeurs : il y a du courant ou il n'y en a pas.

III.1.1.2. La mémoire

Nous savons à présent que notre ordinateur ne sait employer que des nombres représentés en base deux.



Mais comment stocker tout ce fatras de nombres ?

Hé bien, les *bits* sont stockés dans un composant électronique particulier de l'ordinateur : la **mémoire**. Enfin, nous disons « la mémoire », mais il y en a en fait plusieurs.



Mais pourquoi plusieurs mémoires et pas une seule ?

Le fait est qu'il est actuellement impossible de créer des mémoires qui soient à la fois rapides et capables de contenir beaucoup de données. Nous ne pouvons donc utiliser une seule grosse mémoire capable de stocker toutes les données dont nous avons besoin. Ce problème s'est posé dès les débuts de l'informatique, comme en témoigne cette citation des années 1940, provenant des concepteurs d'un des tout premiers ordinateurs.

Idéalement, nous désirerions une mémoire d'une capacité indéfiniment large telle que n'importe quelle donnée soit immédiatement accessible. Nous sommes forcés de reconnaître la possibilité de la construction d'une hiérarchie de mémoire, chacune ayant une capacité plus importante que la précédente, mais accessible moins rapidement.

Burks, Goldstine, et Von Neumann

Mais les chercheurs et ingénieurs du début de l'informatique ont trouvé une solution : segmenter la mémoire de l'ordinateur en plusieurs sous-mémoires, de tailles et de vitesses différentes, utilisées chacune suivant les besoins. Nous aurons donc des mémoires pouvant contenir peu de données et rapides, à côté de mémoires plus importantes et plus lentes.

Nous vous avons dit que l'ordinateur utilisait plusieurs mémoires. Trois d'entre elles méritent à notre sens votre attention :

- les registres ;
- la mémoire vive (ou **RAM** en anglais) ;
- le disque dur.

III. Les bases du langage C

Les **registres** sont des mémoires intégrées dans le processeur, utilisées pour stocker des données temporaires. Elles sont très rapides, mais ne peuvent contenir que des données très simples, comme des nombres.

La **mémoire vive** est une mémoire un peu plus grosse, mais plus lente que les registres. Elle peut contenir pas mal de données et est généralement utilisée pour stocker les programmes en cours d'exécution ainsi que les données qu'ils manipulent.

Ces deux mémoires (les registres et la mémoire vive) ont tout de même un léger défaut : elles perdent leur contenu quand elles ne sont plus alimentées... Autant dire que ce n'est pas le meilleur endroit pour stocker un système d'exploitation ou des fichiers personnels. Ceci est le rôle du **disque dur**, une mémoire avec une capacité très importante, mais très lente qui a toutefois l'avantage d'assurer la persistance des données.

En C, la mémoire la plus manipulée par le programmeur est la mémoire vive. Aussi, nous allons nous y intéresser d'un peu plus près dans ce qui suit.

III.1.1.2.1. Bits, multipléts et octets

Dans la mémoire vive, les *bits* sont regroupés en « paquets » de quantité fixe : des « **cases mémoires** », aussi appelées **multipléts** (ou *bytes* en anglais). À quelques exceptions près, les mémoires utilisent des multipléts de huit *bits*, aussi appelés **octets**. Un octet peut stocker 256 informations différentes (vous pouvez faire le calcul vous-même : combien vaut 11111111 en base deux ? 🍊). Pour stocker plus d'informations, il sera nécessaire d'utiliser plusieurs octets.

III.1.1.2.2. Adresse mémoire

Néanmoins, il est bien beau de stocker des données en mémoire, encore faut-il pouvoir remettre la main dessus.

Dans cette optique, chaque multipléts de la mémoire vive se voit attribuer un nombre unique, **une adresse**, qui va permettre de le sélectionner et de l'identifier parmi tous les autres. Imaginez la mémoire vive de l'ordinateur comme une immense armoire, qui contiendrait beaucoup de tiroirs (les cases mémoires) pouvant chacun contenir un multipléts. Chaque tiroir se voit attribuer un numéro pour le reconnaître parmi tous les autres. Nous pourrions ainsi demander quel est le contenu du tiroir numéro 27. Pour la mémoire, c'est pareil. Chaque case mémoire a un numéro : son adresse.

Adresse	Contenu mémoire
0	11101010
1	01111111
2	00000000
3	01010101
4	10101010
5	00000000

En fait, vous pouvez comparer une adresse à un numéro de téléphone : chacun de vos correspondants a un numéro de téléphone et vous savez que pour appeler telle personne, vous devez composer tel numéro. Les adresses mémoires fonctionnent exactement de la même façon !

i

Plus généralement, toutes les mémoires disposent d'un mécanisme similaire pour retrouver les données. Aussi, vous entendrez souvent le terme de **référence** qui désigne un moyen (comme une adresse) permettant de localiser une donnée. Il s'agit simplement d'une notion plus générale.

III.1.1.3. Les variables

Tout cela est bien sympathique, mais manipuler explicitement des références (des adresses si vous préférez) est un vrai calvaire, de même que de s'évertuer à calculer en base deux. Heureusement pour nous, les langages de programmation (et notamment le C), se chargent d'effectuer les conversions pour nous et remplacent les références par des variables.

Une variable correspondra à une portion de mémoire, appelée **objet**, à laquelle nous donnerons un nom. Ce nom permettra d'identifier notre variable, tout comme une référence permet d'identifier une portion de mémoire parmi toutes les autres. Nous allons ainsi pouvoir nommer les données que nous manipulons, chacun de ces noms étant remplacé lors de la compilation par une référence (le plus souvent une adresse).

III.1.2. Déclarer une variable

Entrons maintenant dans le vif du sujet en apprenant à déclarer nos variables. Tout d'abord, sachez qu'une variable est constituée de deux éléments obligatoires :

- un **type** ;
- un **identificateur** qui est en gros le « nom » de la variable.

Le type d'une variable permet d'indiquer ce qui y sera stocké, par exemple : un caractère, un nombre entier, un nombre réel (ou nombre à **virgule flottante**, parfois tout simplement abrégé en « flottant »), etc. Pour préciser le type d'une variable, il est nécessaire d'utiliser un mot-clé spécifique (il y en a donc un pour chaque type).

Une fois que nous avons décidé du nom et du type de notre variable, nous pouvons la créer (on dit aussi la déclarer) comme suit.

```
1 type identificateur;
```

En clair, il suffit de placer un mot-clé indiquant le type de la variable et de placer le nom qu'on lui a choisi immédiatement après.



Faites bien attention au point-virgule à la fin !

III.1.2.1. Les types

Comme dit précédemment, un type permet d'indiquer au compilateur quel genre de données nous souhaitons stocker. Ce type va permettre de préciser :

- toutes les valeurs que peut prendre la variable ;
- les opérations qu'il est possible d'effectuer avec (il n'est par exemple pas possible de réaliser une division entière avec un nombre à virgule flottante, nous y reviendrons).

Définir le type d'une variable permet donc de préciser son contenu potentiel et ce que nous pouvons faire avec. Le langage C fournit dix types de base.

Type	Sert à stocker
<code>_Bool</code>	un entier
<code>char</code>	un caractère
<code>signed char</code>	un entier
<code>short int</code>	un entier
<code>int</code>	un entier
<code>long int</code>	un entier
<code>long long int</code>	un entier
<code>float</code>	un réel
<code>double</code>	un réel
<code>long double</code>	un réel

Le type `char` sert au stockage de caractères.

Les types `signed char`, `short int`, `int`, `long int` et `long long int` servent tous à stocker des nombres entiers qui peuvent prendre des valeurs positives, négatives, ou nulles. On dit qu'il s'agit de types signés, car ils peuvent comporter un signe. Pour chacun de ces cinq types, il existe un type équivalent dit **non signé**. Un type entier non signé est un type entier qui n'accepte que des valeurs positives ou nulles : il ne peut pas stocker de valeurs négatives. Pour déclarer des variables d'un type non signé, il vous suffit de faire précéder le nom du type entier du mot-clé `unsigned`.



Par défaut, un type entier est signé, le mot-clé `signed` est donc implicite et facultatif, sauf pour le type `char`, car il sert à distinguer le type `char`, qui stocke des caractères, des types `signed char` et `unsigned char`, qui stockent des entiers. Les types `char`, `signed char` et `unsigned char` sont donc *trois types différents*. Ceci constitue une exception par



rapport aux autres types entiers où, par exemple, `int` et `signed int` désignent le même type.



En cas de manque d'information concernant le type d'un entier lors d'une déclaration, c'est le type `int` qui sera utilisé. Ainsi, `short`, `long` et `long long` sont respectivement des raccourcis pour `short int`, `long int` et `long long int`. De même, le mot-clé `unsigned` seul signifie `unsigned int`.

Le type `_Bool`³ est un type entier non signé un peu particulier : il permet de stocker soit 0, soit 1, on parle d'un **booléen** (nous verrons de quoi il s'agit un peu plus tard). À ce titre, `_Bool` est le type entier non signé avec la plus faible capacité.

Les types `float`, `double` et `long double` permettent quant à eux de stocker des nombres réels.⁴

III.1.2.1.1. Capacité d'un type

III.1.2.1.1.1. Les types entiers et flottants Tous les types stockant des nombres ont des bornes, c'est-à-dire une limite aux valeurs qu'ils peuvent stocker. En effet, le nombre de multipliants occupés par une variable est limité suivant son type. En conséquence, il n'est pas possible de mettre tous les nombres possibles dans une variable de type `int`, `float`, ou `double`. Il y aura *toujours* une valeur minimale et une valeur maximale. Ces limites sont les suivantes.

Type	Minimum	Maximum
<code>_Bool</code>	0	1
<code>signed char</code>	-127	127
<code>unsigned char</code>	0	255
<code>short</code>	-32 767	32 767
<code>unsigned short</code>	0	65 535
<code>int</code>	-32 767	32 767
<code>unsigned int</code>	0	65 535
<code>long</code>	-2 147 483 647	2 147 483 647
<code>unsigned long</code>	0	4 294 967 295
<code>long long</code>	-9 223 372 036 854 775 807	9 223 372 036 854 775 807
<code>unsigned long long</code>	0	18 446 744 073 709 551 615
<code>float</code>	-1×10^{37}	1×10^{37}
<code>double</code>	-1×10^{37}	1×10^{37}
<code>long double</code>	-1×10^{37}	1×10^{37}

Si vous regardez bien ce tableau, vous remarquerez que certains types ont des bornes identiques. En vérité, les valeurs présentées ci-dessus sont les minimums garantis par la norme¹ et il est fort probable qu'en réalité, vous puissiez stocker des valeurs plus élevées que ceux-ci. Cependant, dans une optique de portabilité, vous devez considérer ces valeurs comme les minimums et les maximums de ces types, peu importe la capacité réelle de ces derniers sur votre machine.

III.1.2.1.1.2. Le type `char` Le type `char` quant à lui est garanti de pouvoir stocker un ensemble de caractères limités. il s'agit des caractères suivants.

1	A	B	C	D	E	F	G	H	I	J	K	L	M		
2	N	O	P	Q	R	S	T	U	V	W	X	Y	Z		
3	a	b	c	d	e	f	g	h	i	j	k	l	m		
4	n	o	p	q	r	s	t	u	v	w	x	y	z		
5	0	1	2	3	4	5	6	7	8	9					
6	!	"	#	%	&	'	(	)	*	+	,	-	.	/	:
7	;	<	=	>	?	[	\	]	^	_	{		}	~	

Auxquels s'ajoutent l'espace, la tabulation, le retour à la ligne et quelques caractères spéciaux (nous en parlerons davantage plus tard dans ce cours). À nouveau il s'agit du minimum garanti par la norme.

III.1.2.1.2. Taille d'un type

Peut-être vous êtes vous demandé pourquoi il existe autant de types différents. La réponse est toute simple : la taille des mémoires était très limitée à l'époque où le langage C a été créé. En effet, le [PDP-11](#) sur lequel le C a été conçu ne possédait que 24Ko de mémoire (pour comparaison, une calculatrice TI-Nspire possède 100Mo de mémoire, soit environ 4000 fois plus). Il fallait donc l'économiser au maximum en choisissant le type le plus petit possible. Cette taille dépend des machines, mais de manière générale, vous pouvez retenir les deux suites d'inégalités suivantes : `signed char` `short` `int` `long` `long long` et `float` `double` `long double`.

Aujourd'hui ce n'est plus un problème, il n'est pas nécessaire de se casser la tête sur quel type choisir (excepté si vous voulez programmer pour de petits appareils où la mémoire est plus petite). En pratique, nous utiliserons surtout `char` pour les caractères, `int`, `long` ou `long long` pour les entiers et `double` pour les réels.

III.1.2.2. Les identificateurs

Maintenant que nous avons vu les types, parlons des identificateurs. Comme dit précédemment, un identificateur est un nom donné à une variable pour la différencier de toutes les autres. Et ce nom, c'est au programmeur de le choisir. Cependant, il y a quelques limitations à ce choix.

III. Les bases du langage C

- seuls les 26 lettres de l'alphabet latin (majuscules ou minuscules), le trait de soulignement « _ » (*underscore* en anglais) et les chiffres sont acceptés. Pas d'accents, pas de ponctuation ni d'espaces ;
- un identificateur ne peut pas commencer par un chiffre ;
- les mots-clés ne peuvent pas servir à identifier une variable ; il s'agit de :

1	<code>auto</code>	<code>if</code>	<code>unsigned</code>
2	<code>break</code>	<code>inline</code>	<code>void</code>
3	<code>case</code>	<code>int</code>	<code>volatile</code>
4	<code>char</code>	<code>long</code>	<code>while</code>
5	<code>const</code>	<code>register</code>	<code>_Alignas</code>
6	<code>continue</code>	<code>restrict</code>	<code>_Alignof</code>
7	<code>default</code>	<code>return</code>	<code>_Atomic</code>
8	<code>do</code>	<code>short</code>	<code>_Bool</code>
9	<code>double</code>	<code>signed</code>	<code>_Complex</code>
10	<code>else</code>	<code>sizeof</code>	<code>_Generic</code>
11	<code>enum</code>	<code>static</code>	<code>_Imaginary</code>
12	<code>extern</code>	<code>struct</code>	<code>_Noreturn</code>
13	<code>float</code>	<code>switch</code>	<code>_Static_assert</code>
14	<code>for</code>	<code>typedef</code>	<code>_Thread_local</code>
15	<code>goto</code>	<code>union</code>	

- deux variables ne peuvent avoir le même identificateur (le même nom). Il y a parfois quelques exceptions, mais cela n'est pas pour tout de suite ;
- les identificateurs peuvent être aussi longs que l'on désire, toutefois le compilateur ne tiendra compte que des 63 premiers caractères².

Voici quelques exemples pour bien comprendre.

Identificateur correct	Identificateur incorrect	Raison
variable	Nom de variable	Espaces interdits
nombre_de_vie	1nombre_de_vie	Commence par un chiffre
test	test !	Caractère « ! » interdit
un_der- nier_pour_la_route1	<code>continue</code>	Mot-clé réservé par le langage

À noter que le C fait la différence entre les majuscules et les minuscules (on dit qu'il **respecte la casse**). Ainsi les trois identificateurs suivants sont différents.

1	<code>variable</code>
2	<code>Variable</code>
3	<code>VaRiAbLe</code>

III.1.2.3. Déclaration

Maintenant que nous connaissons toutes les bases, entraînons-nous à déclarer quelques variables.

```
1 int main(void)
2 {
3     _Bool booleen;
4     double taille;
5     unsigned int age;
6     char caractere;
7     short petite_valeur;
8
9     return 0;
10 }
```

Il est possible de déclarer plusieurs variables **de même type** sur une même ligne, en séparant leur noms par une virgule.

```
1 int age, taille, nombre;
```

Ceci permet de regrouper les déclarations suivant les rapports que les variables ont entre elles.

```
1 int annee, mois, jour;
2 int age, taille;
3 int x, y, z;
```

III.1.3. Initialiser une variable

En plus de déclarer une variable, il est possible de **l'initialiser**, c'est-à-dire de lui attribuer une valeur. La syntaxe est la suivante.

```
1 type identificateur = valeur;
```

-
1. ISO/IEC 9899:201x, doc. N1570, § 5.2.4.2, Numerical limits, p.26
 2. ISO/IEC 9899:201x, doc. N1570, § 5.2.4, Environmental limits, p.25
 3. Notez que le type `_Bool` a une orthographe différente des autres types (il commence par un trait de soulignement et une majuscule), cela s'explique par le fait qu'il a été introduit après la norme C89.
 4. Depuis la norme C99, il existe pour chaque type réel un type complexe correspondant permettant, comme son nom l'indique, de contenir des [nombres complexes](#) [↗](#). Pour ce faire, il est nécessaire d'ajouter le mot-clé `_Complex` au type souhaité, par exemple `double _Complex`. Toutefois, l'intérêt étant limité, nous n'en parlerons pas plus dans ce cours.

III. Les bases du langage C

III.1.3.0.1. Initialisation des types entiers

Pour les types entiers, c'est assez simple, la valeur à attribuer est... un nombre entier.

```
1 _Bool boolean = 0;
2 unsigned char age = 42;
3 long score = -1;
```

III.1.3.0.2. Initialisation des types réels

Pour les types réels, ceux-ci étant faits pour contenir des nombres à virgule, à l'initialisation, il est nécessaire de placer cette « virgule ». Toutefois, cette dernière est représentée par un point.

```
1 double pi = 3.14;
```

Ceci vient du fait que le C est une invention américaine et que les anglophones utilisent le point à la place de la virgule.



Notez qu'il est important de bien placer ce point, *même si vous voulez stocker un nombre rond*. Par exemple, vous ne devez pas écrire `double a = 5` mais `double a = 5.` (certains préfèrent `double a = 5.0`, cela revient au même).

III.1.3.0.2.1. En notation scientifique Par ailleurs, il est également possible de représenter un nombre à virgule flottante à l'aide de [la notation scientifique](#) [↗](#), c'est-à-dire sous la forme d'un nombre décimal et d'une puissance de dix. Cela se traduit par un nombre réel en notation simple (comme ci-dessus) suivi de la lettre « e » ou « E » et d'un exposant *entier*.

```
1 double f = 1E-1; /* 1x10^-1 = 0.1 */
```

III.1.3.0.3. Initialisation du type char

Nous l'avons précisé : le type `char` peut être utilisé pour contenir un entier ou un caractère. Dans le second cas, le caractère utilisé pour l'initialisation doit être entouré de guillemets simples, comme suit.

```
1 char a = 'a';
```

III.1.3.1. Rendre une variable constante

En plus d'attribuer une valeur à une variable, il est possible de préciser que cette variable ne pourra pas être modifiée par la suite à l'aide du mot-clé `const`. Ceci peut être utile pour stocker une valeur qui ne changera jamais (comme la constante π qui vaut toujours 3,14159265).

```
1 const double pi = 3.14159265;
```

III.1.4. Affecter une valeur à une variable

Nous savons donc déclarer (créer) nos variables et les initialiser (leur donner une valeur à la création). Il ne nous reste plus qu'à voir la dernière manipulation possible : **l'affectation**. Celle-ci permet de modifier la valeur contenue dans une variable pour la remplacer par une autre valeur.

Il va de soi que cette affectation n'est possible que pour les variables qui ne sont pas déclarées avec le mot-clé `const` puisque, par définition, de telles variables sont constantes et ne peuvent voir leur contenu modifié.

Pour faire une affectation, il suffit d'opérer ainsi.

```
1 identificateur = nouvelle_valeur;
```

Nous voyons que la syntaxe est similaire à celle d'une déclaration avec initialisation. La seule différence, c'est que nous n'avons pas à préciser le type. Celui-ci est en effet fixé une fois pour toute lors de la déclaration de notre variable. Aussi, il n'est pas nécessaire de le préciser à nouveau lors d'une affectation.

```
1 age = 30;  
2 taille = 177.5;  
3 petite_valeur = 2;
```

Notez qu'il n'y a aucune limite au nombre d'affectations, comme le démontre l'exemple ci-dessous.

```
1 petite_valeur = 2;  
2 petite_valeur = 4;  
3 petite_valeur = 8;  
4 petite_valeur = 16;  
5 petite_valeur = 8;  
6 petite_valeur = 4;  
7 petite_valeur = 2;
```

III. Les bases du langage C

À chaque affectation, la variable va prendre une nouvelle valeur. Par contre, ne mettez pas le type quand vous voulez changer la valeur, sinon vous aurez le droit à une belle erreur du type « *redefinition of 'nom_de_votre_variable'* » car vous aurez créé deux variables avec le même identificateur !

Le code suivant est donc incorrect.

```
1 int age = 15;
2 int age = 20;
```

Si vous exécutez tous ces codes, vous verrez qu'ils n'affichent toujours rien et c'est normal puisque nous n'avons pas demandé à notre ordinateur d'afficher quoi que ce soit. Nous apprendrons comment faire au chapitre suivant.



Il n'y a pas de valeur par défaut en C. Aussi, sans initialisation ou affectation, la valeur d'une variable est indéterminée ! Veillez donc à ce que vos variables aient une valeur connue avant de les utiliser !

III.1.5. Les représentations octale et hexadécimale

Pour terminer ce chapitre, nous vous proposons un petit aparté sur deux représentations particulières : la représentation octale et la représentation hexadécimale.

Nous avons déjà vu la représentation binaire au début de ce chapitre, les représentations octale et hexadécimale obéissent au même schéma : au lieu d'utiliser dix chiffres pour représenter un nombre, nous en utilisons respectivement huit ou seize.



Seize chiffres ?! Mais... Je n'en connais que dix moi ! 🍊

La représentation hexadécimale est un peu déroutante de prime abord, celle-ci ajoute six chiffres (en fait, six lettres) : a, b, c, d, e et f. Pour vous aider, voici un tableau présentant les nombres de 0 à 16 en binaire, octal, décimal et hexadécimal.

Binaire	Octal	Décimal	Hexadécimal
00000	0	0	0
00001	1	1	1
00010	2	2	2
00011	3	3	3
00100	4	4	4

III. Les bases du langage C

00101	5	5	5
00110	6	6	6
00111	7	7	7
01000	10	8	8
01001	11	9	9
01010	12	10	a
01011	13	11	b
01100	14	12	c
01101	15	13	d
01110	16	14	e
01111	17	15	f
10000	20	16	10

i

Notez que 10 en base 2 (**00010** dans le tableau ci-dessus) équivaut à 2 en base 10, 10 en base octale correspond à 8 en base 10 et 10 en base hexadécimale est égal à 16 en base 10.

?

Quel est l'intérêt de ces deux bases exactement ?

L'avantage des représentations octale et hexadécimale est qu'il est facilement possible de les convertir en binaire contrairement à la représentation décimale. En effet, un chiffre en base huit ou en base seize peut être facilement traduit, respectivement, en trois ou quatre *bits*.

Prenons l'exemple du nombre 35 qui donne 43 en octal et 23 en hexadécimal. Nous pouvons nous focaliser sur chaque chiffre un à un pour obtenir la représentation binaire. Ainsi, du côté de la représentation octale, 4 donne **100** et 3 **011**, ce qui nous donne finalement **00100011**. De même, pour la représentation hexadécimale, 2 nous donne **0010** et 3 **0011** et nous obtenons **00100011**. Il n'est pas possible de faire pareil en décimal.

En résumé, les représentations octale et hexadécimale permettent de représenter un nombre binaire de manière plus concise et plus lisible.

III.1.5.1. Constantes octales et hexadécimales

Il vous est possible de préciser la base d'une constante entière en utilisant des préfixes. Ces préfixes sont **0** pour les constantes octales et **0x** ou **0X** pour les constantes hexadécimales.

```
1 long a = 65535; /* En décimal */
2 int b = 0777; /* En octal */
3 short c = 0xFF; /* En hexadécimal */
```

i

Les lettres utilisées pour la représentation hexadécimale peuvent être aussi bien écrites en minuscule qu'en majuscule.

?

Il n'est pas possible d'utiliser une constante en base deux ?

Malheureusement, non, le langage C ne permet pas d'utiliser de telles constantes.

Conclusion

En résumé

1. Les informations sont numérisées (traduites en nombres) afin de pouvoir être traitées par un ordinateur ;
2. Un ordinateur calcule en base deux ;
3. Un chiffre binaire est appelé un *bit* (contraction de l'expression « *binary digit* ») ;
4. Les *bits* sont regroupés en mutiplets (ou *byte* en anglais) qui sont le plus souvent composés de huit *bits* (on parle alors d'octet) ;
5. Les données sont stockées dans différents types de mémoire, notamment : les registres, la mémoire vive (ou RAM) et les disques durs ;
6. Chaque multiplet de la RAM est identifié par une adresse ;
7. Une variable est déclarée à l'aide d'un type et d'un identificateur ;
8. Le C fournit dix types de base : `_Bool`, `char`, `signed char`, `short int`, `int`, `long int`, `long long int`, `float`, `double`, `long double` ;
9. Tous les types ont une capacité limitée avec un minimum garanti par la norme ;
10. Une variable peut être initialisée en lui fournissant une valeur lors de sa déclaration ;
11. Il est possible d'affecter une valeur à une variable une fois celle-ci déclarée ;
12. Sans initialisation ou affectation, la valeur d'une variable est *indéterminée* ;
13. Il est possible de représenter des nombres entiers en base 8 (octale) ou en base 16 (hexadécimale).

III.2. Manipulations basiques des entrées/sorties

Introduction

Durant l'exécution d'un programme, le processeur a besoin de communiquer avec le reste du matériel. Ces échanges d'informations sont les **entrées** et les **sorties** (ou *input* et *output* pour les anglophones), souvent abrégées E/S (ou I/O par les anglophones).

Les entrées permettent de recevoir une donnée en provenance de certains périphériques. Les données fournies par ces entrées peuvent être une information envoyée par le disque dur, la carte réseau, le clavier, la souris, un CD, un écran tactile, bref par n'importe quel périphérique. Par exemple, notre clavier va transmettre des informations sur les touches enfoncées au processeur : notre clavier est donc une entrée.

À l'inverse, les sorties vont transmettre des données vers ces périphériques. On pourrait citer l'exemple de l'écran : notre ordinateur lui envoie des informations pour qu'elles soient affichées.

Dans ce chapitre, nous allons apprendre différentes fonctions fournies par le langage C qui vont nous permettre d'envoyer des informations vers nos sorties et d'en recevoir depuis nos entrées. Vous saurez ainsi comment demander à un utilisateur de fournir une information au clavier et comment afficher quelque chose sur la console.

III.2.1. Les sorties

Intéressons-nous dans un premier temps aux sorties. Afin d'afficher du texte, nous avons besoin d'une **fonction**.

Une fonction est un morceau de code qui a un but, une *fonction* particulière et qui peut être **appelée** à l'aide d'une référence, le plus souvent le nom de cette fonction (comme pour une variable, finalement). En l'occurrence, nous allons utiliser une fonction qui a pour objectif d'afficher du texte dans la console : la fonction `printf()`.

III.2.1.1. Première approche

Un exemple valant mieux qu'un long discours, voici un premier exemple.

III. Les bases du langage C

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("Bonjour tout le monde !\n");
7     return 0;
8 }
```

Résultat

```
1 Bonjour tout le monde !
```

Deux remarques au sujet de ce code.

```
1 #include <stdio.h>
```

Il s'agit d'une **directive du préprocesseur**, facilement reconnaissable car elles commencent toutes par le symbole `#`. Celle-ci sert à inclure un fichier (« `stdio.h` ») qui contient les références de différentes fonctions d'entrée et sortie (« *stdio* » est une abréviation pour « ***Standard input output*** », soit « Entrée-sortie standard »).

Un fichier se terminant par l'extension « `.h` » est appelé un **fichier d'en-tête** (*header* en anglais) et fait partie avec d'autres d'un ensemble plus large appelée la **bibliothèque standard** (« standard » car elle est prévue par la norme¹).

```
1 printf("Bonjour tout le monde !\n");
```

Ici, nous **appelons** la fonction `printf()` (un appel de fonction est toujours suivi d'un groupe de parenthèses) avec comme **argument** (ce qu'il y a entre les parenthèses de l'appel) un texte (il s'agit plus précisément d'une **chaîne de caractères**, qui est toujours comprise entre deux guillemets double). Le `\n` est un caractère spécial qui représente un retour à la ligne, cela est plus commode pour l'affichage.

Le reste devrait vous être familier.

III.2.1.2. Les formats

Bien, nous savons maintenant afficher une phrase, mais ce serait quand même mieux de pouvoir voir les valeurs de nos variables. Comment faire ? Hé bien, pour y parvenir, la fonction `printf()` met à notre disposition des **formats**. Ceux-ci sont en fait des sortes de repères au sein d'un

III. Les bases du langage C

texte qui indiquent à `printf()` que la valeur d'une variable est attendue à cet endroit. Voici un exemple pour une variable de type `int`.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int variable = 20;
7
8     printf("%d\n", variable);
9     return 0;
10 }
```

Résultat

```
1 20
```

Nous pouvons voir que le texte de l'exemple précédent a été remplacé par `%d`, seul le `\n` a été conservé. Un format commence toujours par le symbole `%` et est suivi par une ou plusieurs lettres qui indiquent le type de données que nous souhaitons afficher. Cette suite de lettres est appelée un **indicateur de conversion**. En voici une liste non exhaustive.

Type	Indicateur(s) de conversion
<code>_Bool</code>	<code>d</code> (ou <code>i</code>)
<code>char</code>	<code>c</code>
<code>signed char</code>	<code>d</code> (ou <code>i</code>)
<code>short</code>	<code>d</code> (ou <code>i</code>)
<code>int</code>	<code>d</code> (ou <code>i</code>)
<code>long</code>	<code>ld</code> (ou <code>li</code>)
<code>long long</code>	<code>lld</code> (ou <code>lli</code>)
<code>unsigned char</code>	<code>u</code> , <code>x</code> (ou <code>X</code>) ou <code>o</code>
<code>unsigned short</code>	<code>u</code> , <code>x</code> (ou <code>X</code>) ou <code>o</code>
<code>unsigned int</code>	<code>u</code> , <code>x</code> (ou <code>X</code>) ou <code>o</code>
<code>unsigned long</code>	<code>lu</code> , <code>lx</code> (ou <code>lX</code>) ou <code>lo</code>
<code>unsigned long long</code>	<code>llu</code> , <code>llx</code> (ou <code>llX</code>) ou <code>llo</code>

float	f , e (ou E) ou g (ou G)
double	f , e (ou E) ou g (ou G)
long double	Lf , Le (ou LE) ou Lg (ou LG)



Notez que les indicateurs de conversions sont identiques pour les types `_Bool`, `char` (s'il stocke un entier), `short` et `int` (pareil pour leurs équivalents non signés) ainsi que pour les types `float` et `double`.

Les indicateurs `x`, `X` et `o` permettent d'afficher un nombre en représentation hexadécimale ou octale (l'indicateur `x` affiche les lettres en minuscules alors que l'indicateur `X` les affiche en majuscules).

Les indicateurs `f`, `e` et `g` permettent quant à eux d'afficher un nombre flottant. L'indicateur `f` affiche un nombre en notation simple avec, par défaut, six décimales ; l'indicateur `e` affiche un nombre flottant en [notation scientifique](#) (l'indicateur `e` utilise la lettre « e » avant l'exposant alors que l'indicateur « E » emploie la lettre « E ») et l'indicateur `g` choisit quant à lui entre les deux notations précédentes suivant le nombre fourni et supprime la partie fractionnaire si elle est nulle de sorte que l'écriture soit concise (la différence entre les indicateurs `g` et `G` est identique à celle entre les indicateurs `e` et `E`).

Allez, un petit exemple pour reprendre tout cela et retravailler le chapitre précédent par la même occasion.

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      char z = 'z';
7      char a = 10;
8      unsigned short b = 20;
9      int c = 30;
10     long d = 40;
11     float e = 50.;
12     double f = 60.0;
13     long double g = 70.0;
14
15     printf("%c\n", z);
16     printf("%d\n", a);
17     printf("%u\n", b);
18     printf("%o\n", b);
19     printf("%x\n", b);
20     printf("%d\n", c);
21     printf("%li\n", d);

```

III. Les bases du langage C

```
22     printf("%f\n", e);
23     printf("%e\n", f);
24     g = 80.0;
25     printf("%Lg\n", g);
26     return 0;
27 }
```

Résultat

1	z
2	10
3	20
4	24
5	14
6	30
7	40
8	50.000000
9	6.000000e+01
10	80



Si vous souhaitez afficher le caractère `%` vous devez le doubler : `%%`.

III.2.1.3. Précision des nombres flottants

Vous avez peut-être remarqué que lorsqu'un flottant est affiché avec le format `f`, il y a un certain nombre de zéros qui suivent (par défaut six) et ce, peu importe qu'ils soient utiles ou non. Afin d'en supprimer certains, vous pouvez spécifier une **précision**. Celle-ci correspond au nombre de chiffres suivant la virgule qui seront affichés. Elle prend la forme d'un point suivi par un nombre : la quantité de chiffres qu'il y aura derrière la virgule.

```
1 double x = 42.42734;
2
3 printf("%.2f\n", x);
```

Résultat

1	42.43
---	-------



Notez que pour respecter la précision demandée, la valeur est arrondie.

III.2.1.4. Les caractères spéciaux

Dans certains cas, nous souhaitons obtenir un résultat à l’affichage (saut de ligne, une tabulation, un retour chariot, etc.). Cependant, ils ne sont pas particulièrement pratiques à insérer dans une chaîne de caractères. Aussi, le C nous permet de le faire en utilisant une **séquence d’échappement**. Il s’agit en fait d’une suite de caractères commençant par le symbole `\` et suivie d’une lettre. En voici une liste non exhaustive.

Séquence d’échappement	Signification
<code>\a</code>	Caractère d’appel
<code>\b</code>	Espacement arrière
<code>\f</code>	Saut de page
<code>\n</code>	Saut de ligne
<code>\r</code>	Retour chariot
<code>\t</code>	Tabulation horizontale
<code>\v</code>	Tabulation verticale
<code>\"</code>	Le symbole « <code>"</code> »
<code>\\</code>	Le symbole « <code>\</code> » lui-même

En général, vous n’utiliserez que le saut de ligne, la tabulation horizontale et de temps à autre le retour chariot, les autres n’ont quasiment plus d’intérêt. Un petit exemple pour illustrer leurs effets.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("Quelques sauts de ligne\n\n\n");
7     printf("\tIl y a une tabulation avant moi !\n");
8     printf("Je voulais dire que...\r");
9     printf("Hey ! Vous pourriez me laisser parler !\n");
10    return 0;
11 }
```

Résultat

```
1 Quelques sauts de ligne
2
3
4     Il y a une tabulation avant moi !
5 Hey ! Vous pourriez me laisser parler !
```



Le retour chariot provoque un retour au début de la ligne courante. Ainsi, il est possible d'écrire par-dessus un texte affiché.

III.2.1.5. Sur plusieurs lignes

Notez qu'il est possible d'écrire un long texte sans appeler plusieurs fois la fonction `printf()`. Pour ce faire, il suffit de le diviser en plusieurs chaînes de caractères.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Texte écrit sur plusieurs "
6           "lignes dans le code source "
7           "mais sur une seule dans la console.\n");
8     return 0;
9 }
```

Résultat

```
1 Texte écrit sur plusieurs lignes dans le code source mais sur
   une seule dans la console.
```

III.2.2. Interagir avec l'utilisateur

Maintenant que nous savons déclarer, utiliser et même afficher des variables, nous sommes fin prêts pour interagir avec l'utilisateur. En effet, jusqu'à maintenant, nous nous sommes contentés

1. ISO/IEC 9899:201x, doc. N1570, § 7, Library, p. 180.

III. Les bases du langage C

d'afficher des informations. Nous allons à présent voir comment en récupérer grâce à la fonction `scanf()`, dont l'utilisation est assez semblable à `printf()`.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int age;
7
8     printf("Quel âge avez-vous ? ");
9     scanf("%d", &age);
10    printf("Vous avez %d an(s)\n", age);
11    return 0;
12 }
```

Résultat

```
1 Quel âge avez-vous ? 15
2 Vous avez 15 an(s).
```



Il est possible que la question « Quel âge avez-vous ? » s'affiche *après* que vous avez entré des données. Ce comportement peut se produire dans le cas où la ligne affichée n'est pas terminée par un retour à la ligne (`\n`) et dépend de la bibliothèque C présente sur votre système. Dans un tel cas, pour afficher la ligne, il vous suffit d'ajouter un appel à la fonction `fflush()` après l'appel à la fonction `printf()`, comme suit.

```
1 printf("Quel âge avez-vous ? ");
2 fflush(stdout);
```

Nous reviendrons sur cette fonction et son argument dans la seconde partie du cours. Pour l'heure, retenez simplement que cet appel est nécessaire si vous rencontrez le comportement décrit. 🍊

Comme vous le voyez, l'appel à `scanf()` ressemble très fort à celui de `printf()` mise à part l'absence du caractère spécial `\n` (qui n'a pas d'intérêt puisque nous récupérons des informations) et le symbole `&`.

À son sujet, souvenez-vous de la brève explication sur la mémoire au début du chapitre précédent. Celle-ci fonctionne comme une armoire avec des tiroirs (les adresses mémoires) et des objets dans ces tiroirs (nos variables). La fonction `scanf()` a besoin de connaître l'emplacement en mémoire de nos variables afin de les modifier. Afin d'effectuer cette opération, nous utilisons

III. Les bases du langage C

le symbole `&` (qui est en fait un opérateur que nous verrons en détail plus tard). Ce concept de transmission d'adresses mémoires est un petit peu difficile à comprendre au début, mais ne vous inquiétez pas, vous aurez l'occasion de bien revoir tout cela en profondeur dans le chapitre traitant des pointeurs.

Ici, `scanf()` attend patiemment que l'utilisateur saisisse un nombre au clavier afin de modifier la valeur de la variable `age`. On dit que c'est une **fonction bloquante**, car elle suspend l'exécution du programme tant que l'utilisateur n'a rien entré.

Pour ce qui est des indicateurs de conversions, ils sont un peu différents de ceux de `printf()`.

Type	Indicateur(s) de conversion
<code>char</code>	<code>c</code>
<code>signed char</code>	<code>hhd</code> (ou <code>hhi</code>)
<code>short</code>	<code>hd</code> ou <code>hi</code>
<code>int</code>	<code>d</code> ou <code>i</code>
<code>long</code>	<code>ld</code> ou <code>li</code>
<code>long long</code>	<code>lld</code> ou <code>lli</code>
<code>unsigned char</code>	<code>hu</code> , <code>hx</code> ou <code>ho</code>
<code>unsigned short</code>	<code>hu</code> , <code>hx</code> ou <code>ho</code>
<code>unsigned int</code>	<code>u</code> , <code>x</code> ou <code>o</code>
<code>unsigned long</code>	<code>lu</code> , <code>lx</code> ou <code>lo</code>
<code>unsigned long long</code>	<code>llu</code> , <code>llx</code> , ou <code>llo</code>
<code>float</code>	<code>f</code>
<code>double</code>	<code>lf</code>
<code>long double</code>	<code>Lf</code>



Faites bien attention aux différences ! Si vous utilisez le mauvais format, le résultat ne sera pas celui que vous attendez. Les changements concernent les types `char` (s'il stocke un entier), `short` et `double`.



Notez que :

- l'indicateur `c` ne peut être employé que pour récupérer un caractère et non un nombre ;
- il n'y a plus qu'un seul indicateur pour récupérer un nombre hexadécimal : `x` (l'utilisation de lettres majuscules ou minuscules n'a pas d'importance) ;
- il n'y a pas d'indicateur pour le type `_Bool`.

III. Les bases du langage C

En passant, sachez qu'il est possible de lire plusieurs entrées en même temps, par exemple comme ceci.

```
1 int x, y;
2
3 scanf("%d %d", &x, &y);
4 printf("x = %d | y = %d\n", x, y);
```

L'utilisateur a deux possibilités, soit insérer un (ou plusieurs) espace(s) entre les valeurs, soit insérer un (ou plusieurs) retour(s) à la ligne entre les valeurs.

Résultat (1)

```
1 14
2 6
3 x = 14 | y = 6
```

Résultat (2)

```
1 14 6
2 x = 14 | y = 6
```

Voici un exemple exploitant et illustrant plusieurs indicateurs de `scanf()`.

```
1 #include <stdio.h>
2
3 int
4 main(void)
5 {
6     char c, hhd;
7
8     scanf("%c %hhd", &c, &hhd);
9     printf("c = %c, n = %d\n", c, hhd);
10
11     int i, o, x;
12
13     scanf("%i %o %x", &i, &o, &x);
14     printf("i = %i, o = %o, x = %x\n", i, o, x);
15
16     float f;
```


III. Les bases du langage C

```
17     double lf;  
18  
19     scanf("%f %lf", &f, &lf);  
20     printf("f = %f, lf = %f\n", f, lf);  
21     return 0;  
22 }
```

Résultat

```
1 c 5  
2 c = c, n = 5  
3 10 777 C7  
4 i = 10, o = 777, x = c7  
5 56.7 2e-3  
6 f = 56.700001, lf = 0.002000
```

La fonction `scanf()` est en apparence simple (oui, *en apparence*), mais son utilisation peut devenir très complexe lorsqu'il est nécessaire de vérifier les entrées de l'utilisateur (entre autres). Cependant, à votre niveau, vous ne pouvez pas encore effectuer de telles vérifications. Ce n'est toutefois pas très grave, nous verrons cela en temps voulu. 🍊

Conclusion

En résumé

1. Les fonctions de la bibliothèque standard sont utilisables en incluant des fichiers d'en-tête ;
2. Il est possible d'afficher du texte à l'aide de la fonction `printf()` ;
3. La fonction `printf()` emploie différents indicateurs de conversion suivant le type de ses arguments ;
4. Il existe un certain nombre de caractères spéciaux permettant de produire différents effets à l'affichage ;
5. La fonction `scanf()` permet de récupérer une entrée ;
6. Les indicateurs de conversions ne sont pas identiques entre `printf()` et `scanf()` ;
7. L'opérateur `&` permet de transmettre l'adresse d'une variable à `scanf()`.

III.3. Les opérations mathématiques

Introduction

Nous savons désormais déclarer, affecter et initialiser une variable, mais que diriez-vous d'apprendre à réaliser des opérations dessus ? Il est en effet possible de réaliser des calculs sur nos variables, comme les additionner, les diviser voire des opérations plus complexes. C'est le but de cette sous-partie. Nous allons donc enfin transformer notre ordinateur en grosse calculatrice programmable !

III.3.1. Les opérations mathématiques de base

Jusqu'à présent, nous nous sommes contentés d'afficher du texte et de manipuler très légèrement les variables. Voyons à présent comment nous pouvons réaliser quelques opérations de base. Le langage C nous permet d'en réaliser cinq :

- l'addition (opérateur `+`) ;
- la soustraction (opérateur `-`) ;
- la multiplication (opérateur `*`) ;
- la division (opérateur `/`) ;
- le modulo (opérateur `%`).



Le langage C fournit bien entendu d'autres fonctions mathématiques et d'autres opérateurs, mais il est encore trop tôt pour vous les présenter.

III.3.1.1. Division et modulo

Les quatre premières opérations vous sont connues depuis l'école primaire. Cependant, une chose importante doit être précisée concernant la division : quand les deux nombres manipulés sont des entiers, il s'agit d'une division entière. Autrement dit, le quotient sera un entier et il peut y avoir un reste. Par exemple, $15 \div 6$, ne donnera pas 2,5 (division réelle), mais un quotient de 2, avec un reste de 3.

```
1 printf("15 / 6 = %d\n", 15 / 6);
```

III. Les bases du langage C

Résultat

```
1 15 / 6 = 2
```

Le modulo est un peu le complément de la division entière : au lieu de donner le quotient, il renvoie le reste d'une division euclidienne. Par exemple, le modulo de 15 par 6 est 3, car $15 = 2 \times 6 + 3$.

```
1 printf("15 %% 6 = %d\n", 15 % 6);
```

Résultat

```
1 15 % 6 = 3
```



Notez que le symbole `%` doit être doublé afin de pouvoir être utilisé littéralement.

Avec des flottants, la division se comporte autrement et n'est pas une division avec reste. La division de deux flottants donnera un résultat « exact », avec potentiellement plusieurs chiffres après la virgule.

```
1 printf("15 / 6 = %f\n", 15. / 6.); /* En C, ce n'est pas la même chose que 15 / 6 */
```

Résultat

```
1 15 / 6 = 2.500000
```

III.3.1.2. Utilisation

Il est possible d'affecter le résultat d'une opération à une variable, comme lorsque nous affichons leur résultat avec `printf()`.

III. Les bases du langage C

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int somme = 5 + 3;
7
8     printf("5 + 3 = %d\n", somme);
9     return 0;
10 }
```

Résultat

```
1 5 + 3 = 8
```

Toute opération peut manipuler :

- des constantes ;
- des variables ;
- les deux à la fois.

Voici un exemple avec des constantes.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("2 + 3 = %d\n", 2 + 3);
7     printf("8 - 12 = %d\n", 8 - 12);
8     printf("6 x 7 = %d\n", 6 * 7);
9     printf("11 %% 4 = %d\n", 11 % 4);
10    return 0;
11 }
```

III. Les bases du langage C

Résultat

1	2 + 3 = 5
2	8 - 12 = -4
3	6 x 7 = 42
4	11 % 4 = 3

Un autre avec des variables.

```
1 int a = 5;
2 int b = 3;
3 int somme = a + b;
4
5 printf("%d + %d = %d\n", a, b, somme);
```

Résultat

1	5 + 3 = 8
---	-----------

Et enfin, un exemple qui mélange variables et constantes.

```
1 int a = 5;
2 int b = 65;
3
4 printf("%d\n", b / a * 2 + 7 % 2);
```

Résultat

1	27
---	----

III.3.1.3. La priorité des opérateurs

Dans l'exemple précédent, nous avons utilisé plusieurs opérations dans une même ligne de code, une même expression. Dans ces cas-là, faites attention à la **priorité des opérateurs** ! Comme en mathématiques, certains opérateurs passent avant d'autres : les opérateurs `*` `/` `%` ont une priorité supérieure par rapport aux opérateurs `+` `-`.

III. Les bases du langage C

Dans le code ci-dessous, c'est `c * 4` qui sera exécuté d'abord, puis `b` sera ajouté au résultat. Faites donc attention sous peine d'avoir de mauvaises surprises. Dans le doute, ajoutez des parenthèses.

```
1 a = b + c * 4;
```

III.3.2. Raccourcis

Dans les exemples précédents, nous avons utilisé des affectations pour sauvegarder le résultat d'une opération dans une variable. Les expressions obtenues ainsi sont assez longues et on peut se demander s'il existe des moyens pour écrire moins de code. Hé bien, le langage C fournit des écritures pour se simplifier la vie. Certains cas particuliers peuvent s'écrire avec des raccourcis, du « **sucre syntaxique** ».

III.3.2.1. Les opérateurs combinés

Comment vous y prendriez-vous pour multiplier une variable par trois ? La solution qui devrait vous venir à l'esprit serait d'affecter à la variable son ancienne valeur multipliée par trois.

```
1 int variable = 3;
2
3 variable = variable * 3;
4 printf("variable * 3 = %d\n", variable);
```

Résultat

```
1 variable * 3 = 9
```

Ce qui est parfaitement correct. Cependant, cela implique de devoir écrire deux fois le nom de la variable, ce qui est quelque peu pénible et source d'erreurs. Aussi, il existe des opérateurs combinés qui réalisent une affectation et une opération en même temps.

Opérateur combiné	Équivalent à
<code>variable += nombre</code>	<code>variable = variable + nombre</code>
<code>variable -= nombre</code>	<code>variable = variable - nombre</code>
<code>variable *= nombre</code>	<code>variable = variable * nombre</code>

III. Les bases du langage C

variable /= nombre	variable = variable / nombre
variable %= nombre	variable = variable % nombre

Avec le code précédent, nous obtenons ceci.

```
1 int variable = 3;
2
3 variable *= 3;
4 printf("variable * 3 = %d\n", variable);
```

Résultat

```
1 variable * 3 = 9
```

III.3.2.2. L'incrément et la décrémentation

L'**incrément** et la **décrément** sont deux opérations qui, respectivement, ajoutent ou enlèvent une unité à une variable. Avec les opérateurs vus précédemment, cela se traduit par le code ci-dessous.

```
1 variable += 1; /* Incrément */
2 variable -= 1; /* Décrément */
```

Cependant, ces deux opérations étant très souvent utilisées, elles ont droit chacune à un opérateur spécifique disponible sous deux formes :

- une forme **préfixée** ;
- une forme **suffixée**.

La forme préfixée s'écrit comme ceci.

```
1 ++variable; /* Incrément */
2 --variable; /* Décrément */
```

La forme suffixée s'écrit comme cela.

III. Les bases du langage C

```
1 variable++; /* Incrémentation */
2 variable--; /* Décrémentation */
```

Le résultat des deux paires d'opérateurs est le même : la variable `variable` est incrémentée ou décrémentée, à une différence près : le résultat de l'opération.

1. Dans le cas de l'opérateur préfixé (`--variable` ou `++variable`), le résultat est la valeur de la variable *après* l'incrément ou la décrémentation.
2. Dans le cas de l'opérateur suffixé (`variable--` ou `variable++`), le résultat est la valeur de la variable *avant* l'incrément ou la décrémentation.

Illustration !

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int x = 1;
6     int y = 1;
7     int a = x++;
8     int b = ++y;
9
10    printf("a = %d\n", a);
11    printf("b = %d\n", b);
12    printf("x = %d\n", x);
13    printf("y = %d\n", y);
14    return 0;
15 }
```

Résultat

```
1 a = 1
2 b = 2
3 x = 2
4 y = 2
```

Comme vous pouvez le constater, le résultat de l'opération `x++` est 1 alors que celui de `++y` est 2. Cela étant, dans les deux cas, les variables `x` et `y` ont bien été incrémentées.

III.3.3. Le type d'une constante

Dans les sections précédentes, nous avons vu plusieurs opérations mathématiques et avons affiché leur résultat. Cependant, peut-être vous êtes-vous posé la question suivante : quel est le type du résultat ?

En effet, nous avons par exemple écrit `printf("2 + 3 = %d\n", 2 + 3)`, ce qui laisse supposer que le résultat de `2 + 3` est de type `int`, sans finalement en savoir plus.

En fait, le type du résultat d'une opération dépend de celui de ses opérandes. Nous allons voir cela en détail un peu après, mais par exemple si les opérandes sont de type `int`, le résultat sera de type `int`.

Toutefois, cela amène une autre question : de quel type sont les constantes comme `2` ou `3` ?

III.3.3.0.1. Les constantes entières

Le type des constantes entières, par défaut, dépend de leur valeur et, de manière plus étonnante, de leur base.

Base	Type
10	<code>int</code>
	<code>long</code>
	<code>long long</code>
8 ou 16	<code>int</code>
	<code>un</code>
	<code>si</code>
	<code>igned</code>
	<code>long</code>
	<code>un</code>
	<code>si</code>
	<code>igned</code>
	<code>long</code>
	<code>long long</code>
16	<code>un</code>
	<code>si</code>
	<code>igned</code>
	<code>long</code>
	<code>long long</code>

Ce tableau doit être compris comme suit :

- Pour un nombre en base 10 (comme `2`), le premier type pouvant stocker la valeur entre les types `int`, `long` et `long long` sera choisi ;

III. Les bases du langage C

- Pour un nombre en base 8 ou 16 (comme 0777 ou 0x16), le premier type pouvant stocker la valeur entre les types `int`, `unsigned`, `long`, `unsigned long`, `long long` et `unsigned long long` sera choisi.

Si aucun de ces types ne peut stocker la valeur indiquée, le comportement des compilateurs varie. En général, ils généreront une erreur de compilation.

Ces règles par défaut sont utiles, mais pas forcément pratiques. En effet, rappelez-vous : les valeurs minimale et maximale d'un type varient d'une machine à l'autre (revoyez le chapitre sur les variables si cela ne vous dit rien), ce qui signifie que le type d'une constante peut varier d'une machine à l'autre.

Il serait préférable de se baser sur les minimums garantis par la norme et de pouvoir fixer le type au regard de ceux-ci. Heureusement pour nous, cela est possible à l'aide de suffixes. Il en existe trois pour les constantes entières.

Suf	Type
fixe	
un	
si	
igned	
u	un
ou	si
U	igned
	long
	un
	si
	igned
	long
	long
l	long
ou	long
L	long
	un
	si
u	igned
ou	long
U	un
	si
	igned
	long
	long
ll	long
ou	long
LL	
	un
u	si
ou	igned
U	long
	long

i

Notez que suivant le cas, le suffixe ne fixe pas un type précis, mais une liste de types. Par exemple, le suffixe `U` précise que le type sera `unsigned` ou `unsigned long` ou `unsigned long long`. Dans un tel cas, le type effectivement utilisé sera le premier pouvant stocker la valeur de la constante.

Ainsi, la constante `1UL` sera par exemple de type `unsigned long`, alors qu'elle aurait été de type `int` par défaut.

i

Nous vous conseillons d'opter pour les lettres majuscules qui ont l'avantage d'être plus lisibles.

III.3.3.0.2. Les constantes flottantes

Pour les constantes flottantes, la règle est simple : elles sont par défaut de type `double`.

Toutefois, comme pour les constantes entières, il est possible de fixer le type à l'aide de suffixes. Il en existe deux pour les constantes flottantes.

Suffixe	Type
f ou F	float
l ou L	long double

Ainsi, la constante `1.L` sera de type `long double` alors qu'elle aurait été de type `double` par défaut.

Si le type choisi ne peut stocker la valeur indiquée, le comportement des compilateurs varie. En général, ils généreront un avertissement.

!

N'oubliez pas le `.` sans quoi la constante sera considérée comme entière.

i

Comme pour les constantes entières, nous vous conseillons d'opter pour les lettres majuscules qui ont l'avantage d'être plus lisibles.

III.3.4. Le type d'une opération

Dans la section précédente, nous vous avons dit que le type d'une opération, dit autrement le type du résultat d'une opération, dépendait de celui de ses opérandes. Or, si c'est évident

III. Les bases du langage C

dans le cas où les opérandes ont le même type, cela ne l'est pas dans le cas où leurs types sont différents.

Par exemple, quel est le type de l'opération `2 + 1.2` ? Intuitivement on serait tenté de choisir un type flottant pour conserver la partie décimale, mais sans règles spécifiques, c'est impossible à dire.

Heureusement pour nous, la norme a prévu ces différents cas et a fixé des règles¹ qui sont appliquées *avant* que l'opération n'ait lieu :

Tout d'abord, si l'un des opérandes est de type `short` (signé ou non), `char` (signé ou non), ou `_Bool`, il est converti en `int` (ou en `unsigned int` si le type `int` n'a pas une capacité suffisante). Cette règle est appelée la **promotion intégrale**² (ou promotion entière). Ensuite, les règles suivantes s'appliquent :

1. si l'un des deux opérandes est de type `long double`, l'autre opérande est converti en `long double` ;
2. si l'un des deux opérandes est de type `double`, l'autre opérande est converti en `double` ;
3. si l'un des deux opérandes est de type `float`, l'autre opérande est converti en `float` ;
4. si l'un des deux opérandes est de type `unsigned long long`, l'autre opérande est converti en `unsigned long long` ;
5. si l'un des deux opérandes est de type `long long`, l'autre opérande est converti en `long long` ;
6. si l'un des deux opérandes est de type `unsigned long`, l'autre opérande est converti en `unsigned long` ;
7. si l'un des deux opérandes est de type `long`, l'autre opérande est converti en `long` ;
8. si l'un des deux opérandes est de type `unsigned int`, l'autre opérande est converti en `unsigned int`.

i

Notez que la promotion intégrale explique pourquoi l'indicateur de conversion de `printf()` reste `d` (ou `u`) pour les types `short`, `char` et `_Bool`, puisque ceux-ci sont convertis en `int`.

Voyons un peu tout ça à l'aide d'un exemple.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     /* long double + int = long double + long double = long double
       */
7     printf("78.56 + 5 = %Lf\n", 78.56L + 5);
8
9     /* long + double = double + double = double */
10    printf("5678 + 2.2 = %f\n", 5678L + 2.2);
```

```
11
12  /* long + unsigned long = unsigned long + unsigned long =
    unsigned long */
13  printf("2 + 5 = %lu\n", 2L + 5UL);
14
15  /* long long + int = long long + long long = long long */
16  printf("1 + 1 = %lld\n", 1LL + 1);
17
18  /*
19   * Promotion intégrale de n qui devient de type int.
20   * Puis int + int = int
21   */
22  signed char n = 3;
23  printf("n + 2 = %d\n", n + 2);
24  return 0;
25 }
```

Résultat

```
1 78.56 + 5 = 83.560000
2 5678 + 2.2 = 5680.200000
3 2 + 5 = 7
4 1 + 1 = 2
5 n + 2 = 5
```

III.3.5. Les conversions

Nous venons de le voir, dans le cadre d'opérations, des conversions peuvent avoir lieu d'un type à l'autre. Ces conversions sont appelées des **conversions implicites**, car elles sont effectuées automatiquement. Il en existe beaucoup en langage C. Par exemple, lors d'une affectation, il y a également une conversion implicite vers le type de la variable assignée.

```
1 int a = 2.; /* Conversion implicite du type double vers le type
    int. */
```

Il est toutefois également possible de demander explicitement une conversion. Pour convertir un opérande vers un type donné, il suffit de le préfixer par le type désiré placé entre parenthèses. Le code ci-dessous convertit la constante 2 vers le type **double** et l'affiche ensuite.

-
1. ISO/IEC 9899:201x, doc. N1570, § 6.3.1.8, Usual arithmetic conversions, p. 52.
 2. ISO/IEC 9899:201x, doc. N1570, § 6.3.1.1, Boolean, characters, and integers, p. 50.

III. Les bases du langage C

```
1 #include <stdio.h>
2
3 int
4 main(void)
5 {
6     printf("%f\n", (double)2);
7     return 0;
8 }
```

Résultat

1	2.000000
---	----------

Une conversion explicite permet également d'imposer le résultat d'une opération en fixant ou en induisant le type de ses opérandes. Ainsi, l'exemple qui suit utilise des conversions explicites pour effectuer une division avec des nombres flottants.

```
1 #include <stdio.h>
2
3 int
4 main(void)
5 {
6     int a = 5;
7     int b = 2;
8
9     /*
10      * b est converti en double.
11      * Comme b est de type double, a est converti implicitement en
12      * double.
13      */
14     printf("%f\n", a / (double)b);
15
16     /* a et b sont explicitement convertis en double. */
17     printf("%f\n", (double)a / (double)b);
18     return 0;
19 }
```

Résultat

1	2.500000
2	2.500000

III.3.6. Exercices

Vous êtes prêts pour un exercice ?

Essayez de réaliser une minicalculatrice qui :

- dit « bonjour » ;
- demande deux nombres entiers à l'utilisateur ;
- les additionne, les soustrait, les multiplie et les divise (au millième près) ;
- dit « au revoir ».

Un exemple d'utilisation pourrait être celui-ci.

```
1 Bonjour !
2 Veuillez saisir le premier nombre : 4
3 Veuillez saisir le deuxième nombre : 7
4 Calculs :
5         4 + 7 = 11
6         4 - 7 = -3
7         4 * 7 = 28
8         4 / 7 = 0.571
9 Au revoir !
```

Bien, vous avez maintenant toutes les cartes en main, donc : au boulot ! 🍊

👁 Contenu masqué n°1

Vous y êtes arrivé sans problèmes ? Bravo ! Dans le cas contraire, ne vous inquiétez pas, ce n'est pas grave. Relisez bien tous les points qui ne vous semblent pas clairs et ça devrait aller mieux.

Conclusion

En résumé

1. Le C fournit des opérateurs pour réaliser les opérations mathématiques de base ;

III. Les bases du langage C

2. Dans le cas où une division est effectuée entre des nombres entiers, le quotient sera également un entier ;
3. La priorité des opérateurs est identique à celle décrite en mathématiques ;
4. Le C fournit des opérateurs combinés et des opérateurs pour l'incrément et la décrémentation afin de gagner en concision ;
5. Le type d'une constante peut-être modifié à l'aide de suffixes ;
6. En cas de mélange des types lors d'une opération, le C prévoit des règles de conversions ;
7. Il est possible de convertir un opérande d'un type vers un autre, certaines conversions sont implicites.

Contenu masqué

Contenu masqué n°1

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int a;
7      int b;
8
9      printf("Bonjour !\n");
10
11     /* Nous demandons deux nombres à l'utilisateur */
12     printf("Veuillez saisir le premier nombre : ");
13     scanf("%d", &a);
14     printf("Veuillez saisir le deuxième nombre : ");
15     scanf("%d", &b);
16
17     /* Puis nous effectuons les calculs */
18     printf("Calculs :\n");
19     printf("\t%d + %d = %d\n", a, b, a + b);
20     printf("\t%d - %d = %d\n", a, b, a - b);
21     printf("\t%d * %d = %d\n", a, b, a * b);
22     printf("\t%d / %d = %.3f\n", a, b, a / (double)b);
23     printf("Au revoir !\n");
24     return 0;
25 }
```

[Retourner au texte.](#)

III.4. Tests et conditions

Introduction

Jusqu'à présent, vous avez appris à écrire du texte, manipuler des nombres et interagir un tout petit peu avec l'utilisateur.

En gros, pour le moment, un programme est quelque chose de sacrément simple et linéaire, ce dernier ne nous permettant que d'exécuter des instructions dans un ordre donné. Techniquement, une simple calculatrice peut en faire autant (voire plus). Cependant et heureusement, les langages de programmation actuels fournissent des moyens permettant de réaliser des tâches plus évoluées.

Pour ce faire, diverses **structures de contrôle** ont été inventées. Celles-ci permettent de modifier le comportement d'un programme suivant la réalisation de différentes conditions. Ainsi, si une condition est vraie, le programme se comportera d'une telle façon et à l'inverse, si elle est fausse, le programme fera telle ou telle chose.

Dans ce chapitre, nous allons voir comment rédiger des conditions à l'aide de deux catégories d'opérateurs :

- les **opérateurs de comparaison**, qui comparent deux nombres ;
- les **opérateurs logiques**, qui permettent de combiner plusieurs conditions.

III.4.1. Les booléens

Comme les opérateurs que nous avons vus précédemment (+, -, *, etc), les opérateurs de comparaison et les opérateurs logiques donnent un résultat : « vrai » si la condition est vérifiée, et « faux » si la condition est fausse. Toutefois, comme vous le savez, notre ordinateur ne voit que des nombres. Aussi, il est nécessaire de représenter ces valeurs à l'aide de ceux-ci.

Certains langages fournissent pour cela un type distinct pour stocker le résultat des opérations de comparaison et deux valeurs spécifiques : `true` (vrai) et `false` (faux). Néanmoins, dans les premières versions du langage C, ce type spécial n'existait pas (le type `_Bool` a été introduit par la norme C99). Il a donc fallu ruser et trouver une solution pour représenter les valeurs « vrai » et « faux ». Pour cela, la méthode la plus simple a été privilégiée : utiliser directement des nombres pour représenter ces deux valeurs. Ainsi, le langage C impose que :

- la valeur « faux » soit représentée par zéro ;
- et que la valeur « vrai » soit représentée par tout sauf zéro.

Les opérateurs de comparaison et les opérateurs logiques suivent cette convention pour représenter leur résultat. Dès lors, une condition vaudra 0 si elle est fausse et 1 si elle est vraie.



Notez qu'il existe un en-tête `<stdbool.h>` qui fournit deux constantes entières : `true` (qui vaut 1) et `false`, qui vaut 0 ainsi qu'un synonyme pour le type `_Bool` nommé `bool`.

```

1 #include <stdbool.h>
2 #include <stdio.h>
3
4
5 int
6 main(void)
7 {
8     bool booleen = true;
9
10    printf("true = %u, false = %u\n", true, false);
11    printf("booleen = %u\n", booleen);
12    return 0;
13 }
```

Résultat

```

1 true = 1, false = 0
2 booleen = 1
```

III.4.2. Les opérateurs de comparaison

Le langage C fournit différents opérateurs qui permettent d'effectuer des comparaisons entre des nombres. Ces opérateurs peuvent s'appliquer aussi bien à des constantes qu'à des variables (ou un mélange des deux). Ces derniers permettent donc par exemple de vérifier si une variable est supérieure à une autre, si elles sont égales, etc.

III.4.2.1. Comparaisons

L'écriture de conditions est similaire aux écritures mathématiques que vous voyez en cours : l'opérateur est entre les deux expressions à comparer. Par exemple, dans le cas de l'opérateur `>` (« strictement supérieur à »), il est possible d'écrire des expressions du type `a > b`, qui vérifie si la variable `a` est strictement supérieure à la variable `b`.

Le tableau ci-dessous reprend les différents opérateurs de comparaison.

Opérateur	Signification
-----------	---------------

III. Les bases du langage C

==	Égalité
!=	Inégalité
<	Strictement inférieur à
<=	Inférieur ou égal à
>	Strictement supérieur à
>=	Supérieur ou égal à

Ces opérateurs ne semblent pas très folichons. En effet, avec, nous ne pouvons faire que quelques tests basiques sur des nombres. Cependant, rappelez-vous : pour un ordinateur, tout n'est que nombre et comme pour le stockage des données (revoyez le début du chapitre sur les variables si cela ne vous dit rien) il est possible de ruser et d'exprimer toutes les conditions possibles avec ces opérateurs (cela vous paraîtra plus clair quand nous passerons aux exercices).

III.4.2.2. Résultat d'une comparaison

Comme dit dans l'extrait plus haut, une opération de comparaison va donner zéro si elle est fausse et un si elle est vraie. Pour illustrer ceci, vérifions quels sont les résultats donnés par différentes comparaisons.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("10 == 20 vaut %d\n", 10 == 20);
7     printf("10 != 20 vaut %d\n", 10 != 20);
8     printf("10 < 20 vaut %d\n", 10 < 20);
9     printf("10 > 20 vaut %d\n", 10 > 20);
10    return 0;
11 }
```

Résultat

```
1 10 == 20 vaut 0
2 10 != 20 vaut 1
3 10 < 20 vaut 1
4 10 > 20 vaut 0
```

III. Les bases du langage C

Le résultat confirme bien ce que nous avons dit ci-dessus.

III.4.3. Les opérateurs logiques

Toutes ces comparaisons sont toutefois un peu faibles seules car il y a des choses qui ne sont pas possibles à vérifier en utilisant une seule comparaison. Par exemple, si un nombre est situé entre zéro et mille (inclus). Pour ce faire, il serait nécessaire de vérifier que celui-ci est supérieur ou égal à zéro **et** inférieur ou égal à mille.

Il nous faudrait donc trouver un moyen de combiner plusieurs comparaisons entre elles pour résoudre ce problème. *Hé* bien rassurez-vous, le langage C fournit de quoi associer plusieurs résultats de comparaisons : les **opérateurs logiques**.

III.4.3.1. Les opérateurs logiques de base

Il existe trois opérateurs logiques. L'opérateur « **et** », l'opérateur « **ou** », et l'opérateur de **négation**. Les deux premiers permettent de combiner deux conditions alors que le troisième permet d'inverser le sens d'une condition.

III.4.3.1.1. L'opérateur « et »

L'opérateur « et » va manipuler deux conditions. Il va donner un si elles sont vraies, et zéro sinon.

Première condition	Seconde condition	Résultat de l'opérateur « et »
Fausse	Fausse	0
Fausse	Vraie	0
Vraie	Fausse	0
Vraie	Vraie	1

Cet opérateur s'écrit `&&` et s'intercale entre les deux conditions à combiner. Si nous reprenons l'exemple vu plus haut, pour combiner les comparaisons `a >= 0` et `a <= 1000`, nous devons placer l'opérateur `&&` entre les deux, ce qui donne l'expression `a >= 0 && a <= 1000`.

III.4.3.1.2. L'opérateur « ou »

L'opérateur « ou » fonctionne exactement comme l'opérateur « et », il prend deux conditions et les combine pour former un résultat. Cependant, l'opérateur « ou » vérifie que l'une des deux conditions (ou que les deux) est (sont) vraie(s).

Première condition	Seconde condition	Résultat de l'opérateur « ou »
Fausse	Fausse	0
Fausse	Vraie	1
Vraie	Fausse	1
Vraie	Vraie	1

Cet opérateur s'écrit `||` et s'intercale entre les deux conditions à combiner. L'exemple suivant permet de déterminer si un nombre est divisible par trois ou par cinq (ou les deux) : `(a % 3 == 0) || (a % 5 == 0)`. Notez que les parenthèses ont été placées par souci de lisibilité.

III.4.3.1.3. L'opérateur de négation

Cet opérateur est un peu spécial : il manipule une seule condition et en inverse le sens.

Condition	Résultat de l'opérateur de négation
Fausse	1
Vraie	0

Cet opérateur se note `!`. Son utilité ? Simplifier certaines expressions. Par exemple, si nous voulons vérifier cette fois qu'un nombre **n'est pas** situé entre zéro et mille, nous pouvons utiliser la condition `a >= 0 && a <= 1000` et lui appliquer l'opérateur de négation, ce qui donne `!(a >= 0 && a <= 1000)`.



Faites bien attention à l'utilisation des parenthèses ! L'opérateur de négation s'applique à l'opérande le plus proche (sans parenthèses, il s'agirait de `a`). Veillez donc à bien entourer de parenthèses l'expression concernée par la négation.



Notez que pour cet exemple, il est parfaitement possible de se passer de cet opérateur à l'aide de l'expression `a < 0 || a > 1000`. Il est d'ailleurs souvent possible d'exprimer une condition de différentes manières.

III.4.3.2. Évaluation en court-circuit

Les opérateurs `&&` et `||` évaluent toujours la première condition avant la seconde. Cela paraît évident, mais ce n'est pas le cas dans tous les langages de programmation. Ce genre de détail permet à ces opérateurs de disposer d'un comportement assez intéressant : **l'évaluation en court-circuit**.

III. Les bases du langage C

De quoi s'agit-il ? Pour illustrer cette notion, reprenons l'exemple précédent : nous voulons vérifier si un nombre est compris entre zéro et mille, ce qui donne l'expression `a >= 0 && a <= 1000`. Si jamais `a` est inférieur à zéro, nous savons dès la vérification de la première condition qu'il n'est pas situé dans l'intervalle voulu. Il n'est donc pas nécessaire de vérifier la seconde. Hé bien, c'est exactement ce qu'il se passe en langage C. Si le résultat de la première condition suffit pour déterminer le résultat de l'opérateur `&&` ou `||`, alors la seconde condition n'est pas évaluée. Voilà pourquoi l'on parle d'évaluation en court-circuit.

Plus précisément, ce sera le cas pour l'opérateur `&&` si la première condition est fausse et pour l'opérateur `||` si la première condition est vraie (relisez les tableaux précédents si cela ne vous semble pas évident).

Ce genre de propriété peut-être utilisé efficacement pour éviter de faire certains calculs, en choisissant intelligemment quelle sera la première condition.

III.4.3.3. Combinaisons

Bien sûr, il est possible de mélanger ces opérateurs pour créer des conditions plus complexes. Voici un exemple un peu plus long (et inutile, soit dit en passant 🍊).

```
1 int a = 3;
2 double b = 64.67;
3 double c = 12.89;
4 int d = 8;
5 int e = -5;
6 int f = 42;
7 int r;
8
9 r = ((a < b && b > 32) || (c < d + b || e == 0)) && (f > d);
10 printf("La valeur logique est égale à : %d\n", r);
```

Ici, la variable `r` est égale à 1, la condition est donc vraie.

III.4.4. Priorité des opérations

Comme pour les opérateurs mathématiques, les opérateurs logiques et de comparaisons ont des règles de priorité (revoyez le chapitre sur les opérations mathématiques si cela ne vous dit rien). En regardant le code écrit plus haut, vous avez d'ailleurs sûrement remarqué la présence de plusieurs parenthèses. Celles-ci permettent d'enlever toute ambiguïté dans les expressions créées avec des opérateurs logiques.

Les règles de priorité sont les suivantes, de la plus forte à la plus faible :

1. l'opérateur de négation (`!`) ;
2. les opérateurs relationnels (`<`, `<=`, `>`, `>=`) ;
3. les opérateurs d'égalité et d'inégalité (`==`, `!=`) ;

III. Les bases du langage C

4. l'opérateur logique `&&` ;
5. l'opérateur logique `||`.

Ainsi, les expressions suivantes seront évaluées comme suit.

- `10 < 20 == 0 : 10 < 20`, puis l'égalité ;
- `!a != 1 < 0 : !a`, puis `1 < 0`, puis l'inégalité ;
- `a && b || c && d`, `a && b`, puis `c && d`, puis le `||`.

Si vous souhaitez un résultat différent, il est nécessaire d'ajouter des parenthèses pour modifier les règles de priorité, par exemple comme suit `a && (b || c) && d` auquel cas l'expression `b || c` est évaluée la première, suivie du premier `&&`, puis du second.

Conclusion

En résumé

1. Une condition est soit vrai, soit fausse.
2. Une condition fausse est représentée par une valeur nulle, une condition vraie par une valeur non nulle ;
3. Les opérateurs `&&` et `||` utilisent l'évaluation en court-circuit : si la première condition est suffisante pour déterminer le résultat, la seconde n'est pas évaluée.
4. Les opérateurs de comparaisons et les opérateurs logiques ont également des règles de priorité.

III.5. Les sélections

Introduction

Comme dit au chapitre précédent, les structures de contrôle permettent de modifier le comportement d'un programme suivant la réalisation de différentes conditions. Parmi ces structures de contrôle se trouvent les **instructions de sélection** (ou **sélections** en abrégé) qui vont retenir notre attention dans ce chapitre.

Le tableau ci-dessous reprend celles dont dispose le langage C.

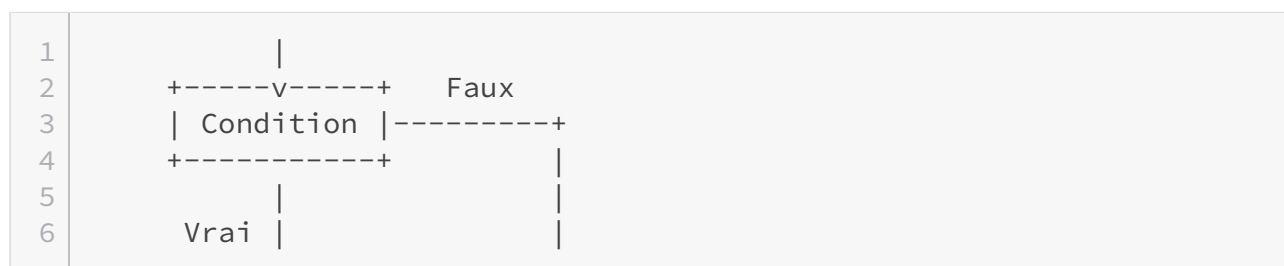
Structure de sélection	Action
if...	exécute une suite d'instructions si une condition est respectée.
if... else...	exécute une suite d'instructions si une condition est respectée, ou une autre suite dans le cas contraire.
switch...	exécute une suite d'instructions différentes selon la valeur testée.

III.5.1. La structure if

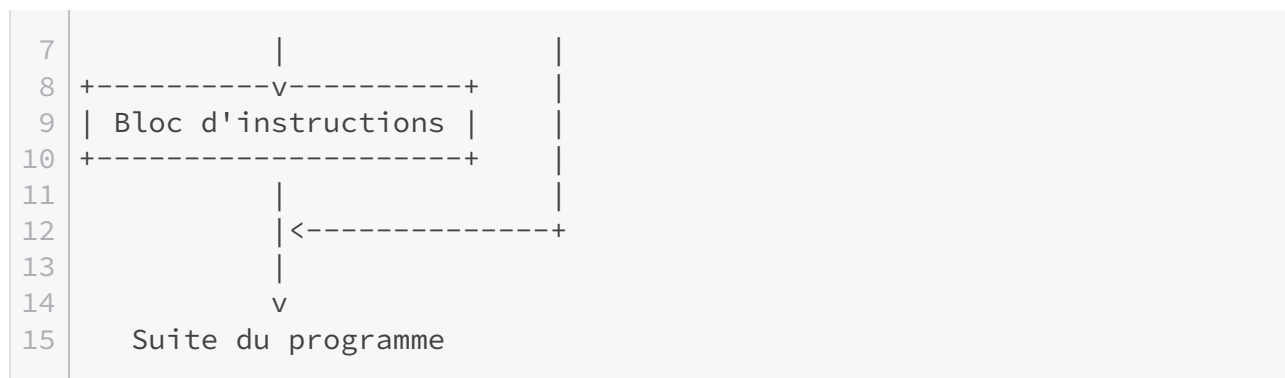
Vous savez désormais manipuler des conditions, c'est bien, cependant l'intérêt de la chose reste assez limité pour l'instant. Rendons à présent cela plus intéressant en voyant comment exécuter un bloc d'instruction quand une ou plusieurs conditions sont respectées. C'est le rôle de l'instruction `if` et de ses consœurs.

III.5.1.1. L'instruction if

L'instruction `if` permet d'exécuter un bloc d'instructions si une condition est vérifiée ou de le passer si ce n'est pas le cas.



III. Les bases du langage C



L'instruction `if` ressemble à ceci.

```
1 if (/* Condition */)
2 {
3     /* Une ou plusieurs instruction(s) */
4 }
```

Si la condition n'est pas vérifiée, le bloc d'instructions est passé et le programme recommence immédiatement à la suite du bloc d'instructions délimité par l'instruction `if`.

Si vous n'avez qu'une seule instruction à réaliser, vous avez la possibilité de ne pas mettre d'accolades.

```
1 if (/* Condition */)
2     /* Une seule instruction */
```

Cependant, nous vous conseillons de mettre les accolades systématiquement afin de vous éviter des problèmes si vous décidez de rajouter des instructions par la suite en oubliant d'ajouter des accolades. Bien sûr, ce n'est qu'un conseil, vous êtes libre de ne pas le suivre.

À présent, voyons quelques exemples d'utilisation.

III.5.1.1.1. Exemple 1

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int a = 10;
7     int b = 20;
8
9     if (a < b)
10    {
```

III. Les bases du langage C

```
11     printf("%d est inférieur à %d\n", a, b);
12 }
13
14     return 0;
15 }
```

Résultat

```
1 10 est inférieur à 20
```

L'instruction `if` évalue l'expression logique `a < b`, conclut qu'elle est valide et exécute le bloc d'instructions.

III.5.1.1.2. Exemple 2

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int a = 10;
7      int b = 20;
8
9      if (a > b)
10     {
11         printf("%d est supérieur à %d\n", a, b);
12     }
13
14     return 0;
15 }
```

Ce code n'affiche rien. La condition étant fausse, le bloc contenant l'appel à la fonction `printf()` est ignoré.

III.5.1.2. La clause else

Avec l'instruction `if`, nous savons exécuter un bloc d'instructions quand une condition est remplie. Toutefois, si nous souhaitons réaliser une action en cas d'échec de l'évaluation de la condition, nous devons ajouter une autre instruction `if` à la suite, comme ci-dessous.

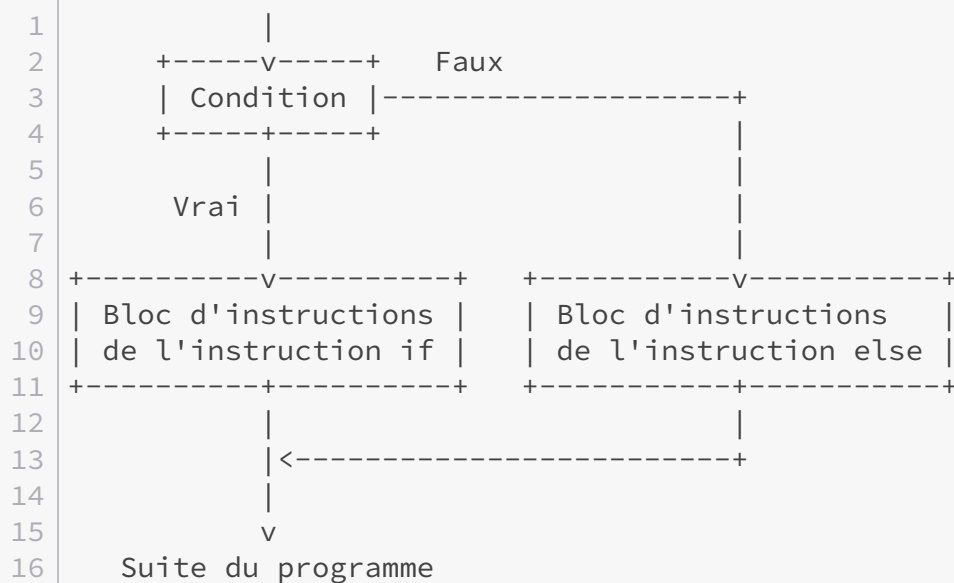
III. Les bases du langage C

```
1  if (a > 5)
2  {
3      /* Du code */
4  }
5
6  if (a <= 5)
7  {
8      /* Code alternatif */
9  }
```

Le seul problème, c'est qu'il est nécessaire d'ajouter une instruction `if` et d'évaluer une nouvelle condition, ce qui n'est pas très efficace et assez long à taper. Pour limiter les dégâts, le C offre la possibilité d'ajouter une clause `else`, qui signifie « sinon ». Celle-ci se place immédiatement après le bloc d'une instruction `if` et permet d'exécuter un bloc d'instructions alternatif si la condition testée n'est pas vérifiée. Sa syntaxe est la suivante.

```
1  if (/* Condition */)
2  {
3      /* Une ou plusieurs instructions */
4  }
5  else
6  {
7      /* Une ou plusieurs instructions */
8  }
```

Et elle doit être comprise comme ceci.



III. Les bases du langage C

III.5.1.2.1. Exemple

Supposons que nous voulions créer un programme très simple auquel nous fournissons une heure et qui indique s'il fait jour ou nuit à cette heure-là. Nous supposons qu'il fait jour de 8 heures à 20 heures et qu'il fait nuit sinon.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int heure;
7
8     scanf("%d", &heure);
9
10    if (heure > 8 && heure < 20)
11    {
12        printf("Il fait jour.\n");
13    }
14    else
15    {
16        printf("Il fait nuit.\n");
17    }
18
19    return 0;
20 }
```

Résultat

```
1 10
2 Il fait jour.
```

III.5.1.3. if, else if

Il est parfois nécessaire d'imbriquer plusieurs instructions `if else` les unes dans les autres, par exemple comme suit.

```
1 if (/* Condition(s) */)
2 {
3     /* Instruction(s) */
4 }
5 else
```

III. Les bases du langage C

```
6 {
7     if (/* Condition(s) */)
8     {
9         /* Instruction(s) */
10    }
11    else
12    {
13        if (/* Condition(s) */)
14        {
15            /* Instruction(s) */
16        }
17        else
18        {
19            /* Instruction(s) */
20        }
21    }
22 }
```

Ce qui est assez long et lourd à écrire, d'autant plus s'il y a beaucoup d'imbrications... Toutefois et heureusement pour nous, il est possible de simplifier cette écriture.

Tout d'abord, rappelez-vous : si un `if` ou un `else` ne comprend qu'une seule instruction, alors les accolades sont facultatives. Nous pouvons donc les retirer pour deux `else` (une suite `if else` compte pour une seule instruction).

```
1 if (/* Condition(s) */)
2 {
3     /* Instruction(s) */
4 }
5 else
6     if (/* Condition(s) */)
7     {
8         /* Instruction(s) */
9     }
10    else
11        if (/* Condition(s) */)
12        {
13            /* Instruction(s) */
14        }
15        else
16        {
17            /* Instruction(s) */
18        }
```

Ensuite, les retours à la ligne n'étant pas obligatoires, nous pouvons « coller » les deux `if` et les deux `else` qui se suivent.

III. Les bases du langage C

```
1  if (/* Condition(s) */)
2  {
3      /* Instruction(s) */
4  }
5  else if (/* Condition(s) */)
6  {
7      /* Instruction(s) */
8  }
9  else if (/* Condition(s) */)
10 {
11     /* Instruction(s) */
12 }
13 else
14 {
15     /* Instruction(s) */
16 }
```

Notez que comme il s'agit toujours d'une suite d'instructions `if else`, il n'y aura qu'un seul bloc d'instructions qui sera finalement exécuté. En effet, l'ordinateur va tester la condition de l'instruction `if`, puis, si elle est fausse, celle de l'instruction `if` de la clause `else` et ainsi de suite jusqu'à ce qu'une condition soit vraie (ou jusqu'à une clause `else` finale si elles sont toutes fausses).

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int heure = 11;
7
8      if (heure > 0 && heure < 7)
9      {
10         printf("Zzz... \n");
11     }
12     else if (heure >= 7 && heure < 12)
13     {
14         printf("C'est le matin !\n");
15     }
16     else if (heure == 12)
17     {
18         printf("Il est midi !\n");
19     }
20     else if (heure > 12 && heure < 18)
21     {
22         printf("C'est l'après-midi !\n");
23     }
24     else if (heure >= 18 && heure < 24)
```

III. Les bases du langage C

```
25     {
26         printf("C'est le soir !\n");
27     }
28     else if (heure == 24 || heure == 0)
29     {
30         printf("Il est minuit, dormez brave gens !\n");
31     }
32     else
33     {
34         printf("Il est l'heure de réapprendre à lire l'heure !\n");
35     }
36
37     return 0;
38 }
```

Résultat

```
1 11
2 C'est le matin !
3 0
4 Il est minuit, dormez brave gens !
5 -2
6 Il est l'heure de réapprendre à lire l'heure !
```

III.5.1.3.1. Exercice

Imaginez que vous avez un score de jeu vidéo sous la main :

- si le score est strictement inférieur à deux mille, affichez « C'est la catastrophe ! » ;
- si le score est supérieur ou égal à deux mille et que le score est strictement inférieur à cinq mille, affichez : « Tu peux mieux faire ! » ;
- si le score est supérieur ou égal à cinq mille et que le score est strictement inférieur à neuf mille, affichez : « Tu es sur la bonne voie ! » ;
- sinon, affichez : « Tu es le meilleur ! ».

Au boulot ! 🍊

👁 Contenu masqué n°2

III.5.2. L'instruction switch

L'instruction `switch` permet de comparer la valeur d'une expression entière par rapport à une liste de constantes entières. Techniquement, elle permet d'écrire de manière plus concise une suite d'instructions `if else` qui auraient pour objectif d'accomplir différentes actions suivant la valeur d'une expression.

```
1  if (a == 1)
2  {
3      /* Instruction(s) */
4  }
5  else if (a == 2)
6  {
7      /* Instruction(s) */
8  }
9  /* Etc. */
10 else
11 {
12     /* Instruction(s) */
13 }
```

Avec l'instruction `switch`, cela donne ceci.

```
1  switch (a)
2  {
3      case 1:
4          /* Instruction(s) */
5          break;
6      case 2:
7          /* Instruction(s) */
8          break;
9
10 /* Etc... */
11
12 default: /* Si aucune comparaison n'est juste */
13     /* Instruction(s) à exécuter dans ce cas */
14     break;
15 }
```

Ici, la valeur de la variable `a` est comparée successivement avec chaque entrée de la liste, indiquées par le mot-clé `case`. En cas de correspondance, les instructions suivant le mot-clé `case` sont exécutées jusqu'à rencontrer une instruction `break` (nous la verrons plus en détail un peu plus tard). Si aucune comparaison n'est bonne, alors ce sont les instructions de l'entrée marquée avec le mot-clé `default` qui seront exécutées.

III.5.2.0.1. Exemple

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int note;
7
8      printf("Quelle note as-tu obtenue (sur cinq) ? ");
9      scanf("%d", &note);
10
11     switch(note)
12     {
13         /* Si note == 0 */
14         case 0:
15             printf("No comment.\n");
16             break;
17
18         /* Si note == 1 */
19         case 1:
20             printf("Cela te fait 4/20, c'est accablant.\n");
21             break;
22
23         /* Si note == 2 */
24         case 2:
25             printf(«
26                 "On se rapproche de la moyenne, mais ce n'est pas encore ça.»
27                 \n");
28             break;
29
30         /* Si note == 3 */
31         case 3:
32             printf("Tu passes.\n");
33             break;
34
35         /* Si note == 4 */
36         case 4:
37             printf("Bon travail, continue ainsi !\n");
38             break;
39
40         /* Si note == 5 */
41         case 5:
42             printf("Excellent !\n");
43             break;
44
45         /* Si note est différente de 0, 1, 2, 3, 4 et 5 */
46         default:
47             printf("Euh... tu possèdes une note improbable...\n");
```

```
46     break;
47 }
48
49     return 0;
50 }
```



Notez que comme pour l'instruction `else`, une entrée marquée avec le mot-clé `default` n'est pas obligatoire.

III.5.2.1. Plusieurs entrées pour une même action

Une même suite d'instructions peut être désignée par plusieurs entrées comme le montre l'exemple suivant.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int note;
7
8      printf("Quelle note as-tu obtenue ? ");
9      scanf("%d", &note);
10
11     switch(note)
12     {
13         /* Si la note est comprise entre zéro et trois inclus */
14         case 0:
15         case 1:
16         case 2:
17         case 3:
18             printf("No comment.\n");
19             break;
20
21         /* Si la note est comprise entre quatre et sept inclus */
22         case 4:
23         case 5:
24         case 6:
25         case 7:
26             printf("C'est accablant.\n");
27             break;
28
29         /* Si la note est comprise entre huit et neuf inclus */
30         case 8:
31         case 9:
```

```
32     printf(_
        "On se rapproche de la moyenne, mais ce n'est pas encore ça._
        \n");
33     break;
34
35     /* Si la note est comprise entre dix et douze inclus */
36     case 10:
37     case 11:
38     case 12:
39         printf("Tu passes.\n");
40         break;
41
42     /* Si la note est comprise entre treize et seize inclus */
43     case 13:
44     case 14:
45     case 15:
46     case 16:
47         printf("Bon travail, continue ainsi !\n");
48         break;
49
50     /* Si la note est comprise entre dix-sept et vingt inclus */
51     case 17:
52     case 18:
53     case 19:
54     case 20:
55         printf("Excellent !\n");
56         break;
57
58     /* Si la note est différente */
59     default:
60         printf("Euh... tu possèdes une note improbable...\n");
61         break;
62 }
63
64 return 0;
65 }
```

III.5.2.2. Plusieurs entrées sans instruction break

Sachez également que l'instruction **break** n'est pas obligatoire. En effet, le but de cette dernière est de sortir du **switch** et donc de ne pas exécuter les actions d'autres entrées. Toutefois, il arrive que les actions à réaliser se chevauchent entre entrées auquel cas l'instruction **break** serait plutôt mal venue.

Prenons un exemple : vous souhaitez réaliser un programme qui affiche entre 1 à dix fois la même phrase, ce nombre étant fourni par l'utilisateur. Vous pourriez écrire une suite de **if** ou différentes entrées d'un **switch** qui, suivant le nombre entré, appelleraient une fois **printf()**, puis deux fois, puis trois fois, etc. mais cela serait horriblement lourd.

III. Les bases du langage C

Dans un tel cas, une meilleure solution consiste à appeler `printf()` à chaque entrée du `switch`, mais de ne pas terminer ces dernières par une instruction `break`.

```
1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      unsigned nb;
8
9      printf(
10         "Combien de fois souhaitez-vous répéter l'affichage (entre 1 à 10 fois,
11         );
12     scanf("%u", &nb);
13
14     switch (nb)
15     {
16     case 10:
17         printf("Cette phrase est répétée une à dix fois.\n");
18     case 9:
19         printf("Cette phrase est répétée une à dix fois.\n");
20     case 8:
21         printf("Cette phrase est répétée une à dix fois.\n");
22     case 7:
23         printf("Cette phrase est répétée une à dix fois.\n");
24     case 6:
25         printf("Cette phrase est répétée une à dix fois.\n");
26     case 5:
27         printf("Cette phrase est répétée une à dix fois.\n");
28     case 4:
29         printf("Cette phrase est répétée une à dix fois.\n");
30     case 3:
31         printf("Cette phrase est répétée une à dix fois.\n");
32     case 2:
33         printf("Cette phrase est répétée une à dix fois.\n");
34     case 1:
35         printf("Cette phrase est répétée une à dix fois.\n");
36     case 0:
37         break;
38     default:
39         printf("Certes, mais encore ?\n");
40         break;
41     }
42
43     return 0;
44 }
```

Résultat

```
1  Combien de fois souhaitez-vous répéter l'affichage (entre 1 à
    10 fois) ? 2
2  Cette phrase est répétée une à dix fois.
3  Cette phrase est répétée une à dix fois.
4
5  Combien de fois souhaitez-vous répéter l'affichage (entre 1 à
    10 fois) ? 5
6  Cette phrase est répétée une à dix fois.
7  Cette phrase est répétée une à dix fois.
8  Cette phrase est répétée une à dix fois.
9  Cette phrase est répétée une à dix fois.
10 Cette phrase est répétée une à dix fois.
```

Comme vous le voyez, la phrase « Cette phrase est répétée une à dix fois » est affichée une à dix fois suivant le nombre initialement fourni. Cela est possible étant donné l'absence d'instruction `break` entre les `case` 10 à 1, ce qui fait que l'exécution du `switch` continue de l'entrée initiale jusqu'au `case` 0.

III.5.3. L'opérateur conditionnel

L'**opérateur conditionnel** ou **opérateur ternaire** est un opérateur particulier dont le résultat dépend de la réalisation d'une condition. Son deuxième nom lui vient du fait qu'il est le seul opérateur du langage C à requérir trois opérandes : une condition et deux expressions.

```
1  (condition) ? expression si vrai : expression si faux
```



Les parenthèses entourant la condition ne sont pas obligatoires, mais préférables.

Grosso modo, cet opérateur permet d'écrire de manière condensée une structure `if {} else {}`. Voyez par vous-mêmes.

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int heure;
6
7      scanf("%d", &heure);
```

III. Les bases du langage C

```
8
9     (heure > 8 && heure < 20) ? printf("Il fait jour.\n") :
10     printf("Il fait nuit.\n");
11     return 0;
12 }
```

Il est également possible de l'écrire sur plusieurs lignes, même si cette pratique est moins courante.

```
1 (heure > 8 && heure < 20)
2   ? printf("Il fait jour.\n")
3   : printf("Il fait nuit.\n");
```

Cet opérateur peut sembler inutile de prime abord, mais il s'avère être un allié de choix pour simplifier votre code quand celui-ci requiert la vérification de conditions simples.

III.5.3.1. Exercice

Pour bien comprendre cette nouvelle notion, nous allons faire un petit exercice. Imaginez que nous voulions faire un mini jeu vidéo dans lequel nous affichons le nombre de coups du joueur. Seulement voilà, vous êtes maniaques du français et vous ne supportez pas qu'il y ait un « s » en trop ou en moins. Essayez de réaliser un programme qui demande à l'utilisateur d'entrer un nombre de coups puis qui affiche celui-ci correctement accordé.

👁 Contenu masqué n°3

Ce programme utilise l'opérateur conditionnel pour condenser l'expression et aller plus vite dans l'écriture du code. Sans lui nous aurions dû écrire quelque chose comme ceci.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int nb_coups;
6
7     printf("Donnez le nombre de coups : ");
8     scanf("%d", &nb_coups);
9
10    if (nb_coups > 1)
11        printf("Vous gagnez en %d coups\n", nb_coups);
12    else
13        printf("Vous gagnez en %d coup\n", nb_coups);
14 }
```

```
15     return 0;
16 }
```

Conclusion

Ce chapitre a été important, il vous a permis d'utiliser les conditions, l'instruction `if`, l'instruction `switch` et l'opérateur conditionnel. Aussi, si vous n'avez pas très bien compris ou que vous n'avez pas tout retenu, nous vous conseillons de relire ce chapitre.

En résumé

1. L'instruction `if` permet de conditionner l'exécution d'une suite d'instructions ;
2. Une clause `else` peut compléter une instruction `if` afin d'exécuter une autre suite d'instructions dans le cas où la condition est fausse ;
3. L'instruction `switch` permet de comparer la valeur d'une expression à une liste de constantes *entières* ;
4. Dans le cas où une comparaison est vraie, les instructions exécutées sont celles suivant un `case` et précédant une instruction `break` ;
5. Plusieurs `case` peuvent précéder la même suite d'instructions ;
6. L'opérateur conditionnel peut être utilisé pour simplifier l'écriture de certaines conditions.

Contenu masqué

Contenu masqué n°2

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int score;
7
8      printf("Quel est le score du joueur ? ");
9      scanf("%d", &score);
10
11     if (score < 2000)
12     {
13         printf("C'est la catastrophe !\n");
14     }
15     else if (score >= 2000 && score < 5000)
16     {
17         printf("Tu peux mieux faire !\n");
```

III. Les bases du langage C

```
18     }
19     else if (score >= 5000 && score < 9000)
20     {
21         printf("Tu es sur la bonne voie !\n");
22     }
23     else
24     {
25         printf("Tu es le meilleur !\n");
26     }
27
28     return 0;
29 }
```

[Retourner au texte.](#)

Contenu masqué n°3

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int nb_coups;
6
7      printf("Donnez le nombre de coups : ");
8      scanf("%d", &nb_coups);
9      printf("Vous gagnez en %d coup%c\n", nb_coups, (nb_coups > 1)
10             ? 's' : ' ');
11     return 0;
12 }
```

[Retourner au texte.](#)

III.6. TP : déterminer le jour de la semaine

Introduction

Avant de poursuivre notre périple, il est à présent temps de nous poser un instant afin de réaliser un petit exercice reprenant tout ce qui vient d'être vu.

III.6.1. Objectif

Votre objectif est de parvenir à réaliser un programme qui, suivant une date fournie par l'utilisateur sous la forme « jj/mm/aaaa », donne le jour de la semaine correspondant. Autrement dit, voici ce que devrait donner l'exécution de ce programme.

```
1 Entrez une date : 11/2/2015
2 C'est un mercredi
3
4 Entrez une date : 13/7/1970
5 C'est un lundi
```

III.6.2. Première étape

Pour cette première étape, vous allez devoir réaliser un programme qui demande à l'utilisateur un jour du mois de janvier de l'an un et qui lui précise de quel jour de la semaine il s'agit, le tout à l'aide de la méthode présentée ci-dessous.

```
1 Entrez un jour : 27
2 C'est un jeudi
```

III.6.2.1. Déterminer le jour de la semaine

Pour déterminer le jour de la semaine correspondant à une date, vous allez devoir partir du premier janvier de l'an un (c'était un samedi) et calculer le nombre de jours qui sépare cette date de celle fournie par l'utilisateur. Une fois ce nombre obtenu, il nous est possible d'obtenir le jour de la semaine correspondant à l'aide de l'opérateur modulo.

III. Les bases du langage C

En effet, comme vous le savez, les jours de la semaine suivent un cycle et se répètent tous les sept jours. Or, le reste de la division entière est justement un nombre cyclique allant de zéro jusqu'au diviseur diminué de un. Voici ce que donne le reste de la division entière des chiffres 1 à 9 par 3.

1	$1 \% 3 = 1$
2	$2 \% 3 = 2$
3	$3 \% 3 = 0$
4	$4 \% 3 = 1$
5	$5 \% 3 = 2$
6	$6 \% 3 = 0$
7	$7 \% 3 = 1$
8	$8 \% 3 = 2$
9	$9 \% 3 = 0$

Comme vous le voyez, le reste de la division oscille toujours entre zéro et deux. Ainsi, si nous attribuons un chiffre de zéro à six à chaque jour de la semaine (par exemple zéro pour samedi et ainsi de suite pour les autres) nous pouvons déduire le jour de la semaine correspondant à un nombre de jours depuis le premier janvier de l'an un.

Prenons un exemple : l'utilisateur entre la date du vingt-sept janvier de l'an un. Il y a vingt-six jours qui le sépare du premier janvier. Le reste de la division entière de vingt-six par 7 est 5, il s'agit donc d'un jeudi.

Si vous le souhaitez, vous pouvez vous aider du calendrier suivant.

👁 Contenu masqué n°4

À présent, à vous de jouer. 🍊

III.6.3. Correction

Alors, cela s'est bien passé ? Si oui, félicitations, si non, la correction devrait vous aider à y voir plus clair. 🍊

👁 Contenu masqué n°5

Tout d'abord, nous demandons à l'utilisateur d'entrer un jour du mois de janvier que nous affectons à la variable `jour`. Ensuite, nous calculons la différence de jours séparant le premier janvier de celui entré par l'utilisateur. Enfin, nous appliquons le modulo à ce résultat afin d'obtenir le jour de la semaine correspondant.

III.6.4. Deuxième étape

Bien, complexifions à présent un peu notre programme et demandons à l'utilisateur de nous fournir un jour *et* un mois de l'an un.

```
1 Entrez une date (jj/mm) : 20/4
2 C'est un mercredi
```

Pour ce faire, vous allez devoir convertir chaque mois en son nombre de jours et ajouter ensuite celui-ci au nombre de jours séparant la date entrée du premier du mois. À cette fin, vous pouvez considérer dans un premier temps que chaque mois compte trente et un jours et ensuite retrancher les jours que vous avez compté en trop suivant le mois fourni.

Par exemple, si l'utilisateur vous demande quel jour de la semaine était le vingt avril de l'an un :

- vous multipliez trente et un par trois puisque trois mois séparent le mois d'avril du mois de janvier (janvier, février et mars) ;
- vous retranchez trois jours (puisque le mois de février ne comporte que vingt-huit jours les années non bissextiles) ;
- enfin, vous ajoutez les dix-neuf jours qui séparent la date fournie du premier du mois.

Au total, vous obtenez alors cent et neuf jours, ce qui nous donne, modulo sept, le nombre quatre, c'est donc un mercredi.

À toutes fins utiles, voici le calendrier complet de l'an un.

👁 Contenu masqué n°6

À vos claviers !

III.6.5. Correction

Bien, passons à la correction. 🍊

👁 Contenu masqué n°7

Nous commençons par demander deux nombres à l'utilisateur qui sont affectés aux variables `jours` et `mois`. Ensuite, nous multiplions le nombre de mois séparant celui entré par l'utilisateur du mois de janvier par trente et un. Après quoi, nous soustrayons les jours comptés en trop suivant le mois fourni. Enfin, nous ajoutons le nombre de jours séparant celui entré du premier du mois, comme pour la première étape.



Notez que nous avons utilisé ici une propriété intéressante de l'instruction `switch` : si la valeur de contrôle correspond à celle d'une entrée, alors les instructions sont exécutées *jusqu'à rencontrer une instruction* `break` (ou jusqu'à la fin du `switch`). Ainsi, si le mois entré est celui de mai, l'instruction `--njours` va être exécutée, puis l'instruction `njours -= 3` va également être exécutée.

III.6.6. Troisième et dernière étape

À présent, il est temps de réaliser un programme complet qui correspond aux objectifs du TP. Vous allez donc devoir demander à l'utilisateur une date entière et lui donner le jour de la semaine correspondant.

```
1 Entrez une date : 11/2/2015
2 C'est un mercredi
```

Toutefois, avant de vous lancer dans la réalisation de celui-ci, nous allons parler calendriers et années bissextiles. 🍌

III.6.6.1. Les calendriers Julien et Grégorien

Vous le savez certainement, une [année bissextile](#) est une année qui comporte 366 jours au lieu de 365 et qui se voit ainsi ajouter un vingt-neuf février. Ce que vous ne savez en revanche peut-être pas, c'est que la détermination des années bissextile a varié au cours du temps.

Jusqu'en 1582, date d'adoption du [calendrier Grégorien](#) (celui qui est en vigueur un peu près partout actuellement), c'est le [calendrier Julien](#) qui était en application. Ce dernier considérait une année comme bissextile si celle-ci était multiple de quatre. Cette méthode serait correcte si une année comportait 365,25 jours. Cependant, il s'est avéré plus tard qu'une année comportait en fait 365,2422 jours.

Dès lors, un décalage par rapport au cycle terrestre s'était lentement installé ce qui posait problème à l'Église catholique pour le calcul de la date de Pâques qui glissait doucement vers l'été. Le calendrier Grégorien fût alors instauré en 1582 pour corriger cet écart en modifiant la règle de calcul des années bissextile : il s'agit d'une année multiple de quatre *et*, s'il s'agit d'une année multiple de 100, également multiple de 400. Par exemple, les années 1000 et 1100 ne sont plus bissextiles à l'inverse de l'année 1200 qui, elle, est divisible par 400.

Toutefois, ce ne sont pas douze années bissextiles qui ont été supprimées lors de l'adoption du calendrier Grégorien (100, 200, 300, 500, 600, 700, 900, 1000, 1100, 1300, 1400, 1500), mais seulement dix afin de rapprocher la date de Pâques de l'équinoxe de printemps.

III.6.6.2. Mode de calcul

Pour réaliser votre programme, vous devrez donc vérifier si la date demandée est antérieure ou postérieure à l'an 1582. Si elle est inférieure ou égale à l'an 1582, alors vous devrez appliquer le calendrier Julien. Si elle est supérieure, vous devrez utiliser le calendrier Grégorien.

Pour vous aider, voici un schéma que vous pouvez suivre.

```
1 Si l'année est supérieure à 1582
2   Multiplier la différence d'années par 365
3   Ajouter au résultat le nombre d'années multiples de 4
4   Soustraire à cette valeur le nombre d'années multiples de 100
5   Ajouter au résultat le nombre d'années multiples de 400
6   Ajouter deux à ce nombre (du fait que seules dix années ont
   été supprimées en 1582)
7 Si l'année est inférieure ou égale à 1582
8   Multiplier la différence d'années par 365
9   Ajouter au résultat le nombre d'années multiples de 4
10
11 Au nombre de jours obtenus, ajouter la différence de jours entre
12 le mois de janvier et le mois fourni. N'oubliez pas que les mois
   comportent
13 trente et un ou trente jours et que le mois de février comporte
   pour sa
14 part vingt-huit jours sauf les années bissextiles où il s'en voit
   ajouter un
15 vingt-neuvième. Également, faites attention au calendrier en
   application pour
16 la détermination des années bissextiles !
17
18 Au résultat obtenu ajouter le nombre de jour qui sépare celui
   entré du premier
19 du mois.
20
21 Appliquer le modulo et déterminer le jour de la semaine.
```

À vous de jouer ! 🍊

III.6.7. Correction

Ça va, vous tenez bon ? 🍊

© Contenu masqué n°8

Tout d'abord, nous demandons à l'utilisateur d'entrer une date au format jj/mm/aaaa et nous attribuons chaque partie aux variables `jour`, `mois` et `an`. Ensuite, nous multiplions par 365 la

III. Les bases du langage C

différence d'années séparant l'année fournie de l'an un. Toutefois, il nous faut encore prendre en compte les années bissextiles pour que le nombre de jours obtenus soit correct. Nous ajoutons donc un jour par année bissextile en prenant soin d'appliquer les règles du calendrier en vigueur à la date fournie.

Maintenant, il nous faut ajouter le nombre de jours séparant le mois de janvier du mois spécifié par l'utilisateur. Pour ce faire, nous utilisons la même méthode que celle vue lors de la deuxième étape à *une différence près* : nous vérifions si l'année courante est bissextile afin de retrancher le bon nombre de jours (le mois de février comportant dans ce cas vingt-neuf jours et non vingt-huit).

Enfin, nous utilisons le même code que celui de la première étape.

Conclusion

Ce chapitre nous aura permis de faire une petite pause et de mettre en application ce que nous avons vu dans les chapitres précédents.

Contenu masqué

Contenu masqué n°4

1	Janvier 1						
2	di	lu	ma	me	je	ve	sa
3							1
4	2	3	4	5	6	7	8
5	9	10	11	12	13	14	15
6	16	17	18	19	20	21	22
7	23	24	25	26	27	28	29
8	30	31					

[Retourner au texte.](#)

Contenu masqué n°5

```
1 #include <stdio.h>
2
3
4 int
5 main(void)
6 {
```

```

7   unsigned jour;
8
9   printf("Entrez un jour : ");
10  scanf("%u", &jour);
11
12  switch ((jour - 1) % 7)
13  {
14  case 0:
15      printf("C'est un samedi\n");
16      break;
17
18  case 1:
19      printf("C'est un dimanche\n");
20      break;
21
22  case 2:
23      printf("C'est un lundi\n");
24      break;
25
26  case 3:
27      printf("C'est un mardi\n");
28      break;
29
30  case 4:
31      printf("C'est un mercredi\n");
32      break;
33
34  case 5:
35      printf("C'est un jeudi\n");
36      break;
37
38  case 6:
39      printf("C'est un vendredi\n");
40      break;
41  }
42
43  return 0;
44 }

```

[Retourner au texte.](#)

Contenu masqué n°6

	Janvier							Février							Mars						
	di	lu	ma	me	je	ve	sa	di	lu	ma	me	je	ve	sa	di	lu	ma	me	je	ve	sa
							1			1	2	3	4	5			1	2	3	4	5
	2	3	4	5	6	7	8	6	7	8	9	10	11	12	6	7	8	9	10	11	12

III. Les bases du langage C

5	9	10	11	12	13	14	15	13	14	15	16	17	18	19	13	14	15	16	17	18	19
6	16	17	18	19	20	21	22	20	21	22	23	24	25	26	20	21	22	23	24	25	26
7	23	24	25	26	27	28	29	27	28						27	28	29	30	31		
8	30	31																			
9																					
10																					
11																					
12																					
13																					
14																					
15																					
16																					
17																					
18																					
19																					
20																					
21																					
22																					
23																					
24																					
25																					
26																					
27																					
28																					
29																					
30																					
31																					

[Retourner au texte.](#)

Contenu masqué n°7

1	<code>#include <stdio.h></code>
2	
3	
4	<code>int</code>
5	<code>main(void)</code>
6	<code>{</code>
7	<code> unsigned jour;</code>
8	<code> unsigned mois;</code>
9	
10	<code> printf("Entrez une date (jj/mm) : ");</code>
11	<code> scanf("%u/%u", &jour, &mois);</code>


```
12
13     unsigned njours = (mois - 1) * 31;
14
15     switch (mois)
16     {
17     case 12:
18         --njours;
19     case 11:
20     case 10:
21         --njours;
22     case 9:
23     case 8:
24     case 7:
25         --njours;
26     case 6:
27     case 5:
28         --njours;
29     case 4:
30     case 3:
31         njours -= 3;
32         break;
33     }
34
35     njours += (jour - 1);
36
37     switch (njours % 7)
38     {
39     case 0:
40         printf("C'est un samedi\n");
41         break;
42
43     case 1:
44         printf("C'est un dimanche\n");
45         break;
46
47     case 2:
48         printf("C'est un lundi\n");
49         break;
50
51     case 3:
52         printf("C'est un mardi\n");
53         break;
54
55     case 4:
56         printf("C'est un mercredi\n");
57         break;
58
59     case 5:
60         printf("C'est un jeudi\n");
61         break;
```

```
62
63     case 6:
64         printf("C'est un vendredi\n");
65         break;
66     }
67
68     return 0;
69 }
```

[Retourner au texte.](#)

Contenu masqué n°8

```
1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      unsigned jour;
8      unsigned mois;
9      unsigned an;
10
11     printf("Entrez une date (jj/mm/aaaa) : ");
12     scanf("%u/%u/%u", &jour, &mois, &an);
13
14     unsigned njours = (an - 1) * 365;
15
16     if (an > 1582) /* Calendrier Grégorien */
17     {
18         njours += ((an - 1) / 4);
19         njours -= ((an - 1) / 100);
20         njours += ((an - 1) / 400);
21         njours += 2;
22     }
23     else /* Calendrier Julien */
24         njours += ((an - 1) / 4);
25
26     njours += (mois - 1) * 31;
27
28     switch (mois)
29     {
30     case 12:
31         --njours;
32     case 11:
33     case 10:
34         --njours;
```

```
35     case 9:
36     case 8:
37     case 7:
38         --njours;
39     case 6:
40     case 5:
41         --njours;
42     case 4:
43     case 3:
44         if (an > 1582)
45         {
46             if (an % 4 == 0 && (an % 100 != 0 || an % 400 == 0))
47                 njours -= 2;
48             else
49                 njours -= 3;
50         }
51         else
52         {
53             if (an % 4 == 0)
54                 njours -= 2;
55             else
56                 njours -= 3;
57         }
58         break;
59     }
60
61
62     njours += (jour - 1);
63
64     switch (njours % 7)
65     {
66     case 0:
67         printf("C'est un samedi\n");
68         break;
69
70     case 1:
71         printf("C'est un dimanche\n");
72         break;
73
74     case 2:
75         printf("C'est un lundi\n");
76         break;
77
78     case 3:
79         printf("C'est un mardi\n");
80         break;
81
82     case 4:
83         printf("C'est un mercredi\n");
84         break;
```

```
85
86     case 5:
87         printf("C'est un jeudi\n");
88         break;
89
90     case 6:
91         printf("C'est un vendredi\n");
92         break;
93     }
94
95     return 0;
96 }
```

[Retourner au texte.](#)

III.7. Les boucles

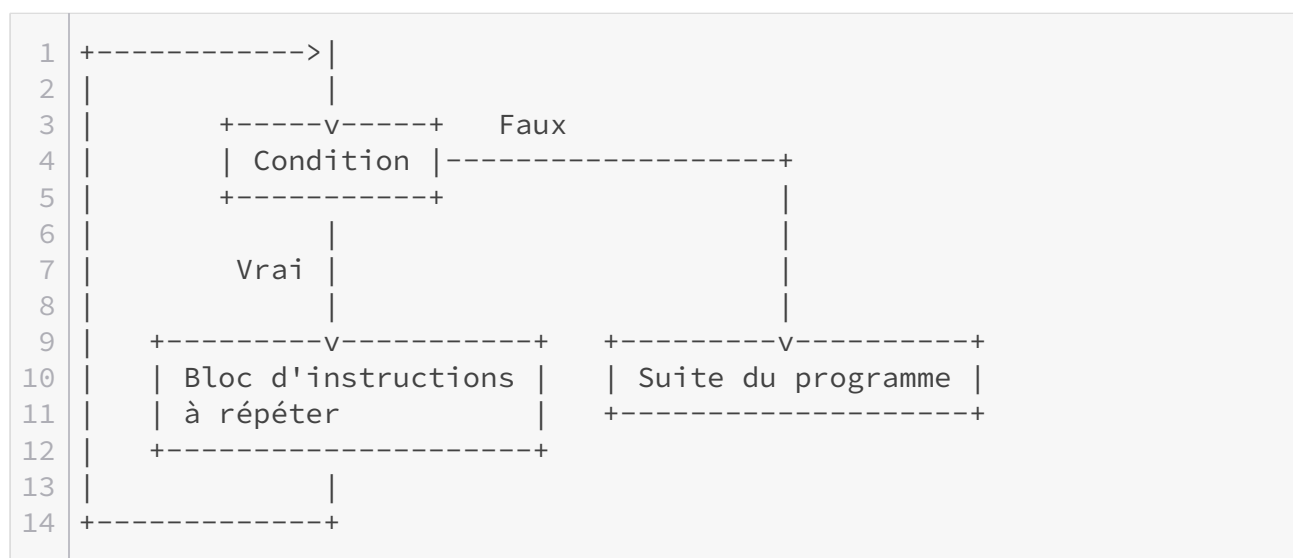
Introduction

Dans ce chapitre, nous allons aborder les **boucles**. Une boucle est un moyen de répéter des instructions suivant le résultat d'une condition. Ces structures, dites **itératives**, que nous allons voir dans ce chapitre sont les suivantes.

Structure itérative	Action
<i>while...</i>	répète une suite d'instructions tant qu'une condition est respectée.
<i>do... while...</i>	répète une suite d'instructions tant qu'une condition est respectée. Le groupe d'instructions est exécuté une fois.
<i>for...</i>	répète un nombre fixé de fois une suite d'instructions.

III.7.1. La boucle while

La première des boucles que nous allons étudier est la boucle **while** (qui signifie « tant que »). Celle-ci permet de répéter un bloc d'instructions tant qu'une condition est remplie.



III.7.1.1. Syntaxe

La syntaxe de notre boucle `while` est assez simple.

```
1 while (/* Condition */)
2 {
3     /* Bloc d'instructions à répéter */
4 }
```

Si vous n'avez qu'une seule instruction à réaliser, vous avez la possibilité de ne pas mettre d'accolades.

```
1 while (/* Condition */)
2     /* Une seule instruction */
```

III.7.1.1.1. Exemple

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int i = 0;
7
8     while (i < 5)
9     {
10         printf("La variable i vaut %d\n", i);
11         i++;
12     }
13
14     return 0;
15 }
```

Résultat

```
1 La variable i vaut 0
2 La variable i vaut 1
3 La variable i vaut 2
4 La variable i vaut 3
5 La variable i vaut 4
```

III. Les bases du langage C

Le fonctionnement est simple à comprendre :

- Au départ, notre variable `i` vaut zéro. Étant donné que zéro est bien inférieur à cinq, la condition est vraie, le corps de la boucle est donc exécuté.
- La valeur de `i` est affichée.
- `i` est augmentée d'une unité et vaut désormais un.
- La condition de la boucle est de nouveau vérifiée.

Ces étapes vont ainsi se répéter pour les valeurs un, deux, trois et quatre. Quand la variable `i` vaudra cinq, la condition sera fausse, et l'instruction `while` sera alors passée.

i

Dans cet exemple, nous utilisons une variable nommée `i`. Ce nom provient des mathématiques où il est utilisé pour la notation des sommes de suite de termes, par exemple comme celle-ci $\sum_{i=2}^4 i^2 = 2^2 + 3^2 + 4^2 = 29$. Ce nom est devenu une convention de nommage en C, vous le rencontrerez très souvent. Dans le cas où d'autres variables sont nécessaires, il est d'usage de poursuivre l'ordre alphabétique, soit `j`, `k`, `l`, etc.

III.7.1.2. Exercice

Essayez de réaliser un programme qui détermine si un nombre entré par l'utilisateur est premier. Pour rappel, un nombre est dit premier s'il n'est divisible que par un et par lui-même. Notez que si un nombre x est divisible par y alors le résultat de l'opération `x % y` est nul.

III.7.1.2.1. Indice

👁 Contenu masqué n°9

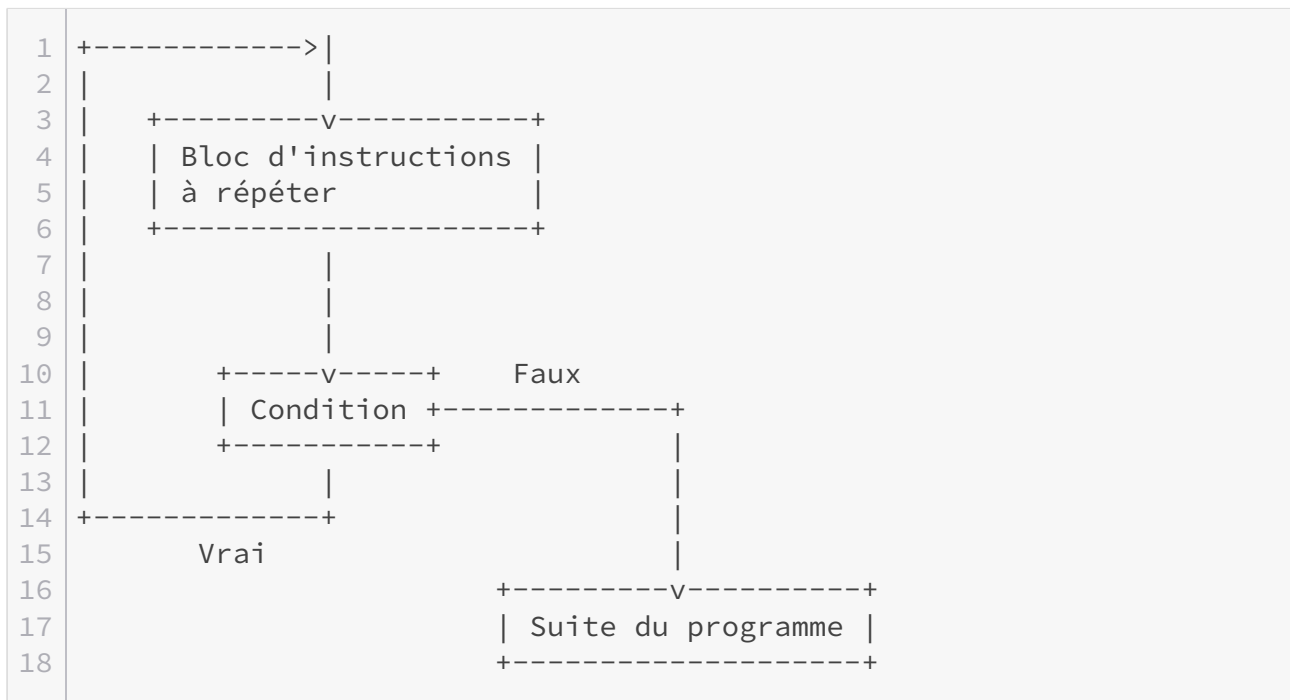
III.7.1.2.2. Correction

👁 Contenu masqué n°10

III.7.2. La boucle do-while

La boucle `do while` fonctionne comme la boucle `while`, à un petit détail près : elle s'exécutera toujours au moins une fois, alors qu'une boucle `while` peut ne pas s'exécuter si la condition est fausse dès le départ.

III. Les bases du langage C



III.7.2.1. Syntaxe

À la différence de la boucle `while`, la condition est placée à la fin du bloc d'instruction à répéter, ce qui explique pourquoi celui-ci est toujours exécuté au moins une fois. Remarquez également la présence d'un point-virgule à la fin de l'instruction qui est obligatoire.

```
1 do
2 {
3     /* Bloc d'instructions à répéter */
4 } while (/* Condition */);
```

Si vous n'avez qu'une seule instruction à exécuter, les accolades sont facultatives.

```
1 do
2     /* Une seule instruction */
3 while (/* Condition */);
```

III.7.2.1.1. Exemple 1

Voici le même code que celui présenté avec l'instruction `while`.


```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int i = 0;
7
8     do
9     {
10         printf("La variable i vaut %d\n", i);
11         ++i;
12     } while (i < 5);
13
14     return 0;
15 }
```

Résultat

```
1 La variable i vaut 0
2 La variable i vaut 1
3 La variable i vaut 2
4 La variable i vaut 3
5 La variable i vaut 4
```

III.7.2.1.2. Exemple 2

Comme nous vous l'avons dit plus haut, une boucle **do while** s'exécute au moins une fois.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     do
7     {
8         printf("Boucle do-while\n");
9     }
10    while (0);
11
12    return 0;
13 }
```

1 Boucle do-while

Comme vous le voyez, bien que la condition est fausse (pour rappel, une valeur nulle correspond à une valeur fausse), le corps de la boucle est exécuté une fois puisque la condition n'est évaluée qu'*après* le parcours du bloc d'instructions.

III.7.3. La boucle for

III.7.3.1. Syntaxe

```
1  for (/* Expression/Déclaration */; /* Condition */; /* Expression  
   */)  
2  {  
3      /* Instructions à répéter */  
4  }
```

Une boucle **for** se décompose en trois parties (ou trois clauses) :

- une expression et/ou une déclaration qui sera le plus souvent l'initialisation d'une variable ;
- une condition ;
- une seconde expression, qui consistera le plus souvent en l'incrémentement d'une variable.

Techniquement, une boucle **for** revient en fait à écrire ceci.

```
1  /* Expression/Déclaration */  
2  
3  while (/* Condition */)   
4  {  
5      /* Bloc d'instructions à répéter */  
6      /* Expression */  
7  }
```

Comme pour les deux autres boucles, si le corps ne comporte qu'une seule instruction, les accolades ne sont pas nécessaires.

```
1  for (/* Expression/Déclaration */; /* Condition */; /* Expression  
   */)  
2      /* Une seule instruction */
```

III. Les bases du langage C

III.7.3.1.1. Exemple

Le fonctionnement de cette boucle est plus simple à appréhender à l'aide d'un exemple.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     for (int i = 0; i < 5; ++i)
7         printf("la variable i vaut %d\n", i);
8
9     return 0;
10 }
```

Résultat

```
1 variable vaut 0
2 variable vaut 1
3 variable vaut 2
4 variable vaut 3
5 variable vaut 4
```

Ce qui, comme dit précédemment, revient exactement à écrire cela.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int i = 0;
7
8     while (i < 5)
9     {
10         printf("la variable i vaut %d\n", i);
11         ++i;
12     }
13
14     return 0;
15 }
```



Notez bien que la déclaration `int i = 0` est située *en dehors* du corps de la boucle. Elle n'est donc pas exécutée à chaque tour.



Si vous déclarez une variable au sein de l'instruction `for`, celle-ci ne sera utilisable *qu'au sein de cette boucle*. Le code suivant est donc incorrect.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     for (int i = 0; i < 5; ++i)
7         printf("la variable i vaut %d\n", i);
8
9     printf("i = %d\n", i); /* Faux, i n'est pas utilisable
10        hors de la boucle for */
11     return 0;
12 }
```

Dans un tel cas, il est nécessaire de déclarer la variable avant et en dehors de la boucle `for`.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int i;
7
8     for (i = 0; i < 5; ++i)
9         printf("la variable i vaut %d\n", i);
10
11     printf("i = %d\n", i); /* Ok */
12     return 0;
13 }
```

III.7.3.1.2. Exercice

Essayez de réaliser un programme qui calcule la somme de tous les nombres compris entre un et n (n étant déterminé par vos soins). Autrement dit, pour un nombre n donné, vous allez devoir calculer $1 + 2 + 3 + \dots + (n - 2) + (n - 1) + n$.

👁 Contenu masqué n°11

i

Notez qu'il est possible de réaliser cet exercice sans boucle en calculant : $\frac{N \times (N+1)}{2}$.

III.7.3.2. Plusieurs compteurs

Notez que le nombre de compteurs, de déclarations ou de conditions n'est pas limité, comme le démontre le code suivant.

```
1 for (int i = 0, j = 2; i < 10 && j < 12; i++, j += 2)
```

Ici, nous définissons deux compteurs `i` et `j` initialisés respectivement à zéro et deux. Le contenu de la boucle est exécuté tant que `i` est inférieur à dix et que `j` est inférieur à douze, `i` étant augmentée de une unité et `j` de deux unités à chaque tour de boucle. Le code est encore assez lisible, cependant la modération est de mise, un trop grand nombre de paramètres rendant la boucle `for` illisible.

III.7.4. Imbrications

Il est parfaitement possible d'**imbriquer** une ou plusieurs boucles en plaçant une boucle dans le corps d'une autre boucle.

```
1 for (int i = 0; i < 1000; ++i)
2 {
3     for (int j = i; j < 1000; ++j)
4     {
5         /* Code */
6     }
7 }
```

Cela peut servir par exemple pour déterminer la liste des nombres dont le produit vaut mille.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     for (int i = 0; i <= 1000; ++i)
```

```
7     for (int j = i; j <= 1000; ++j)
8         if (i * j == 1000)
9             printf ("%d * %d = 1000 \n", i, j);
10
11     return 0;
12 }
```

Résultat

1	1 * 1000 = 1000
2	2 * 500 = 1000
3	4 * 250 = 1000
4	5 * 200 = 1000
5	8 * 125 = 1000
6	10 * 100 = 1000
7	20 * 50 = 1000
8	25 * 40 = 1000



Vous n'êtes bien entendu pas tenu d'imbriquer des types de boucles identiques. Vous pouvez parfaitement placer, par exemple, une boucle `while` dans une boucle `for`.

III.7.5. Boucles infinies

Lorsque vous utilisez une boucle, il y a une chose que vous devez impérativement vérifier : elle doit pouvoir se terminer. Cela paraît évident de prime abord, pourtant il s'agit d'une erreur de programmation assez fréquente qui donne lieu à des **boucles infinies**. Soyez donc vigilants !

L'exemple le plus fréquent est l'oubli d'incrémentement de l'itérateur.

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i = 0;
6
7      while (i < 5)
8      {
9          printf("La variable i vaut %d\n", i);
10         /* Oubli d'incrémentation. */
11     }
12 }
```

III. Les bases du langage C

```
13     return 0;
14 }
```

Résultat

```
1 La variable i vaut 0
2 La variable i vaut 0
3 La variable i vaut 0
4 ...
```

Ce programme continuera son exécution jusqu'à ce qu'il soit arrêté.



Pour stopper l'exécution d'un programme, vous pouvez utiliser la combinaison **Ctrl** + **c** ou simplement fermer votre terminal.

III.7.6. Exercices

III.7.6.1. Calcul du PGCD de deux nombres

Le PGCD de deux nombres est le plus grand nombre qui peut diviser ces derniers. Par exemple, le PGCD de quinze et douze est trois et celui de vingt-quatre et dix-huit est six.

Pour le calculer, nous devons disposer de deux nombres a et b avec a supérieur à b . Ensuite, nous effectuons la division entière de a par b .

- si le reste est nul, alors nous avons terminé ;
- si le reste est non nul, nous revenons au début en remplaçant a par b et b par le reste.

Avec cette explication, vous avez tout ce qu'il vous faut : à vos claviers !

III.7.6.1.1. Correction

👁 Contenu masqué n°12

III.7.6.2. Une overdose de lapins

Au treizième siècle, un mathématicien italien du nom de *Leonardo Fibonacci* posa un petit problème dans un de ses livres, qui mettait en scène des lapins. Ce petit problème mit en avant

III. Les bases du langage C

une suite de nombres particulière, nommée la [suite de Fibonacci](#) , du nom de son inventeur. Il fit les hypothèses suivantes :

- le premier mois, nous plaçons un couple de deux lapins dans un enclos ;
- un couple de lapin ne peut procréer qu'à partir du troisième mois de sa venue dans l'enclos (autrement dit, il ne se passe rien pendant les deux premiers mois) ;
- chaque couple *capable de procréer* donne naissance à un nouveau couple tous les mois ;
- enfin, pour éviter tout problème avec la [SPA](#), les lapins ne meurent jamais.

Le problème est le suivant : combien y a-t-il de couples de lapins dans l'enclos au n-ième mois ? Le but de cet exercice est de réaliser un petit programme qui fasse ce calcul automatiquement.

III.7.6.2.1. Indice

👁 Contenu masqué n°13

III.7.6.2.2. Correction

👁 Contenu masqué n°14

III.7.6.3. Des pieds et des mains pour convertir mille miles

Si vous avez déjà voyagé en Grande-Bretagne ou aux États-unis, vous savez que les unités de mesure utilisées dans ces pays sont différentes des nôtres. Au lieu de notre cher système métrique, dont les *stars* sont les centimètres, mètres et kilomètres, nos amis outre-manche et outre-atlantique utilisent le système impérial, avec ses pouces, pieds et *miles*, voire lieues et *furlongs* ! Et pour empirer les choses, la conversion n'est pas toujours simple à effectuer de tête... Aussi, la lecture d'un ouvrage tel que *Le Seigneur des Anneaux*, dans lequel toutes les distances sont exprimées en unités impériales, peut se révéler pénible.

Grâce au langage C, nous allons aujourd'hui résoudre tous ces problèmes ! Votre mission, si vous l'acceptez, sera d'écrire un programme affichant un tableau de conversion entre *miles* et kilomètres. Le programme ne demande rien à l'utilisateur, mais doit afficher quelque chose comme ceci.

1	Miles	Km
2	5	8
3	10	16
4	15	24
5	20	32
6	25	40
7	30	48

III. Les bases du langage C

Autrement dit, le programme compte les *miles* de cinq en cinq jusqu'à trente et affiche à chaque fois la valeur correspondante en kilomètres. Un *mile* vaut exactement 1.609344 km, cependant nous allons utiliser une valeur approchée : huit-cinquièmes de kilomètre (soit 1.6km). Autrement dit, $1 \text{ miles} = \frac{8}{5} \text{ km}$ ou $(1 \text{ km} = \frac{5}{8} \text{ miles})$.

III.7.6.3.1. Correction

👁 Contenu masqué n°15

III.7.6.4. Puissances de trois

Passons à un exercice un peu plus difficile, du domaine des mathématiques. Essayez de le faire même si vous n'aimez pas les mathématiques.

Vous devez vérifier si un nombre est une puissance de trois, et afficher le résultat. De plus, si c'est le cas, vous devez afficher l'exposant qui va avec.

III.7.6.4.1. Indice

👁 Contenu masqué n°16

III.7.6.4.2. Correction

👁 Contenu masqué n°17

III.7.6.5. La disparition : le retour

Connaissez-vous le roman *La Disparition* ? Il s'agit d'un roman français de Georges Perec, publié en 1969. Sa particularité est qu'il ne contient *pas une seule fois* la lettre « e ». On appelle ce genre de textes privés d'une lettre des *lipogrammes*. Celui-ci est une prouesse littéraire, car la lettre « e » est la plus fréquente de la langue française : elle représente une lettre sur six en moyenne ! Le roman faisant environ trois cents pages, il a sûrement fallu déployer des trésors d'inventivité pour éviter tous les mots contenant un « e ».

Si vous essayez de composer un tel texte, vous allez vite vous rendre compte que vous glissez souvent des « e » dans vos phrases sans même vous en apercevoir. Nous avons besoin d'un vérificateur qui nous sermonnera chaque fois que nous écrirons un « e ». C'est là que le langage C entre en scène !

III. Les bases du langage C

Écrivez un programme qui demande à l'utilisateur de taper une phrase, puis qui affiche le nombre de « e » qu'il y a dans celle-ci. Une phrase se termine toujours par un point « . », un point d'exclamation « ! » ou un point d'interrogation « ? ». Pour effectuer cet exercice, il sera indispensable de lire la phrase caractère par caractère.

```
1 Entrez une phrase : Bonjour, comment allez-vous ?
2 Au moins une lettre 'e' a été repérée (précisément : 2) !
```

III.7.6.5.1. Indice

👁 Contenu masqué n°18

III.7.6.5.2. Correction

👁 Contenu masqué n°19

Conclusion

Les boucles sont assez faciles à comprendre, la seule chose dont il faut se souvenir étant de faire attention de bien avoir une condition de sortie pour ne pas tomber dans une boucle infinie.

En résumé

1. Une boucle permet de répéter l'exécution d'une suite d'instructions tant qu'une condition est vraie ;
2. La boucle **while** évalue une condition *avant* d'exécuter une suite d'instructions ;
3. La boucle **do while** évalue une condition *après* avoir exécuté une suite d'instructions ;
4. La boucle **for** est une forme condensée de la boucle **while** ;
5. Une variable définie dans la première clause d'une boucle **for** ne peut être utilisée qu'au sein de cette boucle ;
6. Une boucle dont la condition est toujours vraie est appelée une boucle infinie.

Contenu masqué

Contenu masqué n°9

Pour savoir si un nombre est premier, il va vous falloir vérifier si celui-ci est uniquement divisible par un et lui-même. Dit autrement, vous allez devoir contrôler qu'aucun nombre compris entre 1 et le nombre entré (tout deux exclus) n'est un diviseur de ce dernier. Pour parcourir ces différentes possibilités, une boucle va vous être nécessaire. [Retourner au texte.](#)

Contenu masqué n°10

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int nombre;
7
8      printf("Entrez un nombre : ");
9      scanf("%d", &nombre);
10
11     int i = 2;
12
13     while ((i < nombre) && (nombre % i != 0))
14     {
15         ++i;
16     }
17
18     if (i == nombre)
19     {
20         printf("%d est un nombre premier\n", nombre);
21     }
22     else
23     {
24         printf("%d n'est pas un nombre premier\n", nombre);
25     }
26
27     return 0;
28 }
```

[Retourner au texte.](#)

Contenu masqué n°11

```
1 #include <stdio.h>
2
3
4 int main (void)
5 {
6     const unsigned int n = 250;
7     unsigned somme = 0;
8
9     for (unsigned i = 1; i <= n; ++i)
10         somme += i;
11
12     printf ("Somme de 1 à %u : %u\n", n, somme);
13     return 0;
14 }
```

[Retourner au texte.](#)

Contenu masqué n°12

```
1 #include <stdio.h>
2
3
4 int main (void)
5 {
6     unsigned int a = 46;
7     unsigned int b = 42;
8     unsigned int reste = a % b;
9
10    while (reste != 0)
11    {
12        a = b;
13        b = reste;
14        reste = a % b;
15    }
16
17    printf("%d", b);
18    return 0;
19 }
```

[Retourner au texte.](#)

Contenu masqué n°13

Allez, un petit coup de pouce : suivant l'énoncé, un couple ne donne naissance à un autre couple qu'au début du troisième mois de son apparition. Combien de couple y a-t-il le premier mois ? Un seul. Combien y en a-t-il le deuxième mois ? Toujours un seul. Combien y en a-t-il le troisième mois (le premier couple étant là depuis deux mois) ? Deux. Avec ceci, vous devriez venir à bout du problème.

[Retourner au texte.](#)

Contenu masqué n°14

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int a = 0;
7     int b = 1;
8     int nb_lapins = 1;
9     int const mois = 10;
10
11     for (int i = 1; i < mois; ++i)
12     {
13         nb_lapins = a + b;
14         a = b;
15         b = nb_lapins;
16     }
17
18     printf("Au mois %d, il y a %d couples de lapins\n", mois,
19           nb_lapins);
20     return 0;
21 }
```

[Retourner au texte.](#)

Contenu masqué n°15

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     unsigned miles = 0;
7
8     printf("Miles\tKm\n");
```

```
9
10 do
11 {
12     ++miles;
13     printf("%u\t%u\n", miles * 5, miles * 8);
14 } while (miles * 5 < 30);
15
16 return 0;
17 }
```

[Retourner au texte.](#)

Contenu masqué n°16

Pour savoir si un nombre est une puissance de trois, vous pouvez utiliser le modulo. Attention cependant : si le reste vaut 0, le nombre n'est pas forcément une puissance de trois (par exemple, le reste de la division de 15 par 3 est nul, mais 15 n'est pas une puissance de trois). [Retourner au texte.](#)

Contenu masqué n°17

```
1 #include <stdio.h>
2
3
4 /* Un petite explication s'impose, notamment au niveau du for. La
   première
5 partie qui correspond à l'initialisation de i ne devrait pas vous
   poser
6 trop de soucis. Ensuite, le i /= 3 sert à diviser i par trois à
   chaque
7 itération. Au tour de la condition, le principe est simple : tant
   que le
8 reste de la division de i par 3 est égal à zéro et que i est
   positif, on
9 incrémente l'exposant. Enfin, pour déterminer si le nombre est une
   puissance
10 de trois, il suffit de vérifier si i est égal à 1 (essayez avec de
   petits
11 nombres dans votre tête ou sur papier si vous n'êtes pas
   convaincu). */
12
13 int main(void)
14 {
15     int number;
16 }
```

```
17     printf("Veuillez entrez un nombre : ");
18     scanf("%d", &number);
19
20     int exposant = 0;
21     int i;
22
23     for (i = number; (i % 3 == 0) && (i > 0); i /= 3)
24         ++exposant;
25
26     /* Chaque division successive divise i par 3, donc si nous
27        obtenons finalement
28        i == 1, c'est que le nombre est bien une puissance de 3 */
29
30     if (i == 1)
31         printf ("%d est égal à 3 ^ %d\n", number, exposant);
32     else
33         printf("%d n'est pas une puissance de 3\n", number);
34
35     return 0;
36 }
```

[Retourner au texte.](#)

Contenu masqué n°18

La première chose à faire est d'afficher un message de bienvenue, afin que l'utilisateur sache quel est votre programme. Ensuite, Il vous faudra lire les caractères tapés (rappelez-vous les différents formats de la fonction `scanf()`), un par un, *jusqu'à ce qu'un point* (normal, d'exclamation ou d'interrogation) soit rencontré. Dans l'intervalle, il faudra compter chaque « e » qui apparaîtra. Enfin, il faudra afficher le nombre de « e » qui ont été comptés (potentiellement aucun).

[Retourner au texte.](#)

Contenu masqué n°19

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Entrez une phrase se terminant par '.', '!' ou '?' : ")
6          );
7
8      unsigned compteur = 0;
9      char c;
10
11     do
```

```
11     {
12         scanf("%c", &c);
13
14         if (c == 'e' || c == 'E')
15             compteur++;
16     } while (c != '.' && c != '!' && c != '?');
17
18     if (compteur == 0)
19         printf("Aucune lettre 'e' repérée. Félicitations !\n");
20     else
21         printf(
22             "Au moins une lettre 'e' a été repérée (précisément : %d) !\n",
23             compteur);
24     return 0;
25 }
```

[Retourner au texte.](#)

III.8. Les sauts

Introduction

Dans les chapitres précédents, nous avons vu comment modifier l'exécution de notre programme en fonction du résultat d'une ou plusieurs conditions. Ainsi, nous avons pu réaliser des tâches plus complexes que de simplement exécuter une suite d'instructions de manière linéaire.

Cette exécution non linéaire est possible grâce à ce que l'on appelle des **sauts**. Un saut correspond au passage d'un point à un autre d'un programme. Bien que cela vous ait été caché, sachez que vous en avez déjà rencontré ! En effet, une instruction `if` réalise par exemple un saut à votre insu.

```
1  if (/* Condition */)
2  {
3      /* Bloc */
4  }
5
6  /* Suite du programme */
```

Dans le cas où la condition est fausse, l'exécution du programme passe le bloc de l'instruction `if` et exécute ce qui suit. Autrement dit, il y a un **saut** jusqu'à la suite du bloc.

Dans la même veine, une boucle `while` réalise également des sauts.

```
1  while (/* Condition */)
2  {
3      /* Bloc à répéter */
4  }
5
6  /* Suite du programme */
```

Dans cet exemple, si la condition est vraie, le bloc qui suit est exécuté puis il y a un saut pour revenir à l'évaluation de la condition. Si en revanche elle est fausse, comme pour l'instruction `if`, il y a un saut au-delà du bloc d'instructions.

Tous ces sauts sont cependant automatiques et vous sont cachés. Dans ce chapitre, nous allons voir comment réaliser manuellement des sauts à l'aide de trois instructions : `break`, `continue` et `goto`.

III.8.1. L'instruction break

Nous avons déjà rencontré l'instruction `break` lors de la présentation de l'instruction `switch`, cette dernière permettait de quitter le bloc d'un `switch` pour reprendre immédiatement après. Cependant, l'instruction `break` peut également être utilisée au sein d'une boucle pour stopper son exécution (autrement dit pour effectuer un saut au-delà du bloc à répéter).

III.8.1.1. Exemple

Le plus souvent, une instruction `break` est employée pour sortir d'une itération lorsqu'une condition (différente de celle contrôlant l'exécution de la boucle) est remplie. Par exemple, si nous souhaitons réaliser un programme qui détermine le plus petit diviseur commun de deux nombres, nous pouvons utiliser cette instruction comme suit.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int a;
7      int b;
8
9      printf("Entrez deux nombres : ");
10     scanf("%d %d", &a, &b);
11
12     int min = (a < b) ? a : b;
13
14     for (int i = 2; i <= min; ++i)
15         if (a % i == 0 && b % i == 0)
16         {
17             printf("le plus petit diviseur de %d et %d est %d\n",
18                 a, b, i);
19             break;
20         }
21     return 0;
22 }
```

Résultat

```
1 Entrez deux nombres : 112 567
2 le plus petit diviseur de 112 et 567 est 7
3
4 Entrez deux nombres : 13 17
```

Comme vous le voyez, la condition principale permet de progresser parmi les diviseurs possibles alors que la seconde détermine si la valeur courante de `i` est un diviseur commun. Si c'est le cas, l'exécution de la boucle est stoppée et le résultat affiché. Dans le cas où il n'y a aucun diviseur commun, la boucle s'arrête lorsque le plus petit des deux nombres est atteint.

III.8.2. L'instruction continue

L'instruction `continue` permet d'arrêter l'exécution de l'itération courante. Autrement dit, celle-ci vous permet de retourner (sauter) directement à l'évaluation de la condition.

III.8.2.0.1. Exemple

Afin d'améliorer un peu l'exemple précédent, nous pourrions passer les cas où le diviseur testé est un multiple de deux (puisque si un des deux nombres n'est pas divisible par deux, il ne peut pas l'être par quatre, par exemple).

Ceci peut s'exprimer à l'aide de l'instruction `continue`.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int a;
7     int b;
8
9     printf("Entrez deux nombres : ");
10    scanf("%d %d", &a, &b);
11
12    int min = (a < b) ? a : b;
13
14    for (int i = 2; i <= min; ++i)
15    {
16        if (i != 2 && i % 2 == 0)
17        {
18            printf("je passe %d\n", i);
```

```
19         continue;
20     }
21     if (a % i == 0 && b % i == 0)
22     {
23         printf("le plus petit diviseur de %d et %d est %d\n",
24             a, b, i);
25         break;
26     }
27
28     return 0;
29 }
```

Résultat

```
1 je passe 4
2 je passe 6
3 le plus petit diviseur de 112 et 567 est 7
```

i

Dans le cas de la boucle `for`, l'exécution reprend à l'évaluation de sa deuxième expression (ici `++i`) et non à l'évaluation de la condition (qui a lieu juste après). Il serait en effet mal venu que la variable `i` ne soit pas incrémentée lors de l'utilisation de l'instruction `continue`.

III.8.3. Boucles imbriquées

Notez bien que les instructions `break` et `continue` n'affectent que l'exécution de la boucle dans laquelle elles sont situées. Ainsi, si vous utilisez l'instruction `break` dans une boucle imbriquée dans une autre, vous sortirez de la première, mais pas de la seconde.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     for (int i = 0 ; i <= 1000 ; ++i)
7         for (int j = i ; j <= 1000 ; ++j)
8             if (i * j == 1000)
9             {
10                 printf ("%d * %d = 1000 \n", i, j);
11             }
```

```
11         break; /* Quitte la boucle courante, mais pas la
12             première. */
13     }
14     return 0;
15 }
```

Résultat

1	1 * 1000 = 1000
2	2 * 500 = 1000
3	4 * 250 = 1000
4	5 * 200 = 1000
5	8 * 125 = 1000
6	10 * 100 = 1000
7	20 * 50 = 1000
8	25 * 40 = 1000

III.8.4. L'instruction goto

Nous venons de voir qu'il était possible de réaliser des sauts à l'aide des instructions `break` et `continue`. Cependant, d'une part ces instructions sont confinées à une boucle ou à une instruction `switch` et, d'autre part, la destination du saut nous est imposée (la condition avec `continue`, la fin du bloc d'instructions avec `break`).

L'instruction `goto` permet de sauter à un point précis du programme que nous aurons déterminé à l'avance. Pour ce faire, le langage C nous permet de marquer des instructions à l'aide d'étiquettes (*labels* en anglais). Une étiquette n'est rien d'autre qu'un nom choisi par nos soins suivi du caractère `:`. Généralement, par souci de lisibilité, les étiquettes sont placées en retrait des instructions qu'elles désignent.

III.8.4.1. Exemple

Reprenons (encore) l'exemple du calcul du plus petit commun diviseur. Ce dernier aurait pu être écrit comme suit à l'aide d'une instruction `goto`.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
```

```
6     int a;
7     int b;
8
9     printf("Entrez deux nombres : ");
10    scanf("%d %d", &a, &b);
11
12    int min = (a < b) ? a : b;
13    int i;
14
15    for (i = 2; i <= min; ++i)
16        if (a % i == 0 && b % i == 0)
17            goto trouve;
18
19    return 0;
20 trouve:
21    printf("le plus petit diviseur de %d et %d est %d\n", a, b, i);
22    return 0;
23 }
```

Comme vous le voyez, l'appel à la fonction `printf()` a été marqué avec une étiquette nommée `trouve`. Celle-ci est utilisée avec l'instruction `goto` pour spécifier que c'est à cet endroit que nous souhaitons nous rendre si un diviseur commun est trouvé. Vous remarquerez également que nous avons désormais deux instructions `return`, la première étant exécutée dans le cas où aucun diviseur commun n'est trouvé.

III.8.4.2. Le dessous des boucles

Maintenant que vous savez cela, vous devriez être capable de réécrire n'importe quelle boucle à l'aide de cette instruction. En effet, une boucle se compose de deux sauts : un vers une condition et l'autre vers l'instruction qui suit le corps de la boucle. Ainsi, les deux codes suivants sont équivalents.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int i = 0;
7
8      while (i < 5)
9      {
10         printf("La variable i vaut %d\n", i);
11         i++;
12     }
13
14     return 0;
```

```
15 }
```

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int i = 0;
7
8      condition:
9      if (i < 5)
10     {
11         printf("La variable i vaut %d\n", i);
12         i++;
13         goto condition;
14     }
15
16     return 0;
17 }
```

III.8.4.3. Goto Hell?

Bien qu'utile dans certaines circonstances, sachez que l'instruction `goto` est fortement décriée, principalement pour deux raisons :

- mis à part dans des cas spécifiques, il est possible de réaliser la même action de manière plus claire à l'aide de structures de contrôles ;
- l'utilisation de cette instruction peut amener votre code à être plus difficilement lisible et, dans les pires cas, en faire un [code spaghetti](#) [↗](#) .

À vrai dire, elle est aujourd'hui surtout utilisée dans le cas de la gestion d'erreur, ce que nous verrons plus tard dans ce cours. Aussi, en attendant, nous vous déconseillons de l'utiliser.

Conclusion

En résumé

1. Une instruction de saut permet de passer directement à l'exécution d'une instruction précise ;
2. L'instruction `break` permet de sortir de la boucle ou de l'instruction `switch` dans laquelle elle est employée ;
3. L'instruction `continue` permet de retourner à l'évaluation de la condition de la boucle dans laquelle elle est utilisée ;
4. L'instruction `goto` permet de passer à l'exécution d'une instruction désignée par une étiquette.

III.9. Les fonctions

Introduction

Nous avons découvert beaucoup de nouveautés dans les chapitres précédents et nos programmes commencent à grossir. C'est pourquoi il est important d'apprendre à les découper en **fonctions**.

III.9.1. Qu'est-ce qu'une fonction ?

Le concept de fonction ne vous est pas inconnu : `printf()`, `scanf()`, et `main()` sont des **fonctions**.

?

Mais qu'est-ce qu'une fonction exactement et quel est leur rôle exactement ?

Une fonction est :

- une suite d'instructions ;
- marquée à l'aide d'un nom (comme une variable finalement) ;
- qui a vocation à être exécutée à plusieurs reprises ;
- qui rassemble des instructions qui permettent d'effectuer une tâche précise (comme afficher du texte à l'écran, calculer la racine carrée d'un nombre, etc).

Pour mieux saisir leur intérêt, prenons un exemple concret.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int a;
7      int b;
8
9      printf("Entrez deux nombres : ");
10     scanf("%d %d", &a, &b);
11
12     int min = (a < b) ? a : b;
13
14     for (int i = 2; i <= min; ++i)
```


III. Les bases du langage C

```
15     if (a % i == 0 && b % i == 0)
16     {
17         printf("Le plus petit diviseur de %d et %d est %d\n",
18             a, b, i);
19         break;
20     }
21     return 0;
22 }
```

Ce code, repris du chapitre précédent, permet de calculer le plus petit commun diviseur de deux nombres donnés. Imaginons à présent que nous souhaitions faire la même chose, mais avec deux paires de nombres. Le code ressemblerait alors à ceci.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int a;
7      int b;
8
9      printf("Entrez deux nombres : ");
10     scanf("%d %d", &a, &b);
11
12     int min = (a < b) ? a : b;
13
14     for (int i = 2; i <= min; ++i)
15         if (a % i == 0 && b % i == 0)
16         {
17             printf("Le plus petit diviseur de %d et %d est %d\n",
18                 a, b, i);
19             break;
20         }
21
22     printf("Entrez deux autres nombres : ");
23     scanf("%d %d", &a, &b);
24     min = (a < b) ? a : b;
25
26     for (int i = 2; i <= min; ++i)
27         if (a % i == 0 && b % i == 0)
28         {
29             printf("Le plus petit diviseur de %d et %d est %d\n",
30                 a, b, i);
31             break;
32         }
33
34     return 0;
35 }
```

```
33 }
```

Comme vous le voyez, ce n'est pas très pratique : nous devons recopier les instructions de calcul deux fois, ce qui est assez dommage et qui plus est source d'erreurs. C'est ici que les fonctions entrent en jeu en nous permettant par exemple de rassembler les instructions dédiées au calcul du plus petit diviseur commun en un seul point que nous solliciterons autant de fois que nécessaire.



Oui, il est aussi possible d'utiliser une boucle pour éviter la répétition, mais l'exemple aurait été moins parlant. 🍊

III.9.2. Définir et utiliser une fonction

Pour définir une fonction, nous allons devoir donner quatre informations sur celle-ci :

- son **nom** : les règles sont les mêmes que pour les variables ;
- son **corps** (son contenu) : le bloc d'instructions à exécuter ;
- son **type de retour** : le type du résultat de la fonction ;
- d'éventuels **paramètres** : des valeurs reçues par la fonction lors de l'appel.

La syntaxe est la suivante.

```
1 type nom(paramètres)
2 {
3     /* Corps de la fonction */
4 }
```

Prenons un exemple en créant une fonction qui affiche « bonjour ! » à l'écran.

```
1 #include <stdio.h>
2
3
4 void bonjour(void)
5 {
6     printf("Bonjour !\n");
7 }
8
9
10 int main(void)
11 {
12     bonjour();
13     return 0;
14 }
```

III. Les bases du langage C

Comme vous le voyez, la fonction se nomme « `bonjour` » et est composée d'un appel à `printf()`. Reste les deux mots-clés `void` :

- dans le cas du type de retour, il spécifie que la fonction ne retourne rien ;
- dans le cas des paramètres, il spécifie que la fonction n'en reçoit aucun (cela se manifeste lors de l'appel : il n'y a rien entre les parenthèses).

III.9.2.1. Le type de retour

Le type de retour permet d'indiquer deux choses : si la fonction retourne une valeur et le type de cette valeur.

```
1 #include <stdio.h>
2
3
4 int deux(void)
5 {
6     return 2;
7 }
8
9
10 int main(void)
11 {
12     printf("Retour : %d\n", deux());
13     return 0;
14 }
```

Résultat

```
1 Retour : 2
```

Dans l'exemple ci-dessus, la fonction `deux()` est définie comme retournant une valeur de type `int`. Vous retrouvez l'instruction `return`, une instruction de saut (comme `break`, `continue` et `goto`). Ce `return` arrête l'exécution de la fonction courante et provoque un retour (techniquement, un saut) vers l'appel à cette fonction qui se voit alors attribuer la valeur de retour (s'il y en a une). Autrement dit, dans notre exemple, l'instruction `return 2` stoppe l'exécution de la fonction `deux()` et ramène l'exécution du programme à l'appel qui vaut désormais 2, ce qui donne finalement `printf("Retour : %d\n", 2)`

III.9.2.2. Les paramètres

Un paramètre sert à fournir des informations à la fonction lors de son exécution. La fonction `printf()` par exemple récupère ce qu'elle doit afficher dans la console à l'aide de paramètres.

III. Les bases du langage C

Ceux-ci sont définis de la même manière que les variables si ce n'est que les définitions sont séparées par des virgules.

```
1 Retour : 2
```

i

Vous pouvez utiliser un maximum de 127 paramètres¹ (si, si 🍊), toutefois nous vous conseillons de vous limiter à *cinq* afin de conserver un code concis et lisible.

Maintenant que nous savons tout cela, nous pouvons réaliser une fonction qui calcule le plus petit commun diviseur entre deux nombres et ainsi simplifier l'exemple du dessus.

```
1 #include <stdio.h>
2
3
4 int ppcd(int a, int b)
5 {
6     int min = (a < b) ? a : b;
7
8     for (int i = 2; i <= min; ++i)
9         if (a % i == 0 && b % i == 0)
10             return i;
11
12     return 0;
13 }
14
15
16 int main(void)
17 {
18     int a;
19     int b;
20
21     printf("Entrez deux nombres : ");
22     scanf("%d %d", &a, &b);
23
24     int resultat = ppcd(a, b);
25
26     if (resultat != 0)
27         printf("Le plus petit diviseur de %d et %d est %d\n", a,
28             b, resultat);
29
30     printf("Entrez deux autres nombres : ");
31     scanf("%d %d", &a, &b);
32     resultat = ppcd(a, b);
33
34     if (resultat != 0)
```

```
34         printf("Le plus petit diviseur de %d et %d est %d\n", a,  
35                b, resultat);  
36     return 0;  
37 }
```

Plus simple et plus lisible, non ?



Remarquez la présence de deux instructions `return` dans la fonction `ppcd()`. La valeur zéro est retournée afin d'indiquer l'absence d'un diviseur commun.

III.9.2.3. Les arguments et les paramètres

À ce stade, il est important de préciser qu'un paramètre est propre à une fonction, il n'est *pas* utilisable en dehors de celle-ci. Par exemple, la variable `a` de la fonction `ppcd()` n'a aucun rapport avec la variable `a` de la fonction `main()`.

Voici un autre exemple plus explicite à ce sujet.

```
1  #include <stdio.h>  
2  
3  
4  void fonction(int nombre)  
5  {  
6      ++nombre;  
7      printf("Variable nombre dans `fonction' : %d\n", nombre);  
8  }  
9  
10  
11 int main(void)  
12 {  
13     int nombre = 5;  
14  
15     fonction(nombre);  
16     printf("Variable nombre dans `main' : %d\n", nombre);  
17     return 0;  
18 }
```

Résultat

```
1 Variable nombre dans `fonction' : 6  
2 Variable nombre dans `main' : 5
```

III. Les bases du langage C

Comme vous le voyez, les deux variables `nombre` sont bel et bien distinctes. En fait, lors d'un appel de fonction, vous spécifiez des **arguments** à la fonction appelée. Ces arguments ne sont rien d'autre que des expressions dont les résultats seront ensuite affectés aux différents **paramètres** de la fonction.



Notez bien cette différence car elle est très importante : un argument est une *expression* alors qu'un paramètre est une *variable*.

Ainsi, la valeur de la variable `nombre` de la fonction `main()` est passée en argument à la fonction `fonction()` et est ensuite affectée au paramètre `nombre`. La variable `nombre` de la fonction `main()` n'est donc en rien modifiée.

III.9.2.4. Les étiquettes et l'instruction `goto`

Nous vous avons présenté les étiquettes et l'instruction `goto` au sein du chapitre précédent. À leur sujet, notez qu'une étiquette n'est utilisable qu'au sein d'une fonction. Autrement dit, il n'est pas possible de sauter d'une fonction à une autre à l'aide de l'instruction `goto`.

```
1 int deux(void)
2 {
3     deux:
4         return 2;
5 }
6
7
8 int main(void)
9 {
10     goto deux; /* Interdit */
11     return 0;
12 }
```

III.9.3. Les prototypes

Jusqu'à présent, nous avons toujours défini notre fonction *avant* la fonction `main()`. Cela paraît de prime abord logique (nous définissons la fonction avant de l'utiliser), cependant cela est surtout indispensable. En effet, si nous déplaçons la définition après la fonction `main()`, le compilateur se retrouve dans une situation délicate : il est face à un appel de fonction dont il ne sait rien (nombre d'arguments, type des arguments et type de retour). Que faire ? Hé bien, il serait possible de stopper la compilation, mais ce n'est pas ce qui a été retenu, le compilateur va considérer que la fonction retourne une valeur de type `int` et qu'elle reçoit un nombre indéterminé d'arguments.

1. ISO/IEC 9899:201x, doc. N1570, § 5.2.4.1, Translation limits, p. 26

III. Les bases du langage C

Toutefois, si cette décision a l'avantage d'éviter un arrêt de la compilation, elle peut en revanche conduire à des problèmes lors de l'exécution si cette supposition du compilateur s'avère inadéquate. Or, il serait pratique de pouvoir définir les fonctions dans l'ordre que nous souhaitons sans se soucier de qui doit être défini avant qui.

Pour résoudre ce problème, il est possible de **déclarer** une fonction à l'aide d'un **prototype**. Celui-ci permet de spécifier le type de retour de la fonction, son nombre d'arguments et leur type, mais ne comporte pas le corps de cette fonction. La syntaxe d'un prototype est la suivante.

```
1 type nom(paramètres);
```

Ce qui donne par exemple ceci.

```
1 #include <stdio.h>
2
3 void bonjour(void);
4
5
6 int main(void)
7 {
8     bonjour();
9     return 0;
10 }
11
12
13 void bonjour(void)
14 {
15     printf("Bonjour !\n");
16 }
```



Notez bien le point-virgule à la fin du prototype qui est obligatoire.



Étant donné qu'un prototype ne comprend pas le corps de la fonction qu'il déclare, il n'est pas obligatoire de préciser le nom des paramètres de celle-ci. Ainsi, le prototype suivant est parfaitement correct.

```
1 int ppcd(int, int);
```

III.9.4. Variables globales et classes de stockage

III.9.4.1. Les variables globales

Il arrive parfois que l'utilisation de paramètres ne soit pas adaptée et que des fonctions soient amenées à travailler sur des données qui doivent leur être communes. Prenons un exemple simple : vous souhaitez compter le nombre d'appels de fonction réalisé durant l'exécution de votre programme. Ceci est impossible à réaliser, sauf à définir une variable dans la fonction `main()`, la passer en argument de chaque fonction et de faire en sorte que chaque fonction retourne sa valeur augmentée de un, ce qui est très peu pratique.

À la place, il est possible de définir une variable dite « **globale** » qui sera utilisable par toutes les fonctions. Pour définir une variable globale, il vous suffit de définir une variable *en dehors de tout bloc*, autrement dit en dehors de toute fonction.

```
1  #include <stdio.h>
2
3  void fonction(void);
4
5  int appels = 0;
6
7
8  void fonction(void)
9  {
10     ++appels;
11 }
12
13
14 int main(void)
15 {
16     fonction();
17     fonction();
18     printf("Ce programme a réalisé %d appel(s) de fonction\n",
19           appels);
19     return 0;
20 }
```

Résultat

```
1 Ce programme a réalisé 2 appel(s) de fonction
```

Comme vous le voyez, nous avons simplement placé la définition de la variable *appels* en dehors de toute fonction et *avant* toute définition de fonction de sorte qu'elle soit partagée entre elles.

i

Le terme « global » est en fait un peu trompeur étant donné que la variable n'est pas globale au programme, mais tout simplement disponible pour toutes les fonctions du fichier dans lequel elle est située. Ce terme est utilisé en opposition aux paramètres et variables des fonctions qui sont dites « **locales** ».

!

N'utilisez les variables globales que lorsque cela vous paraît *vraiment* nécessaire. Ces dernières étant utilisables dans un fichier entier (voire dans plusieurs, nous le verrons un peu plus tard), elles ont tendances à rendre la lecture du code plus difficile.

III.9.4.2. Les classes de stockage

Les variables locales et les variables globales ont une autre différence de taille : leur **classe de stockage**. La classe de stockage détermine (entre autre) la **durée de vie** d'un objet, c'est-à-dire le temps durant lequel celui-ci existera en mémoire.

III.9.4.2.1. Classe de stockage automatique

Les variables locales sont par défaut de classe de stockage **automatique**. Cela signifie qu'elles sont allouées automatiquement à chaque fois que le bloc auquel elles appartiennent est exécuté et qu'elles sont détruites une fois son exécution terminée.

```
1 int ppcd(int a, int b)
2 {
3     int min = (a < b) ? a : b;
4
5     for (int i = 2; i <= min; ++i)
6         if (a % i == 0 && b % i == 0)
7             return i;
8
9     return 0;
10 }
```

Par exemple, à chaque fois que la fonction `ppcd()` est appelée, les variables `a`, `b`, `min` sont allouées en mémoire vive et détruites à la fin de l'exécution de la fonction. La variable `i` quant à elle est allouée au début de la boucle `for` et détruite à la fin de cette dernière.

III.9.4.2.2. Classe de stockage statique

Les variables globales sont *toujours* de classe de stockage **statique**. Ceci signifie qu'elles sont allouées au début de l'exécution du programme et sont détruites à la fin de l'exécution de celui-ci. En conséquence, elles conservent leur valeur tout au long de l'exécution du programme.

III. Les bases du langage C

Également, à l'inverse des autres variables, celles-ci sont *initialisées à zéro* si elles ne font pas l'objet d'une initialisation¹. L'exemple ci-dessous est donc correct et utilise deux variables valant zéro.

```
1 #include <stdio.h>
2
3 int a;
4 double b;
5
6
7 int main(void)
8 {
9     printf("%d, %f\n", a, b);
10    return 0;
11 }
```

Résultat

1	0, 0.000000
---	-------------

Petit bémol tout de même : étant donné que ces variables sont créées au début du programme, elles ne peuvent être initialisées qu'à l'aide de *constantes*. La présence de variables au sein de l'expression d'initialisation est donc proscrite.

```
1 #include <stdio.h>
2
3 int a = 20; /* Correct */
4 double b = a; /* Incorrect */
5
6
7 int main(void)
8 {
9     printf("%d, %f\n", a, b);
10    return 0;
11 }
```

III.9.4.2.3. Modification de la classe de stockage

Il est possible de modifier la classe de stockage d'une variable automatique en précédant sa définition du mot-clé **static** afin d'en faire une variable statique.

1. Dans les cas des pointeurs, que nous verrons plus tard dans ce cours, ils sont initialisés comme pointeurs nuls.

```
1 #include <stdio.h>
2
3
4 int compteur(void)
5 {
6     static int n;
7
8     return ++n;
9 }
10
11
12 int main(void)
13 {
14     compteur();
15     printf("n = %d\n", compteur());
16     return 0;
17 }
```

Résultat

```
1 n = 2
```

III.9.5. Exercices

III.9.5.1. Afficher un rectangle

Le premier exercice que nous vous proposons consiste à afficher un rectangle dans la console. Voici ce que devra donner l'exécution de votre programme.

```
1 Donnez la longueur : 5
2 Donnez la largeur : 3
3
4 ***
5 ***
6 ***
7 ***
8 ***
```

III. Les bases du langage C

III.9.5.1.1. Correction

👁 Contenu masqué n°20

i

Vous pouvez aussi essayer d'afficher le rectangle dans l'autre sens.

III.9.5.2. Afficher un triangle

Même principe, mais cette fois-ci avec un triangle (rectangle). Le programme devra donner ceci.

```
1  Donnez un nombre : 5
2
3  *
4  **
5  ***
6  ****
7  *****
```

Bien entendu, la taille du triangle variera en fonction du nombre entré.

III.9.5.2.1. Correction

👁 Contenu masqué n°21

III.9.5.3. En petites coupures ?

Pour ce dernier exercice, vous allez devoir réaliser un programme qui reçoit en entrée une somme d'argent et donne en sortie la plus petite quantité de coupures nécessaires pour reconstituer cette somme.

Pour cet exercice, vous utiliserez les coupures suivantes :

- des billets de 100€ ;
- des billets de 50€ ;
- des billets de 20€ ;
- des billets de 10€ ;
- des billets de 5€ ;
- des pièces de 2€ ;
- des pièces de 1€ ;

III. Les bases du langage C

Ci dessous un exemple de ce que devra donner votre programme une fois terminé.

```
1 Entrez une somme : 285
2 2 billet(s) de 100.
3 1 billet(s) de 50.
4 1 billet(s) de 20.
5 1 billet(s) de 10.
6 1 billet(s) de 5.
```

III.9.5.3.1. Correction

👁 Contenu masqué n°22

Conclusion

En résumé

1. Une fonction permet de rassembler une suite d'instructions qui est amenée à être utilisée plusieurs fois ;
2. Une fonction peut retourner ou non une valeur et recevoir ou non des paramètres ;
3. Un paramètre est une variable propre à une fonction, il n'est pas utilisable en dehors de celle-ci ;
4. Un argument est une expression passée lors d'un l'appel de fonction et dont la valeur est affectée au paramètre correspondant ;
5. Un prototype permet de déclarer une fonction sans spécifier son corps, il n'est ainsi plus nécessaire de se soucier de l'ordre de définition des fonctions ;
6. Il est possible de définir une variable « globale » en plaçant sa définition en dehors de tout bloc (et donc de toute fonction) ;
7. Une variable définie au sein d'un bloc est de classe de stockage automatique, alors qu'une variable définie en dehors de tout bloc est de classe de stockage statique ;
8. Une variable de classe de stockage automatique est détruite une fois que l'exécution du bloc auquel elle appartient est terminée ;
9. Une variable de classe de stockage statique existe durant toute l'exécution du programme ;
10. Il est possible de modifier la classe de stockage d'une variable de classe de stockage automatique à l'aide du mot-clé `static` ;

Contenu masqué

Contenu masqué n°20

```
1 #include <stdio.h>
2
3 void rectangle(int, int);
4
5
6 int main(void)
7 {
8     int longueur;
9     int largeur;
10
11     printf("Donnez la longueur : ");
12     scanf("%d", &longueur);
13     printf("Donnez la largeur : ");
14     scanf("%d", &largeur);
15     printf("\n");
16     rectangle(longueur, largeur);
17     return 0;
18 }
19
20
21 void rectangle(int longueur, int largeur)
22 {
23     for (int i = 0; i < longueur; i++)
24     {
25         for (int j = 0; j < largeur; j++)
26             printf("*");
27
28         printf("\n");
29     }
30 }
```

[Retourner au texte.](#)

Contenu masqué n°21

```
1 #include <stdio.h>
2
3 void triangle(int);
4
5
```

III. Les bases du langage C

```
6 int main(void)
7 {
8     int nombre;
9
10    printf("Donnez un nombre : ");
11    scanf("%d", &nombre);
12    printf("\n");
13    triangle(nombre);
14    return 0;
15 }
16
17 void triangle(int nombre)
18 {
19     for (int i = 0; i < nombre; i++)
20     {
21         for (int j = 0; j <= i; j++)
22             printf("*");
23
24         printf("\n");
25     }
26 }
```

[Retourner au texte.](#)

Contenu masqué n°22

```
1 #include <stdio.h>
2
3
4 int coupure_inferieure(int valeur)
5 {
6     switch (valeur)
7     {
8         case 100:
9             return 50;
10
11         case 50:
12             return 20;
13
14         case 20:
15             return 10;
16
17         case 10:
18             return 5;
19
20         case 5:
21             return 2;
```

```
22
23     case 2:
24         return 1;
25
26     default:
27         return 0;
28     }
29 }
30
31
32 void coupure(int somme)
33 {
34     int valeur = 100;
35
36     while (valeur != 0)
37     {
38         int nb_coupure = somme / valeur;
39
40         if (nb_coupure > 0)
41         {
42             if (valeur >= 5)
43                 printf("%d billet(s) de %d.\n", nb_coupure,
44                     valeur);
45             else
46                 printf("%d pièce(s) de %d.\n", nb_coupure, valeur);
47
48             somme -= nb_coupure * valeur;
49         }
50
51         valeur = coupure_inferieure(valeur);
52     }
53 }
54
55 int main(void)
56 {
57     int somme;
58
59     printf("Entrez une somme : ");
60     scanf("%d", &somme);
61     coupure(somme);
62     return 0;
63 }
```

[Retourner au texte.](#)

III.10. TP : une calculatrice basique

Introduction

Après tout ce que vous venez de découvrir, il est temps de faire une petite pause et de mettre en pratique vos nouveaux acquis. Pour ce faire, rien de tel qu'un exercice récapitulatif : réaliser une calculatrice basique.

III.10.1. Objectif

Votre objectif sera de réaliser une calculatrice basique pouvant calculer une somme, une soustraction, une multiplication, une division, le reste d'une division entière, une puissance, une factorielle, le PGCD et le PPCD.

Celle-ci attendra une entrée formatée suivant la [notation polonaise inverse](#) [↗](#). Autrement dit, les opérandes d'une opération seront entrés *avant* l'opérateur, par exemple comme ceci pour la somme de quatre et cinq : `4 5 +`.

Elle devra également retenir le résultat de l'opération précédente et déduire l'utilisation de celui-ci en cas d'omission d'un opérande. Plus précisément, si l'utilisateur entre par exemple `5 +`, vous devrez déduire que le premier opérande de la somme est le résultat de l'opération précédente (ou zéro s'il n'y en a pas encore eu).

Chaque opération sera identifiée par un symbole ou une lettre, comme suit :

- addition : `+` ;
- soustraction : `-` ;
- multiplication : `*` ;
- division : `/` ;
- reste de la division entière : `%` ;
- puissance : `^` ;
- factorielle : `!` ;
- PGCD : `g` ;
- PPCD : `p`.

Le programme doit s'arrêter lorsque la lettre « q » est spécifiée comme opération (avec ou sans opérande).

III.10.2. Préparation

III.10.2.1. Précisions concernant scanf



Pourquoi utiliser la notation polonaise inverse et non l'écriture habituelle ?

Parce qu'elle va vous permettre de bénéficier d'une caractéristique intéressante de la fonction `scanf()` : sa valeur de retour. Nous anticipons un peu sur les chapitres suivants, mais sachez que la fonction `scanf()` retourne une valeur entière qui correspond au nombre de conversions réussies. Une conversion est réussie si ce qu'entre l'utilisateur correspond à l'indicateur de conversion.

Ainsi, si nous souhaitons récupérer un entier à l'aide de l'indicateur `d`, la conversion sera réussie si l'utilisateur entre un nombre (par exemple 2) alors qu'elle échouera s'il entre une lettre ou un signe de ponctuation.

Grâce à cela, vous pourrez détecter facilement s'il manque ou non un opérande pour une opération.



Lorsqu'une conversion échoue, la fonction `scanf()` arrête son exécution. Aussi, s'il y avait d'autres conversions à effectuer après celle qui a avorté, elles ne seront pas réalisées.

```
1 double a;  
2 double b;  
3 char op;  
4  
5 scanf("%lf %lf %c", &a, &b, &op);
```

Dans le code ci-dessus, si l'utilisateur entre `7 *`, la fonction `scanf()` retournera 1 et n'aura lu que le nombre 7. Il sera nécessaire de l'appeler une seconde fois pour que le symbole `*` soit récupéré.



Petit bémol tout de même : les symboles `+` et `-` sont considérés comme des débuts de nombre valables (puisque vous pouvez par exemple entrer -2). Dès lors, si vous souhaitez additionner ou soustraire un nombre au résultat de l'opération précédente, vous devrez doubler ce symbole. Pour ajouter cinq cela donnera donc : `5 ++`.

III.10.2.2. Les puissances

Pour élever un nombre à une puissance donnée (autrement dit, pour calculer x^y), nous allons avoir besoin d'une nouvelle partie de la bibliothèque standard dédiée aux fonctions mathématiques

III. Les bases du langage C

de base. Le fichier d'en-tête de la bibliothèque mathématique se nomme `<math.h>` et contient, entre autres, la déclaration de la fonction `pow()`.

```
1 double pow(double x, double y);
```

Cette dernière prend deux arguments : la base et l'exposant.



L'utilisation de la bibliothèque mathématique requiert d'ajouter l'option `-lm` lors de la compilation, comme ceci : `gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin main.c -lm` (faites bien en sorte de placer `-lm` après le ou les fichiers sources).

III.10.2.3. La factorielle

La factorielle d'un nombre est égale au produit des nombres entiers *positifs et non nuls* inférieurs ou égaux à ce nombre. La factorielle de quatre équivaut donc à $1 * 2 * 3 * 4$, donc vingt-quatre. Cette fonction n'est pas fournie par la bibliothèque standard, il vous faudra donc la programmer (pareil pour le PGCD et le PPCD).



Par convention, la factorielle de zéro est égale à un.

III.10.2.4. Le PGCD

Le **plus grand commun diviseur de deux entiers** [↗](#) (abrégé PGCD) est, parmi les diviseurs communs à ces entiers, le plus grand d'entre eux. Par exemple, le PGCD de 60 et 18 est 6.



Par convention, le PGCD de 0 et 0 est 0 et le PGCD d'un entier non nul et de zéro est cet entier non nul.

III.10.2.5. Le PPCD

Le plus petit commun dénominateur (ou le **plus petit commun multiple** [↗](#)), abrégé PPCD, de deux entiers est le plus petit entier strictement positif qui soit multiple de ces deux nombres. Par exemple, le PPCD de 2 et 3 est 6.



Par convention, si l'un des deux entiers (ou les deux) sont nuls, le résultat est zéro.

III.10.2.6. Exemple d'utilisation

```
1 > 5 6 +
2 11.000000
3 > 4 *
4 44.000000
5 > 2 /
6 22.000000
7 > 5 2 %
8 1.000000
9 > 2 5 ^
10 32.000000
11 > 1 ++
12 33.000000
13 > 5 !
14 120.000000
```

III.10.2.7. Derniers conseils

Nous vous conseillons de récupérer les nombres sous forme de **double**. Cependant, gardez bien à l'esprit que certaines opérations ne peuvent s'appliquer qu'à des entiers : le reste de la division entière, la factorielle, le PGCD et le PPCD. Il vous sera donc nécessaire d'effectuer des conversions.

Également, notez bien que la factorielle ne s'applique qu'à *un seul* opérande à l'inverse de toutes les autres opérations.

Bien, vous avez à présent toutes les cartes en main : au travail ! 🍊

III.10.3. Correction

Alors ? Pas trop secoué ? 🍊

Bien, voyons à présent la correction.

```
1 #include <math.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 long long pgcd(long long, long long);
6 long long ppcd(long long, long long);
7 unsigned long long factorielle(unsigned long long);
8
9
10 long long pgcd(long long a, long long b)
```

```
11 {
12     if (b == 0)
13         return a;
14
15     long long r = a % b;
16
17     while (r != 0)
18     {
19         a = b;
20         b = r;
21         r = a % b;
22     }
23
24     return b;
25 }
26
27
28 long long ppcd(long long a, long long b)
29 {
30     if (a == 0 || b == 0)
31         return 0;
32
33     long long max = (a > b) ? a : b;
34     long long i = max;
35
36     while (i % a != 0 || i % b != 0)
37         ++i;
38
39     return i;
40 }
41
42
43 unsigned long long factorielle(unsigned long long a)
44 {
45     unsigned long long r = 1;
46
47     for (unsigned long long i = 2; i <= a; ++i)
48         r *= i;
49
50     return r;
51 }
52
53
54 int
55 main(void)
56 {
57     double res = 0.;
58
59     while (1)
60     {
```

```
61     double a;
62     double b;
63     char op;
64
65     printf("> ");
66     int n = scanf("%lf %lf %c", &a, &b, &op);
67
68     if (n <= 1)
69     {
70         scanf("%c", &op);
71         b = a;
72         a = res;
73     }
74     if (op == 'q')
75         break;
76
77     switch (op)
78     {
79     case '+':
80         res = a + b;
81         break;
82
83     case '-':
84         res = a - b;
85         break;
86
87     case '*':
88         res = a * b;
89         break;
90
91     case '/':
92         res = a / b;
93         break;
94
95     case '%':
96         res = (long long)a % (long long)b;
97         break;
98
99     case '^':
100        res = pow(a, b);
101        break;
102
103     case '!':
104        res = factorielle((n == 0) ? a : b);
105        break;
106
107     case 'g':
108        res = pgcd(a, b);
109        break;
110
```

```
111         case 'p':
112             res = ppcd(a, b);
113             break;
114         }
115
116         printf("%lf\n", res);
117     }
118     return 0;
119 }
```

Commençons par la fonction `main()`. Nous définissons plusieurs variables :

- `res`, qui correspond au résultat de la dernière opération réalisée (ou zéro s'il n'y en a pas encore eu) ;
- `a` et `b`, qui représentent les éventuels opérandes fournis ;
- `op`, qui retient l'opération demandée ; et
- `n`, qui est utilisée pour retenir le retour de la fonction `scanf()`.

Ensuite, nous entrons dans une boucle infinie (la condition étant toujours vraie puisque valant un) où nous demandons à l'utilisateur d'entrer l'opération à réaliser et les éventuels opérandes. Nous vérifions ensuite si un seul opérande est fourni ou aucun (ce qui se déduit, respectivement, d'un retour de la fonction `scanf()` valant un ou zéro). Si c'est le cas, nous appelons une seconde fois `scanf()` pour récupérer l'opérateur. Puis, la valeur de `a` est attribuée à `b` et la valeur de `res` à `a`.

Si l'opérateur utilisé est `q`, alors nous quittons la boucle et par la même occasion le programme. Notez que nous n'avons pas pu effectuer cette vérification dans le corps de l'instruction `switch` qui suit puisque l'instruction `break` nous aurait fait quitter celui-ci et non la boucle.

Enfin, nous réalisons l'opération demandée au sein de l'instruction `switch`, nous stockons le résultat dans la variable `res` et l'affichons. Remarquez que l'utilisation de conversions explicites n'a été nécessaire que pour le calcul du reste de la division entière. En effet, dans les autres cas (par exemple lors de l'affectation à la variable `res`), il y a des conversions implicites.



Nous avons utilisé le type `long long` lors des calculs nécessitant des nombres entiers afin de disposer de la plus grande capacité possible. Par ailleurs, nous avons employé le type `unsigned long long` pour la fonction factorielle puisque celle-ci n'opère que sur des nombres strictement positifs.

Conclusion

Ce chapitre nous aura permis de revoir la plupart des notions des chapitres précédents.

III.11. Découper son projet

Introduction

Ce chapitre est la suite directe de celui consacré aux fonctions : nous allons voir comment découper nos projets en plusieurs fichiers. En effet, même si l'on découpe bien son projet en fonctions, ce dernier est difficile à relire si tout est contenu dans le même fichier. Ce chapitre a donc pour but de vous apprendre à découper vos projets efficacement.

III.11.1. Portée et masquage

III.11.1.1. La notion de portée

Avant de voir comment diviser nos programmes en plusieurs fichiers, il est nécessaire de vous présenter une notion importante, celle de **portée**. La portée d'une variable est la partie du programme où cette dernière est utilisable. Il existe plusieurs types de portées, cependant nous n'en verrons que deux¹ :

- au niveau d'un bloc ;
- au niveau d'un fichier.

III.11.1.1.1. Au niveau d'un bloc

Une portée au niveau d'un bloc signifie qu'une variable est utilisable, visible de sa déclaration jusqu'à la fin du bloc dans lequel elle est déclarée. Illustration.

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      {
6          int nombre = 3;
7          printf("%d\n", nombre);
8      }
9
10     /* Incorrect ! */
```

1. Il existe en vérité 4 types de portées, si vous souhaitez approfondir le sujet, voyez le cours [les identificateurs en langage C](#) ↗.

III. Les bases du langage C

```
11     printf("%d\n", nombre);
12     return 0;
13 }
```

Dans ce code, la variable `nombre` est déclarée dans un sous-bloc. Sa portée est donc limitée à ce dernier et elle ne peut pas être utilisée en dehors.

III.11.1.1.2. Au niveau d'un fichier

Une portée au niveau d'un fichier signifie qu'une variable est utilisable, visible, de sa déclaration jusqu'à la fin du fichier dans lequel elle est déclarée. En fait, il s'agit de la portée des variables « globales » dont nous avons parlé dans le chapitre sur les fonctions.

```
1  #include <stdio.h>
2
3  int nombre = 3;
4
5  int triple(void)
6  {
7      return nombre * 3;
8  }
9
10 int main(void)
11 {
12     nombre = triple();
13     printf("%d\n", nombre);
14     return 0;
15 }
```

Dans ce code, la variable `nombre` a une portée au niveau du fichier et peut par conséquent être aussi bien utilisée dans la fonction `triple()` que dans la fonction `main()`.

III.11.1.2. La notion de masquage

En voyant les deux types de portées, vous vous êtes peut-être posé la question suivante : que se passe-t-il s'il existe plusieurs variables de même nom ? Hé bien, cela dépend de la portée de ces dernières. Si elles ont la même portée comme dans l'exemple ci-dessous, alors le compilateur sera incapable de déterminer à quelle variable ou à quelle fonction le nom fait référence et, dès lors, retournera une erreur.

```
1  int main(void)
2  {
3      int nombre = 10;
4      int nombre = 20;
```

III. Les bases du langage C

```
5  
6     return 0;  
7 }
```

En revanche, si elles ont des portées différentes, alors celle ayant la portée la plus petite sera privilégiée, on dit qu'elle **masque** celle · s de portée · s plus grande · s. Autrement dit, dans l'exemple qui suit, c'est la variable du bloc de la fonction `main()` qui sera affichée.

```
1 #include <stdio.h>  
2  
3 int nombre = 10;  
4  
5 int main(void)  
6 {  
7     int nombre = 20;  
8  
9     printf("%d\n", nombre);  
10    return 0;  
11 }
```

Notez que nous disons : « celle · s de portée plus petite », car les variables déclarées dans un sous-bloc ont une portée plus limitée (car confinée au sous-bloc) que celle déclarée dans un bloc supérieur. Ainsi, le code ci-dessous est parfaitement valide et affichera 30.

```
1 #include <stdio.h>  
2  
3 int nombre = 10;  
4  
5 int main(void)  
6 {  
7     int nombre = 20;  
8  
9     if (nombre == 20)  
10    {  
11        int nombre = 30;  
12  
13        printf("%d\n", nombre);  
14    }  
15  
16    return 0;  
17 }
```

III.11.2. Partager des fonctions et variables

Voyons à présent comment répartir nos fonctions et variables entre différents fichiers.

III.11.2.1. Les fonctions

Dans l'extrait précédent, nous avons, entre autres, créé une fonction `triple()` que nous avons placée dans le même fichier que la fonction `main()`. Essayons à présent de les répartir dans deux fichiers distincts. Pour ce faire, il vous suffit de créer un second fichier avec l'extension « `.c` ». Dans notre cas, il s'agira de « `main.c` » et de « `autre.c` ».

autre.c

```
1 int triple(int nombre)
2 {
3     return nombre * 3;
4 }
```

main.c

```
1 int main(void)
2 {
3     int nombre = triple(3);
4     return 0;
5 }
```

La compilation se réalise de la même manière qu'auparavant, si ce n'est qu'il vous est nécessaire de spécifier les deux noms de fichier.

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin
   main.c autre.c
```

À noter que vous pouvez également utiliser une forme raccourcie où `*.c` correspond à tous les fichiers portant l'extension « `.c` » du dossier courant.

```
1 gcc -Wall -Wextra -pedantic -std=c11 -fno-common -fno-builtin *.c
```

Si vous testez ce code, vous aurez droit à un bel avertissement de votre compilateur du type « *implicit declaration of function 'triple'* ». Quel est le problème ? Le problème est que la fonction

III. Les bases du langage C

`triple()` n'est pas déclarée dans le fichier `main.c` et que le compilateur ne la connaît donc pas lorsqu'il compile le fichier. Pour corriger cette situation, nous devons déclarer la fonction en signalant au compilateur que cette dernière se situe dans un autre fichier. Pour ce faire, nous allons inclure le prototype de la fonction `triple()` dans le fichier `main.c` en le précédant du mot-clé `extern`, qui peut être compris comme « la fonction se trouve dans ce fichier *ou* dans un autre » (le « ou dans un autre » explique l'emploi du terme « externe »).

autre.c

```
1 int triple(int nombre)
2 {
3     return nombre * 3;
4 }
```

main.c

```
1 extern int triple(int nombre);
2
3 int main(void)
4 {
5     int nombre = triple(3);
6
7     return 0;
8 }
```

En termes techniques, on dit que la fonction `triple()` est **définie** dans le fichier « autre.c » (car c'est là que se situe le corps de la fonction) et qu'elle est **déclarée** dans le fichier « main.c ». Sachez qu'une fonction ne peut être définie qu'*une seule et unique fois*.

Pour information, notez que le mot-clé `extern` est facultatif devant un prototype (il est implicitement inséré par le compilateur). Nous vous conseillons cependant de l'utiliser par souci de clarté et de symétrie avec les déclarations de variables (voyez ci-dessous).

III.11.2.2. Les variables

La même méthode peut être appliquée aux variables, mais *uniquement à celle ayant une portée au niveau d'un fichier*. Également, à l'inverse des fonctions, il est plus difficile de distinguer une définition d'une déclaration de variable (elles n'ont pas de corps comme les fonctions). La règle pour les différencier est qu'une déclaration sera précédée du mot-clé `extern` alors que la définition non¹. C'est à vous de voir dans quel fichier vous souhaitez définir la variable, mais elle ne peut être définie qu'*une seule et unique fois*. Enfin, sachez que seule la définition peut

III. Les bases du langage C

comporter une initialisation. Ainsi, cet exemple est tout à fait valide.

autre.c

```
1 int nombre = 10; /* Une définition */
2 extern int autre; /* Une déclaration */
```

main.c

```
1 extern int nombre; /* Une déclaration */
2 int autre = 10; /* Une définition */
```

Alors que celui-ci, non.

autre.c

```
1 int nombre = 10; /* Il existe une autre définition */
2 extern int autre = 10; /* Une déclaration ne peut pas
    comprendre une initialisation */
```

main.c

```
1 int nombre = 20; /* Il existe une autre définition */
2 int autre = 10; /* Une définition */
```

i

Notez qu'une déclaration de fonction ou de variable peut parfaitement être insérée au sein d'un bloc, auquel cas elle aura une portée au niveau d'un bloc.

1. La règle est en vérité un peu plus complexe que cela, voyez le cours sur [les identificateurs en langage C](#) [↗](#) si vous souhaitez davantage de détails.



```
1 int
2 main(void)
3 {
4     extern int nombre;
5
6     {
7         extern int triple(int nombre);
8         printf("%d\n", triple(nombre));
9     }
10
11     printf("%d\n", triple(nombre)); /* triple() n'existe plus.
12                                     */
13     return 0;
14 }
```

III.11.3. Fonctions et variables exclusives

Dans la section précédente, nous vous avons dit qu'il n'était possible de définir une variable ou une fonction qu'une seule fois. Toutefois, ce n'est pas tout à fait vrai... Il est possible de rendre une variable (ayant une portée au niveau d'un fichier) ou une fonction exclusive à un fichier donné (on peut aussi dire « locale » à un fichier).

Une variable ou une fonction exclusive à un fichier est propre à ce dernier et ne peut être utilisée qu'au sein de celui-ci. Dans un tel cas, l'existence de plusieurs définitions ne pose pas de problèmes puisque soit chacune d'entre elles est confinée à un fichier, soit l'une d'entre elle est partagée et les autres sont exclusives.

III.11.3.1. Les fonctions

Afin de rendre une fonction exclusive à un fichier, il est nécessaire de précéder sa définition du mot-clé `static`.

main.c

```
1 #include <stdio.h>
2
3 extern void dis_bonjour(void);
4
5 static void bonjour(void)
6 {
7     printf("Bonjour depuis main.c.\n");
8 }
9
10 int main(void)
11 {
12     bonjour();
13     dis_bonjour();
14     return 0;
15 }
```

bonjour.c

```
1 #include <stdio.h>
2
3 static void bonjour(void)
4 {
5     printf("Bonjour depuis bonjour.c.\n");
6 }
7
8 void dis_bonjour(void)
9 {
10     bonjour();
11 }
```

Résultat

```
1 Bonjour depuis main.c.
2 Bonjour depuis bonjour.c.
```

Dans l'exemple ci-dessus, nous avons défini deux fois la fonction `bonjour()`, en précédant chaque définition du mot-clé `static`, rendant ces définitions propres, respectivement, au fichier `main.c` et au fichier `bonjour.c`.

III. Les bases du langage C

De même, il est possible d'avoir une définition partagée et une ou plusieurs définitions exclusives. Dans l'exemple ci-dessous, il existe deux définitions de la fonction `bonjour()` : une propre au fichier `main.c` et une partagée dont la définition est dans le fichier `bonjour.c` et est utilisée dans le fichier `dis_bonjour.c`.

main.c

```
1 #include <stdio.h>
2
3 extern void dis_bonjour(void);
4
5 static void bonjour(void)
6 {
7     printf("Bonjour depuis main.c.\n");
8 }
9
10 int main(void)
11 {
12     bonjour();
13     dis_bonjour();
14     return 0;
15 }
```

bonjour.c

```
1 #include <stdio.h>
2
3 void bonjour(void)
4 {
5     printf("Bonjour depuis bonjour.c.\n");
6 }
```

dis_bonjour.c

```
1 extern void bonjour(void);
2
3 void dis_bonjour(void)
4 {
5     bonjour();
6 }
```


Résultat

```
1 Bonjour depuis main.c.  
2 Bonjour depuis bonjour.c.
```

III.11.3.2. Les variables

Comme pour les fonctions, une variable ayant une portée au niveau d'un fichier peut être rendue exclusive à un fichier en précédant sa définition du mot-clé `static`.



Distinguez bien les deux utilisations différentes du mot-clé `static` :

- Si la variable a une portée au niveau d'un bloc, le mot-clé `static` *modifie sa classe de stockage* pour la rendre statique (sa durée de vie s'étend à la durée d'exécution du programme) ;
- Si la variable a une portée au niveau d'un fichier, le mot-clé `static` *rend la variable exclusive au fichier*.

main.c

```
1 #include <stdio.h>  
2  
3 extern void affiche_nombre(void);  
4 static int nombre = 20;  
5  
6 int main(void)  
7 {  
8     printf("nombre dans main.c = %d\n", nombre);  
9     affiche_nombre();  
10    return 0;  
11 }
```

III. Les bases du langage C

nombre.c

```
1 #include <stdio.h>
2
3 static int nombre = 10;
4
5 void affiche_nombre(void)
6 {
7     printf("nombre dans nombre.c = %d\n", nombre);
8 }
```

Résultat

```
1 nombre dans main.c = 20
2 nombre dans nombre.c = 10
```

De même, comme pour les fonctions, une définition partagée peut coexister avec une ou plusieurs définitions exclusives.

main.c

```
1 #include <stdio.h>
2
3 extern void affiche_nombre(void);
4 static int nombre = 20;
5
6 int main(void)
7 {
8     printf("nombre dans main.c = %d\n", nombre);
9     affiche_nombre();
10    return 0;
11 }
```

nombre.c

```
1 int nombre = 10;
```

affiche_nombre.c

```
1 #include <stdio.h>
2
3 extern int nombre;
4
5 void affiche_nombre(void)
6 {
7     printf("nombre dans affiche_nombre.c = %d\n", nombre);
8 }
```

Résultat

```
1 nombre dans main.c = 20
2 nombre dans affiche_nombre.c = 10
```

i

Notez que si vous essayez d'utiliser une fonction ou une variable exclusive dans un autre fichier, vous obtiendrez, assez logiquement, une erreur lors de la compilation du type « *undefined reference to [...]* ».

III.11.4. Les fichiers d'en-têtes

Pour terminer ce chapitre, il ne nous reste plus qu'à voir les fichiers d'en-têtes.

Jusqu'à présent, lorsque vous voulez utiliser une fonction ou une variable définie dans un autre fichier, vous insérez sa déclaration dans le fichier ciblé. Seulement voilà, si vous utilisez dix fichiers et que vous décidez un jour d'ajouter ou de supprimer une fonction ou une variable ou encore de modifier une déclaration, vous vous retrouvez Gros-Jean comme devant et vous êtes bon pour modifier les dix fichiers, ce qui n'est pas très pratique...

Pour résoudre ce problème, on utilise des fichiers d'en-têtes (d'extension « .h »). Ces derniers contiennent conventionnellement des déclarations de fonctions et de variables et sont inclus via la directive `#include` dans les fichiers qui utilisent les fonctions et variables en question.

i

Les fichiers d'en-têtes n'ont pas besoin d'être spécifiés lors de la compilation, ils seront automatiquement inclus.

La structure d'un fichier d'en-tête est généralement de la forme suivante.

III. Les bases du langage C

```
1 #ifndef CONSTANCE_H
2 #define CONSTANCE_H
3
4 /* Les déclarations */
5
6 #endif
```

Les directives du préprocesseur sont là pour éviter les inclusions multiples ou en cascades. Cela arrive typiquement si vous incluez un en-tête A, qui inclue un en-tête B et qui à son tour inclue l'en-tête A. Vous voyez qu'une boucle se forme (A inclue B, B inclue A, A inclue B, etc.). C'est ce type de situation que ces directives évitent. Aussi, prenez soin de les inclure dans chacun de vos fichiers d'en-têtes.

Vous pouvez remplacer **CONSTANTE** par ce que vous voulez, le plus simple et le plus fréquent étant le nom de votre fichier, par exemple **AUTRE_H** si votre fichier se nomme « autre.h ». Voici un exemple d'utilisation de fichiers d'en-têtes.

autre.h

```
1 #ifndef AUTRE_H
2 #define AUTRE_H
3
4 extern int triple(int nombre);
5 extern int variable;
6
7 #endif
```

autre.c

```
1 #include "autre.h"
2
3 int variable = 42;
4
5
6 int triple(int nombre)
7 {
8     return nombre * 3;
9 }
```

main.c

```
1 #include <stdio.h>
2
3 #include "autre.h"
4
5
6 int main(void)
7 {
8     printf("%d\n", triple(variable));
9     return 0;
10 }
```

Plusieurs remarques à propos de ce code :

- dans la directive d’inclusion, les fichiers d’en-têtes sont entre guillemets et non entre chevrons comme les fichiers d’en-têtes de la bibliothèque standard ;
- les fichiers sources et d’en-têtes correspondants portent le même nom ;
- nous vous conseillons d’inclure le fichier d’en-tête dans le fichier source correspondant (dans mon cas « autre.h » dans « autre.c ») afin d’éviter des problèmes de portée.

Conclusion

En résumé

1. Une variable avec une portée au niveau d’un bloc est utilisable de sa déclaration jusqu’à la fin du bloc dans lequel elle est déclarée ;
2. Une variable avec une portée au niveau d’un fichier est utilisable de sa déclaration jusqu’à la fin du fichier dans lequel elle est déclarée ;
3. Dans le cas où deux variables ont le même nom, mais des portées différentes, celle ayant la portée la plus petite *masque* celle ayant la portée la plus grande ;
4. Il est possible de répartir les définitions de fonction et de variable (ayant une portée au niveau d’un fichier) entre différents fichiers ;
5. Pour utiliser une fonction ou une variable (ayant une portée au niveau d’un fichier) définie dans un autre fichier, il est nécessaire d’insérer une déclaration dans le fichier où elle doit être utilisée ;
6. Une fonction ou une variable (ayant une portée au niveau d’un fichier) dont la définition est précédée du mot-clé **static** ne peut être utilisée que dans le fichier où elle est définie ;
7. Les fichiers d’en-tête peuvent être utilisés pour inclure facilement les déclarations nécessaires à un ou plusieurs fichiers source.

III.12. La gestion d'erreurs (1)

Introduction

Dans les chapitres précédents, nous vous avons présenté des exemples simplifiés afin de vous familiariser avec le langage. Aussi, nous avons pris soin de ne pas effectuer de vérifications quant à d'éventuelles rencontres d'erreurs.

Mais à présent, il nous est possible de réaliser ces vérifications. Vous disposez désormais d'un bagage suffisant pour affronter la dure réalité d'un programmeur : des fois, il y a des dysfonctionnements et il est nécessaire de les prévoir. Nous allons voir comment dans ce chapitre.

III.12.1. Détection d'erreurs

La première chose à faire pour gérer d'éventuelles erreurs lors de l'exécution, c'est avant tout de les détecter. Par exemple, quand vous exécutez une fonction et qu'une erreur a lieu lors de son exécution, celle-ci doit vous prévenir d'une manière ou d'une autre. Et elle peut le faire de deux manières différentes.

III.12.1.1. Valeurs de retour

Nous l'avons vu dans les chapitres précédents : certaines fonctions, comme `scanf()`, retournent un nombre (souvent un entier) alors qu'elles ne calculent pas un résultat comme la fonction `pow()` par exemple. Vous savez dans le cas de `scanf()` que cette valeur représente le nombre de conversions réussies, cependant cela va plus loin que cela : cette valeur vous signifie si l'exécution de la fonction s'est bien déroulée.

III.12.1.1.1. Scanf

En fait, la fonction `scanf()` retourne le nombre de conversions réussies ou un nombre inférieur si elles n'ont pas toutes été réalisées ou, enfin, *un nombre négatif en cas d'erreur*.

Ainsi, si nous souhaitons récupérer deux entiers et être certains que `scanf()` les a récupérés, nous pouvons utiliser le code suivant.

III. Les bases du langage C

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int x;
7     int y;
8
9     printf("Entrez deux nombres : ");
10
11     if (scanf("%d %d", &x, &y) == 2)
12         printf("Vous avez entre : %d et %d\n", x, y);
13
14     return 0;
15 }
```

Résultat

```
1 Entrez deux nombres : 1 2
2 Vous avez entre : 1 et 2
3
4 Entrez deux nombres : 1 a
```

Comme vous pouvez le constater, le programme n'exécute pas l'affichage des nombres dans le dernier cas, car `scanf()` n'a pas réussi à réaliser deux conversions.

III.12.1.1.2. Main

Maintenant que vous savez cela, regardez bien votre fonction `main()`.

```
1 int main(void)
2 {
3     return 0;
4 }
```

Vous ne voyez rien qui vous interpelle ? 🍊

Oui, vous avez bien vu, elle retourne un entier qui, comme pour `scanf()`, sert à indiquer la présence d'erreur. En fait, il y a deux valeurs possibles :

- `EXIT_SUCCESS` (ou zéro, cela revient au même), qui indique que tout s'est bien passé ; et
- `EXIT_FAILURE`, qui indique un échec du programme.

Ces deux constantes sont définies dans l'en-tête `<stdlib.h>`.

III.12.1.1.3. Les autres fonctions

Sachez que `scanf()`, `printf()` et `main()` ne sont pas les seules fonctions qui retournent des entiers, en fait quasiment toutes les fonctions de la bibliothèque standard le font.

?

Ok, mais je fais comment pour savoir ce que retourne une fonction ?

À l'aide de la documentation. Vous disposez de [la norme](#) (enfin, du brouillon de celle-ci) qui reste la référence ultime, sinon vous pouvez également utiliser un moteur de recherche avec la requête `man nom_de_fonction` afin d'obtenir les informations dont vous avez besoin.

i

Si vous êtes anglophobe, une traduction française de diverses descriptions est disponible à [cette adresse](#), vous les trouverez à la section trois.

III.12.1.2. Variable globale `errno`

Le retour des fonctions est un vecteur très pratique pour signaler une erreur. Cependant, il n'est pas toujours utilisable. En effet, nous avons vu lors du second TP la fonction mathématique `pow()`. Or, cette dernière utilise *déjà* son retour pour transmettre le résultat d'une opération. Comment faire dès lors pour signaler un problème ?

Une première idée serait d'utiliser une valeur particulière, comme zéro par exemple. Toutefois, ce n'est pas satisfaisant puisque, dans le cas de la fonction `pow()`, elle peut parfaitement retourner zéro lors d'un fonctionnement normal. Que faire alors ?

Dans une telle situation, il ne reste qu'une seule solution : utiliser un autre canal, en l'occurrence une variable globale. La bibliothèque standard fournit une variable globale nommée `errno` (elle est déclarée dans l'en-tête `<errno.h>`) qui permet à différentes fonctions d'indiquer une erreur en modifiant la valeur de celle-ci.

i

Une valeur de zéro indique qu'aucune erreur n'est survenue.

Les fonctions mathématiques recourent abondamment à cette fonction. Prenons l'exemple suivant.

```
1 #include <errno.h>
2 #include <math.h>
3 #include <stdio.h>
4
5
6 int main(void)
7 {
8     errno = 0;
```



```
9     double x = pow(-1, 0.5);
10
11     if (errno == 0)
12         printf("x = %f\n", x);
13
14     return 0;
15 }
```

L'appel revient à demander le résultat de l'expression $-1^{\frac{1}{2}}$, autrement dit, de cette expression : $\sqrt{-1}$, ce qui est impossible dans l'ensemble des réels. Aussi, la fonction `pow()` modifie la variable `errno` pour vous signifier qu'elle n'a pas pu calculer l'expression demandée.

Une petite précision concernant ce code et la variable `errno` : celle-ci doit *toujours* être mise à zéro *avant* d'appeler une fonction qui est susceptible de la modifier, ceci afin de vous assurer qu'elle ne contient pas la valeur qu'une autre fonction lui a assignée auparavant. Imaginez que vous ayez précédemment appelé la fonction `pow()` et que cette dernière a échoué, si vous l'appellez à nouveau, la valeur de `errno` sera toujours celle assignée lors de l'appel précédent.



Notez que la norme ne prévoit en fait que trois valeurs d'erreurs possibles pour `errno` : `EDOM` (pour le cas où le résultat d'une fonction mathématique est impossible), `ERANGE` (en cas de dépassement de capacité, nous y reviendrons plus tard) et `EILSEQ` (pour les erreurs de conversions, nous en reparlerons plus tard également). Ces trois constantes sont définies dans l'en-tête `<errno.h>`. Toutefois, il est fort probable que votre système supporte et définisse davantage de valeurs.

III.12.2. Prévenir l'utilisateur

Savoir qu'une erreur s'est produite, c'est bien, le signaler à l'utilisateur, c'est mieux. Ne laissez pas votre utilisateur dans le vide, s'il se passe quelque chose, dites le lui.

```
1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int x;
7      int y;
8
9      printf("Entrez deux nombres : ");
10
11     if (scanf("%d %d", &x, &y) == 2)
12         printf("Vous avez entré : %d et %d\n", x, y);
13     else
14         printf("Vous devez saisir deux nombres !\n");
```

```
15  
16     return 0;  
17 }
```

Résultat

```
1 Entrez deux nombres : a b  
2 Vous devez saisir deux nombres !  
3  
4 Entrez deux nombres : 1 2  
5 Vous avez entre : 1 et 2
```

Simple, mais tellement plus agréable. 🍊

III.12.3. Un exemple d'utilisation des valeurs de retour

Maintenant que vous savez tout cela, il vous est possible de modifier le code utilisant la fonction `scanf()` pour vérifier si celle-ci a réussi et, si ce n'est pas le cas, préciser à l'utilisateur qu'une erreur est survenue *et* quitter la fonction `main()` en retournant la valeur `EXIT_FAILURE`.

```
1 #include <stdio.h>  
2 #include <stdlib.h>  
3  
4  
5 int main(void)  
6 {  
7     int x;  
8     int y;  
9  
10    printf("Entrez deux nombres : ");  
11  
12    if (scanf("%d %d", &x, &y) != 2)  
13    {  
14        printf("Vous devez saisir deux nombres !\n");  
15        return EXIT_FAILURE;  
16    }  
17  
18    printf("Vous avez entré : %d et %d\n", x, y);  
19    return 0;  
20 }
```

Ceci nous permet de réduire un peu la taille de notre code en éliminant directement les cas d'erreurs.

Conclusion

Bien, vous voilà à présent fin prêt pour la suite du cours et ses *vrais* exemples. Plus de pitié donc : gare à vos fesses si vous ne vérifiez pas le comportement des fonctions que vous appelez !



En résumé

1. Certaines fonctions comme `scanf()` ou `main()` utilisent leur valeur de retour pour signaler la survenance d'erreurs ;
2. Dans le cas où la valeur de retour ne peut pas être utilisée (comme pour la fonction mathématique `pow()`), les fonctions de la bibliothèque standard utilisent la variable globale `errno` ;
3. Tâchez de prévenir l'utilisateur à l'aide d'un message explicite dans le cas où vous détectez une erreur.

Quatrième partie

Agrégats, mémoire et fichiers

IV.1. Les pointeurs

Introduction

Dans ce chapitre, nous allons aborder une notion centrale du langage C : les pointeurs.

Les pointeurs constituent ce qui est appelé une **fonctionnalité bas niveau**, c'est-à-dire un mécanisme qui nécessite de connaître quelques détails sur le fonctionnement d'un ordinateur pour être compris et utilisé correctement. Dans le cas des pointeurs, il s'agira surtout de disposer de quelques informations sur la mémoire vive.

IV.1.1. Présentation

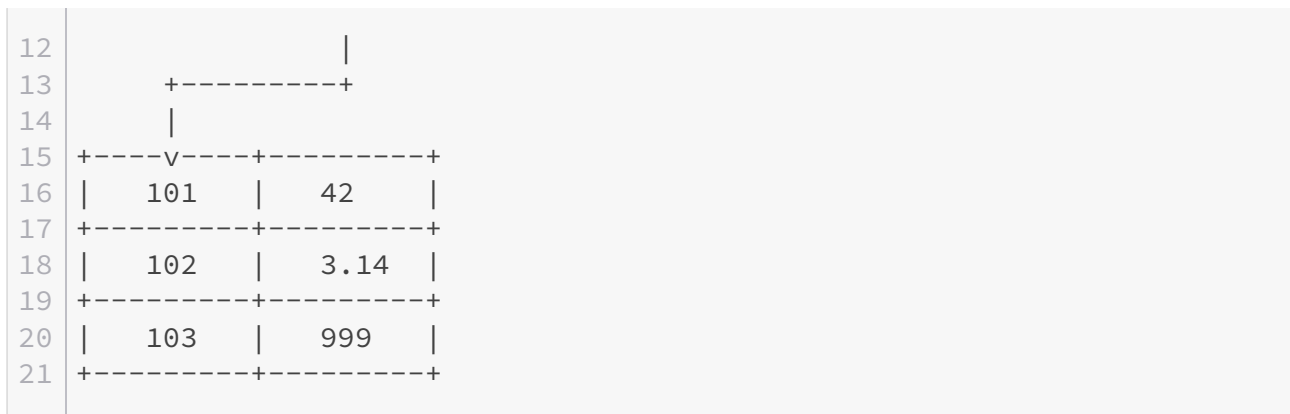
Avant de vous présenter le concept de pointeur, un petit rappel concernant la mémoire s'impose (n'hésitez pas à relire le chapitre sur les variables si celui-ci s'avère insuffisant).

Souvenez-vous : toute donnée manipulée par l'ordinateur est stockée dans sa **mémoire**, plus précisément dans une de ses différentes mémoires (registre(s), mémoire vive, disque(s) dur(s), etc.). Cependant, pour utiliser une donnée, nous avons besoin de savoir où elle se situe, nous avons besoin d'une **référence** vers cette donnée. Dans la plupart des cas, cette référence est en fait une **adresse mémoire** qui indique la position de la donnée dans la mémoire vive.

IV.1.1.1. Les pointeurs

Si l'utilisation des références peut être implicite (c'est par exemple le cas lorsque vous manipulez des variables), il est des cas où elle doit être explicite. C'est à cela que servent les **pointeurs** : ce sont des variables dont le contenu est une adresse mémoire (une référence, donc).

1	+-----+-----+		
2	Adresse Contenu		
3	+-----+-----+		
4	1 56.7		
5	+-----+-----+		
6	2 78		
7	+-----+-----+		
8	3 0		
9	+-----+-----+		
10	4 101		
11	+-----+-----+		



Comme vous pouvez le voir sur le schéma ci-dessus, le contenu à l'adresse 4 est lui-même une adresse et référence le contenu situé à l'adresse 101.

IV.1.1.2. Utilité des pointeurs

Techniquement, il y a trois utilisations majeures des pointeurs en C :

- le passage de références à des fonctions ;
- la manipulation de données complexes ;
- l'allocation dynamique de mémoire.

IV.1.1.2.1. Passage de références à des fonctions

Rappelez-vous du chapitre sur les fonctions : lorsque vous fournissez un argument lors d'un appel, la valeur de celui-ci est affectée au paramètre correspondant, paramètre qui est une variable propre à la fonction appelée. Toutefois, il est parfois souhaitable de modifier une variable de la fonction *appelante*. Dès lors, plutôt que de passer la valeur de la variable en argument, c'est une référence vers celle-ci qui sera envoyée à la fonction.

IV.1.1.2.2. Manipulation de données complexes

Jusqu'à présent, nous avons manipulé des données simples : `int`, `double`, `char`, etc. Cependant, le C nous permet également d'utiliser des données plus complexes qui sont en fait des **agrégats** (un regroupement si vous préférez) de données simples. Or, il n'est possible de manipuler ces agrégats qu'en les parcourant donnée simple par donnée simple, ce qui requiert de disposer d'une référence vers les données qui le composent.

i

Nous verrons les agrégats plus en détails lorsque nous aborderons les structures et les tableaux.

IV.1.1.2.3. L'allocation dynamique de mémoire

Il n'est pas toujours possible de savoir quelle quantité de mémoire sera utilisée par un programme. En effet, si vous prenez le cas d'un logiciel de dessin, ce dernier ne peut pas prévoir quelle sera la taille des images qu'il va devoir manipuler. Pour pallier ce problème, les programmes recourent au mécanisme de l'**allocation dynamique de mémoire** : ils demandent de la mémoire au système d'exploitation lors de leur exécution. Pour que cela fonctionne, le seul moyen est que le système d'exploitation fournisse au programme une référence vers la zone allouée.

IV.1.2. Déclaration et initialisation

La syntaxe pour déclarer un pointeur est la suivante.

```
1 type *nom_du_pointeur;
```

Par exemple, si nous souhaitons créer un pointeur sur `int` (c'est-à-dire un pointeur pouvant stocker l'adresse d'un objet de type `int`) et que nous voulons le nommer « ptr », nous devons écrire ceci.

```
1 int *ptr;
```

L'astérisque peut être entourée d'espaces et placée n'importe où entre le type et l'identificateur. Ainsi, les trois définitions suivantes sont identiques.

```
1 int *ptr;  
2 int * ptr;  
3 int* ptr;
```



Notez bien qu'un pointeur est toujours typé. Autrement dit, vous aurez toujours un pointeur sur (ou vers) un objet d'un certain type (`int`, `double`, `char`, etc.).

IV.1.2.1. Initialisation

Un pointeur, comme une variable, ne possède pas de valeur par défaut, il est donc important de l'initialiser pour éviter d'éventuels problèmes. Pour ce faire, il est nécessaire de recourir à l'**opérateur d'adressage** (ou de référencement) : `&` qui permet d'obtenir l'adresse d'un objet. Ce dernier se place devant l'objet dont l'adresse souhaite être obtenue. Par exemple comme ceci.

```
1 int a = 10;
2 int *p;
3
4 p = &a;
```

Ou, plus directement, comme cela.

```
1 int a = 10;
2 int *p = &a;
```



Faites bien attention à ne pas mélanger différents types de pointeurs ! Un pointeur sur `int` n'est pas le même qu'un pointeur sur `long` ou qu'un pointeur sur `double`. De même, n'affectez l'adresse d'un objet qu'à un pointeur du même type.

```
1 int a;
2 double b;
3 int *p = &b; /* Faux */
4 int *q = &a; /* Correct */
5 double *r = p; /* Faux */
```

IV.1.2.2. Pointeur nul

Vous souvenez-vous du chapitre sur la gestion d'erreurs ? Dans ce dernier, nous vous avons dit que, le plus souvent, les fonctions retournaient une valeur particulière en cas d'erreur. *Quid* de celles qui retournent un pointeur ? Existe-t-il une valeur spéciale qui puisse représenter une erreur ou bien sommes-nous condamnés à utiliser une variable globale comme `errno` ?

Heureusement pour nous, il existe un cas particulier : les pointeurs nuls. Un pointeur nul est tout simplement un pointeur contenant une adresse invalide. Cette adresse invalide dépend de votre machine, mais elle est la même pour tous les pointeurs nuls. Ainsi, deux pointeurs nuls ont une valeur égale.

Pour obtenir cette adresse invalide, il vous suffit de convertir explicitement la constante entière zéro vers le type de pointeur voulu. Ainsi, le pointeur suivant est un pointeur nul.

```
1 int *p = (int *)0;
```




Rappelez-vous qu'il y a conversion implicite vers le type de destination dans le cas d'une affectation. La conversion est donc superflue dans ce cas-ci.

IV.1.2.2.1. La constante NULL

Afin de clarifier un peu les codes sources, il existe une constante définie dans l'en-tête `<std def.h>` : `NULL`. Celle-ci peut être utilisée partout où un pointeur nul est attendu.

```
1 int *p = NULL; /* Un pointeur nul */
```

IV.1.3. Utilisation

IV.1.3.1. Indirection (ou déréférencement)

Maintenant que nous savons récupérer l'adresse d'un objet et l'affecter à un pointeur, voyons le plus intéressant : accéder à cet objet ou le modifier via le pointeur. Pour y parvenir, nous avons besoin de l'**opérateur d'indirection** (ou de déréférencement) : `*`.



Le symbole `*` n'est pas celui de la multiplication ? 🍊

Si, c'est aussi le symbole de la multiplication. Toutefois, à l'inverse de l'opérateur de multiplication, l'opérateur d'indirection ne prend qu'un seul opérande (il n'y a donc pas de risque de confusion).

L'opérateur d'indirection attend un pointeur comme opérande et se place juste derrière celui-ci. Une fois appliqué, ce dernier nous donne accès à la valeur de l'objet référencé par le pointeur, aussi bien pour la lire que pour la modifier.

Dans l'exemple ci-dessous, nous accédons à la valeur de la variable `a` via le pointeur `p`.

```
1 int a = 10;
2 int *p = &a;
3
4 printf("a = %d\n", *p);
```

Résultat

```
1 a = 10
```

À présent, modifions la variable `a` à l'aide du pointeur `p`.

```
1 int a = 10;
2 int *p = &a;
3
4 *p = 20;
5 printf("a = %d\n", a);
```

Résultat

```
1 a = 20
```

IV.1.3.2. Passage comme argument

Voici un exemple de passage de pointeurs en arguments d'une fonction.

```
1 #include <stdio.h>
2
3 void test(int *pa, int *pb)
4 {
5     *pa = 10;
6     *pb = 20;
7 }
8
9
10 int main(void)
11 {
12     int a;
13     int b;
14     int *pa = &a;
15     int *pb = &b;
16
17     test(&a, &b);
18     test(pa, pb);
19     printf("a = %d, b = %d\n", a, b);
```

```
20     printf("a = %d, b = %d\n", *pa, *pb);
21     return 0;
22 }
```

Résultat

1	a = 10, b = 20
2	a = 10, b = 20

Remarquez que les appels `test(&a, &b)` et `test(pa, pb)` réalisent la même opération.

IV.1.3.3. Retour de fonction

Pour terminer, sachez qu'une fonction peut également retourner un pointeur. Cependant, faites attention : *l'objet référencé par le pointeur doit toujours exister au moment de son utilisation !* L'exemple ci-dessous est donc incorrect étant donnée que la variable `n` est de classe de stockage automatique et qu'elle n'existe donc plus après l'appel à la fonction `ptr()`.

```
1  #include <stdio.h>
2
3
4  int *ptr(void)
5  {
6      int n;
7
8      return &n;
9  }
10
11
12 int main(void)
13 {
14     int *p = ptr();
15
16     *p = 10;
17     printf("%d\n", *p);
18     return 0;
19 }
```

L'exemple devient correct si `n` est de classe de stockage statique.

IV.1.3.4. Pointeur de pointeur

Au même titre que n'importe quel autre objet, un pointeur a lui aussi une adresse. Dès lors, il est possible de créer un objet pointant sur ce pointeur : un pointeur de pointeur.

```
1 int a = 10;
2 int *pa = &a;
3 int **pp = &pa;
```

Celui-ci s'utilise de la même manière qu'un pointeur si ce n'est qu'il est possible d'opérer deux indirections : une pour atteindre le pointeur référencé et une seconde pour atteindre la variable sur laquelle pointe le premier pointeur.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int a = 10;
7     int *pa = &a;
8     int **pp = &pa;
9
10    printf("a = %d\n", **pp);
11    return 0;
12 }
```



Ceci peut continuer à l'infini pour concevoir des pointeurs de pointeur de pointeur de pointeur de... Bref, vous avez compris le principe. 🍊

IV.1.4. Pointeurs génériques et affichage

IV.1.4.1. Le type `void`

Vous avez déjà rencontré le mot-clé `void` lorsque nous avons parlé des fonctions, ce dernier permet d'indiquer qu'une fonction n'utilise aucun paramètre et/ou ne retourne aucune valeur. Toutefois, nous n'avons pas tout dit à son sujet : `void` est en fait un type, au même titre que `int` ou `double`. 🍊



Et il représente quoi ce type, alors ?

Hum... rien (d'où son nom). 🍊

En fait, il s'agit d'un type dit « **incomplet** », c'est à dire que la taille de ce dernier n'est pas calculable et qu'il n'est pas utilisable dans des expressions. Quel est l'intérêt de la chose me direz-vous ? Permettre de créer des pointeurs « **génériques** » (ou « universels »).

En effet, nous venons de vous dire qu'un pointeur devait toujours être typé. Cependant, cela peut devenir gênant si vous souhaitez créer une fonction qui doit pouvoir travailler avec n'importe quel type de pointeur (nous verrons un exemple très bientôt). C'est ici que le type `void` intervient : un pointeur sur `void` est considéré comme un pointeur générique, ce qui signifie qu'il peut référencer n'importe quel type d'objet.

En conséquence, il est possible d'affecter n'importe quelle adresse d'objet à un pointeur sur `void` et d'affecter un pointeur sur `void` à n'importe quel autre pointeur (et inversement).

```
1 int a;
2 double b;
3 void *p;
4 double *r;
5
6 p = &a; /* Correct */
7 p = &b; /* Correct */
8 r = p; /* Correct */
```

IV.1.4.2. Afficher une adresse

Il est possible d'afficher une adresse à l'aide de l'indicateur de conversion `p` de la fonction `printf()`. Ce dernier attend en argument un pointeur sur `void`. Vous voyez ici l'intérêt d'un pointeur générique : un seul indicateur suffit pour afficher tous les types de pointeurs.

Notez que l'affichage s'effectue le plus souvent en hexadécimal.

```
1 int a;
2 int *p = &a;
3
4 printf("%p == %p\n", (void *)&a, (void *)p);
```

Tant que nous y sommes, profitons en pour voir quelle est l'adresse invalide de notre système.

```
1 printf("%p\n", (void *)0); /* Comme ceci */
2 printf("%p\n", (void *)NULL); /* Ou comme cela */
```

IV.1.5. Exercice

Pour le moment, tout ceci doit sans doute vous paraître quelques peu abstrait et sans doute inutile. Toutefois, rassurez-vous, cela vous semblera plus clair après les chapitres suivants.

En attendant, nous vous proposons un petit exercice mettant en pratique les pointeurs : programmez une fonction nommée « swap », dont le rôle est d'échanger la valeur de deux variables de type `int`. Autrement dit, la valeur de la variable « a » doit devenir celle de « b » et la valeur de « b », celle de « a ».

IV.1.5.1. Correction

👁 Contenu masqué n°24

Conclusion

Nous en avons fini avec les pointeurs, du moins, pour le moment. En effet, les pointeurs sont omniprésents en langage C et nous n'avons pas fini d'en entendre parler. 🍌

En résumé

1. Un pointeur est une variable dont le contenu est une adresse ;
2. L'opérateur d'adressage `&` permet de récupérer l'adresse d'une variable ;
3. Un pointeur d'un type peut uniquement contenir l'adresse d'un objet du même type ;
4. Un pointeur nul contient une adresse invalide qui dépend de votre système d'exploitation ;
5. Un pointeur nul est obtenu en convertissant zéro vers un type pointeur ou en recourant à la macroconstante `NULL`.
6. L'opérateur d'indirection (`*`) permet d'accéder à l'objet référencé par un pointeur ;
7. En cas de retour d'un pointeur par une fonction, il est *impératif* de s'assurer que l'objet référencé existe toujours ;
8. Le type `void` permet de construire des pointeurs génériques ;
9. L'indicateur de conversion `%p` peut être utilisé pour afficher une adresse. Une conversion vers le type `void *` est pour cela nécessaire.

Contenu masqué

Contenu masqué n°23

Oui, le plus souvent, il s'agit de zéro. 🍌

[Retourner au texte.](#)

Contenu masqué n°24

```
1 #include <stdio.h>
2
3 void swap(int *, int *);
4
5
6 void swap(int *pa, int *pb)
7 {
8     int tmp = *pa;
9     *pa = *pb;
10    *pb = tmp;
11 }
12
13
14 int main(void)
15 {
16     int a = 10;
17     int b = 20;
18
19     swap(&a, &b);
20     printf("a = %d, b = %d\n", a, b);
21     return 0;
22 }
```

[Retourner au texte.](#)

IV.2. Les structures

Introduction

Dans le chapitre précédent, nous vous avons dit que les pointeurs étaient entre autres utiles pour manipuler des données complexes. Les structures sont les premières que nous allons étudier.

Une **structure** est un regroupement de plusieurs objets, de types différents ou non. *Grosso modo*, une structure est finalement une boîte qui regroupe plein de données différentes.

IV.2.1. Définition, initialisation et utilisation

IV.2.1.1. Définition d'une structure

Une structure étant un regroupement d'objets, la première chose à réaliser est la description de celle-ci (techniquement, sa **définition**), c'est-à-dire préciser de quel(s) objet(s) cette dernière va se composer.

La syntaxe de toute définition est la suivante.

```
1 struct étiquette
2 {
3     /* Objet(s) composant(s) la structure. */
4 };
```

Prenons un exemple concret : vous souhaitez demander à l'utilisateur deux mesures de temps sous la forme heure(s):minute(s):seconde(s).milliseconde(s) et lui donner la différence entre les deux en secondes. Vous pourriez utiliser six variables pour stocker ce que vous fournit l'utilisateur, toutefois cela reste assez lourd. À la place, nous pourrions représenter chaque mesure à l'aide d'une structure composée de trois objets : un pour les heures, un pour les minutes et un pour les secondes.

```
1 struct temps {
2     unsigned heures;
3     unsigned minutes;
4     double secondes;
5 };
```


Comme vous le voyez, nous avons donné un nom (plus précisément, une **étiquette**) à notre structure : « temps ». Les règles à respecter sont les mêmes que pour les noms de variable et de fonction.

Pour le reste, la composition de la structure (sa **définition**) est décrite à l'aide d'une suite de déclarations de variables. Ces différentes déclarations constituent les **membres** ou **champs** de la structure.

Enfin, notez la présence d'un point-virgule *obligatoire* à la fin de la définition de la structure.



Une structure ne peut pas comporter plus de cent vingt-sept membres.

IV.2.1.2. Définition d'une variable de type structure

Une fois notre structure décrite, il ne nous reste plus qu'à créer une variable de ce type. Pour ce faire, la syntaxe est la suivante.

```
1 struct étiquette identificateur;
```

La méthode est donc la même que pour définir n'importe quelle variable, si ce n'est que le type de la variable est précisé à l'aide du mot-clé `struct` et de l'étiquette de la structure. Avec notre exemple de la structure `temps`, cela donne ceci.

```
1 struct temps {
2     unsigned heures;
3     unsigned minutes;
4     double secondes;
5 };
6
7
8 int main(void)
9 {
10     struct temps t;
11
12     return 0;
13 }
```



Notez qu'il est parfaitement possible d'utiliser le même identificateur (le même nom si vous préférez) pour une variable, une étiquette de structure et un membre de structure. Ainsi, le code suivant est parfaitement valide.



```

1 struct test {
2     int test;
3 };
4
5 int main(void)
6 {
7     struct test test;
8     test.test = 10;
9     return 0;
10 }

```

De part la présence du mot-clé `struct` pour désigner l'étiquette de la structure ou de l'opérateur `.` (ou `->` que nous verrons bientôt) pour désigner un membre de la structure, il n'y a pas de risque de confusion avec une variable. Du point de vue du langage, on dit que les identificateurs d'étiquette de structure, de membre de structure et de variable appartiennent à des **espaces de nom** (*namespace* en anglais) différents.

IV.2.1.3. Initialisation

Comme pour n'importe quelle autre variable, il est possible d'initialiser une variable de type structure dès sa définition. Toutefois, à l'inverse des autres, l'initialisation s'effectue à l'aide d'une liste fournissant une valeur pour un ou plusieurs membres de la structure.

IV.2.1.3.1. Initialisation séquentielle

L'initialisation séquentielle permet de spécifier une valeur pour un ou plusieurs membres de la structure en suivant l'ordre de la définition. Ainsi, l'exemple ci-dessous initialise le membre `heures` à 1, `minutes` à 45 et `secondes` à 30.560.

```

1 struct temps t = { 1, 45, 30.560 };

```

IV.2.1.3.2. Initialisation sélective

L'initialisation séquentielle n'est toutefois pas toujours pratique, surtout si les champs que vous souhaitez initialiser sont par exemple au milieu d'une grande structure. Pour éviter ce problème, il est possible de recourir à une initialisation sélective en spécifiant explicitement le ou les champs à initialiser. L'exemple ci-dessous est identique au premier si ce n'est qu'il recourt à une initialisation sélective.

```
1 struct temps t = { .secondes = 30.560, .minutes = 45, .heures = 1
    };
```

Notez qu'il n'est plus nécessaire de suivre l'ordre de la définition dans ce cas.

i

Il est parfaitement possible de mélanger les initialisations séquentielles et sélectives. Dans un tel cas, l'initialisation séquentielle reprend au dernier membre désigné par une initialisation sélective. Partant, le code suivant initialise le membre `heures` à `1` et le membre `secondes` à `30.560`.

```
1 struct temps t = { 1, .secondes = 30.560 };
```

Alors que le code ci-dessous initialise le membre `minutes` à `45` et le membre `secondes` à `30.560`.

```
1 struct temps t = { .minutes = 45, 30.560 };
```

!

Dans le cas où vous n'initialisez pas tous les membres de la structure, les autres seront initialisés à zéro ou, s'il s'agit de pointeurs, seront des pointeurs nuls.

```
1 #include <stdio.h>
2
3 struct temps {
4     unsigned heures;
5     unsigned minutes;
6     double secondes;
7 };
8
9
10 int main(void)
11 {
12     struct temps t = { 1 };
13
14     printf("%d, %d, %f\n", t.heures, t.minutes, t.secondes);
15     return 0;
16 }
```



Résultat	
1	1, 0, 0.000000



Notez que si vous n'initialisez aucun champ, mais que votre structure est de classe de stockage statique, tous les champs seront initialisés à zéro ou, s'il s'agit de pointeurs, seront des pointeurs nuls.

IV.2.1.4. Accès à un membre

L'accès à un membre d'une structure se réalise à l'aide de la variable de type structure et de l'opérateur `.` suivi du nom du champ visé.

```
1 variable.membre
```

Cette syntaxe peut être utilisée aussi bien pour obtenir la valeur d'un champ que pour en modifier le contenu. L'exemple suivant effectue donc la même action que l'initialisation présentée précédemment.

```
1 t.heures = 1;
2 t.minutes = 45;
3 t.secondes = 30.560;
```

IV.2.1.5. Exercice

Afin d'assimiler tout ceci, voici un petit exercice.

Essayez de réaliser ce qui a été décrit plus haut : demandez à l'utilisateur de vous fournir deux mesures de temps sous la forme heure(s):minute(s):seconde(s).milliseconde(s) et donnez lui la différence en seconde entre celles-ci.

Voici un exemple d'utilisation.

```
1 Première mesure (hh:mm:ss.xxx) : 12:45:50.640
2 Deuxième mesure (hh:mm:ss.xxx) : 13:30:35.480
3 Il y 2684.840 seconde(s) de différence.
```

IV.2.1.5.1. Correction

👁 Contenu masqué n°25

IV.2.2. Structures et pointeurs

Certains d'entre vous s'en étaient peut-être doutés : s'il existe un objet d'un type, il doit être possible de créer un pointeur vers un objet de ce type. Si oui, sachez que vous aviez raison. 🍊

```
1 struct temps *p;
```

La définition ci-dessus crée un pointeur `p` vers un objet de type `struct temps`.

IV.2.2.1. Accès via un pointeur

L'utilisation d'un pointeur sur structure est un peu plus complexe que celle d'un pointeur vers un type de base. En effet, il y a deux choses à gérer : l'accès via le pointeur et l'accès à un membre. Intuitivement, vous combineriez sans doute les opérateurs `*` et `.` comme ceci.

```
1 *p.heures = 1;
```

Toutefois, cette syntaxe ne correspond pas à ce que nous voulons car l'opérateur `.` s'applique *prioritairement* à l'opérateur `*`. Autrement dit, le code ci-dessus accède au champ `heures` et tente de lui appliquer l'opérateur d'indirection, ce qui est incorrect puisque le membre `heures` est un entier non signé.

Pour résoudre ce problème, nous devons utiliser des parenthèses afin que l'opérateur `.` soit appliqué *après* le déréférencement, ce qui donne la syntaxe suivante.

```
1 (*p).heures = 1;
```

Cette écriture étant un peu lourde, le C fournit un autre opérateur qui combine ces deux opérations : l'opérateur `->`.

```
1 p->heures = 1;
```

Le code suivant initialise donc la structure `t` via le pointeur `p`.

```
1 struct temps t;  
2 struct temps *p = &t;  
3  
4 p->heures = 1;  
5 p->minutes = 45;  
6 p->secondes = 30.560;
```

IV.2.2.2. Adressage

Il est important de préciser que l'opérateur d'adressage peut s'appliquer aussi bien à une structure qu'à un de ses membres. Ainsi, dans l'exemple ci-dessous, nous définissons un pointeur `p` pointant sur la structure `t` et un pointeur `q` pointant sur le champ `heures` de la structure `t`.

```
1 struct temps t;  
2 struct temps *p = &t;  
3 int *q = &t.heures;
```

IV.2.2.3. Pointeurs sur structures et fonctions

Indiquons enfin que l'utilisation de pointeurs est particulièrement propice dans le cas du passage de structures à des fonctions. En effet, rappelez-vous, lorsque vous fournissez un argument lors d'un appel de fonction, la valeur de celui-ci est affectée au paramètre correspondant. Cette règle s'applique également aux structures : la valeur de chacun des membres est copiée une à une. Dès lors, si la structure passée en argument comporte beaucoup de champs, la copie risque d'être longue. L'utilisation d'un pointeur évite ce problème puisque seule la valeur du pointeur (autrement dit, l'adresse vers la structure) sera copiée et non toute la structure.

IV.2.3. Portée et déclarations

IV.2.3.1. Portée

Dans les exemples précédents, nous avons toujours placé notre définition de structure en dehors de toute fonction. Cependant, sachez que celle-ci peut être circonscrite à un bloc de sorte de limiter sa **portée**, comme pour les définitions de variables et les déclarations de variables et fonctions.

Dans l'exemple ci-dessous, la structure `temps` ne peut être utilisée que dans le bloc de la fonction `main()` et les éventuels sous-blocs qui la composent.

```

1 int main(void)
2 {
3     struct temps {
4         unsigned heures;
5         unsigned minutes;
6         double secondes;
7     };
8
9     struct temps t;
10
11     return 0;
12 }

```

Notez qu'il est possible de combiner une définition de structure et une définition de variable. En effet, une définition de structure n'étant rien d'autre que la définition d'un nouveau type, celle-ci peut être placée là où est attendu un type dans une définition de variable. Avec l'exemple précédent, cela donne ceci.

```

1 int main(void)
2 {
3     struct temps {
4         unsigned heures;
5         unsigned minutes;
6         double secondes;
7     } t1;
8     struct temps t2;
9
10    return 0;
11 }

```

Il y a trois définitions dans ce code : celle du type `struct temps`, celle de la variable `t1` et celle de la variable `t2`. Après sa définition, le type `struct temps` peut tout à fait être utilisé pour définir d'autres variables de ce type, ce qui est le cas de `t2`.

i

Notez qu'il est possible de condenser la définition du type `struct temps` et de la variable `t1` sur une seule ligne, comme ceci.

```

1 struct temps { unsigned heures; unsigned minutes; double
    secondes; } t1;

```

S'agissant d'une définition de variable, il est également parfaitement possible de l'initialiser.

```
1 int main(void)
2 {
3     struct temps {
4         unsigned heures;
5         unsigned minutes;
6         double secondes;
7     } t1 = { 1, 45, 30.560 };
8     struct temps t2;
9
10    return 0;
11 }
```

Enfin, précisons que l'étiquette d'une structure peut être omise lors de sa définition. Néanmoins, cela ne peut avoir lieu que si la définition de la structure est combinée avec une définition de variable. C'est assez logique étant donné qu'il ne peut pas être fait référence à cette définition à l'aide d'une étiquette.

```
1 int main(void)
2 {
3     struct {
4         unsigned heures;
5         unsigned minutes;
6         double secondes;
7     } t;
8
9     return 0;
10 }
```

IV.2.3.2. Déclarations

Jusqu'à présent, nous avons parlé de définitions de structures, toutefois, comme pour les variables et les fonctions, il existe également des déclarations de structures. Une déclaration de structure est en fait une définition sans le corps de la structure.

```
1 struct temps;
```

Quel est l'intérêt de la chose me direz-vous ? Résoudre deux types de problèmes : les structures interdépendantes et les structures comportant un ou des membres qui sont des pointeurs vers elle-même.

IV.2.3.2.1. Les structures interdépendantes

Deux structures sont interdépendantes lorsque l'une comprend un pointeur vers l'autre et inversement.

```
1 struct a {  
2     struct b *p;  
3 };  
4  
5 struct b {  
6     struct a *p;  
7 };
```

Voyez-vous le problème ? Si le type `struct b` ne peut être utilisé qu'après sa définition, alors le code ci-dessus est faux et le cas de structures interdépendantes est insoluble. Heureusement, il est possible de **déclarer** le type `struct b` afin de pouvoir l'utiliser *avant* sa définition.



Une déclaration de structure crée un type dit **incomplet** (comme le type `void`). Dès lors, il ne peut pas être utilisé pour définir une variable (puisque les membres qui composent la structure sont inconnus). Ceci n'est utilisable que pour définir des pointeurs.

Le code ci-dessous résout le « problème » en déclarant le type `struct b` avant son utilisation.

```
1 struct b;  
2  
3 struct a {  
4     struct b *p;  
5 };  
6  
7 struct b {  
8     struct a *p;  
9 };
```

Nous avons entouré le mot « problème » de guillemets car le premier code que nous vous avons montré n'en pose en fait aucun et compile sans sourciller. 🍊

En fait, afin d'éviter ce genre d'écritures, le langage C prévoit l'ajout de **déclarations implicites**. Ainsi, lorsque le compilateur rencontre un pointeur vers un type de structure qu'il ne connaît pas, il ajoute implicitement une déclaration de cette structure juste avant. Ainsi, le premier et le deuxième code sont équivalents si ce n'est que le premier comporte une déclaration implicite et non une déclaration explicite.

IV.2.3.2.2. Structure qui pointe sur elle-même

Le deuxième cas où les déclarations de structure s'avèrent nécessaires est celui d'une structure qui comporte un pointeur vers elle-même.

```
1 struct a {  
2     struct a *p;  
3 };
```

De nouveau, si le type `struct a` n'est utilisable qu'après sa définition, c'est grillé. Toutefois, comme pour l'exemple précédent, le code est correct, mais pas tout à fait pour les mêmes raisons. Dans ce cas ci, le type `struct a` est connu car nous sommes en train de le définir. Techniquement, dès que la définition commence, le type est déclaré.



Notez que, comme les définitions, les déclarations (implicites ou non) ont une portée.

IV.2.4. Les structures littérales

Nous venons de le voir, manipuler une structure implique de définir une variable. Toutefois, il est des cas où devoir définir une variable est peu utile, par exemple si la structure n'est pas amenée à être modifiée par la suite. Aussi est-il possible de construire des structures dites « littérales », c'est-à-dire sans passer par la définition d'une variable.

La syntaxe est la suivante.

```
1 (type) { élément1, élément2, ... }
```

Où `type` est le type de la structure et où `élément1` et `élément2` représentent les éléments d'une liste d'initialisation (comme dans le cas de l'initialisation d'une variable de type structure).

Ces structures littérales peuvent être utilisées partout où une structure est attendue, par exemple comme argument d'un appel de fonction.

```
1 #include <stdio.h>  
2  
3 struct temps {  
4     unsigned heures;  
5     unsigned minutes;  
6     double secondes;  
7 };  
8  
9
```

```

10 void affiche_temps(struct temps temps)
11 {
12     printf("%d:%d:%f\n", temps.heures, temps.minutes,
13             temps.secondes);
14 }
15
16 int
17 main(void)
18 {
19     affiche_temps((struct temps) { .heures = 12, .minutes = 0,
20                                   .secondes = 1. });
21     return 0;
22 }

```

Résultat

1	12:0:1.000000
---	---------------

Comme vous le voyez, au lieu de définir une variable et de passer celle-ci en argument lors de l'appel à la fonction `affiche_temps()`, nous avons directement fourni une structure littérale.



Classe de stockage

Les structures littérales sont de classe de stockage *automatique*, elles sont donc détruites à la fin du bloc dans lequel elles ont été créées.

Notez qu'à l'inverse des chaînes de caractères littérales, les structures littérales peuvent parfaitement être modifiées. L'exemple suivant est donc parfaitement correct.

```

1  #include <stdio.h>
2
3  struct temps {
4      unsigned heures;
5      unsigned minutes;
6      double secondes;
7  };
8
9
10 int
11 main(void)
12 {
13     struct temps *p = &(struct temps) { .heures = 12, .minutes =
14                                         0, .secondes = 1. };

```

```

14
15     p->secondes = 42.; /* Ok. */
16     return 0;
17 }

```

IV.2.5. Un peu de mémoire

?

Savez-vous comment sont représentées les structures en mémoire ?

Les membres d'une structure sont placés les uns après les autres en mémoire. Par exemple, prenons cette structure.

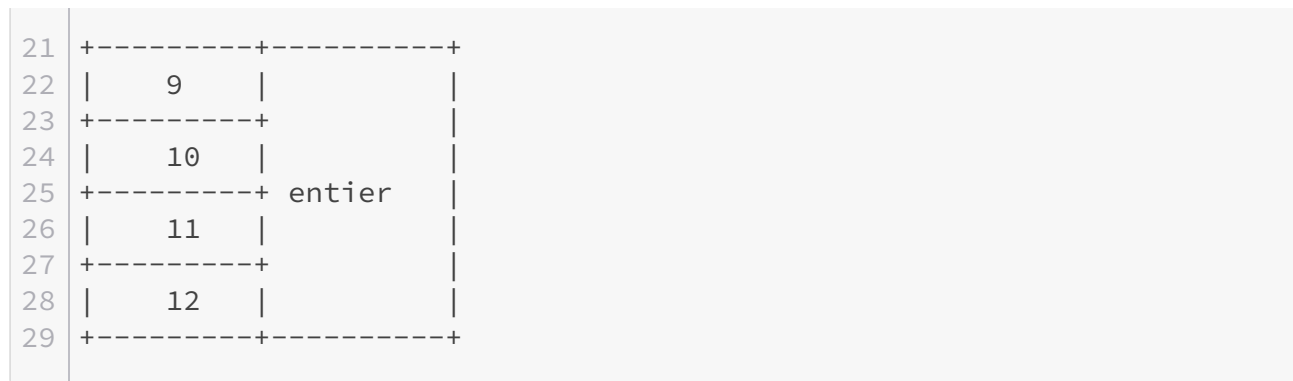
```

1 struct exemple
2 {
3     double flottant;
4     char lettre;
5     unsigned entier;
6 };

```

Si nous supposons qu'un `double` a une taille de huit octets, un `char` d'un octet, et un `unsigned int` de quatre octets, voici ce que devrait donner cette structure en mémoire.

1	+-----+-----+	
2	Adresse Champ	
3	+-----+-----+	
4	0	
5	+-----+	
6	1	
7	+-----+	
8	2	
9	+-----+	
10	3	
11	+-----+ flottant	
12	4	
13	+-----+	
14	5	
15	+-----+	
16	6	
17	+-----+	
18	7	
19	+-----+	
20	8 lettre	



Les adresses spécifiées dans le schéma sont fictives et ne servent qu'à titre d'illustration.

IV.2.5.1. L'opérateur sizeof

Voyons à présent comment déterminer cela de manière plus précise, en commençant par la taille des types. L'opérateur `sizeof` permet de connaître la taille en multiplètes (*bytes* en anglais) de son opérande. Cet opérande peut être soit un type (qui doit alors être entre parenthèses), soit une expression (auquel cas les parenthèses sont facultatives).

Le résultat de cet opérateur est de type `size_t`. Il s'agit d'un type entier *non signé* défini dans l'en-tête `<stddef.h>` qui est capable de contenir la taille de n'importe quel objet. L'exemple ci-dessous utilise l'opérateur `sizeof` pour obtenir la taille des types de bases.



Une expression de type `size_t` peut être affichée à l'aide de l'indicateur de conversion `zu`.

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      double f;
7
8      printf("_Bool : %zu\n", sizeof(_Bool));
9      printf("char : %zu\n", sizeof(char));
10     printf("short : %zu\n", sizeof(short));
11     printf("int : %zu\n", sizeof(int));
12     printf("long : %zu\n", sizeof(long));
13     printf("float : %zu\n", sizeof(float));
14     printf("double : %zu\n", sizeof(double));
15     printf("long double : %zu\n", sizeof(long double));
16
17     printf("int : %zu\n", sizeof 5);
18     printf("double : %zu\n", sizeof f);

```

```
19     return 0;
20 }
```

Résultat

```
1  _Bool : 1
2  char : 1
3  short : 2
4  int : 4
5  long : 8
6  float : 4
7  double : 8
8  long double : 16
9  int : 4
10 double : 8
```



Le type `char` a *toujours* une taille d'un multipliet.



Il est parfaitement possible que vous n'obteniez pas les même valeurs que nous, celles-ci dépendent de votre machine.

Remarquez que les parenthèses ne sont pas obligatoires dans le cas où l'opérande de l'opérateur `sizeof` est une expression (dans notre exemple : `5` et `f`).

Ceci étant dit, voyons à présent ce que donne la taille de la structure présentée plus haut. En toute logique, elle devrait être égale à la somme des tailles de ses membres, chez nous : 13 (8 + 1 + 4).

```
1  #include <stddef.h>
2  #include <stdio.h>
3
4  struct exemple
5  {
6      double flottant;
7      char lettre;
8      unsigned int entier;
9  };
10
11
12 int main(void)
13 {
```

```

14     printf("struct exemple : %zu\n", sizeof(struct exemple));
15     return 0;
16 }

```

Résultat

```
1 struct exemple : 16
```

Ah ! Il semble que nous avons loupé quelque chose... 🍊

Pourquoi obtenons-nous 16 et non 13, comme attendu ? Pour répondre à cette question, nous allons devoir plonger un peu dans les entrailles de notre machine.

IV.2.5.2. Alignement en mémoire

En fait, il faut savoir qu'il n'est dans les faits pas possible de lire ou modifier un multipllet spécifique de la mémoire vive. Techniquement, il est uniquement possible de lire ou écrire des blocs de données de taille fixe appelés **mots mémoire**. Dès lors, *il est nécessaire que les données à lire ou à écrire soient contenues dans un de ces mots mémoires*.

Quel rapport avec notre structure me direz-vous ? Supposons que notre RAM utilise des blocs de huit octets. Si notre structure faisait treize octets, voici comment elle serait vue en termes de blocs.

1		+-----+-----+
2		Adresse Champ
3	+->	+-----+-----+
4		0
5		+-----+
6		1
7		+-----+
8		2
9		+-----+
10		3
11	1	+-----+ flottant
12		4
13		+-----+
14		5
15		+-----+
16		6
17		+-----+
18		7
19	+->	+-----+-----+
20		8 lettre

IV. Agrégats, mémoire et fichiers

21			+-----+-----+
22			9
23			+-----+
24			10
25			+-----+ entier
26			11
27	2		+-----+
28			12
29			+-----+
30			
31			
32			
33			
34	.		
35	.		

Comme vous le voyez, le champ `flottant` remplit complètement le premier mot, alors que les champs `lettre` et `entier` ne remplissent que partiellement le second. Dans le seul cas de notre structure, cela ne pose pas de problème, sauf que celle-ci n'est pas seule en mémoire. En effet, imaginons maintenant qu'un autre `int` suive cette structure, nous obtiendrions alors ceci.

1			+-----+-----+
2			Adresse Champ
3	+->		+-----+
4			0
5			+-----+
6			1
7			+-----+
8			2
9			+-----+
10			3
11	1		+-----+ flottant
12			4
13			+-----+
14			5
15			+-----+
16			6
17			+-----+
18			7
19	+->		+-----+
20			8 lettre
21			+-----+
22			9
23			+-----+
24			10
25			+-----+ entier
26			11
27	2		+-----+

IV. Agrégats, mémoire et fichiers

28			12			
29			+-----+	+-----+		
30			13			
31			+-----+			
32			14			
33			+-----+	int		
34			15			
35	+->		+-----+			
36			16			
37			+-----+	+-----+		
38						
39						
40	3					
41			.			
42			.			
43			.			

Et là, c'est le drame, le second entier est à cheval sur le deuxième et le troisième mot. Pour éviter ces problèmes, les compilateurs ajoutent des multipliets dit « de bourrage » (*padding* en anglais), afin que les données ne soient jamais à cheval sur plusieurs mots. Dans le cas de notre structure, le compilateur a ajouté trois octets de bourrage juste après le membre `lettre` afin que les deux derniers champs occupent un mot complet.

1			+-----+	+-----+		
2			Adresse	Champ		
3	+->		+-----+	+-----+		
4			0			
5			+-----+			
6			1			
7			+-----+			
8			2			
9			+-----+			
10			3			
11	1		+-----+	flottant		
12			4			
13			+-----+			
14			5			
15			+-----+			
16			6			
17			+-----+			
18			7			
19	+->		+-----+	+-----+		
20			8	lettre		
21			+-----+			
22			9			
23			+-----+			
24			10	bourrage		
25			+-----+			

26				11			
27	2		+	-----	+	-----	+
28				12			
29			+	-----	+		
30				13			
31			+	-----	+	entier	
32				14			
33			+	-----	+		
34				15			
35		+->	+	-----	+	-----	+
36				16			
37			+	-----	+		
38				17			
39			+	-----	+	int	
40				18			
41			+	-----	+		
42				19			
43	3		+	-----	+	-----	+
44							
45							
46							
47	.						
48	.						
49	.						

Ainsi, il n'y a plus de problèmes, l'entier suivant la structure n'est plus à cheval sur deux mots.

?

Mais, comment le compilateur sait-il qu'il doit ajouter des multipliants de bourrage avant le champ `entier` et précisément trois ?

Cela est dû à la position du champ `entier` dans la structure. Dans notre exemple, il y a 9 multipliants qui précèdent le champ `entier`, or le compilateur sait que la taille d'un `unsigned int` est de quatre multipliants et que la RAM utilise des blocs de huit octets, il en déduit alors qu'il y aura un problème. En effet, pour qu'il n'y ait pas de soucis, il est nécessaire qu'un champ de type `unsigned int` soit précédé d'un nombre de multipliants qui soit multiple de quatre (d'où les trois multipliants de bourrage pour arriver à 12).

i

Notez que la même réflexion peut s'opérer en termes d'adresses : dans notre exemple, un champ de type `unsigned int` doit commencer à une adresse multiple de 4.

Cette contrainte est appelée une **contrainte d'alignement** (car il est nécessaire d'« aligner » les données en mémoire). Chaque type possède sa propre contrainte d'alignement, dans notre exemple le type `unsigned int` a une contrainte d'alignement (ou simplement un alignement) de 4.

IV.2.5.3. L'opérateur `_Alignof`

Il est possible de connaître les contraintes d'alignement d'un type, à l'aide de l'opérateur `_Alignof` (ou `alignof` si l'en-tête `<stdalign.h>` est inclus)¹. Ce dernier s'utilise de la même manière que l'opérateur `sizeof`, si ce n'est que seul un type peut lui être fourni et non une expression. Le résultat de l'opérateur est, comme pour `sizeof`, une expression de type `size_t`.

Le code suivant vous donne les contraintes d'alignement pour chaque type de base.

```

1 #include <stdalign.h>
2 #include <stdio.h>
3
4
5 int main(void)
6 {
7     printf("_Bool: %zu\n", _Alignof(_Bool));
8     printf("char: %zu\n", _Alignof(char));
9     printf("short: %zu\n", _Alignof(short));
10    printf("int : %zu\n", _Alignof(int));
11    printf("long : %zu\n", _Alignof(long));
12    printf("long long : %zu\n", _Alignof(long long));
13    printf("float : %zu\n", alignof(float));
14    printf("double : %zu\n", alignof(double));
15    printf("long double : %zu\n", alignof(long double));
16    return 0;
17 }
```

Résultat

```

1 _Bool: 1
2 char: 1
3 short: 2
4 int : 4
5 long : 8
6 long long : 8
7 float : 4
8 double : 8
9 long double : 16
```



Le type `char` ayant une taille d'un multiplet, il peut *toujours* être contenu dans un mot. Il n'a donc pas de contraintes d'alignement ou, plus précisément, une contrainte d'alignement de 1.



Comme pour l'opérateur `sizeof`, il est possible que vous n'obteniez pas les mêmes valeurs que nous, celles-ci dépendent de votre machine.

Conclusion

Les structures en elles-mêmes ne sont pas compliquées à comprendre, mais l'intérêt est parfois plus difficile à saisir. Ne vous en faites pas, nous aurons bientôt l'occasion de découvrir des cas où les structures se trouvent être bien pratiques. En attendant, n'hésitez pas à relire le chapitre s'il vous reste des points obscurs.

En résumé

1. Une structure est un regroupement de plusieurs objets, potentiellement de types différents ;
2. La composition d'une structure est précisée à l'aide d'une définition ;
3. Si certains membres d'une structure ne se voient pas attribuer de valeurs durant l'initialisation d'une variable, ils seront initialisés à zéro ou, s'il s'agit de pointeurs, seront des pointeurs nuls ;
4. L'opérateur `.` permet d'accéder aux champs d'une variable de type structure ;
5. L'opérateur `->` peut être utilisé pour simplifier l'accès aux membres d'une structure via un pointeur ;
6. Une structure peut être déclarée en omettant sa composition (son corps), le type ainsi déclaré est incomplet jusqu'à ce qu'il soit défini ;
7. Tous les types ont des contraintes d'alignement, c'est-à-dire un nombre dont les adresses d'un type doivent être le multiple ;
8. Les contraintes d'alignement peuvent induire l'ajout de multipliants de bourrage entre certains champs d'une structure.

Contenu masqué

Contenu masqué n°25

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct temps {
5     unsigned heures;
6     unsigned minutes;
```

1. l'opérateur `_Alignof` a été introduit par la norme C11. Auparavant, pour connaître les contraintes d'alignement, il était nécessaire de se reposer sur les structures et la macrofonction `offsetof` (définie dans l'en-tête `<stddef.h>`).

```
7     double secondes;
8 };
9
10
11 int main(void)
12 {
13     struct temps t1;
14     struct temps t2;
15
16     printf("Première mesure (hh:mm:ss.xxx) : ");
17
18     if (scanf("%u:%u:%lf", &t1.heures, &t1.minutes, &t1.secondes)
19         != 3)
20     {
21         printf("Mauvaise saisie\n");
22         return EXIT_FAILURE;
23     }
24
25     printf("Deuxième mesure (hh:mm:ss.xxx) : ");
26
27     if (scanf("%u:%u:%lf", &t2.heures, &t2.minutes, &t2.secondes)
28         != 3)
29     {
30         printf("Mauvaise saisie\n");
31         return EXIT_FAILURE;
32     }
33
34     t1.minutes += t1.heures * 60;
35     t1.secondes += t1.minutes * 60;
36     t2.minutes += t2.heures * 60;
37     t2.secondes += t2.minutes * 60;
38
39     printf("Il y a %.3f seconde(s) de différence.\n",
40           t2.secondes - t1.secondes);
41     return 0;
42 }
```

[Retourner au texte.](#)

IV.3. Les tableaux

Introduction

Poursuivons notre tour d’horizon des données complexes avec les **tableaux**.

Comme les structures, les tableaux sont des regroupements de plusieurs objets. Cependant, à l’inverse de celles-ci, les tableaux regroupent des données de *même type* et de manière *contiguë* (ce qui exclut la présence de multiplets de bourrage).

Un tableau est donc un gros bloc de mémoire de *taille finie* qui commence à une adresse déterminée : celle de son premier élément.

IV.3.1. Les tableaux simples (à une dimension)

IV.3.1.1. Définition

La définition d’un tableau nécessite trois informations :

- le type des éléments du tableau (rappelez-vous : un tableau est une suite de données de *même type*) ;
- le nom du tableau (en d’autres mots, son identificateur) ;
- la longueur du tableau (autrement dit, le nombre d’éléments qui le composent). Cette dernière doit être une expression *entière*.

```
1 type identificateur[longueur];
```

Comme vous le voyez, la syntaxe de la déclaration d’un tableau est similaire à celle d’une variable, la seule différence étant qu’il est nécessaire de préciser le nombre d’éléments entre crochets à la suite de l’identificateur du tableau.

Ainsi, si nous souhaitons par exemple définir un tableau contenant vingt `int`, nous devons procéder comme suit.

```
1 int tab[20];
```

IV.3.1.2. Initialisation

Comme pour les variables, il est possible d'initialiser un tableau ou, plus précisément, tout ou une partie de ses éléments. L'initialisation se réalise de la même manière que pour les structures, c'est-à-dire à l'aide d'une liste d'initialisation, séquentielle ou sélective.

IV.3.1.2.1. Initialisation séquentielle

IV.3.1.2.1.1. Initialisation avec une longueur explicite L'initialisation séquentielle permet de spécifier une valeur pour un ou plusieurs membres du tableau en partant du premier élément. Ainsi, l'exemple ci-dessous initialise les trois membres du tableau avec les valeurs 1, 2 et 3.

```
1 int tab[3] = { 1, 2, 3 };
```

IV.3.1.2.1.2. Initialisation avec une longueur implicite Lorsque vous initialisez un tableau, il vous est permis d'omettre la longueur de celui-ci, car le compilateur sera capable d'en déterminer la taille en comptant le nombre d'éléments présents dans la liste d'initialisation. Ainsi, l'exemple ci-dessous est correct et définit un tableau de trois `int` valant respectivement 1, 2 et 3.

```
1 int tab[] = { 1, 2, 3 };
```

IV.3.1.2.2. Initialisation sélective

IV.3.1.2.2.1. Initialisation avec une longueur explicite Comme pour les structures, il est par ailleurs possible de désigner spécifiquement les éléments du tableau que vous souhaitez initialiser. Cette initialisation sélective est réalisée à l'aide du numéro du ou des éléments.



Faites attention ! Les éléments sont numérotés à partir de *zéro*. Nous y viendrons dans la section suivante.

L'exemple ci-dessous définit un tableau de trois `int` et initialise le troisième élément.

```
1 int tab[3] = { [2] = 3 };
```

IV.3.1.2.2.2. Initialisation avec une longueur implicite Dans le cas où la longueur du tableau n'est pas précisée, le compilateur déduira la taille du tableau du plus grand indice utilisé lors de l'initialisation sélective. Partant, le code ci-dessous crée un tableau de cent `int`.

```
1 int tab[] = { [0] = 42, [1] = 64, [99] = 100 };
```



Comme pour les structures, dans le cas où vous ne fournissez pas un nombre suffisant de valeurs, les éléments oubliés seront initialisés à zéro ou, s'il s'agit de pointeurs, seront des pointeurs nuls.



Également, il est possible de mélanger initialisations séquentielles et sélectives. Dans un tel cas, l'initialisation séquentielle reprend au dernier élément désigné par une initialisation sélective. Dès lors, le code ci-dessous définit un tableau de dix `int` et initialise le neuvième élément à 9 et le dixième à 10.

```
1 int tab[] = { [8] = 9, 10 };
```



Pour un tableau de structures, la liste d'initialisation comportera elle-même une liste d'initialisation pour chaque structure composant le tableau, par exemple comme ceci.

```
1 struct temps tab[2] = { { 12, 45, 50.6401 }, { 13, 30, 35.480 } } ;
```

Notez par ailleurs que, inversement, une structure peut comporter des tableaux comme membres.

IV.3.1.3. Accès aux éléments d'un tableau

L'accès aux éléments d'un tableau se réalise à l'aide d'un **indice**, un nombre entier correspondant à la position de chaque élément dans le tableau (premier, deuxième, troisième, etc). Cependant, il y a une petite subtilité : *les indices commencent toujours à zéro*. Ceci tient à une raison historique : les performances étant limitées à l'époque de la conception du langage C, il était impératif d'éviter des calculs inutiles, y compris lors de la compilation¹.

Plus précisément, l'accès aux différents éléments d'un tableau est réalisé à l'aide de l'adresse de son premier élément, à laquelle est ajouté l'indice. En effet, étant donné que tous les éléments ont la même taille et se suivent en mémoire, leurs adresses peuvent se calculer à l'aide de l'adresse du premier élément et d'un décalage par rapport à celle-ci (l'indice, donc).

Or, si le premier indice n'est pas zéro, mais par exemple un, cela signifie que le compilateur doit soustraire une unité à chaque indice lors de la compilation pour retrouver la bonne adresse, ce qui implique des calculs supplémentaires, chose impensable quand les ressources sont limitées.

IV. Agrégats, mémoire et fichiers

Prenons un exemple avec un tableau composé de `int` (ayant une taille de quatre octets) et dont le premier élément est placé à l'adresse 1008. Si vous déterminez à la main les adresses de chaque élément, vous obtiendrez ceci.

Indice	Adresse de l'élément
0	1008 (1008 + 0)
1	1012 (1008 + 4)
2	1016 (1008 + 8)
3	1020 (1008 + 12)
4	1024 (1008 + 16)
5	1028 (1008 + 20)
...	...

En fait, il est possible de reformuler ceci à l'aide d'une multiplication entre l'indice et la taille d'un `int`.

Indice	Adresse de l'élément
0	1008 (1008 + (0 * 4))
1	1012 (1008 + (1 * 4))
2	1016 (1008 + (2 * 4))
3	1020 (1008 + (3 * 4))
4	1024 (1008 + (4 * 4))
5	1028 (1008 + (5 * 4))
...	...

Nous pouvons désormais formaliser mathématiquement tout ceci en posant T la taille d'un élément du tableau, i l'indice de cet élément, et A l'adresse de début du tableau (l'adresse du premier élément, donc). L'adresse de l'élément d'indice i s'obtient en calculant $A + T \times i$. Ceci étant posé, voyons à présent comment mettre tout cela en œuvre en C.

IV.3.1.3.1. Le premier élément

Pour commencer, nous avons besoin de l'adresse du premier élément du tableau. Celle-ci s'obtient en fait d'une manière plutôt contre-intuitive : lorsque vous utilisez une variable de type tableau dans une expression, celle-ci est convertie implicitement en un pointeur sur son premier élément. Comme vous pouvez le constater dans l'exemple qui suit, nous pouvons utiliser la variable `tab` comme nous l'aurions fait s'il s'agissait d'un pointeur.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int tab[3] = { 1, 2, 3 };
6
7     printf("Premier élément : %d\n", *tab);
8     return 0;
9 }
```

Résultat

```
1 Premier élément : 1
```



Notez toutefois qu'il n'est pas possible d'affecter une valeur à une variable de type tableau (nous y viendrons bientôt). Ainsi, le code suivant est incorrect.

```
1 int t1[3];
2 int t2[3];
3
4 t1 = t2; /* Incorrect */
```

La règle de conversion implicite comprend néanmoins deux exceptions : l'opérateur `&` et l'opérateur `sizeof`.

Lorsqu'il est appliqué à une variable de type tableau, l'opérateur `&` produit comme résultat l'adresse du premier élément du tableau. Si vous exécutez le code ci-dessous, vous constaterez que les deux expressions donnent un résultat identique.

```
1 #include <stdio.h>
2
3
```

```

4 int main(void)
5 {
6     int tab[3];
7
8     printf("%p == %p\n", (void *)tab, (void *)&tab);
9     return 0;
10 }

```

i

Notez toutefois que si l'adresse référencée par les deux pointeurs est identique, leurs types sont différents. `tab` est un pointeur sur `int` et `&tab` est un pointeur sur *un tableau* de 3 `int`. Nous en parlerons lorsque nous verrons les tableaux multidimensionnels un peu plus tard dans ce chapitre.

Dans le cas où une expression de type tableau est fournie comme opérande de l'opérateur `sizeof`, le résultat de celui-ci sera bien la taille totale du tableau (en multiplés) et non la taille d'un pointeur.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int tab[3];
7     int *ptr;
8
9     printf("sizeof tab = %zu\n", sizeof tab);
10    printf("sizeof ptr = %zu\n", sizeof ptr);
11    return 0;
12 }

```

Résultat

```

1 sizeof tab = 12
2 sizeof ptr = 8

```

Cette propriété vous permet d'obtenir le nombre d'éléments d'un tableau à l'aide de l'expression suivante.

```

1 sizeof tab / sizeof tab[0]

```



Notez que pour obtenir la taille d'un type tableau, la syntaxe est la suivante.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("sizeof int[3] = %zu\n", sizeof(int[3]));
7     printf("sizeof double[42] = %zu\n", sizeof(double[42]));
8     return 0;
9 }
```

Résultat

```

1 sizeof int[3] = 12
2 sizeof double[42] = 336
```

IV.3.1.3.2. Les autres éléments

Pour accéder aux autres éléments, il va nous falloir ajouter la position de l'élément voulu à l'adresse du premier élément et ensuite utiliser l'adresse obtenue. Toutefois, recourir à la formule présentée au-dessus ne marchera pas car, en C, les pointeurs sont typés. Dès lors, lorsque vous additionnez un nombre à un pointeur, le compilateur multiplie automatiquement ce nombre par la taille du type d'objet référencé par le pointeur. Ainsi, pour un tableau de `int`, l'expression `tab + 1` est implicitement convertie en `tab + sizeof(int)`.

Voici un exemple affichant la valeur de tous les éléments d'un tableau.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int tab[3] = { 1, 2, 3 };
7
8     printf("Premier élément : %d\n", *tab);
9     printf("Deuxième élément : %d\n", *(tab + 1));
10    printf("Troisième élément : %d\n", *(tab + 2));
11    return 0;
12 }
```

Résultat

```
1 Premier élément : 1
2 Deuxième élément : 2
3 Troisième élément : 3
```

L'expression `*(tab + i)` étant quelque peu lourde, il existe un opérateur plus concis pour réaliser cette opération : l'opérateur `[]`. Celui-ci s'utilise de cette manière.

```
1 expression[indice]
```

Ce qui est équivalent à l'expression suivante.

```
1 *(expression + indice)
```

L'exemple suivant est donc identique au précédent.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int tab[3] = { 1, 2, 3 };
7
8     printf("Premier élément : %d\n", tab[0]);
9     printf("Deuxième élément : %d\n", tab[1]);
10    printf("Troisième élément : %d\n", tab[2]);
11    return 0;
12 }
```

IV.3.1.4. Parcours et débordement

Une des erreurs les plus fréquentes en C consiste à dépasser la taille d'un tableau, ce qui est appelé un cas de **débordement** (*overflow* en anglais). En effet, si vous tentez d'accéder à un objet qui ne fait pas partie de votre tableau, vous réalisez un accès mémoire non autorisé, ce qui provoquera un comportement indéfini. Cela arrive généralement lors d'un parcours de tableau à l'aide d'une boucle.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int tableau[5] = { 784, 5, 45, -12001, 8 };
7     int somme = 0;
8
9     for (unsigned i = 0; i <= 5; ++i)
10         somme += tableau[i];
11
12     printf("%d\n", somme);
13     return 0;
14 }
```

Le code ci-dessus est volontairement erroné et tente d'accéder à un élément qui se situe au-delà du tableau. Ceci provient de l'utilisation de l'opérateur `<=` à la place de l'opérateur `<` ce qui entraîne un tour de boucle avec `i` qui est égal à 5, alors que le dernier indice du tableau doit être quatre.



N'oubliez pas : les indices d'un tableau commencent *toujours* à zéro. En conséquence, les indices valides d'un tableau de n éléments vont de 0 à $n - 1$.

IV.3.1.5. Tableaux et fonctions

IV.3.1.5.1. Passage en argument

Étant donné qu'un tableau peut être utilisé comme un pointeur sur son premier élément, lorsque vous passez un tableau en argument d'une fonction, celle-ci reçoit un pointeur vers le premier élément du tableau. Le plus souvent, il vous sera nécessaire de passer également la taille du tableau afin de pouvoir le parcourir.

Le code suivant utilise une fonction pour parcourir un tableau d'entiers et afficher la valeur de chacun de ses éléments.

```
1 #include <stdio.h>
2
3
4 void affiche_tableau(int *tab, unsigned taille)
5 {
6     for (unsigned i = 0; i < taille; ++i)
7         printf("tab[%u] = %d\n", i, tab[i]);
8 }
9
```

```

10
11 int main(void)
12 {
13     int tab[5] = { 2, 45, 67, 89, 123 };
14
15     affiche_tableau(tab, 5);
16     return 0;
17 }

```

Résultat

```

1 tab[0] = 2
2 tab[1] = 45
3 tab[2] = 67
4 tab[3] = 89
5 tab[4] = 123

```



Notez qu'il existe une syntaxe alternative pour déclarer un paramètre de type tableau héritée du langage B (voyez la section suivante).

```

1 void affiche_tableau(int tab[], unsigned taille)

```

Toutefois, nous vous conseillons de recourir à la première écriture, cette dernière étant plus explicite.

IV.3.1.5.2. Retour de fonction

De la même manière que pour le passage en argument, retourner un tableau revient à retourner un pointeur sur le premier élément de celui-ci. Toutefois, n'oubliez pas les problématiques de classe de stockage ! Si vous retournez un tableau de classe de stockage automatique, vous fournissez à la fonction appelante un pointeur vers un objet qui n'existe plus (puisque l'exécution de la fonction appelée est terminée).

```

1 #include <stdio.h>
2
3
4 int *tableau(void)
5 {
6     int tab[5] = { 1, 2, 3, 4, 5 };

```

```

7
8     return tab;
9 }
10
11
12 int main(void)
13 {
14     int *p = tableau(); /* Incorrect */
15
16     printf("%d\n", p[0]);
17     return 0;
18 }

```

IV.3.2. La vérité sur les tableaux

Nous vous avons dit qu'il n'est pas possible d'affecter une valeur à une variable de type tableau.

```

1 int t1[3];
2 int t2[3];
3
4 t1 = t2; /* Incorrect */

```

Dans cette section, nous allons vous expliquer pourquoi, mais pour cela, il va nous falloir faire un peu d'histoire. 🍊

IV.3.2.1. Un peu d'histoire

Le prédécesseur du langage C était le langage B. Lorsque le développement du C a commencé, un des objectifs était de le rendre autant que possible compatible avec le B, afin de ne pas devoir (trop) modifier les codes existants (un code écrit en B pourrait ainsi être compilé avec un compilateur C sans ou avec peu de modifications). Or, en B, un tableau se définissait comme suit.

```

1 auto tab[3];

```

i

Le langage B était un langage non typé, ce qui explique l'absence de type dans la définition. Le mot-clé `auto` (toujours présent en langage C, mais devenu obsolète) servait à indiquer que la variable définie était de classe de stockage automatique.

1. Pour les curieux, vous pouvez lire [ce billet](#) (en anglais) qui explique en détail les raisons de ce choix.

IV. Agrégats, mémoire et fichiers

Toutefois, à la différence du langage C, cette définition crée un tableau de trois éléments *et* un pointeur initialisé avec l'adresse du premier élément. Ainsi, pour créer un pointeur, il suffisait de définir une variable comme un tableau de taille nulle.

```
1 auto ptr[];
```

Le langage C, toujours en gestation, avait repris ce mode de fonctionnement. Cependant, les structures sont arrivées et les problèmes avec. En effet, prenez ce bout de code.

```
1 #include <stdio.h>
2
3 struct exemple {
4     int tab[3];
5 };
6
7
8 struct exemple exemple_init(void)
9 {
10     struct exemple init = { { 1, 2, 3 } };
11
12     return init;
13 }
14
15
16 int main(void)
17 {
18     struct exemple s = exemple_init();
19
20     printf("%d\n", s.tab[0]);
21     return 0;
22 }
```

La fonction `exemple_init()` retourne une structure qui est utilisée pour initialiser la variable de la fonction `main()`. Dans un tel cas, comme pour n'importe quelle variable, le contenu de la première structure sera intégralement copié dans la deuxième. Le souci, c'est que si une définition de tableau crée un tableau *et* un pointeur initialisé avec l'adresse du premier élément de celui-ci, alors il est nécessaire de modifier le champ `tab` de la structure `s` lors de la copie sans quoi son champ `tab` pointera vers le tableau de la structure `init` (qui n'existera plus puisque de classe de stockage automatique) et non vers le sien. Voilà qui complexifie la copie de structures, particulièrement si sa définition comprend plusieurs tableaux possiblement imbriqués...

Pour contourner ce problème, les concepteurs du langage C ont imaginé une solution (tordue) qui est à l'origine d'une certaine confusion dans l'utilisation des tableaux : une variable de type tableau *ne sera plus un pointeur*, mais sera *convertie en un pointeur sur son premier élément lors de son utilisation*.

IV.3.2.2. Conséquences de l'absence d'un pointeur

Étant donné qu'il n'y a plus de pointeur alloué, la copie de structures s'en trouve simplifiée et peut être réalisée sans opération particulière (ce qui était l'objectif recherché).

Toutefois, cela entraîne une autre conséquence : il n'est plus possible d'assigner une valeur à une variable de type tableau, seuls ses éléments peuvent se voir affecter une valeur. Ainsi, le code suivant est incorrect puisqu'il n'y a aucun pointeur pour recevoir l'adresse du premier élément du tableau `t2`.

```
1 int t1[3];
2 int t2[3];
3
4 t1 = t2; /* Incorrect */
```

Également, puisqu'une variable de type tableau n'est plus un pointeur, celle-ci n'a pas d'adresse. Dès lors, lorsque l'opérateur `&` est appliqué à une variable de type tableau, le résultat sera l'adresse du premier élément du tableau puisque seuls les éléments du tableau ont une adresse.

IV.3.3. Les tableaux multidimensionnels

Jusqu'à présent, nous avons travaillé avec des tableaux linéaires, c'est-à-dire dont les éléments se suivaient les uns à la suite des autres. Il s'agit de tableaux dit à une dimension ou **unidimensionnels**.

Cependant, certaines données peuvent être représentées plus simplement sous la forme de tableaux à deux dimensions (autrement dit, organisées en lignes et en colonnes). C'est par exemple le cas des images (non vectorielles) qui sont des matrices de pixels ou, plus simplement, d'une grille de Sudoku qui est organisée en neuf lignes et en neuf colonnes.

Le langage C vous permet de créer et de gérer ce type de tableaux dit **multidimensionnels** (en fait, des tableaux de tableaux) et ce, bien au-delà de deux dimensions.

IV.3.3.1. Définition

La définition d'un tableau multidimensionnel se réalise de la même manière que celle d'un tableau unidimensionnel si ce n'est que vous devez fournir la taille des différentes dimensions.

Par exemple, si nous souhaitons définir un tableau de `int` de vingt lignes et trente-cinq colonnes, nous procéderons comme suit.

```
1 int tab[20][35];
```

De même, pour un tableau de `double` à trois dimensions.

```
1 double tab[3][4][5];
```

IV.3.3.2. Initialisation

IV.3.3.2.1. Initialisation avec une longueur explicite

L'initialisation d'un tableau multidimensionnel s'effectue à l'aide d'une liste d'initialisation comprenant elle-même des listes d'initialisations.

```
1 int t1[2][2] = { { 1, 2 }, { [0] = 3, [1] = 4 } };
2 int t2[2][2][2] = { { { 1, 2 }, { 3, 4 } }, { { 5, 6 }, { 7, 8 } }
    };
```

IV.3.3.2.2. Initialisation avec une longueur implicite

Lorsque vous initialisez un tableau multidimensionnel, il vous est permis d'omettre la taille de la *première dimension*. La taille des autres dimensions doit en revanche être spécifiée, le compilateur ne pouvant déduire la taille de toutes les dimensions.

```
1 int t1[][2] = { { 1, 2 }, { [0] = 3, [1] = 4 } };
2 int t2[][2][2] = { { { 1, 2 }, { 3, 4 } }, { { 5, 6 }, { 7, 8 } }
    };
```



Comme pour les tableaux unidimensionnels, dans le cas où vous ne fournissez pas un nombre suffisant de valeurs, les éléments omis seront initialisés à zéro ou, s'il s'agit de pointeurs, seront des pointeurs nuls.

IV.3.3.3. Utilisation

Techniquement, un tableau multidimensionnel est un tableau dont les éléments sont eux-mêmes des tableaux. Dès lors, vous avez besoin d'autant d'indices qu'il y a de dimensions. Par exemple, pour un tableau à deux dimensions, vous avez besoin d'un premier indice pour accéder à l'élément souhaité du premier tableau, mais comme cet élément est lui-même un tableau, vous devez utiliser un second indice pour sélectionner un élément de celui-ci. Illustration.

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int tab[2][2] = { { 1, 2 }, { 3, 4 } };
7
8      printf("tab[0][0] = %d\n", tab[0][0]);
9      printf("tab[0][1] = %d\n", tab[0][1]);
10     printf("tab[1][0] = %d\n", tab[1][0]);
11     printf("tab[1][1] = %d\n", tab[1][1]);
12     return 0;
13 }

```

Résultat

```

1  tab[0][0] = 1
2  tab[0][1] = 2
3  tab[1][0] = 3
4  tab[1][1] = 4

```

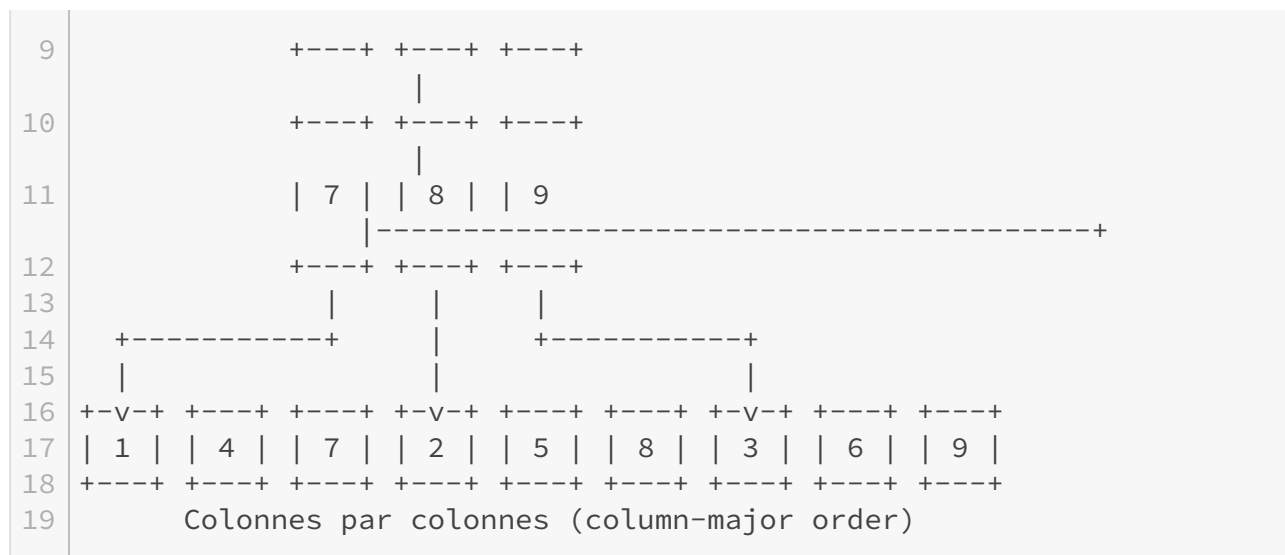
IV.3.3.3.1. Représentation en mémoire

Techniquement, les données d'un tableau multidimensionnel sont stockées les unes à côté des autres en mémoire : elles sont rassemblées dans un tableau à une seule dimension. Si les langages comme le FORTRAN mémorisent les colonnes les unes après les autres (*column-major order* en anglais), le C mémorise les tableaux lignes par lignes (*row-major order*).

```

1      Un exemple de tableau
2      à deux dimensions
3
4      Lignes par lignes
5      (row-major order)
6
7      +---+ +---+ +---+ +---+ +---+ +---+ +---+ +---+
8      | 1 | | 2 | | 3 | ---> 1 | | 2 | | 3 | | 4 | | 5 | | 6
9      | | 7 | | 8 | | 9 |
10     +---+ +---+ +---+ +---+ +---+ +---+ +---+
11     +---+ +---+ +---+ +---+
12     +---+ +---+ +---+
13     |
14     | 4 | | 5 | | 6 | -----+
15     |

```



Le calcul d'adresse à effectuer est une généralisation du calcul vu au chapitre précédent.

IV.3.3.4. Parcours

Le même exemple peut être réalisé à l'aide de deux boucles imbriquées afin de parcourir le tableau.

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int tab[2][2] = { { 1, 2 }, { 3, 4 } };
7
8      for (unsigned i = 0; i < 2; ++i)
9          for (unsigned j = 0; j < 2; ++j)
10             printf("tab[%u][%u] = %d\n", i, j, tab[i][j]);
11
12     return 0;
13 }
```

IV.3.3.5. Tableaux multidimensionnels et fonctions

IV.3.3.5.1. Passage en argument

Souvenez-vous : sauf exceptions, un tableau est converti en un pointeur sur son premier élément. Dès lors, qu'obtenons-nous lors du passage d'un tableau à deux dimensions en argument d'une fonction ? Le premier élément du tableau est un tableau, donc un pointeur sur... Un tableau (hé oui). 🍊

La syntaxe d'un pointeur sur tableau est la suivante.

```
1 type (*identificateur)[taille];
```

Vous remarquerez la présence de parenthèses autour du symbole `*` et de l'identificateur afin de signaler au compilateur qu'il s'agit d'un pointeur sur un tableau et non d'un tableau de pointeurs. Également, notez que la taille du tableau référencé doit être spécifiée. En effet, sans celle-ci, le compilateur ne pourrait pas opérer correctement le calcul d'adresses puisqu'il ne connaîtrait pas la taille des éléments composant le tableau référencé par le pointeur.

La même logique peut être appliquée pour créer des pointeurs sur des tableaux de tableaux.

```
1 type (*identificateur)[N][M];
```

Et ainsi de suite jusqu'à ce que mort s'ensuive... 🍊

L'exemple ci-dessous illustre ce qui vient d'être dit en utilisant une fonction pour afficher le contenu d'un tableau à deux dimensions de `int`.

```
1 #include <stdio.h>
2
3
4 void affiche_tableau(int (*tab)[2], unsigned n, unsigned m)
5 {
6     for (unsigned i = 0; i < n; ++i)
7         for (unsigned j = 0; j < m; ++j)
8             printf("tab[%u][%u] = %d\n", i, j, tab[i][j]);
9 }
10
11
12 int main(void)
13 {
14     int tab[2][2] = { { 1, 2 }, { 3, 4 } };
15
16     affiche_tableau(tab, 2, 2);
17     return 0;
18 }
```

IV.3.3.5.2. Retour de fonction

Même remarque que pour les tableaux unidimensionnels : attention à la classe de stockage ! Pour le reste, nous vous laissons admirer la syntaxe particulièrement dégoûtante d'une fonction retournant un pointeur sur un tableau de deux `int`.

```

1  #include <stdio.h>
2
3
4  int (*tableau(void))[2] /* Ouh ! Que c'est laid ! */
5  {
6      int tab[2][2] = { { 1, 2 }, { 3, 4 } };
7
8      return tab;
9  }
10
11
12 int main(void)
13 {
14     int (*p)[2] = tableau(); /* Incorrect */
15
16     printf("%d\n", p[0][0]);
17     return 0;
18 }

```

IV.3.4. Les tableaux littéraux

Comme pour les structures, il est possible de construire des tableaux « littéraux » c'est-à-dire sans besoin de déclarer une variable. Comme pour les structures, ces tableaux littéraux ont une classe de stockage automatique, ils sont donc détruits à la fin du bloc qui les a vu naître.

La syntaxe est similaire à celle des structures.

```

1  (type[taille]) { élément1, élément2 }

```

Comme pour l'initialisation de n'importe quel tableau, il est possible d'omettre la taille afin que le compilateur la calcule en se basant sur les éléments fournis. L'exemple ci-dessous utilise un tableau littéral en lieu et place d'une variable.

```

1  #include <stdio.h>
2
3
4  static void affiche_tableau(int *tab, unsigned taille)
5  {
6      for (unsigned i = 0; i < taille; ++i)
7          printf("tab[%u] = %d\n", i, tab[i]);
8  }
9
10

```

```
11 int
12 main(void)
13 {
14     affiche_tableau((int[]) { 1, 2, 3, 4, 5 }, 5);
15     return 0;
16 }
```

Résultat

```
1 tab[0] = 1
2 tab[1] = 2
3 tab[2] = 3
4 tab[3] = 4
5 tab[4] = 5
```

IV.3.5. Exercices

IV.3.5.1. Somme des éléments

Réalisez une fonction qui calcule la somme de tous les éléments d'un tableau de `int`.

👁 Contenu masqué n°26

IV.3.5.2. Maximum et minimum

Créez deux fonctions : une qui retourne le plus petit élément d'un tableau de `int` et une qui renvoie le plus grand élément d'un tableau de `int`.

👁 Contenu masqué n°27

IV.3.5.3. Recherche d'un élément

Construisez une fonction qui teste la présence d'une valeur dans un tableau de `int`. Celle-ci retournera 1 si un ou plusieurs éléments du tableau sont égaux à la valeur recherchée, 0 sinon.

👁 Contenu masqué n°28

IV.3.5.4. Inverser un tableau

Produisez une fonction qui inverse le contenu d'un tableau (le premier élément devient le dernier, l'avant-dernier le deuxième et ainsi de suite).

IV.3.5.4.1. Indice

👁 Contenu masqué n°29

IV.3.5.4.2. Correction

👁 Contenu masqué n°30

IV.3.5.5. Produit des lignes

Composez une fonction qui calcule le produit de la somme des éléments de chaque ligne d'un tableau de `int` à deux dimensions (ce tableau comprend cinq lignes et cinq colonnes).

👁 Contenu masqué n°31

IV.3.5.6. Triangle de Pascal

Les triangles de Pascal sont des objets mathématiques amusants. Voici une petite animation qui vous expliquera le fonctionnement de ceux-ci.

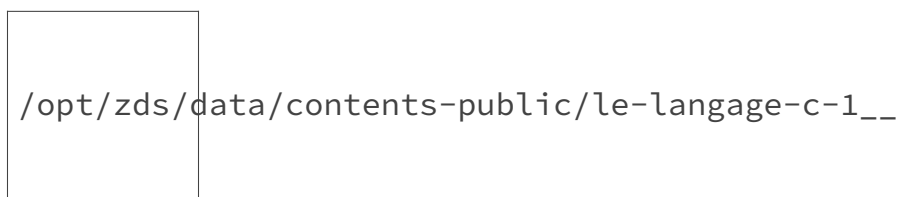


FIGURE IV.3.1. – Explication des triangles de Pascal en image

Votre objectif va être de réaliser un programme qui affiche un triangle de Pascal de la taille souhaitée par l'utilisateur. Pour ce faire, nous allons diviser le triangle en lignes afin de pouvoir le représenter sous la forme d'un tableau à deux dimensions. Chaque élément du tableau se verra attribué soit une valeur du triangle soit zéro pour signaler qu'il n'est pas utilisé.

La première chose que nous allons faire est donc définir un tableau à deux dimensions (nous fixerons la taille des dimensions à dix) dont tous les éléments sont initialisés à zéro. Ensuite,

IV. Agrégats, mémoire et fichiers

nous allons demander à l'utilisateur d'entrer la taille du triangle qu'il souhaite obtenir (celle-ci ne devra pas être supérieure aux dimensions du tableau).

À présent, passons à la fonction de création du triangle de Pascal. Celle-ci devra mettre en œuvre l'algorithme suivant.

```
1 N = taille du triangle de Pascal fournie par l'utilisateur
2
3 Mettre la première case du tableau à 1
4
5 Pour i = 1, i < N, i = i + 1
6     Mettre la première case de la ligne à 1
7
8     Pour j = 1, j < i, j = j + 1
9         La case [i,j] prend la valeur [i - 1, j - 1] + [i - 1, j]
10
11 Mettre la dernière case de la ligne à 1
```

Enfin, il vous faudra écrire une petite fonction pour afficher le tableau ainsi créé.

Bon courage ! 🍊

👁 Contenu masqué n°32

Conclusion

En résumé

1. Un tableau est une suite contiguë d'objets de même type ;
2. La taille d'un tableau est précisée à l'aide d'une expression entière ;
3. L'accès aux éléments d'un tableau se réalise à l'aide d'un indice ;
4. Le premier élément d'un tableau a l'indice zéro ;
5. Il n'est pas possible d'affecter une valeur à une variable de type tableau, seul ses éléments sont modifiables ;
6. Un tableau est implicitement converti en un pointeur sur son premier élément, sauf lorsqu'il est l'opérande de l'opérateur `sizeof` ou `&`.

Contenu masqué

Contenu masqué n°26

```
1 int somme(int *tableau, unsigned taille)
2 {
3     int res = 0 ;
4
5     for (unsigned i = 0; i < taille; ++i)
6         res += tableau[i];
7
8     return res;
9 }
```

[Retourner au texte.](#)

Contenu masqué n°27

```
1 int minimum(int *tab, unsigned taille)
2 {
3     int min = tab[0];
4
5     for (unsigned i = 1; i < taille; ++i)
6         if (tab[i] < min)
7             min = tab[i];
8
9     return min;
10 }
11
12
13 int maximum(int *tab, unsigned taille)
14 {
15     int max = tab[0];
16
17     for (unsigned i = 1; i < taille; ++i)
18         if (tab[i] > max)
19             max = tab[i];
20
21     return max;
22 }
```

[Retourner au texte.](#)

Contenu masqué n°28

```
1 int find(int *tab, unsigned taille, int val)
2 {
3     for (unsigned i = 0; i < taille; ++i)
4         if (tab[i] == val)
5             return 1;
6
7     return 0;
8 }
```

[Retourner au texte.](#)

Contenu masqué n°29

Pensez à la fonction `swap()` présentée dans le chapitre sur les pointeurs. [Retourner au texte.](#)

Contenu masqué n°30

```
1 void swap(int *pa, int *pb)
2 {
3     int tmp = *pa;
4     *pa = *pb;
5     *pb = tmp;
6 }
7
8
9 void invert(int *tab , unsigned taille)
10 {
11     for (unsigned i = 0; i < (taille / 2); ++i)
12         swap(tab + i , tab + taille - 1 - i);
13 }
```

[Retourner au texte.](#)

Contenu masqué n°31

```
1 int produit(int (*tab)[5])
2 {
3     int res = 1;
4
5     for (unsigned i = 0; i < 5; ++i)
6     {
7         int tmp = 0;
8
9         for (unsigned j = 0; j < 5; ++j)
10             tmp += tab[i][j];
11
12         res *= tmp;
13     }
14
15     return res;
16 }
```

[Retourner au texte.](#)

Contenu masqué n°32

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 void triangle_pascal(int (*tab)[10], unsigned taille)
6 {
7     tab[0][0] = 1;
8
9     for (unsigned i = 1; i < taille; ++i)
10    {
11        tab[i][0] = 1;
12
13        for (unsigned j = 1; j < i; ++j)
14            tab[i][j] = tab[i - 1][j - 1] + tab[i - 1][j];
15
16        tab[i][i] = 1;
17    }
18 }
19
20
21 void print_triangle(int (*tab)[10], unsigned taille)
22 {
```

```
23     for (unsigned i = 0; i < taille; ++i)
24     {
25         for (int sp = taille - 1 - i; sp > 0; --sp)
26             printf(" ");
27
28         for (unsigned j = 0; j < taille; ++j)
29             if (tab[i][j] != 0)
30                 printf("%d ", tab[i][j]);
31
32         printf("\n");
33     }
34 }
35
36
37 int main(void)
38 {
39     int tab[10][10] = { { 0 } };
40     unsigned taille;
41
42     printf("Taille du triangle: ");
43
44     if (scanf("%u", &taille) != 1)
45     {
46         printf("Mauvaise saisie\n");
47         return EXIT_FAILURE;
48     }
49     else if (taille > 10)
50     {
51         printf("La taille ne doit pas être supérieure à 10\n");
52         return EXIT_FAILURE;
53     }
54
55     triangle_pascal(tab, taille);
56     print_triangle(tab, taille);
57     return 0;
58 }
```

[Retourner au texte.](#)

IV.4. Les chaînes de caractères

Introduction

Dans ce chapitre, nous allons apprendre à manipuler du texte ou, en langage C, des **chaînes de caractères**.

IV.4.1. Qu'est-ce qu'une chaîne de caractères ?

Dans le chapitre sur les variables, nous avons mentionné le type `char`. Pour rappel, nous vous avons dit que le type `char` servait surtout au stockage de caractères, mais que comme ces derniers étaient stockés dans l'ordinateur sous forme de nombres, il était également possible d'utiliser ce type pour mémoriser des nombres.

Le seul problème, c'est qu'une variable de type `char` ne peut stocker qu'une seule lettre, ce qui est insuffisant pour stocker une phrase ou même un mot. Si nous voulons mémoriser un texte, nous avons besoin d'un outil pour rassembler plusieurs lettres dans un seul objet, manipulable dans notre langage. Cela tombe bien, nous en avons justement découvert un au chapitre précédent : les tableaux.

C'est ainsi que le texte est géré en C : sous forme de tableaux de `char` appelés **chaînes de caractères** (*strings* en anglais).

IV.4.1.1. Représentation en mémoire

Néanmoins, il y a une petite subtilité. Une chaîne de caractères est un peu plus qu'un tableau : c'est un objet à part entière qui doit être manipulable directement. Or, ceci n'est possible que si nous connaissons sa taille.

IV.4.1.1.1. Avec une taille intégrée

Dans certains langages de programmation, les chaînes de caractères sont stockées sous la forme d'un tableau de `char` auquel est adjoint un entier pour indiquer sa longueur. Plus précisément, lors de l'allocation du tableau, le compilateur réserve un élément supplémentaire pour conserver la taille de la chaîne. Ainsi, il est aisé de parcourir la chaîne et de savoir quand la fin de celle-ci est atteinte. De telles chaînes de caractères sont souvent appelées des *Pascal strings*, s'agissant d'une convention apparue avec le langage de programmation Pascal.

IV.4.1.1.2. Avec une sentinelle

Toutefois, une telle technique limite la taille des chaînes de caractères à la capacité du type entier utilisé pour mémoriser la longueur de la chaîne. Dans la majorité des cas, il s'agit d'un `unsigned char`, ce qui donne une limite de 255 caractères maximum sur la plupart des machines. Pour ne pas subir cette limitation, les concepteurs du langage C ont adopté une autre solution : la fin de la chaîne de caractères sera indiquée par un caractère spécial, en l'occurrence zéro (noté `'\0'`). Les chaînes de caractères qui fonctionnent sur ce principe sont appelées *null terminated strings*, ou encore *C strings*.

Cette solution a toutefois deux inconvénients :

- la taille de chaque chaîne doit être calculée en la parcourant jusqu'au caractère nul ;
- le programmeur doit s'assurer que chaque chaîne qu'il construit se termine bien par un caractère nul.

IV.4.2. Définition, initialisation et utilisation

IV.4.2.1. Définition

Définir une chaîne de caractères, c'est avant tout définir un tableau de `char`, ce que nous avons vu au chapitre précédent. L'exemple ci-dessous définit un tableau de vingt-cinq `char`.

```
1 char tab[25];
```

IV.4.2.2. Initialisation

Il existe deux méthodes pour initialiser une chaîne de caractères :

- de la même manière qu'un tableau ;
- à l'aide d'une chaîne de caractères littérale.

IV.4.2.2.1. Avec une liste d'initialisation

IV.4.2.2.1.1. Initialisation avec une longueur explicite Comme pour n'importe quel tableau, l'initialisation se réalise à l'aide d'une liste d'initialisation. L'exemple ci-dessous définit donc un tableau de vingt-cinq `char` et initialise les sept premiers avec la suite de lettres « Bonjour ».

```
1 char chaine[25] = { 'B', 'o', 'n', 'j', 'o', 'u', 'r' };
```

Étant donné que seule une partie des éléments est initialisée, les autres sont implicitement mis à zéro, ce qui nous donne une chaîne de caractères valide puisqu'elle est bien terminée par un

caractère nul. Faites cependant attention à ce qu'il y ait *toujours* de la place pour un caractère nul.

IV.4.2.2.1.2. Initialisation avec une longueur implicite Dans le cas où vous ne spécifiez pas de taille lors de la définition, il vous faudra ajouter le caractère nul à la fin de la liste d'initialisation pour obtenir une chaîne valide.

```
1 char chaine[] = { 'B', 'o', 'n', 'j', 'o', 'u', 'r', '\0' };
```

IV.4.2.2.2. Avec une chaîne littérale

Bien que tout à fait valide, cette première solution est toutefois assez fastidieuse. Aussi, il en existe une seconde : recourir à une **chaîne de caractères littérale** pour initialiser un tableau. Une chaîne de caractères littérale est une suite de caractères entourée par le symbole ". Nous en avons déjà utilisé auparavant comme argument des fonctions `printf()` et `scanf()`.

Techniquement, une chaîne littérale est un tableau de `char` terminé par un caractère nul. Elles peuvent donc s'utiliser comme n'importe quel autre tableau. Si vous exécutez le code ci-dessous, vous remarquerez que l'opérateur `sizeof` retourne bien le nombre de caractères composant la chaîne littérale (n'oubliez pas de compter le caractère nul) et que l'opérateur `[]` peut effectivement leur être appliqué.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("%zu\n", sizeof "Bonjour");
7     printf("%c\n", "Bonjour"[3]);
8     return 0;
9 }
```

Résultat

```
1 8
2 j
```

Ces chaînes de caractères littérales peuvent également être utilisées à la place des listes d'initialisation. En fait, il s'agit de la troisième et dernière exception à la règle de conversion implicite des tableaux.

IV.4.2.2.2.1. Initialisation avec une longueur explicite Dans le cas où vous spécifiez la taille de votre tableau, faites bien attention à ce que celui-ci dispose de suffisamment de place pour accueillir la chaîne entière, c'est-à-dire les caractères qui la composent *et* le caractère nul.

```
1 char chaine[25] = "Bonjour";
```

IV.4.2.2.2.2. Initialisation avec une longueur implicite L'utilisation d'une chaîne littérale pour initialiser un tableau dont la taille n'est pas spécifiée vous évite de vous soucier du caractère nul puisque celui-ci fait partie de la chaîne littérale.

```
1 char chaine[] = "Bonjour";
```

IV.4.2.2.3. Utilisation de pointeurs

Nous vous avons dit que les chaînes littérales n'étaient rien d'autre que des tableaux de `char` terminés par un caractère nul. Dès lors, comme pour n'importe quel tableau, il vous est loisible de les référencer à l'aide de pointeurs.

```
1 char *ptr = "Bonjour";
```

Néanmoins, les chaînes littérales sont des *constantes*, il vous est donc impossible de les modifier. L'exemple ci-dessous est donc incorrect.

```
1 int main(void)
2 {
3     char *ptr = "bonjour";
4
5     ptr[0] = 'B'; /* Incorrect */
6     return 0;
7 }
```



Notez bien la différence entre les exemples précédents qui initialisent un tableau avec le contenu d'une chaîne littérale (il y a donc copie de la chaîne littérale) et cet exemple qui initialise un pointeur avec l'adresse du premier élément d'une chaîne littérale.

IV.4.2.3. Utilisation

Pour le reste, une chaîne de caractères s'utilise comme n'importe quel autre tableau. Aussi, pour modifier son contenu, il vous faudra accéder à ses éléments un à un.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     char chaine[25] = "Bonjour";
7
8     printf("%s\n", chaine);
9     chaine[0] = 'A';
10    chaine[1] = 'u';
11    chaine[2] = ' ';
12    chaine[3] = 'r';
13    chaine[4] = 'e';
14    chaine[5] = 'v';
15    chaine[6] = 'o';
16    chaine[7] = 'i';
17    chaine[8] = 'r';
18    chaine[9] = '\0'; /* N'oubliez pas le caractère nul ! */
19    printf("%s\n", chaine);
20    return 0;
21 }
```

Résultat

```
1 Bonjour
2 Au revoir
```

IV.4.3. Afficher et récupérer une chaîne de caractères

Les chaînes littérales n'étant rien d'autre que des tableaux de `char`, il vous est possible d'utiliser des chaînes de caractères là où vous employiez des chaînes littérales. Ainsi, les deux exemples ci-dessous afficheront la même chose.

```
1 #include <stdio.h>
2
3
```

```
4 int main(void)
5 {
6     char chaine[] = "Bonjour\n";
7
8     printf(chaine);
9     return 0;
10 }
```

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("Bonjour\n");
7     return 0;
8 }
```

Résultat

```
1 Bonjour
```

Toutefois, les fonctions `printf()` et `scanf()` disposent d'un indicateur de conversion vous permettant d'afficher ou de demander une chaîne de caractères : `s`.

IV.4.3.1. Printf

L'exemple suivant illustre l'utilisation de cet indicateur de conversion avec `printf()` et affiche la même chose que les deux codes précédents.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     char chaine[] = "Bonjour";
7
8     printf("%s\n", chaine);
9     return 0;
10 }
```

Résultat

1	Bonjour
---	---------



Afin d'éviter de mauvaises surprises et notamment le cas où un utilisateur vous fournirait en entrée une chaîne de caractères comprenant des indicateurs de conversions, préférez *toujours* afficher une chaîne à l'aide de l'indicateur de conversion `s` et non directement comme premier argument de `printf()`.

IV.4.3.2. Scanf

Le même indicateur de conversion peut être utilisé avec `scanf()` pour demander à l'utilisateur d'entrer une chaîne de caractères. Cependant, un problème se pose : étant donné que nous devons créer un tableau de taille finie pour accueillir la saisie de l'utilisateur, nous devons impérativement limiter la longueur des données que nous fournit l'utilisateur.

Pour éviter ce problème, il est possible de spécifier une taille maximale à la fonction `scanf()`. Pour ce faire, il vous suffit de placer un nombre entre le symbole `%` et l'indicateur de conversion `s`. L'exemple ci-dessous demande à l'utilisateur d'entrer son prénom (limité à 254 caractères) et affiche ensuite celui-ci.



La fonction `scanf()` ne décompte pas le caractère nul final de la limite fournie ! Il vous est donc nécessaire de lui indiquer la taille de votre tableau diminuée de un.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      char chaine[255];
8
9      printf("Quel est votre prénom ? ");
10
11     if (scanf("%254s", chaine) != 1)
12     {
13         printf("Erreur lors de la saisie\n");
14         return EXIT_FAILURE;
15     }
16

```

```

17     printf("Bien le bonjour %s !\n", chaine);
18     return 0;
19 }

```

Résultat

```

1 Quel est votre prénom ? Albert
2 Bien le bonjour Albert !

```

IV.4.3.2.1. Chaîne de caractères avec des espaces

Sauf qu'en fait, l'indicateur `s` signifie : « la plus longue suite de caractère ne comprenant *pas* d'espaces » (les espaces étant ici entendus comme une suite d'un ou plusieurs des caractères suivants : `' '`, `'\f'`, `'\n'`, `'\r'`, `'\t'`, `'\v'`)...

Autrement dit, si vous entrez « Bonjour tout le monde », la fonction `scanf()` va s'arrêter au mot « Bonjour », ce dernier étant suivi par un espace.

?

Comment peut-on récupérer une chaîne complète alors ? 🍌

Eh bien, il va falloir nous débrouiller avec l'indicateur `c` de la fonction `scanf()` et réaliser nous même une fonction employant ce dernier au sein d'une boucle. Ainsi, nous pouvons par exemple créer une fonction recevant un pointeur sur `char` et la taille du tableau référencé qui lit un caractère jusqu'à être arrivé à la fin de la ligne ou à la limite du tableau.

?

Et comment sait-on que la lecture est arrivée à la fin d'une ligne ?

La fin de ligne est indiquée par le caractère `\n`.

Avec ceci, vous devriez pouvoir construire une fonction adéquate.

Allez, *hop*, au boulot et faites gaffe aux retours d'erreurs ! 🍌

👁 Contenu masqué n°33

i

Gardez bien cette fonction sous le coude, nous allons en avoir besoin pour la suite. 🍌



Si vous tentez d'utiliser plusieurs fois la fonction `lire_ligne()` en entrant un nombre de caractères supérieur à la limite fixée, vous constaterez que les caractères excédentaires ne sont pas perdus, mais sont lus lors d'un appel subséquent à `lire_ligne()`. Ce comportement sera expliqué plus tard lorsque nous verrons les fichiers.

IV.4.4. Lire et écrire depuis et dans une chaîne de caractères

S'il est possible d'afficher et récupérer une chaîne de caractères, il est également possible de lire depuis une chaîne et d'écrire dans une chaîne. À cette fin, deux fonctions qui devraient vous sembler familières existent : `snprintf()` et `sscanf()`.

IV.4.4.1. La fonction `snprintf`

```
1 int snprintf(char *chaîne, size_t taille, char *format, ...);
```



Les trois points à la fin du prototype de la fonction signifient que celle-ci attend un nombre variable d'arguments. Nous verrons ce mécanisme plus en détail dans la troisième partie du cours.

La fonction `snprintf()` est identique à la fonction `printf()` mis à part que celle-ci écrit les données produites dans une chaîne de caractères au lieu de les afficher à l'écran. Elle écrit au maximum `taille - 1` caractères dans la chaîne `chaîne` et ajoute elle-même un caractère nul à la fin. La fonction retourne le nombre de caractères écrit *sans compter le caractère nul final* ou bien un nombre négatif en cas d'erreur. Toutefois, dans le cas où la chaîne de destination est trop petite pour accueillir toutes les données à écrire, la fonction retourne le nombre de caractères qui *aurait dû être écrit* (hors caractère nul, à nouveau) si la limite n'avait pas été atteinte.

Cette fonction peut vous permettre, entre autres, d'écrire un nombre dans une chaîne de caractères.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     char chaîne[8];
8     unsigned long long n = 64;
9 }
```

```

10     if (snprintf(chaine, sizeof chaine, "%llu", n) < 0)
11     {
12         printf("Erreur lors de la conversion\n");
13         return EXIT_FAILURE;
14     }
15
16     printf("chaîne : %s\n", chaine);
17     n = 18446744073709551615LLU;
18     int ret = snprintf(chaine, sizeof chaine, "%llu", n);
19
20     if (ret < 0)
21     {
22         printf("Erreur lors de la conversion\n");
23         return EXIT_FAILURE;
24     }
25     if (ret + 1 > sizeof chaine) // snprintf ne compte pas le
        caractère nul
26         printf(
            "La chaîne est trop petite, elle devrait avoir une taille de : %d\n", ret +
            1);
27
28     printf("chaîne : %s\n", chaine);
29     return 0;
30 }

```

Résultat

```

1 chaîne : 64
2 La chaîne est trop petite, elle devrait avoir une taille de :
  21
3 chaîne : 1844674

```

Comme vous le voyez, dans le cas où il n'y a pas assez d'espace, la fonction `snprintf()` retourne bien le nombre de caractères qui auraient dû être écrit (ici 20, caractère nul exclu). Cette propriété peut d'ailleurs être utilisée pour calculer la taille nécessaire d'une chaîne à l'avance en précisant une taille de zéro. Dans un tel cas, aucun caractère ne sera écrit, un pointeur nul peut alors être passé en argument.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)

```



```

6 {
7     unsigned long long n = 18446744073709551615LLU;
8     int ret = snprintf(NULL, 0, "%llu", n);
9
10    if (ret < 0)
11    {
12        printf("Erreur lors de la conversion\n");
13        return EXIT_FAILURE;
14    }
15    else
16        printf("Il vous faut une chaîne de %d caractère(s)\n", ret
17              + 1);
18    return 0;
19 }

```

Résultat

```
1 Il vous faut une chaîne de 21 caractère(s)
```

IV.4.4.2. La fonction `sscanf`

```
1 int sscanf(char *chaine, char *format, ...);
```

La fonction `sscanf()` est identique à la fonction `scanf()` si ce n'est que celle-ci extrait les données depuis une chaîne de caractères plutôt qu'en provenance d'une saisie de l'utilisateur. Cette dernière retourne le nombre de conversions réussies *ou* un nombre inférieur si elles n'ont pas toutes été réalisées *ou* enfin un nombre négatif en cas d'erreur.

Voici un exemple d'utilisation.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     char chaine[10];
8     int n;
9
10    if (sscanf("5 abcd", "%d %9s", &n, chaine) != 2)

```

```

11     {
12         printf("Erreur lors de l'examen de la chaîne\n");
13         return EXIT_FAILURE;
14     }
15
16     printf("%d %s\n", n, chaine);
17     return 0;
18 }

```

Résultat

```
1 5 abcd
```

IV.4.5. Les classes de caractères

Pour terminer cette partie théorique sur une note un peu plus légère, sachez que la bibliothèque standard fournit un en-tête `<ctype.h>` qui permet de classer les caractères. Onze fonctions sont ainsi définies.

```

1 int isalnum(int c);
2 int isalpha(int c);
3 int isblank(int c);
4 int iscntrl(int c);
5 int isdigit(int c);
6 int isgraph(int c);
7 int islower(int c);
8 int isprint(int c);
9 int ispunct(int c);
10 int isspace(int c);
11 int isupper(int c);
12 int isxdigit(int c);

```

Chacune d'entre elles attend en argument un caractère et retourne un nombre positif ou zéro suivant que le caractère fourni appartienne ou non à la catégorie déterminée par la fonction.

Fonction	Catégorie	Description
isupper	Majuscule	Détermine si le caractère est une lettre majuscule

islower	Minuscule	Détermine si le caractère est une lettre minuscule
isdigit	Chiffre décimal	Détermine si le caractère est un chiffre décimal
isxdigit	Chiffre hexadécimal	Détermine si le caractère est un chiffre hexadécimal
isblank	Espace	Détermine si le caractère est un espace blanc
isspace	Espace	Détermine si le caractère est un espace
isctrl	Contrôle	Détermine si le caractère est un caractère de contrôle
ispunct	Ponctuation	Détermine si le caractère est un signe de ponctuation
isalpha	Alphabétique	Détermine si le caractère est une lettre alphabétique
isalnum	Alphanumérique	Détermine si le caractère est une lettre alphabétique ou un chiffre
isgraph	Graphique	Détermine si le caractère est un caractère graphique
isprint	Affichable	Détermine si le caractère est un caractère imprimable



La suite `<barre_droite>` symbolise le caractère `|`.

Le tableau ci-dessus vous présente chacune de ces onze fonctions ainsi que les ensembles de caractères qu'elles décrivent. La colonne « par défaut » vous détaille leur comportement en cas d'utilisation de la **locale C**. Nous reviendrons sur les *locales* dans la troisième partie de ce cours ; pour l'heure, considérez que ces fonctions ne retournent un nombre positif que si un des caractères de leur ensemble « par défaut » leur est fourni en argument.

L'exemple ci-dessous utilise la fonction `isdigit()` pour déterminer si l'utilisateur a bien entré une suite de chiffres.

```

1  #include <ctype.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  /* lire_ligne() */
6
7
8  int main(void)
9  {
10     char suite[255];
11
12     if (!lire_ligne(suite, sizeof suite))
13     {
14         printf("Erreur lors de la saisie.\n");
15         return EXIT_FAILURE;
16     }
17
18     for (unsigned i = 0; suite[i] != '\0'; ++i)
19         if (!isdigit(suite[i]))
20         {
21             printf(
22                 "Veuillez entrer une suite de chiffres.\n");
23             return EXIT_FAILURE;
24         }
25     printf("C'est bien une suite de chiffres.\n");
26     return 0;
27 }

```

Résultat

```

1  122334
2  C'est bien une suite de chiffres.
3
4  5678a
5  Veuillez entrer une suite de chiffres.

```



Notez que cet en-tête fournit également deux fonctions : `tolower()` et `toupper()` retournant respectivement la version minuscule ou majuscule de la lettre entrée. Dans le cas où un caractère autre qu'une lettre est entrée (ou que celle-ci est déjà en minuscule ou en majuscule), la fonction retourne celui-ci.

IV.4.6. Exercices

IV.4.6.1. Palindrome

Un palindrome est un texte identique lorsqu'il est lu de gauche à droite et de droite à gauche. Ainsi, le mot *radar* est un palindrome, de même que les phrases *engage le jeu que je le gagne* et *élu par cette crapule*. Normalement, il n'est pas tenu compte des accents, trémas, cédilles ou des espaces. Toutefois, pour cet exercice, nous nous contenterons de vérifier si un mot donné est un palindrome.

☞ Contenu masqué n°34

IV.4.6.2. Compter les parenthèses

Écrivez un programme qui lit une ligne et vérifie que chaque parenthèse ouvrante est bien refermée par la suite.

```
1 Entrez une ligne : printf("%zu\n", sizeof(int);  
2 Il manque des parenthèses.
```

☞ Contenu masqué n°35

Conclusion

En résumé

1. Une chaîne de caractères est un tableau de `char` terminé par un élément nul ;
2. Une chaîne littérale ne peut pas être modifiée ;
3. L'indicateur de conversion `s` permet d'afficher ou de récupérer une chaîne de caractères ;
4. Les fonctions `sscanf()` et `snprintf()` permet de lire ou d'écrire depuis et dans une chaîne de caractères ;
5. Les fonctions de l'en-tête `<ctype.h>` permettent de classer les caractères.

Contenu masqué

Contenu masqué n°33

```
1 #include <stdbool.h>
2 #include <stddef.h>
3 #include <stdio.h>
4
5
6 bool lire_ligne(char *chaine, size_t max)
7 {
8     size_t i;
9
10    for (i = 0; i < max - 1; ++i)
11    {
12        char c;
13
14        if (scanf("%c", &c) != 1)
15            return false;
16        else if (c == '\\n')
17            break;
18
19        chaine[i] = c;
20    }
21
22    chaine[i] = '\\0';
23    return true;
24 }
25
26
27 int main(void)
28 {
29     char chaine[255];
30
31     printf("Quel est votre prénom ? ");
32
33     if (lire_ligne(chaine, sizeof chaine))
34         printf("Bien le bonjour %s !\\n", chaine);
35
36     return 0;
37 }
```

Résultat

```
1 Quel est votre prénom ? Charles Henri
2 Bien le bonjour Charles Henri !
```

[Retourner au texte.](#)

Contenu masqué n°34

```
1 bool palindrome(char *s)
2 {
3     size_t len = 0;
4
5     while (s[len] != '\0')
6         ++len;
7
8     for (size_t i = 0; i < len; ++i)
9         if (s[i] != s[len - 1 - i])
10            return false;
11
12     return true;
13 }
```

[Retourner au texte.](#)

Contenu masqué n°35

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* lire_ligne() */
5
6
7 int main(void)
8 {
9     char s[255];
10
11     printf("Entrez une ligne : ");
12
13     if (!lire_ligne(s, sizeof s))
14     {
```

```
15     printf("Erreur lors de la saisie.\n");
16     return EXIT_FAILURE;
17 }
18
19 int n = 0;
20
21 for (char *t = s; *t != '\0'; ++t)
22 {
23     if (*t == '(')
24         ++n;
25     else if (*t == ')')
26         --n;
27
28     if (n < 0)
29         break;
30 }
31
32 if (n == 0)
33     printf("Le compte est bon.\n");
34 else
35     printf("Il manque des parenthèses.\n");
36
37 return 0;
38 }
```

[Retourner au texte.](#)

IV.5. TP : l'en-tête <string.h>

Introduction

Le dernier chapitre ayant été riche en nouveautés, posons-nous un instant pour mettre en pratique tout cela.

IV.5.1. Préparation

Dans le chapitre précédent, nous vous avons dit qu'une chaîne de caractères se manipulait comme un tableau, à savoir en parcourant ses éléments un à un. Cependant, si cela s'arrêtait là, cela serait assez gênant. En effet, la moindre opération sur une chaîne nécessiterait d'accéder à ses différents éléments, que ce soit une modification, une copie, une comparaison, etc. Heureusement pour nous, la bibliothèque standard fournit une suite de fonctions nous permettant de réaliser plusieurs opérations de base sur des chaînes de caractères. Ces fonctions sont déclarées dans l'en-tête <string.h>.

L'objectif de ce TP sera de réaliser une version pour chacune des fonctions de cet en-tête qui vont vous être présentées. 🍊

IV.5.1.1. strlen

```
1 size_t strlen(char *chaine);
```

La fonction `strlen()` vous permet de connaître la taille d'une chaîne fournie en argument. Celle-ci retourne une valeur de type `size_t`. Notez bien que la longueur retournée ne comprend *pas* le caractère nul. L'exemple ci-dessous affiche la taille de la chaîne « Bonjour ».

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     printf("Longueur : %zu\n", strlen("Bonjour"));
8     return 0;
```

```
9 }
```

Résultat

```
1 Longueur : 7
```

IV.5.1.2. strcpy

```
1 char *strcpy(char *destination, char *source);
```

La fonction `strcpy()` copie le contenu de la chaîne `source` dans la chaîne `destination`, caractère nul compris. La fonction retourne l'adresse de `destination`. L'exemple ci-dessous copie la chaîne « Au revoir » dans la chaîne `chaine`.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     char chaine[25] = "Bonjour\n";
8
9     strcpy(chaine, "Au revoir");
10    printf("%s\n", chaine);
11    return 0;
12 }
```

Résultat

```
1 Au revoir
```



La fonction `strcpy()` n'effectue *aucune* vérification. Vous devez donc vous assurer que la chaîne de destination dispose de suffisamment d'espace pour accueillir la chaîne qui doit être copiée (caractère nul compris!).

IV.5.1.3. strcat

```
1 char *strcat(char *destination, char *source);
```

La fonction `strcat()` ajoute le contenu de la chaîne `source` à celui de la chaîne `destination`, caractère nul compris. La fonction retourne l'adresse de `destination`. L'exemple ci-dessous ajoute la chaîne « tout le monde » au contenu de la chaîne `chaine`.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     char chaine[25] = "Bonjour";
8
9     strcat(chaine, " tout le monde");
10    printf("%s\n", chaine);
11    return 0;
12 }
```

Résultat

```
1 Bonjour tout le monde
```



Comme `strcpy()`, la fonction `strcat()` n'effectue *aucune* vérification. Vous devez donc vous assurer que la chaîne de destination dispose de suffisamment d'espace pour accueillir la chaîne qui doit être ajoutée (caractère nul compris!).

IV.5.1.4. strcmp

```
1 int strcmp(char *chaine1, char *chaine2);
```

La fonction `strcmp()` compare deux chaînes de caractères. Cette fonction retourne :

- une valeur positive si la première chaîne est « plus grande » que la seconde ;
- zéro si elles sont égales ;
- une valeur négative si la seconde chaîne est « plus grande » que la première.



Ne vous inquiétez pas au sujet des valeurs positives et négatives, nous y reviendrons en temps voulu lorsque nous aborderons les notions de jeux de caractères et d'encodages dans la troisième partie du cours. En attendant, effectuez simplement une comparaison entre les deux caractères qui diffèrent.

```

1  #include <stdio.h>
2  #include <string.h>
3
4
5  int main(void)
6  {
7      char chaine1[] = "Bonjour";
8      char chaine2[] = "Au revoir";
9
10     if (strcmp(chaine1, chaine2) == 0)
11         printf("Les deux chaînes sont identiques\n");
12     else
13         printf("Les deux chaînes sont différentes\n");
14
15     return 0;
16 }
```

Résultat

```
1 Les deux chaînes sont différentes
```

IV.5.1.5. strchr

```
1 char *strchr(char *chaine, int ch);
```

La fonction `strchr()` recherche la présence du caractère `ch` dans la chaîne `chaine`. Si celui-ci est rencontré, la fonction retourne l'adresse de la première occurrence de celui-ci au sein de la chaîne. Dans le cas contraire, la fonction renvoie un pointeur nul.

```

1  #include <stddef.h>
2  #include <stdio.h>
3  #include <string.h>
4
```

```

5
6 int main(void)
7 {
8     char chaine[] = "Bonjour";
9     char *p = strchr(chaine, 'o');
10
11     if (p != NULL)
12         printf("La chaîne '%s' contient la lettre %c\n", chaine,
13                *p);
14
15     return 0;
16 }

```

Résultat

```

1 La chaîne 'Bonjour' contient la lettre o

```

IV.5.1.6. strpbrk

```

1 char *strpbrk(char *chaine, char *liste);

```

La fonction `strpbrk()` recherche la présence *d'un* des caractères de la chaîne `liste` dans la chaîne `chaine`. Si un de ceux-ci est rencontré, la fonction retourne l'adresse de la première occurrence au sein de la chaîne. Dans le cas contraire, la fonction renvoie un pointeur nul.

```

1 #include <stddef.h>
2 #include <stdio.h>
3 #include <string.h>
4
5
6 int main(void)
7 {
8     char chaine[] = "Bonjour";
9     char *p = strpbrk(chaine, "aeiouy");
10
11     if (p != NULL)
12         printf("La première voyelle de la chaîne '%s' est : %c\n",
13                chaine, *p);
14
15     return 0;
16 }

```

Résultat

```
1 La première voyelle de la chaîne `Bonjour' est : o
```

IV.5.1.7. strstr

```
1 char *strstr(char *chaine1, char *chaine2);
```

La fonction `strstr()` recherche la présence de la chaîne `chaine2` dans la chaîne `chaine1`. Si celle-ci est rencontrée, la fonction retourne l'adresse de la première occurrence de celle-ci au sein de la chaîne. Dans le cas contraire, la fonction renvoie un pointeur nul.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <string.h>
4
5
6 int main(void)
7 {
8     char chaine[] = "Bonjour";
9     char *p = strstr(chaine, "jour");
10
11     if (p != NULL)
12         printf("La chaîne `%s' contient la chaîne `%s'\n", chaine,
13             p);
14     return 0;
15 }
```

Résultat

```
1 La chaîne `Bonjour' contient la chaîne `jour'
```

Cela étant dit, à vous de jouer. 🍊



Par convention, nous commencerons le nom de nos fonctions par la lettre « x » afin d'éviter des collisions avec ceux de l'en-tête `<string.h>`.

IV.5.2. Correction

Bien, l'heure de la correction est venue. 🍌

IV.5.2.1. strlen

👁️ Contenu masqué n°36

IV.5.2.2. strcpy

👁️ Contenu masqué n°37

IV.5.2.3. strcat

👁️ Contenu masqué n°38

IV.5.2.4. strcmp

👁️ Contenu masqué n°39

IV.5.2.5. strchr

👁️ Contenu masqué n°40

IV.5.2.6. strpbrk

👁️ Contenu masqué n°41

IV.5.2.7. strstr

👁 Contenu masqué n°42

IV.5.3. Pour aller plus loin : strtok

Pour les plus aventureux d'entre-vous, nous vous proposons de réaliser en plus la mise en œuvre de la fonction `strtok()` qui est un peu plus complexe que ses congénères. 🍊

```
1 char *strtok(char *chaine, char *liste);
```

La fonction `strtok()` est un peu particulière : elle divise la chaîne `chaine` en une suite de sous-chaînes délimitée par *un* des caractères présents dans la chaîne `liste`. En fait, cette dernière remplace les caractères de la chaîne `liste` par des caractères nuls dans la chaîne `chaine` (elle modifie donc la chaîne `chaine` !) et renvoie les différentes sous-chaînes ainsi créées au fur et à mesure de ses appels.

Lors des appels subséquents, la chaîne `chaine` doit être un pointeur nul afin de signaler à `strtok()` que nous souhaitons la sous-chaîne suivante. S'il n'y a plus de sous-chaîne ou qu'aucun caractère de la chaîne `liste` n'est trouvé, celle-ci retourne un pointeur nul.

L'exemple ci-dessous tente de scinder la chaîne « un, deux, trois » en trois sous-chaînes.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <string.h>
4
5
6 int main(void)
7 {
8     char chaine[] = "un, deux, trois";
9     char *p = strtok(chaine, ", ");
10
11     for (size_t i = 0; p != NULL; ++i)
12     {
13         printf("%zu : %s\n", i, p);
14         p = strtok(NULL, ", ");
15     }
16
17     return 0;
18 }
```


Résultat

1	0 : un
2	1 : deux
3	2 : trois

La fonction `strtok()` étant un peu plus complexe que les autres, voici une petite marche à suivre pour réaliser cette fonction.

- regarder si la chaîne fournie (ou la sous-chaîne suivante) contient, à *son début*, des caractères présents dans la seconde chaîne. Si oui, ceux-ci doivent être passés ;
- si la fin de la (sous-)chaîne est atteinte, retourner un pointeur nul ;
- parcourir la chaîne fournie (ou la sous-chaîne courante) jusqu'à rencontrer un des caractères composant la seconde chaîne. Si un caractère est rencontré, le remplacer par un caractère nul, conserver la position actuelle dans la chaîne et retourner la sous-chaîne ainsi créée (*sans* les éventuels caractères passés au début) ;
- si la fin de la (sous-)chaîne est atteinte, retourner la (sous-)chaîne (*sans* les éventuels caractères passés au début).

i

Vous aurez besoin d'une variable de classe de stockage statique pour réaliser cette fonction. Également, l'instruction `goto` pourra vous être utile.

IV.5.3.1. Correction

👁 Contenu masqué n°43

Contenu masqué

Contenu masqué n°36

```

1 size_t xstrlen(char *chaine)
2 {
3     size_t i;
4
5     for (i = 0; chaine[i] != '\0'; ++i)
6         ;
7
8     return i;

```

```
9 }
```

[Retourner au texte.](#)

Contenu masqué n°37

```
1 char *xstrcpy(char *destination, char *source)
2 {
3     size_t i;
4
5     for (i = 0; source[i] != '\0'; ++i)
6         destination[i] = source[i];
7
8     destination[i] = '\0'; // N'oubliez pas le caractère nul final
9     !
10    return destination;
11 }
```

[Retourner au texte.](#)

Contenu masqué n°38

```
1 char *xstrcat(char *destination, char *source)
2 {
3     size_t i = 0, j = 0;
4
5     while (destination[i] != '\0')
6         ++i;
7
8     while (source[j] != '\0') {
9         destination[i] = source[j];
10        ++i;
11        ++j;
12    }
13
14    destination[i] = '\0'; // N'oubliez pas le caractère nul
15    final !
16    return destination;
17 }
```

[Retourner au texte.](#)

Contenu masqué n°39

```
1 int xstrcmp(char *chaine1, char *chaine2)
2 {
3     while (*chaine1 == *chaine2)
4     {
5         if (*chaine1 == '\0')
6             return 0;
7
8         ++chaine1;
9         ++chaine2;
10    }
11
12    return (*chaine1 < *chaine2) ? -1 : 1;
13 }
```

[Retourner au texte.](#)

Contenu masqué n°40

```
1 char *xstrchr(char *chaine, int ch)
2 {
3     while (*chaine != '\0')
4         if (*chaine == ch)
5             return chaine;
6         else
7             ++chaine;
8
9     return NULL;
10 }
```

[Retourner au texte.](#)

Contenu masqué n°41

```
1 char *xstrpbrk(char *chaine, char *liste)
2 {
3     while (*chaine != '\0')
4     {
5         char *p = liste;
6
7         while (*p != '\0')
```

```
8      {
9          if (*chaine == *p)
10             return chaine;
11
12         ++p;
13     }
14
15     ++chaine;
16 }
17
18 return NULL;
19 }
```

[Retourner au texte.](#)

Contenu masqué n°42

```
1 char *xstrstr(char *chaine1, char *chaine2)
2 {
3     while (*chaine1 != '\0')
4     {
5         char *s = chaine1;
6         char *t = chaine2;
7
8         while (*s != '\0' && *t != '\0')
9         {
10             if (*s != *t)
11                 break;
12
13             ++s;
14             ++t;
15         }
16
17         if (*t == '\0')
18             return chaine1;
19
20         ++chaine1;
21     }
22
23     return NULL;
24 }
```

[Retourner au texte.](#)

Contenu masqué n°43

```
1 char *xstrtok(char *chaine, char *liste)
2 {
3     static char *dernier;
4     char *base = (chaine != NULL) ? chaine : dernier;
5
6     if (base == NULL)
7         return NULL;
8
9     separateur_au_debut:
10    for (char *sep = liste; *sep != '\0'; ++sep)
11        if (*base == *sep)
12            {
13                ++base;
14                goto separateur_au_debut;
15            }
16
17    if (*base == '\0')
18    {
19        dernier = NULL;
20        return NULL;
21    }
22
23    for (char *s = base; *s != '\0'; ++s)
24        for (char *sep = liste; *sep != '\0'; ++sep)
25            if (*s == *sep)
26                {
27                    *s = '\0';
28                    dernier = s + 1;
29                    return base;
30                }
31
32    dernier = NULL;
33    return base;
34 }
```

[Retourner au texte.](#)

IV.6. L'allocation dynamique

Introduction

Il est à présent temps d'aborder la troisième et dernière utilisation majeure des pointeurs : l'allocation dynamique de mémoire.

Comme nous vous l'avons dit dans le chapitre sur les pointeurs, il n'est pas toujours possible de savoir quelle quantité de mémoire sera utilisée par un programme. Par exemple, si vous demandez à l'utilisateur de vous fournir un tableau, vous devrez lui fixer une limite, ce qui pose deux problèmes :

- la limite en elle-même, qui ne convient peut-être pas à votre utilisateur ;
- l'utilisation excessive de mémoire du fait que vous réservez un tableau d'une taille fixée à l'avance.

De plus, si vous utilisez un tableau de classe de stockage statique, alors cette quantité de mémoire superflue sera inutilisable jusqu'à la fin de votre programme.

Or, un ordinateur ne dispose que d'une quantité limitée de mémoire vive, il est donc important de ne pas en réserver abusivement. L'allocation dynamique permet de réserver une partie de la mémoire vive inutilisée pour stocker des données et de libérer cette même partie une fois qu'elle n'est plus nécessaire.

IV.6.1. La notion d'objet

Jusqu'à présent, nous avons toujours recouru au système de types et de variables du langage C pour stocker nos données sans jamais vraiment nous soucier de ce qui se passait « en dessous ». Il est à présent temps de lever une partie de ce voile en abordant la notion d'**objet**.

En C, un objet est une zone mémoire pouvant contenir des données et est composée d'un ou plusieurs multipliets contigus. En fait, tous les types du langage C manipulent des objets. La différence entre les types tient simplement en la manière dont ils répartissent les données au sein de ces objets, ce qui est appelé leur **représentation** (celle-ci sera abordée dans la troisième partie du cours). Ainsi, la valeur 1 n'est pas représentée de la même manière dans un objet de type `int` que dans un objet de type `double`.

Un objet étant composé d'un ou plusieurs multipliets contigus, il est possible d'en examiner le contenu en lisant ses multipliets un à un. Ceci peut se réaliser en C à l'aide de l'adresse de l'objet et d'un pointeur sur `unsigned char`, les types `signed char`, `char` et `unsigned char` du C ayant *toujours* une taille d'un multipliet. Notez qu'il est *impératif* d'utiliser le type `unsigned char` afin d'éviter des problèmes de conversions.

L'exemple ci-dessous affiche les multipliets composant un objet de type `int` et un objet de type `double` en hexadécimal.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int n = 1;
7     double f = 1.;
8     unsigned char *byte = (unsigned char *)&n;
9
10    for (unsigned i = 0; i < sizeof n; ++i)
11        printf("%x ", byte[i]);
12
13    printf("\n");
14    byte = (unsigned char *)&f;
15
16    for (unsigned i = 0; i < sizeof f; ++i)
17        printf("%x ", byte[i]);
18
19    printf("\n");
20    return 0;
21 }
```

Résultat

1	1 0 0 0
2	0 0 0 0 0 0 f0 3f



Il se peut que vous n'obteniez pas le même résultat que nous, ce dernier dépend de votre machine.

Comme vous le voyez, la représentation de la valeur 1 n'est pas du tout la même entre le type `int` et le type `double`.

IV.6.2. Malloc et consœurs

La bibliothèque standard fournit trois fonctions vous permettant d'allouer de la mémoire : `malloc()`, `calloc()` et `realloc()` et une vous permettant de la libérer : `free()`. Ces quatre fonctions sont déclarées dans l'en-tête `<stdlib.h>`.

IV.6.2.1. malloc

```
1 void *malloc(size_t taille);
```

La fonction `malloc()` vous permet d'allouer un objet de la taille fournie en argument (qui représente un nombre de multipliants) et retourne l'adresse de cet objet sous la forme d'un pointeur générique. En cas d'échec de l'allocation, elle retourne un pointeur nul.



Vous devez *toujours* vérifier le retour d'une fonction d'allocation afin de vous assurer que vous manipulez bien un pointeur valide.

IV.6.2.1.1. Allocation d'un objet

Dans l'exemple ci-dessous, nous réservons un objet de la taille d'un `int`, nous y stockons ensuite la valeur dix et l'affichons. Pour cela, nous utilisons un pointeur sur `int` qui va se voir affecter l'adresse de l'objet ainsi alloué et qui va nous permettre de le manipuler comme nous le ferions s'il référençait une variable de type `int`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    *p = 10;
16    printf("%d\n", *p);
17    return 0;
18 }
```

Résultat

```
1 10
```


IV.6.2.1.2. Allocation d'un tableau

Pour allouer un tableau, vous devez réserver un bloc mémoire de la taille d'un élément multiplié par le nombre d'éléments composant le tableau. L'exemple suivant alloue un tableau de dix `int`, l'initialise et affiche son contenu.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int) * 10);
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    for (unsigned i = 0; i < 10; ++i)
16    {
17        p[i] = i * 10;
18        printf("p[%u] = %d\n", i, p[i]);
19    }
20
21    return 0;
22 }
```

Résultat

```
1 p[0] = 0
2 p[1] = 10
3 p[2] = 20
4 p[3] = 30
5 p[4] = 40
6 p[5] = 50
7 p[6] = 60
8 p[7] = 70
9 p[8] = 80
10 p[9] = 90
```

i

Autrement dit et de manière plus générale : pour allouer dynamiquement un objet de type T , il vous faut créer un pointeur sur le type T qui conservera son adresse.

i

Notez qu'il est possible de simplifier l'expression fournie à `malloc()` en écrivant `sizeof(int[10])` qui a le mérite d'être plus concise et plus claire.

x

La fonction `malloc()` n'effectue *aucune* initialisation, le contenu du bloc alloué est donc indéterminé.

IV.6.2.2. free

```
1 void free(void *ptr);
```

La fonction `free()` libère le bloc précédemment alloué par une fonction d'allocation dont l'adresse est fournie en argument. Dans le cas où un pointeur nul lui est fourni, elle n'effectue aucune opération.

!

Retenez bien la règle suivante : à chaque appel à une fonction d'allocation doit correspondre un appel à la fonction `free()`.

Dès lors, nous pouvons compléter les exemples précédents comme suit.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    *p = 10;
16    printf("%d\n", *p);
17    free(p);
```

```

18     return 0;
19 }

```

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      int *p = malloc(sizeof(int[10]));
8
9      if (p == NULL)
10     {
11         printf("Échec de l'allocation\n");
12         return EXIT_FAILURE;
13     }
14
15     for (unsigned i = 0; i < 10; ++i)
16     {
17         p[i] = i * 10;
18         printf("p[%u] = %d\n", i, p[i]);
19     }
20
21     free(p);
22     return 0;
23 }

```



Remarquez que même si le deuxième exemple alloue un tableau, il n'y a bien eu qu'une seule allocation. Un seul appel à la fonction `free()` est donc nécessaire.

IV.6.2.3. `calloc`

```

1 void *calloc(size_t nombre, size_t taille);

```

La fonction `calloc()` attend deux arguments : le nombre d'éléments à allouer et la taille de chacun de ces éléments. Techniquement, elle revient au même que d'appeler `malloc()` comme suit.

```

1 malloc(nombre * taille);

```

à un détail près : chaque multiplet de l'objet ainsi alloué est initialisé à zéro.



Faites attention : cette initialisation n'est pas similaire à celle des variables de classe de stockage statique ! En effet, le fait que chaque multipliet ait pour valeur zéro ne signifie pas qu'il s'agit de la représentation du zéro dans le type donné. Si cela est par exemple vrai pour les types entiers, ce n'est pas le cas pour les pointeurs (rappelez-vous : un pointeur nul référence une adresse invalide qui dépend de votre système) ou pour les nombres réels (nous y reviendrons plus tard). De manière générale, ne considérez cette initialisation utile que pour les entiers et les chaînes de caractères.

IV.6.2.4. realloc

```
1 void *realloc(void *p, size_t taille);
```

La fonction `realloc()` libère un bloc de mémoire précédemment alloué, en réserve un nouveau de la taille demandée et copie le contenu de l'ancien objet dans le nouveau. Dans le cas où la taille demandée est inférieure à celle du bloc d'origine, le contenu de celui-ci sera copié à hauteur de la nouvelle taille. À l'inverse, si la nouvelle taille est supérieure à l'ancienne, l'excédent n'est pas initialisé.

La fonction attend deux arguments : l'adresse d'un bloc précédemment alloué à l'aide d'une fonction d'allocation et la taille du nouveau bloc à allouer. Elle retourne l'adresse du nouveau bloc ou un pointeur nul en cas d'erreur.

L'exemple ci-dessous alloue un tableau de dix `int` et utilise `realloc()` pour agrandir celui-ci à vingt `int`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int[10]));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    for (unsigned i = 0; i < 10; ++i)
16        p[i] = i * 10;
17
18    int *tmp = realloc(p, sizeof(int[20]));
19
```

```

20     if (tmp == NULL)
21     {
22         free(p);
23         printf("Échec de l'allocation\n");
24         return EXIT_FAILURE;
25     }
26
27     p = tmp;
28
29     for (unsigned i = 10; i < 20; ++i)
30         p[i] = i * 10;
31
32     for (unsigned i = 0; i < 20; ++i)
33         printf("p[%u] = %d\n", i, p[i]);
34
35     free(p);
36     return 0;
37 }

```

Résultat

```

1  p[0] = 0
2  p[1] = 10
3  p[2] = 20
4  p[3] = 30
5  p[4] = 40
6  p[5] = 50
7  p[6] = 60
8  p[7] = 70
9  p[8] = 80
10 p[9] = 90
11 p[10] = 100
12 p[11] = 110
13 p[12] = 120
14 p[13] = 130
15 p[14] = 140
16 p[15] = 150
17 p[16] = 160
18 p[17] = 170
19 p[18] = 180
20 p[19] = 190

```

Remarquez que nous avons utilisé une autre variable, `tmp`, pour vérifier le retour de la fonction `realloc()`. En effet, si nous avions procéder comme ceci.

```
1 p = realloc(p, sizeof(int[20]));
```

Il nous aurait été impossible de libérer le bloc mémoire référencé par `p` en cas d'erreur puisque celui-ci serait devenu un pointeur nul. Il est donc *impératif* d'utiliser une seconde variable afin d'éviter des [fuites de mémoire](#) ☞ .

IV.6.3. Les tableaux multidimensionnels

L'allocation de tableaux multidimensionnels est un petit peu plus complexe que celles des autres objets. Techniquement, il existe deux solutions : l'allocation d'un seul bloc de mémoire (comme pour les tableaux simples) et l'allocation de plusieurs tableaux eux-mêmes référencés par les éléments d'un autre tableau.

IV.6.3.1. Allocation en un bloc

Comme pour un tableau simple, il vous est possible d'allouer un bloc de mémoire dont la taille correspond à la multiplication des longueurs de chaque dimension, elle-même multipliée par la taille d'un élément. Toutefois, cette solution vous contraint à effectuer une partie du calcul d'adresse vous-même puisque vous allouez en fait un seul tableau.

L'exemple ci-dessous illustre ce qui vient d'être dit en allouant un tableau à deux dimensions de trois fois trois `int`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     int *p = malloc(sizeof(int[3][3]));
8
9     if (p == NULL)
10    {
11        printf("Échec de l'allocation\n");
12        return EXIT_FAILURE;
13    }
14
15    for (unsigned i = 0; i < 3; ++i)
16        for (unsigned j = 0; j < 3; ++j)
17        {
18            p[(i * 3) + j] = (i * 3) + j;
19            printf("p[%u][%u] = %d\n", i, j, p[(i * 3) + j]);
20        }
21
```

```

22     free(p);
23     return 0;
24 }

```

Résultat

```

1  p[0][0] = 0
2  p[0][1] = 1
3  p[0][2] = 2
4  p[1][0] = 3
5  p[1][1] = 4
6  p[1][2] = 5
7  p[2][0] = 6
8  p[2][1] = 7
9  p[2][2] = 8

```

Comme vous le voyez, une partie du calcul d'adresse doit être effectuée en multipliant le premier indice par la longueur de la première dimension, ce qui permet d'arriver à la bonne « ligne ». Ensuite, il ne reste plus qu'à sélectionner le bon élément de la « colonne » à l'aide du second indice.

Bien qu'un petit peu plus complexe quant à l'accès aux éléments, cette solution a l'avantage de n'effectuer qu'une seule allocation de mémoire.

i

Dans ce cas ci, la taille du tableau étant connue à l'avance, nous aurions pu définir le pointeur `p` comme un pointeur sur un tableau de 3 `int` et ainsi nous affranchir du calcul d'adresse.

```

1  int (*p)[3] = malloc(sizeof(int[3][3]));

```

IV.6.3.2. Allocation de plusieurs tableaux

La seconde solution consiste à allouer plusieurs tableaux plus un autre qui les référencera. Dans le cas d'un tableau à deux dimensions, cela signifie allouer un tableau de pointeurs dont chaque élément se verra affecter l'adresse d'un tableau également alloué dynamiquement. Cette technique nous permet d'accéder aux éléments des différents tableaux de la même manière que pour un tableau multidimensionnel puisque nous utilisons cette fois plusieurs tableaux.

L'exemple ci-dessous revient au même que le précédent, mais utilise le procédé qui vient d'être décrit. Notez que puisque nous réservons un tableau de pointeurs sur `int`, l'adresse de celui-ci doit être stockée dans un pointeur de pointeur sur `int` (rappelez-vous la règle générale lors de la présentation de la fonction `malloc()`).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      int **p = malloc(sizeof(int*[3]));
8
9      if (p == NULL)
10     {
11         printf("Échec de l'allocation\n");
12         return EXIT_FAILURE;
13     }
14
15     for (unsigned i = 0; i < 3; ++i)
16     {
17         p[i] = malloc(sizeof(int[3]));
18
19         if (p[i] == NULL)
20         {
21             printf("Échec de l'allocation\n");
22             return EXIT_FAILURE;
23         }
24     }
25
26     for (unsigned i = 0; i < 3; ++i)
27         for (unsigned j = 0; j < 3; ++j)
28         {
29             p[i][j] = (i * 3) + j;
30             printf("p[%u][%u] = %d\n", i, j, p[i][j]);
31         }
32
33     for (unsigned i = 0; i < 3; ++i)
34         free(p[i]);
35
36     free(p);
37     return 0;
38 }

```

Si cette solution permet de faciliter l'accès aux différents éléments, elle présente toutefois l'inconvénient de réaliser plusieurs allocations et donc de nécessiter plusieurs appels à la fonction `free()`.

IV.6.4. Les tableaux de longueur variable

À côté de l'allocation dynamique manuelle (matérialisée par des appels aux fonctions `malloc()` et consœurs) que nous venons de voir, il existe une autre solution pour allouer dynamiquement

de la mémoire : les **tableaux de longueur variable** (ou *variable length arrays* en anglais, souvent abrégé en VLA).

IV.6.4.1. Définition

En fait, jusqu'à présent, nous avons toujours défini des tableaux dont la taille était soit déterminée, soit déterminable lors de la compilation.

```
1 int t1[3]; /* Taille déterminée */
2 int t2[] = { 1, 2, 3 }; // Taille déterminable
```

Toutefois, la taille d'un tableau étant spécifiée par une expression entière, il est parfaitement possible d'employer une variable à la place d'une constante entière. Ainsi, le code suivant est parfaitement correct.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int
7 main(void)
8 {
9     size_t n;
10
11     printf("Entrez la taille du tableau : ");
12
13     if (scanf("%zu", &n) != 1)
14     {
15         printf("Erreur lors de la saisie\n");
16         return EXIT_FAILURE;
17     }
18
19     int tab[n];
20
21     for (size_t i = 0; i < n; ++i)
22     {
23         tab[i] = i * 10;
24         printf("tab[%zu] = %d\n", i, tab[i]);
25     }
26
27     return 0;
28 }
```

Résultat

```

1  Entrez la taille du tableau : 10
2  tab[0] = 0
3  tab[1] = 10
4  tab[2] = 20
5  tab[3] = 30
6  tab[4] = 40
7  tab[5] = 50
8  tab[6] = 60
9  tab[7] = 70
10 tab[8] = 80
11 tab[9] = 90

```

Dans le cas où la taille du tableau ne peut pas être déterminée à la compilation, le compilateur va lui-même ajouter des instructions en vue d'allouer et de libérer de la mémoire dynamiquement.



Un tableau de longueur variable ne peut pas être initialisé lors de sa définition. Ceci tient au fait que le compilateur ne peut effectuer aucune vérification quant à la taille de la liste d'initialisation étant donné que la taille du tableau ne sera connue qu'à l'exécution du programme.

```
1 int tab[n] = { 1, 2, 3 }; // Incorrect
```



Une structure ne peut pas contenir de tableaux de longueur variable.

IV.6.4.2. Utilisation

Un tableau de longueur variable s'utilise de la même manière qu'un tableau, avec toutefois quelques ajustements. En effet, s'agissant d'un tableau, s'il est employé dans une expression, il sera converti en un pointeur sur son premier élément. Toutefois, se pose alors la question suivante.

```

1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5

```

```

6 void affiche_tableau(int (*tab)[/* Heu ... ? */], unsigned n,
  unsigned m)
7 {
8     for (unsigned i = 0; i < n; ++i)
9         for (unsigned j = 0; j < m; ++j) {
10             tab[i][j] = i * m + j;
11             printf("tab[%u][%u] = %d\n", i, j, tab[i][j]);
12         }
13 }
14
15
16 int
17 main(void)
18 {
19     size_t n, m;
20
21     printf("Entrez la longueur et la largeur du tableau : ");
22
23     if (scanf("%zu %zu", &n, &m) != 2)
24     {
25         printf("Erreur lors de la saisie\n");
26         return EXIT_FAILURE;
27     }
28
29     int tab[n][m];
30
31     affiche_tableau(tab, n, m);
32     return 0;
33 }

```

Comment peut-on préciser le type du paramètre `tab` étant donné que la taille du tableau ne sera connue qu'à l'exécution ? 🍊

Dans un tel cas, il est nécessaire d'inverser l'ordre des paramètres afin d'utiliser ceux-ci pour préciser au compilateur que le type ne sera déterminé qu'à l'exécution. Partant, la définition de la fonction `affiche_tableau()` devient celle-ci.

```

1 void affiche_tableau(size_t n, size_t m, int (*tab)[m]);
2 {
3     for (unsigned i = 0; i < n; ++i)
4         for (unsigned j = 0; j < m; ++j) {
5             tab[i][j] = i * m + j;
6             printf("tab[%u][%u] = %d\n", i, j, tab[i][j]);
7         }
8 }

```

Ainsi, nous précisons au compilateur que le paramètre `tab` sera un pointeur sur un tableau de `m` `int`, `m` n'étant connu qu'à l'exécution.

IV.6.4.3. Application en dehors des tableaux de longueur variable

Notez que cette syntaxe n'est pas réservée aux tableaux de longueur variable. En effet, si elle est obligatoire dans le cas de leur utilisation, elle peut être employée dans d'autres situations. Si nous reprenons notre exemple d'allocation d'un tableau multidimensionnel en un bloc et le modifions de sorte que l'utilisateur puisse spécifier la taille de ses dimensions, nous pouvons écrire ceci et dès lors éviter le calcul d'adresse manuel.

```
1  #include <stddef.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5
6  int main(void)
7  {
8      size_t n, m;
9
10     printf("Entrez la longueur et la largeur du tableau : ");
11
12     if (scanf("%zu %zu", &n, &m) != 2)
13     {
14         printf("Erreur lors de la saisie\n");
15         return EXIT_FAILURE;
16     }
17
18     int (*p)[m] = malloc(sizeof(int[n][m]));
19
20     if (p == NULL)
21     {
22         printf("Échec de l'allocation\n");
23         return EXIT_FAILURE;
24     }
25
26     for (size_t i = 0; i < n; ++i)
27         for (size_t j = 0; j < m; ++j)
28         {
29             p[i][j] = (i * n) + j;
30             printf("p[%zu][%zu] = %d\n", i, j, p[i][j]);
31         }
32
33     free(p);
34     return 0;
35 }
```

Résultat

```

1 Entrez la longueur et la largeur du tableau : 3 3
2 p[0][0] = 0
3 p[0][1] = 1
4 p[0][2] = 2
5 p[1][0] = 3
6 p[1][1] = 4
7 p[1][2] = 5
8 p[2][0] = 6
9 p[2][1] = 7
10 p[2][2] = 8

```

IV.6.4.4. Limitations

Pourquoi dans ce cas s'ennuyer avec l'allocation dynamique manuelle me direz-vous ? En effet, si le compilateur peut se charger de ce fardeau, autant le lui laisser, non ? *Eh* bien, parce que si cette méthode peut s'avérer salvatrice dans certains cas, elle souffre de limitations par rapport à l'allocation dynamique manuelle.

IV.6.4.4.1. Durée de vie

La première limitation est que la classe de stockage des tableaux de longueur variable est automatique. Finalement, c'est assez logique, puisque le compilateur se charge de tout, il faut bien fixer le moment où la mémoire sera libérée. Du coup, comme pour n'importe quelle variable automatique, il est incorrect de retourner l'adresse d'un tableau de longueur variable puisque celui-ci n'existera plus une fois la fonction terminée.

```

1 int *ptr(size_t n)
2 {
3     int tab[n];
4
5     return tab; /* Incorrect */
6 }

```

IV.6.4.4.2. Goto

La seconde limitation tient au fait qu'une allocation dynamique est réalisée lors de la définition du tableau de longueur variable. En effet, finalement, la définition d'un tableau de longueur variable peut être vue comme suit.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int
7 main(void)
8 {
9     size_t n;
10
11     printf("Entrez la taille du tableau : ");
12
13     if (scanf("%zu", &n) != 1)
14     {
15         printf("Erreur lors de la saisie\n");
16         return EXIT_FAILURE;
17     }
18
19     int tab[n]; /* int *tab = malloc(sizeof(int) * n); */
20
21     /* free(tab); */
22     return 0;
23 }
```

Or, assez logiquement, si l'allocation est passée, typiquement avec une instruction de saut comme `goto`, nous risquons de rencontrer des problèmes... Dès lors, le code suivant est incorrect.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int
7 main(void)
8 {
9     size_t n;
10
11     printf("Entrez la taille du tableau : ");
12
13     if (scanf("%zu", &n) != 1)
14     {
15         printf("Erreur lors de la saisie\n");
16         return EXIT_FAILURE;
17     }
18
19     goto boucle; /* Faux, car on saute l'allocation */
20 }
```

```
21     if (n < 1000)
22     {
23         int tab[n]; /* int *tab = malloc(sizeof(int) * n); */
24
25         boucle:
26         for (size_t i = 0; i < n; ++i)
27             tab[i] = i * 10;
28
29         /* free(tab); */
30     }
31
32     return 0;
33 }
```

Dit plus formellement, il n'est pas permis de sauter après la définition d'un tableau de longueur variable si le saut est effectué en dehors de sa portée.

IV.6.4.4.3. Taille des objets alloués

Une autre limitation vient du fait que la quantité de mémoire pouvant être allouée de cette manière est plus retreinte qu'avec `malloc()` et consœurs. Ceci tient à la manière dont la mémoire est allouée par le compilateur dans le cas des tableaux de longueur variable, qui diffère de celle employée par `malloc()`. Nous n'entrerons pas dans les détails ne s'agissant pas de l'objet de ce cours, mais globalement et de manière générale, *évit*ez d'allouer un tableau de longueur variable si vous avez besoin de plus de 4 194 304 multipl^{ets} (soit le plus souvent environ 4 mégaoctets).

IV.6.4.4.4. Gestion d'erreur

La dernière limitation tient à l'impossibilité de mettre en place une gestion d'erreur. En effet, il n'est pas possible de savoir si l'allocation a réussi ou échoué et votre programme est dès lors condamné à s'arrêter brutalement si cette dernière échoue.

Conclusion

En résumé

1. Un objet est une zone mémoire pouvant contenir des données composée d'une suite contiguë d'un ou plusieurs multipl^{ets} ;
2. La représentation d'un type détermine de quelle manière les données sont réparties au sein d'un objet de ce type ;
3. Il est possible de lire le contenu d'un objet multipl^{et} par multipl^{et} à l'aide d'un pointeur sur `unsigned char` ;
4. La fonction `malloc()` permet d'allouer dynamiquement un objet d'une taille donnée ;
5. La fonction `calloc()` fait de même en initialisant chaque multipl^{et} à zéro ;

6. La valeur zéro n'est pas forcément représentée en initialisant tous les multiplets d'un objet à zéro, cela dépend des types ;
7. La fonction `realloc()` permet d'augmenter ou de diminuer la taille d'un bloc précédemment alloué avec `malloc()` ou `calloc()` ;
8. Il est possible de préciser la taille d'un tableau à l'aide d'une variable auquel cas le compilateur se charge pour nous d'allouer et de libérer de la mémoire pour ce tableau ;
9. L'utilisation de tels tableaux souffre de limitations par rapport à l'allocation dynamique manuelle.

IV.7. Les fichiers (1)

Introduction

Jusqu'à présent, nous n'avons manipulé que des données en mémoire, ce qui nous empêchait de les stocker de manière permanente. Dans ces deux chapitres, nous allons voir comment conserver des informations de manière permanente à l'aide des fichiers.

IV.7.1. Les fichiers

En informatique, un fichier est un ensemble d'informations stockées sur un support, réuni sous un même nom et manipulé comme une unité.

IV.7.1.1. Extension de fichier

Le contenu de chaque fichier est lui-même organisé suivant un format qui dépend des données qu'il contient. Il en existe une pléthore pour chaque type de données :

- audio : Ogg, MP3, MP4, FLAC, Wave, etc. ;
- vidéo : Ogg, WebM, MP4, AVI, etc. ;
- documents : ODT, DOC et DOCX, XLS et XLSX, PDF, Postscript, etc.

Afin d'aider l'utilisateur, ces formats sont le plus souvent indiqués à l'aide d'une **extension** qui se traduit par un suffixe ajouté au nom de fichier. Toutefois, cette extension est purement *indicative* et *facultative*, elle n'influence *en rien* le contenu du fichier. Son seul objectif est d'aider à déterminer le type de contenu d'un fichier.

IV.7.1.2. Système de fichiers

Afin de faciliter leur localisation et leur gestion, les fichiers sont classés et organisés sur leur support suivant un **système de fichiers**. C'est lui qui permet à l'utilisateur de répartir ses fichiers dans une arborescence de dossiers et de localiser ces derniers à partir d'un chemin d'accès.

IV.7.1.2.1. Le chemin d'accès

Le chemin d'accès d'un fichier est une suite de caractères décrivant la position de celui-ci dans le système de fichiers. Un chemin d'accès se compose au minimum du nom du fichier visé et au maximum de la suite de tous les dossiers qu'il est nécessaire de traverser pour l'atteindre depuis le **répertoire racine**. Ces éventuels dossiers à traverser sont séparés par un caractère spécifique qui est `/` sous Unixöide et `\` sous Windows.

La **racine** d'un système de fichier est le point de départ de l'arborescence des fichiers et dossiers. Sous Unixöides, il s'agit du répertoire `/` tandis que sous Windows, chaque lecteur est une racine (comme `C:` par exemple). Si un chemin d'accès commence par la racine, alors celui-ci est dit **absolu** car il permet d'atteindre le fichier depuis n'importe quelle position dans l'arborescence. Si le chemin d'accès ne commence pas par la racine, il est dit **relatif** et ne permet de parvenir au fichier que depuis un point précis dans la hiérarchie de fichiers.

Ainsi, si nous souhaitons accéder à un fichier nommé « `texte.txt` » situé dans le dossier « documents » lui-même situé dans le dossier « utilisateur » qui est à la racine, alors le chemin d'accès absolu vers ce fichier serait `/utilisateur/documents/texte.txt` sous Unixöide et `C:\utilisateur\documents\texte.txt` sous Windows (à supposer qu'il soit sur le lecteur `C:`). Toutefois, si nous sommes déjà dans le dossier « utilisateur », alors nous pouvons y accéder à l'aide du chemin relatif `documents/texte.txt` ou `documents\texte.txt`. Néanmoins, ce chemin relatif n'est utilisable que si nous sommes dans le dossier « utilisateur ».

IV.7.1.2.2. Métadonnées

Également, le système de fichier se charge le plus souvent de conserver un certain nombre de données concernant chaque fichier comme :

- sa taille ;
- ses droits d'accès (les utilisateurs autorisés à le manipuler) ;
- la date de dernier accès ;
- la date de dernière modification ;
- etc.

IV.7.2. Les flux : un peu de théorie

La bibliothèque standard vous fournit différentes fonctions pour manipuler les fichiers, toutes déclarées dans l'en-tête `<stdio.h>`. Toutefois, celles-ci manipulent non pas des fichiers, mais des **flux** de données en provenance ou à destination de fichiers. Ces flux peuvent être de deux types :

- des flux de textes qui sont des suites de caractères terminées par un caractère de fin de ligne (`\n`) et formant ainsi des lignes ;
- des flux binaires qui sont des suites de multiplets.

IV.7.2.1. Pourquoi utiliser des flux ?

Pourquoi recourir à des flux plutôt que de manipuler directement des fichiers nous demanderez-vous ? Pour deux raisons : les disparités entre systèmes d'exploitation quant à la représentation des lignes et la lourdeur des opérations de lecture et d'écriture.

IV.7.2.1.1. Disparités entre systèmes d'exploitation

Les différents systèmes d'exploitation ne représentent pas les lignes de la même manière :

- sous Mac OS (avant Mac OS X), la fin d'une ligne était indiquée par le caractère `\r` ;
- sous Windows, la fin de ligne est indiquée par la suite de caractères `\r\n` ;
- sous Unixôides (GNU/Linux, *BSD, Mac OS X, Solaris, etc.), la fin de ligne est indiquée par le caractère `\n`.

Aussi, si nous manipulions directement des fichiers, nous devrions prendre en compte ces disparités, ce qui rendrait notre code nettement plus pénible à rédiger. En passant par les fonctions de la bibliothèque standard, nous évitons ce casse-tête car celle-ci remplace le ou les caractères de fin de ligne par un `\n` (ce que nous avons pu expérimenter avec les fonctions `printf()` et `scanf()`) et inversement.

IV.7.2.1.2. Lourdeur des opérations de lecture et d'écriture

Lire depuis un fichier ou écrire dans un fichier signifie le plus souvent accéder au disque dur. Or, rappelez-vous, il s'agit de la mémoire la plus lente d'un ordinateur. Dès lors, si pour lire une ligne il était nécessaire de récupérer les caractères un à un depuis le disque dur, cela prendrait un temps fou.

Pour éviter ce problème, les flux de la bibliothèque standard recourent à un mécanisme appelé la **temporisation** ou **mémorisation** (*buffering* en anglais). La bibliothèque standard fournit deux types de temporisation :

- la temporisation par blocs ;
- et la temporisation par lignes.

Avec la **temporisation par blocs**, les données sont récupérées depuis le fichier et écrites dans le fichier sous forme de blocs d'une taille déterminée. L'idée est la suivante : plutôt que de lire les caractères un à un, nous allons demander un bloc de données d'une taille déterminée que nous conserverons en mémoire vive pour les accès suivants. Cette technique est également utilisée lors des opérations d'écritures : les données sont stockées en mémoire jusqu'à ce qu'elles atteignent la taille d'un bloc. Ainsi, le nombre d'accès au disque dur sera limité à la taille totale du fichier à lire (ou des données à écrire) divisée par la taille d'un bloc.

La **temporisation par lignes**, comme son nom l'indique, se contente de mémoriser une ligne. Techniquement, celle-ci est utilisée lorsque le flux est associé à un périphérique interactif comme un terminal. En effet, si nous appliquions la temporisation par blocs pour les entrées de l'utilisateur, ce dernier devrait entrer du texte jusqu'à atteindre la taille d'un bloc. Pareillement, nous devrions attendre d'avoir écrit une quantité de données égale à la taille d'un bloc pour que du texte soit affiché à l'écran. Cela serait assez gênant et peu interactif... À la place, c'est la

temporisation par lignes qui est utilisée : les données sont stockées jusqu'à ce qu'un caractère de fin de ligne soit rencontré (`\n`, donc) ou jusqu'à ce qu'une taille maximale soit atteinte.

La bibliothèque standard vous permet également de ne pas temporiser les données en provenance ou à destination d'un flux.

IV.7.2.2. `stdin`, `stdout` et `stderr`

Par défaut, trois flux *de texte* sont ouverts lors du démarrage d'un programme et sont déclarés dans l'en-tête `<stdio.h>` : `stdin`, `stdout` et `stderr`.

- `stdin` correspond à l'**entrée standard**, c'est-à-dire le flux depuis lequel vous pouvez récupérer les informations fournies par l'utilisateur.
- `stdout` correspond à la **sortie standard**, il s'agit du flux vous permettant de transmettre des informations à l'utilisateur qui seront le plus souvent affichées dans le terminal.
- `stderr` correspond à la **sortie d'erreur standard**, c'est ce flux que vous devez privilégier lorsque vous souhaitez transmettre des messages d'erreurs ou des avertissements à l'attention de l'utilisateur (nous verrons comment l'utiliser un peu plus tard dans ce chapitre). Comme pour `stdout`, ces données sont le plus souvent affichées dans le terminal.

Les flux `stdin` et `stdout` sont temporisés par lignes (sauf s'ils sont associés à des fichiers au lieu de périphériques interactifs, auxquels cas ils seront temporisés par blocs) tandis que le flux `stderr` est *au plus* temporisé par lignes (ceci afin que les informations soient transmises le plus rapidement possible à l'utilisateur).

IV.7.3. Ouverture et fermeture d'un flux

IV.7.3.1. La fonction `fopen`

```
1 FILE *fopen(char *chemin, char *mode);
```

La fonction `fopen()` permet d'ouvrir un flux. Celle-ci attend deux arguments : un **chemin d'accès** vers un fichier qui sera associé au flux et un **mode** qui détermine le type de flux (texte ou binaire) et la nature des opérations qui seront réalisées sur le fichier via le flux (lecture, écriture ou les deux). Elle retourne un pointeur vers un flux en cas de succès et un pointeur nul en cas d'échec.

IV.7.3.1.1. Le mode

Le mode est une chaîne de caractères composée d'une ou plusieurs lettres qui décrit le type du flux et la nature des opérations qu'il doit réaliser.

Cette chaîne commence *obligatoirement* par la seconde de ces informations. Il existe six possibilités reprises dans le tableau ci-dessous.

	Type(s) d'opé- rations
r	Lec- ture Néant
r+	Lec- ture et Néant
w	écri- ture Si le fi- chier n'existe pas, il est créé. Écri- ture Si le fi- chier existe, son contenu est ef- facé.
w+	Lec- ture et <i>Idem</i> écri- ture

		Si
		le
		fi-
		chier
		n'existe
		pas,
		il
		est
		créé.
a	Écrir	Place
	ture	les
		don-
		nées
		à
		la
		fin
		du
		fi-
		chier
	Lec-	
	ture	
a	et	Idem
	écri-	
	ture	

Par défaut, les flux sont des flux de textes. Pour obtenir un flux binaire, il suffit d'ajouter la lettre **b** à la fin de la chaîne décrivant le mode.

IV.7.3.1.2. Exemple

Le code ci-dessous tente d'ouvrir un fichier nommé « texte.txt » en lecture seule dans le dossier courant. Notez que dans le cas où il n'existe pas, la fonction **fopen()** retournera un pointeur nul (seul le mode **r** permet de produire ce comportement).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      FILE *fp = fopen("texte.txt", "r");
8
9      if (fp == NULL)
10     {
11         printf("Le fichier texte.txt n'a pas pu être ouvert\n");
12         return EXIT_FAILURE;

```

```

13     }
14
15     printf("Le fichier texte.txt existe\n");
16     return 0;
17 }

```

IV.7.3.2. La fonction `fclose`

```

1 int fclose(FILE *flux);

```

La fonction `fclose()` termine l'association entre un flux et un fichier. S'il reste des données temporisées, celles-ci sont écrites. La fonction retourne zéro en cas de succès et `EOF` en cas d'erreur.



`EOF` est une constante définie dans l'en-tête `<stdio.h>` et est utilisée par les fonctions déclarées dans ce dernier pour indiquer soit l'arrivée à la fin d'un fichier (nous allons y venir) soit la survenance d'une erreur. La valeur de cette constante est *toujours* un entier négatif.

Nous pouvons désormais compléter l'exemple précédent comme suit.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     FILE *fp = fopen("texte.txt", "r");
8
9     if (fp == NULL)
10    {
11        printf("Le fichier texte.txt n'a pas pu être ouvert\n");
12        return EXIT_FAILURE;
13    }
14
15    printf("Le fichier texte.txt existe\n");
16
17    if (fclose(fp) == EOF)
18    {
19        printf("Erreur lors de la fermeture du flux\n");
20        return EXIT_FAILURE;
21    }
22

```

```
23     return 0;
24 }
```



Veillez qu'à chaque appel à la fonction `fopen()` corresponde un appel à la fonction `fclose()`.

IV.7.4. Écriture vers un flux de texte

IV.7.4.1. Écrire un caractère

```
1 int putc(int ch, FILE *flux);
2 int fputc(int ch, FILE *flux);
3 int putchar(int ch);
```

Les fonctions `putc()` et `fputc()` écrivent un caractère dans un flux. Il s'agit de l'opération d'écriture la plus basique sur laquelle reposent toutes les autres fonctions d'écriture. Ces deux fonctions retournent soit le caractère écrit, soit `EOF` si une erreur est rencontrée. La fonction `putchar()`, quant à elle, est identique aux fonctions `putc()` et `fputc()` si ce n'est qu'elle écrit dans le flux `stdout`.



Techniquement, `putc()` et `fputc()` sont identiques, si ce n'est que `putc()` est en fait le plus souvent une macrofonction. Étant donné que nous n'avons pas encore vu de quoi il s'agit, préférez utiliser la fonction `fputc()` pour l'instant.

L'exemple ci-dessous écrit le caractère « C » dans le fichier « `texte.txt` ». Étant donné que nous utilisons le mode `w`, le fichier est soit créé s'il n'existe pas, soit vidé de son contenu s'il existe (revoyez le tableau des modes à la section précédente si vous êtes perdus).

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     FILE *fp = fopen("texte.txt", "w");
8
9     if (fp == NULL)
10    {
11        printf("Le fichier texte.txt n'a pas pu être ouvert\n");
```



```

12     return EXIT_FAILURE;
13 }
14 if (fputc('C', fp) == EOF)
15 {
16     printf("Erreur lors de l'écriture d'un caractère\n");
17     return EXIT_FAILURE;
18 }
19 if (fclose(fp) == EOF)
20 {
21     printf("Erreur lors de la fermeture du flux\n");
22     return EXIT_FAILURE;
23 }
24
25 return 0;
26 }

```

IV.7.4.2. Écrire une ligne

```

1 int fputs(char *ligne, FILE *flux);
2 int puts(char *ligne);

```

La fonction `fputs()` écrit une ligne dans le flux `flux`. La fonction retourne un nombre positif ou nul en cas de succès et `EOF` en cas d'erreurs. La fonction `puts()` est identique si ce n'est qu'elle ajoute automatiquement un caractère de fin de ligne et qu'elle écrit sur le flux `stdout`.



Maintenant que nous savons comment écrire une ligne dans un flux précis, nous allons pouvoir diriger nos messages d'erreurs vers le flux `stderr` afin que ceux-ci soient affichés le plus rapidement possible.

L'exemple suivant écrit le mot « Bonjour » suivi d'un caractère de fin de ligne au sein du fichier « `texte.txt` ».

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     FILE *fp = fopen("texte.txt", "w");
8
9     if (fp == NULL)
10    {

```

```

11     fputs("Le fichier texte.txt n'a pas pu être ouvert\n",
12           stderr);
13     return EXIT_FAILURE;
14 }
15 if (fputs("Bonjour\n", fp) == EOF)
16 {
17     fputs("Erreur lors de l'écriture d'une ligne\n", stderr);
18     return EXIT_FAILURE;
19 }
20 if (fclose(fp) == EOF)
21 {
22     fputs("Erreur lors de la fermeture du flux\n", stderr);
23     return EXIT_FAILURE;
24 }
25 return 0;
26 }

```



La norme¹ vous garantit qu'une ligne peut contenir jusqu'à 254 caractères (caractère de fin de ligne inclus). Aussi, veillez à ne pas écrire de ligne d'une taille supérieure à cette limite.

IV.7.4.3. La fonction fprintf

```

1 int fprintf(FILE *flux, char *format, ...);

```

La fonction `fprintf()` est la même que la fonction `printf()` si ce n'est qu'il est possible de lui spécifier sur quel flux écrire (au lieu de `stdout` pour `printf()`). Elle retourne le nombre de caractères écrits ou une valeur négative en cas d'échec.

IV.7.5. Lecture depuis un flux de texte

IV.7.5.1. Récupérer un caractère

```

1 int getc(FILE *flux);
2 int fgetc(FILE *flux);
3 int getchar(void);

```

1. ISO/IEC 9899:201x, doc. N1570, § 7.21.2, Streams, al. 9.

Les fonctions `getc()` et `fgetc()` sont les exacts miroirs des fonctions `putc()` et `fputc()` : elles récupèrent un caractère depuis le flux fourni en argument. Il s'agit de l'opération de lecture la plus basique sur laquelle reposent toutes les autres fonctions de lecture. Ces deux fonctions retournent soit le caractère lu, soit `EOF` si la fin de fichier est rencontrée *ou* si une erreur est rencontrée. La fonction `getchar()`, quant à elle, est identique à ces deux fonctions si ce n'est qu'elle récupère un caractère depuis le flux `stdin`.



Comme `putc()`, la fonction `getc()` est le plus souvent une macrofonction. Utilisez donc plutôt la fonction `fgetc()` pour le moment.

L'exemple ci-dessous lit un caractère provenant du fichier `texte.txt`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      FILE *fp = fopen("texte.txt", "r");
8
9      if (fp == NULL)
10     {
11         fprintf(stderr,
12             "Le fichier texte.txt n'a pas pu être ouvert\n");
13         return EXIT_FAILURE;
14     }
15
16     int ch = fgetc(fp);
17
18     if (ch != EOF)
19         printf("%c\n", ch);
20     if (fclose(fp) != EOF)
21     {
22         fprintf(stderr, "Erreur lors de la fermeture du flux\n");
23         return EXIT_FAILURE;
24     }
25
26     return 0;
27 }
```

IV.7.5.2. Récupérer une ligne

```
1 char *fgets(char *tampon, int taille, FILE *flux);
```

La fonction `fgets()` lit une ligne depuis le flux `flux` et la stocke dans le tableau `tampon`. Cette dernière lit au plus un nombre de caractères égal à `taille` diminué de un afin de laisser la place pour le caractère nul, qui est automatiquement ajouté. Dans le cas où elle rencontre un caractère de fin de ligne : *celui-ci est conservé au sein du tableau*, un caractère nul est ajouté et la lecture s'arrête.

La fonction retourne l'adresse du tableau `tampon` en cas de succès et un pointeur nul si la fin du fichier est atteinte *ou si une erreur est survenue*.

L'exemple ci-dessous réalise donc la même opération que le code précédent, mais en utilisant la fonction `fgets()`.

i

Étant donné que la norme nous garantit qu'une ligne peut contenir jusqu'à 254 caractères (caractère de fin de ligne inclus), nous utilisons un tableau de 255 caractères pour les contenir (puisque'il est nécessaire de prévoir un espace pour le caractère nul).

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     char buf[255];
8     FILE *fp = fopen("texte.txt", "r");
9
10    if (fp == NULL)
11    {
12        fprintf(stderr,
13            "Le fichier texte.txt n'a pas pu être ouvert\n");
14        return EXIT_FAILURE;
15    }
16    if (fgets(buf, sizeof buf, fp) != NULL)
17        printf("%s\n", buf);
18    if (fclose(fp) == EOF)
19    {
20        fprintf(stderr, "Erreur lors de la fermeture du flux\n");
21        return EXIT_FAILURE;
22    }
23    return 0;
24 }
```

Toutefois, il y a un petit problème : la fonction `fgets()` conserve le caractère de fin de ligne qu'elle rencontre. Dès lors, nous affichons deux retours à la ligne : celui contenu dans la chaîne `buf` et celui affiché par `printf()`. Aussi, il serait préférable d'en supprimer un, de préférence celui de la chaîne de caractères. Pour ce faire, nous pouvons faire appel à une petite fonction (que nous appellerons `chomp()` en référence à la fonction éponyme du langage Perl) qui se chargera de remplacer le caractère de fin de ligne par un caractère nul.

```
1 void chomp(char *s)
2 {
3     while (*s != '\n' && *s != '\0')
4         ++s;
5
6     *s = '\0';
7 }
```

IV.7.5.3. La fonction `fscanf`

```
1 int fscanf(FILE *flux, char *format, ...);
```

La fonction `fscanf()` est identique à la fonction `scanf()` si ce n'est qu'elle récupère les données depuis le flux fourni en argument (au lieu de `stdin` pour `scanf()`).

La fonction `fscanf()` retourne le nombre de conversions réussies (voire zéro, si aucune n'est demandée ou n'a pu être réalisée) ou `EOF` si une erreur survient *avant* qu'une conversion n'ait eu lieu.

IV.7.6. Écriture vers un flux binaire

IV.7.6.1. Écrire un multiplét

```
1 int putc(int ch, FILE *flux);
2 int fputc(int ch, FILE *flux);
```

Comme pour les flux de texte, il vous est possible de recourir aux fonctions `putc()` et `fputc()`. Dans le cas d'un flux binaire, ces fonctions écrivent un multiplét (sous la forme d'un `int` converti en `unsigned char`) dans le flux spécifié.

IV.7.6.2. Écrire une suite de multipliets

```
1 size_t fwrite(void *ptr, size_t taille, size_t nombre, FILE *flux);
```

La fonction `fwrite()` écrit le tableau référencé par `ptr` composé de `nombre` éléments de `taille` multipliets dans le flux `flux`. Elle retourne une valeur égale à `nombre` en cas de succès et une valeur inférieure en cas d'échec.

L'exemple suivant écrit le contenu du tableau `tab` dans le fichier `binaire.bin`. Dans le cas où un `int` fait 4 octets, 20 octets seront donc écrits.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int main(void)
7 {
8     int tab[5] = { 1, 2, 3, 4, 5 };
9     const size_t n = sizeof tab / sizeof tab[0];
10    FILE *fp = fopen("binaire.bin", "wb");
11
12    if (fp == NULL)
13    {
14        fprintf(stderr,
15            "Le fichier binaire.bin n'a pas pu être ouvert\n");
16        return EXIT_FAILURE;
17    }
18    if (fwrite(&tab, sizeof tab[0], n, fp) != n)
19    {
20        fprintf(stderr,
21            "Erreur lors de l'écriture du tableau\n");
22        return EXIT_FAILURE;
23    }
24    if (fclose(fp) != EOF)
25    {
26        fprintf(stderr, "Erreur lors de la fermeture du flux\n");
27        return EXIT_FAILURE;
28    }
29    return 0;
30 }
```

IV.7.7. Lecture depuis un flux binaire

IV.7.7.1. Lire un multiplet

```
1 int getc(FILE *flux);
2 int fgetc(FILE *flux);
```

Lors de la lecture depuis un flux binaire, les fonctions `fgetc()` et `getc()` permettent de récupérer un multiplet (sous la forme d'un `unsigned char` converti en `int`) depuis un flux.

IV.7.7.2. Lire une suite de multiplets

```
1 size_t fread(void *ptr, size_t taille, size_t nombre, FILE *flux);
```

La fonction `fread()` est l'inverse de la fonction `fwrite()` : elle lit `nombre` éléments de `taille` multiplets depuis le flux `flux` et les stocke dans l'objet référencé par `ptr`. Elle retourne une valeur égale à `nombre` en cas de succès ou une valeur inférieure en cas d'échec.

Dans le cas où nous disposons du fichier `binaire.bin` produit par l'exemple de la section précédente, nous pouvons reconstituer le tableau `tab`.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int main(void)
7 {
8     int tab[5] = { 0 };
9     const size_t n = sizeof tab / sizeof tab[0];
10    FILE *fp = fopen("binaire.bin", "rb");
11
12    if (fp == NULL)
13    {
14        fprintf(stderr,
15            "Le fichier binaire.bin n'a pas pu être ouvert\n");
16        return EXIT_FAILURE;
17    }
18    if (fread(&tab, sizeof tab[0], n, fp) != n)
19    {
20        fprintf(stderr,
21            "Erreur lors de la lecture du tableau\n");
22        return EXIT_FAILURE;
23    }
24 }
```

```
21     }
22     if (fclose(fp) == EOF)
23     {
24         fprintf(stderr, "Erreur lors de la fermeture du flux\n");
25         return EXIT_FAILURE;
26     }
27
28     for (unsigned i = 0; i < n; ++i)
29         printf("tab[%u] = %d\n", i, tab[i]);
30
31     return 0;
32 }
```

Résultat

```
1 tab[0] = 1
2 tab[1] = 2
3 tab[2] = 3
4 tab[3] = 4
5 tab[4] = 5
```

Conclusion

En résumé

1. L'extension d'un fichier fait partie de son nom et est purement indicative ;
2. Les fichiers sont manipulés à l'aide de flux afin de s'abstraire des disparités entre systèmes d'exploitation et de recourir à la temporisation ;
3. Trois flux de texte sont disponibles par défaut : `stdin`, `stdout` et `stderr` ;
4. La fonction `fopen()` permet d'associer un fichier à un flux, la fonction `fclose()` met fin à cette association ;
5. Les fonctions `fputc()`, `fputs()` et `fprintf()` écrivent des données dans un flux de texte ;
6. Les fonctions `fgetc()`, `fgets()` et `fscanf()` lisent des données depuis un flux de texte ;
7. Les fonctions `fputc()` et `fwrite()` écrivent un ou plusieurs multipliets dans un flux binaire ;
8. Les fonctions `fgetc()` et `fread()` lisent un ou plusieurs multipliets depuis un flux binaire.

IV.8. Les fichiers (2)

Introduction

Dans ce chapitre, nous allons poursuivre notre lancée et vous présenter plusieurs points un peu plus avancés en rapport avec les fichiers et les flux.

IV.8.1. Détection d'erreurs et fin de fichier

Lors de la présentation des fonctions `fgetc()`, `getc()` et `fgets()` vous avez peut-être remarqué que ces fonctions utilisent un même retour pour indiquer soit la rencontre de la fin du fichier, soit la survenance d'une erreur. Du coup, comment faire pour distinguer l'un et l'autre cas ?

Il existe deux fonctions pour clarifier une telle situation : `feof()` et `ferror()`.

IV.8.1.1. La fonction `feof`

```
1 int feof(FILE *flux);
```

La fonction `feof()` retourne une valeur non nulle dans le cas où la fin du fichier associé au flux spécifié est atteinte.

IV.8.1.2. La fonction `ferror`

```
1 int ferror(FILE *flux);
```

La fonction `ferror()` retourne une valeur non nulle dans le cas où une erreur s'est produite lors d'une opération sur le flux visé.

IV.8.1.3. Exemple

L'exemple ci-dessous utilise ces deux fonctions pour déterminer le type de problème rencontré par la fonction `fgets()`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      char buf[255];
8      FILE *fp = fopen("texte.txt", "r");
9
10     if (fp == NULL)
11     {
12         fprintf(stderr,
13             "Le fichier texte.txt n'a pas pu être ouvert\n");
14         return EXIT_FAILURE;
15     }
16     if (fgets(buf, sizeof buf, fp) != NULL)
17         printf("%s\n", buf);
18     else if (ferror(fp))
19     {
20         fprintf(stderr, "Erreur lors de la lecture\n");
21         return EXIT_FAILURE;
22     }
23     else
24     {
25         fprintf(stderr, "Fin de fichier rencontrée\n");
26         return EXIT_FAILURE;
27     }
28     if (fclose(fp) == EOF)
29     {
30         fprintf(stderr, "Erreur lors de la fermeture du flux\n");
31         return EXIT_FAILURE;
32     }
33     return 0;
34 }

```

IV.8.2. Position au sein d'un flux

De manière imagée, un fichier peut être vu comme une longue bande magnétique depuis laquelle des données sont lues ou sur laquelle des informations sont écrites. Ainsi, au fur et à mesure que les données sont lues ou écrites, nous avançons le long de cette bande.

Toutefois, il est parfois nécessaire de se rendre directement à un point précis de cette bande (par exemple à la fin) afin d'éviter des lectures inutiles pour parvenir à un endroit souhaité. À cet effet, la bibliothèque standard fournit plusieurs fonctions.

IV.8.2.1. La fonction `ftell`

```
1 long ftell(FILE *flux);
```

La fonction `ftell()` retourne la position actuelle au sein du fichier sous forme d'un `long`. Elle retourne un nombre négatif en cas d'erreur. Dans le cas des flux *binaires*, cette valeur correspond au nombre de multipliants séparant la position actuelle du début du fichier.



Lors de l'ouverture d'un flux, la position courante correspond au début du fichier, *sauf* pour les modes `a` et `a+` pour lesquels la position initiale est soit le début, soit la fin du fichier.

IV.8.2.2. La fonction `fseek`

```
1 int fseek(FILE *flux, long distance, int repere);
```

La fonction `fseek()` permet d'effectuer un déplacement d'une distance fournie en argument depuis un repère donné. Elle retourne zéro en cas de succès et une autre valeur en cas d'échec. Il existe trois repères possibles :

- `SEEK_SET` qui correspond au début du fichier ;
- `SEEK_CUR` qui correspond à la position courante ;
- `SEEK_END` qui correspond à la fin du fichier.

Cette fonction s'utilise différemment suivant qu'elle opère sur un flux de texte ou sur un flux binaire.

IV.8.2.2.1. Les flux de texte

Dans le cas d'un flux de texte, il y a deux possibilités :

- la distance fournie est nulle ;
- la distance est une valeur fournie par un précédent appel à la fonction `ftell()` et le repère est `SEEK_SET`.

IV.8.2.2.2. Les flux binaires

Dans le cas d'un flux binaire, seuls les repères `SEEK_SET` et `SEEK_CUR` peuvent être utilisés. La distance correspond à un nombre de multipliants à passer depuis le repère fourni.



Dans le cas des modes `a` et `a+`, un déplacement a lieu à la fin du fichier *avant chaque opération d'écriture* et ce, *qu'il y ait eu déplacement auparavant ou non*.



Un fichier peut-être vu comme une bande magnétique : si vous rembobinez pour ensuite écrire, vous *remplacez* les données existantes par celles que vous écrivez.

IV.8.2.3. La fonction `fgetpos`

```
1 int fgetpos(FILE *flux, fpos_t *position);
```

La fonction `fgetpos()` écrit la position courante dans un objet de type `fpos_t` référencé par `position`. Elle retourne zéro en cas de succès et un nombre non nul dans le cas contraire.

IV.8.2.4. La fonction `fsetpos`

```
1 int fsetpos(FILE *flux, fpos_t *position);
```

La fonction `fsetpos()` effectue un déplacement à la position précédemment écrite dans un objet de type `fpos_t` référencé par `position`. En cas de succès, elle retourne zéro, sinon un nombre entier non nul.

IV.8.2.5. La fonction `rewind`

```
1 void rewind(FILE *flux);
```

La fonction `rewind()` vous ramène au début du fichier (autrement dit, elle rembobine la bande).



Si vous utilisez un flux ouvert en lecture *et* écriture vous *devez* appeler une fonction de déplacement entre deux opérations de natures différentes ou utiliser la fonction `fflush()` (présentée dans l'extrait suivant) entre une opération d'écriture et de lecture. Si vous ne souhaitez pas vous déplacer, vous pouvez utiliser l'appel `fseek(flux, 0, SEEK_CUR)` afin de respecter cette condition sans réellement effectuer un déplacement.

IV.8.3. La temporisation

Dans le chapitre précédent, nous vous avons précisé que les flux étaient le plus souvent temporisés afin d'optimiser les opérations de lecture et d'écriture sous-jacentes. Dans cette section, nous allons nous pencher un peu plus sur cette notion.

IV.8.3.1. Introduction

Nous vous avons dit auparavant que deux types de temporisations existaient : la temporisation par lignes et celle par blocs. Une des conséquences logiques de cette temporisation est que les fonctions de lecture/écriture récupèrent les données et les inscrivent dans ces tampons. Ceci peut paraître évident, mais peut avoir des conséquences parfois surprenantes si ce fait est oublié ou inconnu.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      char nom[64];
8      char prenom[64];
9
10     printf("Quel est votre nom ? ");
11
12     if (scanf("%63s", nom) != 1)
13     {
14         fprintf(stderr, "Erreur lors de la saisie\n");
15         return EXIT_FAILURE;
16     }
17
18     printf("Quel est votre prénom ? ");
19
20     if (scanf("%63s", prenom) != 1)
21     {
22         fprintf(stderr, "Erreur lors de la saisie\n");
23         return EXIT_FAILURE;
24     }
25
26     printf("Votre nom est %s\n", nom);
27     printf("Votre prénom est %s\n", prenom);
28     return 0;
29 }
```

Résultat

```

1 Quel est votre nom ? Charles Henri
2 Quel est votre prénom ? Votre nom est Charles
3 Votre prénom est Henri

```

Comme vous le voyez, le programme ci-dessus réalise deux saisies, mais si l'utilisateur entre par exemple « Charles Henri », il n'aura l'occasion d'entrer des données qu'une seule fois. Ceci est dû au fait que l'indicateur `s` récupère une suite de caractères exempte d'espaces (ce qui fait qu'il s'arrête à « Charles ») et que la suite « Henri » *demeure dans le tampon du flux `stdin`*. Ainsi, lors de la deuxième saisie, il n'est pas nécessaire de récupérer de nouvelles données depuis le terminal puisqu'il y en a déjà en attente dans le tampon, d'où le résultat obtenu.

Le même problème peut se poser si par exemple les données fournies ont une taille supérieure par rapport à l'objet qui doit les accueillir.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     char chaine1[16];
8     char chaine2[16];
9
10    printf("Un morceau : ");
11
12    if (fgets(chaine1, sizeof chaine1, stdin) == NULL)
13    {
14        printf("Erreur lors de la saisie\n");
15        return EXIT_FAILURE;
16    }
17
18    printf("Un autre morceau : ");
19
20    if (fgets(chaine2, sizeof chaine2, stdin) == NULL)
21    {
22        printf("Erreur lors de la saisie\n");
23        return EXIT_FAILURE;
24    }
25
26    printf("%s ; %s\n", chaine1, chaine2);
27    return 0;
28 }

```

Résultat

- | | |
|---|--|
| 1 | Un morceau : Une chaîne de caractères, vraiment, mais alors vraiment trop longue |
| 2 | Un autre morceau : Une chaîne de ; caractères, vr |

Ici, la chaîne entrée est trop importante pour être contenue dans `chaine1`, les données non lues sont alors conservées dans le tampon du flux `stdin` et lues lors de la seconde saisie (qui ne lit par l'entière non plus).

IV.8.3.2. Interagir avec la temporisation

IV.8.3.2.1. Vider un tampon

Si la temporisation nous évite des coûts en terme d'opérations de lecture/écriture, il nous est parfois nécessaire de passer outre cette mécanique pour vider manuellement le tampon d'un flux.

IV.8.3.2.1.1. Opération de lecture Si le tampon contient des données qui proviennent d'une opération de lecture, celles-ci peuvent être abandonnées soit en appelant une fonction de positionnement soit en lisant les données, tout simplement. Il y a toutefois un bémol avec la première solution : les fonctions de positionnement ne fonctionnent pas dans le cas où le flux ciblé est lié à un périphérique « interactif », c'est-à-dire le plus souvent un terminal.

Autrement dit, pour que l'exemple précédent recoure bien à deux saisies, il nous est nécessaire de vérifier que la fonction `fgetc()` a bien lu un caractère `\n` (qui signifie que la fin de ligne est atteinte et donc celle du tampon s'il s'agit du flux `stdin`). Si ce n'est pas le cas, alors il nous faut lire les caractères restants jusqu'au `\n` final (ou la fin du fichier).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5
6  int vider_tampon(FILE *fp)
7  {
8      int c;
9
10     do
11         c = fgetc(fp);
12     while (c != '\n' && c != EOF);
13
14     return ferror(fp) ? 0 : 1;

```

```

15 }
16
17
18 int main(void)
19 {
20     char chaine1[16];
21     char chaine2[16];
22
23     printf("Un morceau : ");
24
25     if (fgets(chaine1, sizeof chaine1, stdin) == NULL)
26     {
27         printf("Erreur lors de la saisie\n");
28         return EXIT_FAILURE;
29     }
30     if (strchr(chaine1, '\n') == NULL)
31         if (!vider_tampon(stdin))
32         {
33             fprintf(stderr,
34                 "Erreur lors de la vidange du tampon.\n");
35             return EXIT_FAILURE;
36         }
37     printf("Un autre morceau : ");
38
39     if (fgets(chaine2, sizeof chaine2, stdin) == NULL)
40     {
41         printf("Erreur lors de la saisie\n");
42         return EXIT_FAILURE;
43     }
44     if (strchr(chaine2, '\n') == NULL)
45         if (!vider_tampon(stdin))
46         {
47             fprintf(stderr,
48                 "Erreur lors de la vidange du tampon.\n");
49             return EXIT_FAILURE;
50         }
51     printf("%s ; %s\n", chaine1, chaine2);
52     return 0;
53 }

```


Résultat

- | | |
|---|--|
| 1 | Un morceau : Une chaîne de caractères vraiment, mais alors vraiment longue |
| 2 | Un autre morceau : Une autre chaîne de caractères |
| 3 | Une chaîne de ; Une autre chaî |

IV.8.3.2.1.2. Opération d'écriture Si, en revanche, le tampon comprend des données en attente d'écriture, il est possible de forcer celle-ci soit à l'aide d'une fonction de positionnement, soit à l'aide de la fonction `fflush()`.

```
1 int fflush(FILE *flux);
```

Celle-ci vide le tampon du flux spécifié et retourne zéro en cas de succès ou `EOF` en cas d'erreur.



Notez bien que cette fonction ne peut être employée que pour vider un tampon comprenant des données en attente d'écriture.

IV.8.3.2.2. Modifier un tampon

Techniquement, il ne vous est pas possible de modifier directement le contenu d'un tampon, ceci est réalisé par les fonctions de la bibliothèque standard au gré de vos opérations de lecture et/ou d'écriture. Il y a toutefois une exception à cette règle : la fonction `ungetc()`.

```
1 int ungetc(int ch, FILE *flux);
```

Cette fonction est un peu particulière : elle ajoute un caractère ou un multiplète `ch` au début du tampon du flux `flux`. Celui-ci pourra être lu lors d'un appel ultérieur à une fonction de lecture. Elle retourne le caractère ou le multiplète ajouté en cas de succès et `EOF` en cas d'échec.

Cette fonction est très utile dans le cas où les actions d'un programme dépendent du contenu d'un flux. Imaginez par exemple que votre programme doit déterminer si l'entrée standard contient une suite de caractères ou un nombre et doit ensuite afficher celui-ci. Vous pourriez utiliser `getchar()` pour récupérer le premier caractère et déterminer s'il s'agit d'un chiffre. Toutefois, le premier caractère du flux est alors lu et cela complique votre tâche pour la suite... La fonction `ungetc()` vous permet de résoudre ce problème en remplaçant ce caractère dans le tampon du flux `stdin`.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  static void afficher_nombre(void);
6  static void afficher_chaine(void);
7
8
9  static void afficher_nombre(void)
10 {
11     double f;
12
13     if (scanf("%lf", &f) == 1)
14         printf("Vous avez entré le nombre : %f\n", f);
15     else
16         fprintf(stderr, "Erreur lors de la saisie\n");
17 }
18
19
20 static void afficher_chaine(void)
21 {
22     char chaine[255];
23
24     if (scanf("%254s", chaine) == 1)
25         printf("Vous avez entré la chaine : %s\n", chaine);
26     else
27         fprintf(stderr, "Erreur lors de la saisie\n");
28 }
29
30
31 int main(void)
32 {
33     int ch = getchar();
34
35     if (ungetc(ch, stdin) == EOF)
36     {
37         fprintf(stderr, "Impossible de remplacer un caractère\n");
38         return EXIT_FAILURE;
39     }
40
41     switch (ch)
42     {
43     case '0':
44     case '1':
45     case '2':
46     case '3':
47     case '4':
48     case '5':
49     case '6':
```

```
50     case '7':
51     case '8':
52     case '9':
53         afficher_nombre();
54         break;
55
56     default:
57         afficher_chaine();
58         break;
59 }
60
61 return 0;
62 }
```

Résultat

```
1 Test
2 Vous avez entré la chaine : Test
3
4 52.5
5 Vous avez entré le nombre : 52.500000
```



La fonction `ungetc()` ne vous permet de replacer qu'un seul caractère ou multiplét avant une opération de lecture.

IV.8.4. Flux ouverts en lecture et écriture

Pour clore ce chapitre, un petit mot sur le cas des flux ouverts en lecture *et* en écriture (soit à l'aide des modes `r+`, `w+`, `a+` et leurs équivalents binaires).

Si vous utilisez un tel flux, vous *devez* appeler une fonction de déplacement entre deux opérations de natures différentes ou utiliser la fonction `fflush()` entre une opération d'écriture et de lecture. Si vous ne souhaitez pas vous déplacer, vous pouvez utiliser l'appel `fseek(flux, 0L, SEEK_CUR)` afin de respecter cette condition sans réellement effectuer un déplacement.



Vous l'aurez sans doute compris : cette règle impose en fait de vider le tampon du flux entre deux opérations de natures différentes.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      char chaine[255];
8      FILE *fp = fopen("texte.txt", "w+");
9
10     if (fp == NULL)
11     {
12         fprintf(stderr,
13             "Le fichier texte.txt n'a pas pu être ouvert.\n");
14         return EXIT_FAILURE;
15     }
16     if (fputs("Une phrase.\n", fp) == EOF)
17     {
18         fprintf(stderr,
19             "Erreur lors de l'écriture d'une ligne.\n");
20         return EXIT_FAILURE;
21     }
22
23     /* Retour au début : la condition est donc remplie. */
24
25     if (fseek(fp, 0L, SEEK_SET) != 0)
26     {
27         fprintf(stderr,
28             "Impossible de revenir au début du fichier.\n");
29         return EXIT_FAILURE;
30     }
31     if (fgets(chaine, sizeof chaine, fp) == NULL)
32     {
33         fprintf(stderr, "Erreur lors de la lecture.\n");
34         return EXIT_FAILURE;
35     }
36
37     printf("%s\n", chaine);
38
39     if (fclose(fp) == EOF)
40     {
41         fputs("Erreur lors de la fermeture du flux\n", stderr);
42         return EXIT_FAILURE;
43     }
44
45     return 0;
46 }

```

Résultat

1	Une phrase.
---	-------------

Conclusion

En résumé

1. La fonction `feof()` permet de savoir si la fin du fichier associé à un flux a été atteinte ;
2. La fonction `ferror()` permet de savoir si une erreur est survenue lors d'une opération sur un flux ;
3. La fonction `ftell()` retourne la position actuelle au sein d'un flux ;
4. La fonction `fseek()` effectue un déplacement au sein d'un fichier depuis un point donné ;
5. La fonction `rewind()` réalise un déplacement jusqu'au début d'un fichier ;
6. Pour vider un tampon contenant des données en attente de lecture, il est nécessaire de lire ces dernières ;
7. Afin de vider un tampon contenant des données en attente d'écriture, il est possible d'employer soit une fonction de positionnement, soit la fonction `fflush()` ;
8. La fonction `ungetc()` permet de placer un caractère ou un multiplète dans un tampon en vue d'une *lecture* ;
9. Lorsqu'un flux est ouvert en lecture et écriture, il est nécessaire d'appeler une fonction de déplacement entre deux opérations de natures différentes (ou d'appeler la fonction `fflush()` entre une opération d'écriture et de lecture).

IV.9. Le préprocesseur

Introduction

Le **préprocesseur** est un programme qui réalise des traitements sur le code source *avant* que ce dernier ne soit réellement compilé. Globalement, il a trois grands rôles :

- réaliser des **inclusions** (la fameuse directive `#include`) ;
- définir des **macros** qui sont des substituts à des morceaux de code. Après le passage du préprocesseur, tous les appels à ces macros seront remplacés par le code associé ;
- permettre la **compilation conditionnelle**, c'est-à-dire de moduler le contenu d'un fichier source suivant certaines conditions.

IV.9.1. Les inclusions

Nous avons vu dès le début du cours comment inclure des fichiers d'en-tête avec la directive `#include`, sans toutefois véritablement expliquer son rôle. Son but est très simple : inclure le contenu d'un fichier dans un autre. Ainsi, si nous nous retrouvons par exemple avec deux fichiers comme ceux-ci avant la compilation.

fichier.h

```
1 #ifndef FICHIER_H
2 #define FICHIER_H
3
4 extern int glob_var;
5
6 extern void f1(int);
7 extern long f2(double, char);
8
9 #endif
```

fichier.c

```

1  #include "fichier.h"
2
3  void f1(int arg)
4  {
5      /* du code */
6  }
7
8  long f2(double arg, char c)
9  {
10     /* du code */
11 }

```

Après le passage du préprocesseur et avant la compilation à proprement parler, le code obtenu sera le suivant :

fichier.c

```

1  extern int glob_var;
2
3  extern void f1(int arg);
4  extern long f2(double arg, char c);
5
6  void f1(int arg)
7  {
8      /* du code */
9  }
10
11 long f2(double arg, char c)
12 {
13     /* du code */
14 }

```

Nous pouvons voir que les déclarations contenues dans le fichier « fichier.h » ont été incluses dans le fichier « fichier.c » et que toutes les directives du préprocesseur (les lignes commençant par le symbole #) ont disparu.



Vous pouvez utiliser l'option `-E` lors de la compilation pour requérir uniquement l'utilisation du préprocesseur. Ainsi, vous pouvez voir ce que donne votre code *après* son passage.



```
1 $ gcc -E fichier.c
```

IV.9.2. Les macroconstantes

Comme nous l'avons dit dans l'introduction, le préprocesseur permet la définition de macros, c'est-à-dire de substituts à des morceaux de code. Une macro est constituée des éléments suivants.

- La directive `#define`.
- Le nom de la macro qui, par convention, est souvent écrit en majuscules. Notez que vous pouvez choisir le nom que vous voulez, à condition de respecter les mêmes règles que pour les noms de variable ou de fonction.
- Une liste *optionnelle* de paramètres.
- La définition, c'est-à-dire le code par lequel la macro sera remplacée.

Dans le cas particulier où la macro n'a pas de paramètre, on parle de **macroconstante** (ou constante de préprocesseur). Leur substitution donne toujours le même résultat (d'où l'adjectif « constante »).

IV.9.2.1. Substitutions de constantes

Par exemple, pour définir une macroconstante `TAILLE` avec pour valeur `100`, nous utiliserons le code suivant.

```
1 #define TAILLE 100
```

L'exemple qui suit utilise cette macroconstante afin de ne pas multiplier l'usage de constantes entières. À chaque fois que la macroconstante `TAILLE` est utilisée, le préprocesseur remplacera celle-ci par sa définition, à savoir `100`.

```
1 #include <stdio.h>
2 #define TAILLE 100
3
4 int main(void)
5 {
6     int variable = 5;
7
8     /* On multiplie par TAILLE */
9     variable *= TAILLE;
10    printf("Variable vaut : %d\n", variable);
11
```



```
12     /* On additionne TAILLE */
13     variable += TAILLE;
14     printf("Variable vaut : %d\n", variable);
15     return 0;
16 }
```

Ce code sera remplacé, après le passage du préprocesseur, par celui-ci.

```
1  int main(void)
2  {
3      int variable = 5;
4
5      variable *= 100;
6      printf("Variable vaut : %d\n", variable);
7      variable += 100;
8      printf("Variable vaut : %d\n", variable);
9      return 0;
10 }
```

Nous n'avons pas inclus le contenu de `<stdio.h>` car celui-ci est trop long et trop compliqué pour le moment. Néanmoins, l'exemple permet d'illustrer le principe des macros et surtout de leur avantage : il suffit de changer la définition de la macroconstante `TAILLE` pour que le reste du code s'adapte.

?

D'accord, mais je peux aussi très bien utiliser une variable constante, non ?

Dans certains cas (comme dans l'exemple ci-dessus), utiliser une variable constante donnera le même résultat. Toutefois, une variable nécessitera le plus souvent de réserver de la mémoire et, si celle-ci doit être partagée par différents fichiers, de jouer avec les déclarations et les définitions.

i

Notez qu'il vous est possible de définir une macroconstante lors de la compilation à l'aide de l'option `-D`.

```
1  zcc -DTAILLE=100
```

Ceci revient à définir une macroconstante `TAILLE` au début de chaque fichier qui sera substituée par `100`.

IV.9.2.2. Substitutions d'instructions

Si les macroconstantes sont souvent utilisées en vue de substituer des constantes, il est toutefois aussi possible que leur définition comprenne des suites d'instructions. L'exemple ci-dessous

IV. Agrégats, mémoire et fichiers

définit une macroconstante `BONJOUR` qui sera remplacée par `puts("Bonjour !")`.

```
1 #include <stdio.h>
2
3 #define BONJOUR puts("Bonjour !")
4
5 int main(void)
6 {
7     BONJOUR;
8     return 0;
9 }
```

Notez que nous n'avons pas placé de point-virgule dans la définition de la macroconstante, tout d'abord afin de pouvoir en placer un lors de son utilisation, ce qui est plus naturel et, d'autre part, pour éviter des doublons qui peuvent s'avérer fâcheux.

En effet, si nous ajoutons un point virgule à la fin de la définition, il ne faut pas en rajouter un lors de l'appel, sous peine de risquer des erreurs de syntaxe.

```
1 #include <stdio.h>
2
3 #define BONJOUR puts("Bonjour !");
4 #define AUREVOIR puts("Au revoir !");
5
6 int main(void)
7 {
8     if (1)
9         BONJOUR; /* erreur ! */
10    else
11        AUREVOIR;
12
13    return 0;
14 }
```

Le code donnant en réalité ceci.

```
1 int main(void)
2 {
3     if (1)
4         puts("Bonjour !");;
5     else
6         puts("Au revoir !");;
7
8     return 0;
9 }
```

Ce qui est incorrect. Pour faire simple : le corps d'une condition ne peut comprendre qu'une instruction, or `puts("Bonjour !");` en est une, le point-virgule seul (`;`) en est une autre. Dès lors, il y a une instruction entre le `if` et le `else`, ce qui n'est pas permis. L'exemple ci-dessous pose le même problème en étant un peu plus limpide.

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     if (1)
7         puts("Bonjour !");
8
9     puts("Je suis une instruction mal placée !");
10
11    else
12        puts("Au revoir !");
13
14    return 0;
15 }
```

IV.9.2.3. Macros dans d'autres macros

La définition d'une macro peut parfaitement comprendre une ou plusieurs autres macros (qui, à leurs tours, peuvent également comprendre des macros dans leur définition et ainsi de suite). L'exemple suivant illustre cela en définissant une macroconstante `NB_PIXELS` correspondant à la multiplication entre les deux macroconstantes `LONGUEUR` et `HAUTEUR`.

```
1 #define LONGUEUR 1024
2 #define HAUTEUR 768
3 #define NB_PIXELS (LONGUEUR * HAUTEUR)
```

La règle suivante permet d'éviter de répéter les opérations de remplacements « à l'infini » :



Une macro n'est pas substituée si elle est utilisée dans sa propre définition.

Ainsi, dans le code ci-dessous, `LONGUEUR` donnera `LONGUEUR * 2 / 2` et `HAUTEUR` sera substituée par `HAUTEUR / 2 * 2`. En effet, lors de la substitution de la macroconstante `LONGUEUR`, la macroconstante `HAUTEUR` est remplacée par sa définition : `LONGUEUR * 2`. Or, comme nous sommes en train de substituer la macroconstante `LONGUEUR`, `LONGUEUR` sera laissé tel quel. Le même raisonnement peut être appliqué pour la macroconstante `HAUTEUR`.

```
1 #define LONGUEUR (HAUTEUR / 2)
2 #define HAUTEUR (LONGUEUR * 2)
```

IV.9.2.4. Définition sur plusieurs lignes

La définition d'une macro doit normalement tenir sur une seule ligne. C'est-à-dire que dès que la fin de ligne est atteinte, la définition est considérée comme terminée. Ainsi, dans le code suivant, la définition de la macroconstante `BONJOUR_AUREVOIR` ne comprend en fait que `puts("Bonjour");`.

```
1 #define BONJOUR_AUREVOIR puts("Bonjour");
2 puts("Au revoir")
```

Pour que cela fonctionne, nous sommes donc contraints de la rédiger comme ceci.

```
1 #define BONJOUR_AUREVOIR puts("Bonjour"); puts("Au revoir")
```

Ce qui est assez peu lisible et peu pratique si notre définition doit comporter une multitude d'instructions ou, pire, des blocs d'instructions... Heureusement, il est possible d'indiquer au préprocesseur que plusieurs lignes doivent être fusionnées. Cela se fait à l'aide du symbole `\`, que nous plaçons en fin de ligne.

```
1 #define BONJOUR_AUREVOIR puts("Bonjour");\
2 puts("Au revoir")
```

Lorsque le préprocesseur rencontre une barre oblique inverse (`\`) suivie d'une fin de ligne, celles-ci sont supprimées et la ligne qui suit est fusionnée avec la ligne courante.



Notez bien qu'*aucun espace* n'est ajouté durant l'opération !

```
1 #define BONJOUR_AUREVOIR\
2 puts("Bonjour");\
3 puts("Au revoir")
```

Le code ci-dessus est donc incorrect car il donne en vérité ceci.



```
1 #define BONJOUR_AUREVOIRputs("Bonjour");puts("Au revoir")
```



L'utilisation de la fusion ne se limite pas aux définitions de macros, il est possible de l'employer n'importe où dans le code source. Bien que cela ne soit pas obligatoire (une instruction pouvant être répartie sur plusieurs lignes), il est possible d'y recourir pour couper une ligne un peu trop longue tout en indiquant cette césure de manière claire.

```
1 printf(
    "Une phrase composée de plusieurs résultats à présenter : %d, %f, %f, %d
    \n",\
2 a, b, c, d, e, f);
```

Si vous souhaitez inclure un bloc d'instructions au sein d'une définition, ne perdez pas de vue que celui-ci constitue une instruction et donc que vous retombez sur le problème exposé plus haut.

```
1 #include <stdio.h>
2
3 #define BONJOUR\
4     {\
5         puts("Bonjour !");\
6     }
7 #define AUREVOIR\
8     {\
9         puts("Au revoir !");\
10    }
11
12 int main(void)
13 {
14     if (1)
15         BONJOUR; /* erreur ! */
16     else
17         AUREVOIR;
18
19     return 0;
20 }
```

Une solution fréquente à ce problème consiste à recourir à une boucle `do {} while` avec une condition nulle (de sorte qu'elle ne soit exécutée qu'une seule fois), celle-ci ne constituant qu'une seule instruction *avec* le point-virgule final.

```
1 #include <stdio.h>
2
3 #define BONJOUR\
4     do {\
5         puts("Bonjour !");\
6     } while(0)
7 #define AUREVOIR\
8     do {\
9         puts("Au revoir !");\
10    } while(0)
11
12 int main(void)
13 {
14     if (1)
15         BONJOUR; /* ok */
16     else
17         AUREVOIR;
18
19     return 0;
20 }
```

IV.9.2.4.1. Définition nulle

Sachez que le corps d'une macro peut parfaitement être vide.

```
1 #define MACRO
```

Dans un tel cas, la macro ne sera tout simplement pas substituée par quoi que ce soit. Bien que cela puisse paraître étrange, cette technique est souvent utilisée, notamment pour la compilation conditionnelle que nous verrons bientôt.

IV.9.2.5. Annuler une définition

Enfin, une définition peut être annulée à l'aide de la directive `#undef` en lui spécifiant le nom de la macro qui doit être détruite.

```
1 #define TAILLE 100
2 #undef TAILLE
3
4 /* TAILLE n'est à présent plus utilisable. */
```

IV.9.3. Les macrofonctions

Pour l'instant, nous n'avons manipulé que des macroconstantes, c'est-à-dire des macros n'employant pas de paramètres. Comme vous vous en doutez, une **macrofonction** est une macro qui accepte des paramètres et les emploie dans sa définition.

```
1 #define MACRO(paramètre, autre_paramètre) définition
```

Pour ce faire, le nom de la macro est suivi de parenthèses contenant le nom des paramètres séparés par une virgule. Chacun d'eux peut ensuite être utilisé dans la définition de la macrofonction et sera remplacé par la suite fournie en argument lors de l'appel à la macrofonction.



Notez bien que nous n'avons pas parlé de « valeur » pour les arguments. En effet, n'importe quelle suite de symboles peut-être passée en argument d'une macrofonction, y compris du code. C'est d'ailleurs ce qui fait la puissance du préprocesseur.

Illustrons ce nouveau concept avec un exemple : nous allons écrire deux macros : **EUR** qui convertira une somme en euro en francs (français) et **FRF** qui fera l'inverse. Pour rappel, un euro équivaut à 6,55957 francs français.

👁 Contenu masqué n°44

Appliquons encore ce concept avec un deuxième exercice : essayez de créer la macro **MIN** qui renvoie le minimum entre deux nombres.

👁 Contenu masqué n°45

IV.9.3.1. Priorité des opérations

Quelque chose vous a peut-être frappé dans les corrections : pourquoi écrire **(x)** et pas simplement **x** ?

En fait, il s'agit d'une protection en vue d'éviter certaines ambiguïtés. En effet, si l'on n'y prend pas garde, on peut par exemple avoir des surprises dues à la priorité des opérateurs. Prenons l'exemple d'une macro **MUL** qui effectue une multiplication.

```
1 #define MUL(a, b) (a * b)
```

Tel quel, le code peut poser des problèmes. En effet, si nous appelons la macrofonction comme ceci.

```
1 MUL(2+3, 4+5)
```

Nous obtenons comme résultat 19 (la macro sera remplacée par `2 + 3 * 4 + 5`) et non 45, qui est le résultat attendu. Pour garantir la bonne marche de la macrofonction, nous devons rajouter des parenthèses.

```
1 #define MUL(a, b) ((a) * (b))
```

Dans ce cas, nous obtenons bien le résultat souhaité, c'est-à-dire 45 `((2 + 3) * (4 + 5))`.



Nous vous conseillons de rajouter des parenthèses en cas de doute pour éviter toute erreur.

IV.9.3.2. Les effets de bords

Pour finir, une petite mise en garde : évitez d'utiliser plus d'une fois un paramètre dans la définition d'une macro en vue d'éviter de multiplier d'éventuels **effets de bord**.



Des effets de quoi ?

Un effet de bord est une modification du contexte d'exécution. Vous voilà bien avancé nous direz-vous... En fait, vous en avez déjà rencontré, l'exemple le plus typique étant une affectation.

```
1 a = 10;
```

Dans cet exemple, le contexte d'exécution du programme (qui comprend ses variables) est modifié puisque la valeur d'une variable est changée. Ainsi, imaginez que la macro `MUL` soit appelée comme suit.

```
1 MUL(a = 10, a = 20)
```

Après remplacement, celle-ci donnerait l'expression suivante : `((a = 10) * (a = 20))` qui est assez problématique... En effet, quelle sera la valeur de `a`, finalement ? 10 ou 20 ? Ceci est impossible à dire sans fixer une règle d'évaluation et... la norme n'en prévoit aucune dans ce cas ci. 🍊

Aussi, pour éviter ce genre de problèmes tordus, veillez à n'utiliser chaque paramètre qu'une seule fois.

IV.9.4. Les directives conditionnelles

Le préprocesseur dispose de cinq directives conditionnelles : `#if`, `#ifdef`, `#ifndef`, `#elif` et `#else`. Ces dernières permettent de conserver ou non une portion de code en fonction de la validité d'une condition (si la condition est vraie, le code est gardé sinon il est passé). Chacune de ces directives (ou suite de directives) doit être terminée par une directive `#endif`.

IV.9.4.1. Les directives `#if`, `#elif` et `#else`

IV.9.4.1.1. Les directives `#if` et `#elif`

Les directives `#if` et `#elif`, comme l'instruction `if`, attendent une expression conditionnelle ; toutefois, celles-ci sont plus restreintes étant donné que nous sommes dans le cadre du préprocesseur. Ce dernier se contentant d'effectuer des substitutions, il n'a par exemple aucune connaissance des mots-clés du langage C ou des variables qui sont employées.

Ainsi, les conditions ne peuvent comporter que des expressions *entières* (ce qui exclut les nombres flottants et les pointeurs) et *constantes* (ce qui exclut l'utilisation d'opérateurs à effets de bord comme `=`). Aussi, les mots-clés et autres identificateurs (hormis les macros) présents dans la définition *sont ignorés* (plus précisément, ils sont remplacés par `0`).

L'exemple ci-dessous explicite cela.

```
1 #if 1.89 > 1.88 /* Incorrect, ce ne sont pas des entiers */
2 #if sizeof(int) == 4 /* Équivalent à 0(0) == 4, 'sizeof' et 'int'
   étant des mots-clés */
3 #if (a = 20) == 20 /* Équivalent à (0 = 20) == 4, 'a' étant un
   identificateur */
```

Techniquement, seuls les opérateurs suivants peuvent être utilisés : `+` (binaire ou unaire), `-` (binaire ou unaire), `*`, `/`, `%`, `==`, `!=`, `<=`, `<`, `>`, `>=`, `!`, `&&`, `||`, `,`, l'opérateur ternaire et les opérateurs de manipulations des *bits*, que nous verrons au chapitre suivant.



Bien entendu, vous pouvez également utiliser des parenthèses afin de régler la priorité des opérations.

IV.9.4.1.1.1. L'opérateur `defined` Le préprocesseur fournit un opérateur supplémentaire utilisable dans les conditions : `defined`. Celui-ci prend comme opérande un nom de macro et retourne 1 ou 0 suivant que ce nom corresponde ou non à une macro définie.

```
1 #if defined TAILLE
```

Celui-ci est fréquemment utilisé pour produire des programmes portables. En effet, chaque système et chaque compilateur définissent généralement une ou des macroconstantes qui lui sont propres. En vérifiant si une de ces constantes existe, nous pouvons déterminer sur quelle plate-forme et avec quel compilateur nous compilons et adapter le code en conséquence.



Notez que ces constantes sont propres à un système d'exploitation et/ou compilateur donné, elles ne sont donc pas spécifiées par la norme du langage C.

IV.9.4.1.2. La directive `#else`

La directive `#else` quant à elle, se comporte comme l'instruction éponyme.

IV.9.4.1.3. Exemple

Voici un exemple d'utilisation.

```
1 #include <stdio.h>
2
3 #define A 2
4
5 int main(void)
6 {
7     #if A < 0
8         puts("A < 0");
9     #elif A > 0
10        puts("A > 0");
11    #else
12        puts("A == 0");
13    #endif
14    return 0;
15 }
```

Résultat

```
1 A > 0
```



Notez que les directives conditionnelles peuvent être utilisées à la place des commentaires en vue d'empêcher la compilation d'un morceau de code. L'avantage de cette technique par rapport aux commentaires est qu'elle vous évite de produire des commentaires imbriqués.



```

1  #if 0
2  /* On multiplie par TAILLE */
3  variable *= TAILLE;
4  printf("Variable vaut : %d\n", variable);
5  #endif

```

IV.9.4.2. Les directives `#ifdef` et `#ifndef`

Ces deux directives sont en fait la contraction, respectivement, de la directive `#if defined` et `#if !defined`. Si vous n'avez qu'une seule constante à tester, il est plus rapide d'utiliser ces deux directives à la place de leur version longue.

IV.9.4.2.1. Protection des fichiers d'en-tête

Cela étant dit, vous devriez à présent être en mesure de comprendre le fonctionnement des directives jusqu'à présent utilisées dans les fichiers en-têtes.

```

1  #ifndef CONSTANCE_H
2  #define CONSTANCE_H
3
4  /* Les déclarations */
5
6  #endif

```

La première directive vérifie que la macroconstante `CONSTANTE_H` n'a pas encore été définie. Si ce n'est pas le cas, elle est définie (ici avec une valeur nulle) et le bloc de la condition (comprenant le contenu du fichier d'en-tête) est inclus. Si elle a déjà été définie, le bloc est sauté et le contenu du fichier n'est ainsi pas à nouveau inclus.



Pour rappel, ceci est nécessaire pour éviter des problèmes d'inclusions multiples, par exemple lorsqu'un fichier A inclut un fichier B, qui lui-même inclut le fichier A.

Conclusion

Avec ce chapitre, vous devriez pouvoir utiliser le préprocesseur de manière basique et éviter plusieurs de ses pièges fréquents. 🍊

En résumé

- La directive `#include` permet d'inclure le contenu d'un fichier dans un autre ;
- La directive `#define` permet de définir des macros qui sont des substituts à des morceaux de code ;
- Les macros sans paramètres sont appelées macroconstantes ;
- Les macros qui acceptent des paramètres et les emploient dans leur définition sont appelées macrofonctions ;
- Des effets de bord sont possibles si un paramètre est utilisé plus d'une fois dans la définition d'une macrofonction ;
- Grâce aux directives conditionnelles, il est possible de conserver ou non une portion de code, en fonction de conditions.

Contenu masqué

Contenu masqué n°44

```
1 #include <stdio.h>
2
3 #define EUR(x) ((x) / 6.55957)
4 #define FRF(x) ((x) * 6.55957)
5
6 int main(void)
7 {
8     printf("Dix francs français valent %f euros.\n", EUR(10));
9     printf("Dix euros valent %f francs français.\n", FRF(10));
10    return 0;
11 }
```

Résultat

```
1 Dix francs français valent 1.524490 euros.
2 Dix euros valent 65.595700 francs français.
```

[Retourner au texte.](#)

Contenu masqué n°45

```
1 #include <stdio.h>
2
3 #define MIN(a, b) ((a) < (b) ? (a) : (b))
4
5 int main(void)
6 {
7     printf("Le minimum entre 16 et 32 est %d.\n", MIN(16, 32));
8     printf("Le minimum entre 2+9+7 et 3*8 est %d.\n", MIN(2+9+7,
9         3*8));
10    return 0;
11 }
```

Résultat

```
1 Le minimum entre 16 et 32 est 16.
2 Le minimum entre 2+9+7 et 3*8 est 18.
```



Remarquez que nous avons utilisé des expressions composées lors de la deuxième utilisation de la macrofonction `MIN`.

[Retourner au texte.](#)

IV.10. TP : un Puissance 4

Introduction

Après ce que nous venons de découvrir, voyons comment mettre tout cela en musique à l'aide d'un exercice récapitulatif : réaliser un *Puissance 4*.

IV.10.1. Première étape : le jeu

Dans cette première partie, vous allez devoir réaliser un « Puissance 4 » pour deux joueurs *humains*. Le programme final devra ressembler à ceci.

1	1	2	3	4	5	6	7
2	+---+---+---+---+---+---+						
3							
4	+---+---+---+---+---+---+						
5							
6	+---+---+---+---+---+---+						
7							
8	+---+---+---+---+---+---+						
9							
10	+---+---+---+---+---+---+						
11							
12	+---+---+---+---+---+---+						
13							
14	+---+---+---+---+---+---+						
15	1	2	3	4	5	6	7
16							
17	Joueur 1 : 1						
18							
19	1	2	3	4	5	6	7
20	+---+---+---+---+---+---+						
21							
22	+---+---+---+---+---+---+						
23							
24	+---+---+---+---+---+---+						
25							
26	+---+---+---+---+---+---+						
27							
28	+---+---+---+---+---+---+						

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
		0														
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
		1		2		3		4		5		6		7		
Joueur 2 : 7																
		1		2		3		4		5		6		7		
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
		0											X			
	+	-	-	+	-	-	+	-	-	+	-	-	+	-	-	+
		1		2		3		4		5		6		7		
Joueur 1 :																

Pour ceux qui ne connaissent pas le jeu Puissance 4, ce dernier se joue à l'aide d'une grille verticale de sept colonnes sur six lignes. Chaque joueur dispose de vingt et un jetons d'une couleur (le plus souvent, rouge et jaune traduit dans notre exemple par les caractères « O » et « X ») et place ceux-ci au sein de la grille à tour de rôle.

Pour gagner le jeu, un joueur doit aligner quatre jetons verticalement, horizontalement ou en oblique. Il s'agit donc du même principe que le Morpion à une différence prêt : la grille est *verticale* ce qui signifie que les jetons tombent au fond de la colonne choisie par le joueur. Pour plus de détails, je vous renvoie à [l'article dédié sur Wikipédia](#) .

Maintenant que vous savez cela, il est temps de passer à la réalisation de celui-ci.

Bon travail ! 🍊

IV.10.2. Correction

Bien, l'heure de la correction est venue. 🍊

👁 Contenu masqué n°46

Le programme commence par initialiser la grille (tous les caractères la composant sont des espaces au début, symbolisant une case vide) et l'afficher.

Ensuite, il est demandé au premier joueur d'effectuer une action. Cela est réalisé via la fonction `demande_action()` qui attend du joueur soit qu'il entre un numéro de colonne, soit la lettre Q ou q. Si le joueur entre autre chose ou que la colonne indiquée est invalide, une nouvelle saisie lui est demandée.

Dans le cas où le numéro de colonne est valable, la fonction `calcule_position()` est appelée afin de calculer le numéro de ligne et de stocker celui-ci et le numéro de colonne dans une structure. Après quoi la grille est mise à jour en ajoutant un nouveau jeton et à nouveau affichée.

La fonction `statut_jeu()` entre alors en action et détermine si quatre jetons adjacents sont présents ou non ou si la grille est complète. Cela est notamment réalisé à l'aide de la fonction `calcule_nb_jetons_depuis()` qui fait elle-même appel à la fonction `calcule_nb_jetons_depuis_vers()`. Si aucune des deux conditions n'est remplie, c'est reparti pour un tour sinon la boucle est quittée et le programme se termine.

La fonction `calcule_nb_jetons_depuis_vers()` mérite que l'on s'attarde un peu sur elle. Celle-ci reçoit une position dans la grille et se déplace suivant les valeurs attribuées à `dpl_hrz` (soit « déplacement horizontal ») et `dpl_vrt` (pour « déplacement vertical »). Celle-ci effectue son parcours aussi longtemps que des jetons identiques sont rencontrés et que le déplacement a lieu dans la grille. Elle retourne ensuite le nombre de jetons identiques rencontrés (notez que nous commençons le décompte à un puisque, par définition, la position indiquée est celle qui vient d'être jouée par un des joueurs).

La fonction `calcule_nb_jetons_depuis()` appelle à plusieurs reprises la fonction `calcule_nb_jetons_depuis_vers()` afin d'obtenir le nombre de jetons adjacents de la colonne courante (déplacement de un vers le bas), de la ligne (deux déplacements : un vers la gauche et un vers la droite) et des deux obliques (deux déplacements pour chacune d'entre elles).

Notez qu'afin d'éviter de nombreux passages d'arguments, nous avons employé la variable globale `grille` (dont le nom est explicite).



Nous insistons sur le fait que nous vous présentons *une* solution parmi d'autres. Ce n'est pas parce que votre façon de procéder est différente qu'elle est incorrecte.

IV.10.3. Deuxième étape : une petite IA

IV.10.3.1. Introduction

Dans cette seconde partie, votre objectif va être de construire une petite intelligence artificielle. Votre programme va donc devoir demander si un ou deux joueurs sont présents et jouer à la place du second joueur s'il n'y en a qu'un seul.

Afin de vous aider dans la réalisation de cette tâche, nous allons vous présenter un algorithme simple que vous pourrez mettre en œuvre. Le principe est le suivant : pour chaque emplacement possible (il y en aura toujours au maximum sept), nous allons calculer combien de pièces composeraient la ligne si nous jouions à cet endroit *sans tenir compte de notre couleur* (autrement dit, le jeton que nous jouons est considéré comme étant des *deux* couleurs). Si une valeur est plus

IV. Agrégats, mémoire et fichiers

élevée que les autres, l'emplacement correspondant sera choisi. Si en revanche il y a plusieurs nombres égaux, une des cases sera choisie « au hasard » (nous y reviendrons).

Cet algorithme connaît toutefois une exception : s'il s'avère lors de l'analyse qu'un coup permet à l'ordinateur de gagner, alors le traitement s'arrêtera là et le mouvement est immédiatement joué.

Par exemple, dans la grille ci-dessous, il est possible de jouer à sept endroits (indiqués à l'aide d'un point d'interrogation).

1		1	2	3	4	5	6	7
2	+---+---+---+---+---+---+---+							
3					?			
4	+---+---+---+---+---+---+---+							
5					0		?	
6	+---+---+---+---+---+---+---+							
7					0	X		?
8	+---+---+---+---+---+---+---+							
9					0	0	X	
10	+---+---+---+---+---+---+---+							
11			?		?	X	0	0
12	+---+---+---+---+---+---+---+							
13		?	X		0	0	X	X
14	+---+---+---+---+---+---+---+							
15		1	2	3	4	5	6	7

Si nous appliquons l'algorithme que nous venons de décrire, nous obtenons les valeurs suivantes.

1		1	2	3	4	5	6	7
2	+---+---+---+---+---+---+---+							
3					4			
4	+---+---+---+---+---+---+---+							
5					0	2		
6	+---+---+---+---+---+---+---+							
7					0	X	2	
8	+---+---+---+---+---+---+---+							
9					0	0	X	
10	+---+---+---+---+---+---+---+							
11			2		2	X	0	0
12	+---+---+---+---+---+---+---+							
13		2	X		0	0	X	X
14	+---+---+---+---+---+---+---+							
15		1	2	3	4	5	6	7

Le programme jouera donc dans la colonne quatre, cette dernière ayant la valeur la plus élevée (ce qui tombe bien puisque l'autre joueur gagnerait si l'ordinateur jouait ailleurs 🍊).



Pour effectuer le calcul, vous pouvez vous aider de la fonction `calcule_nb_jetons_depuis()`.

IV.10.3.2. Tirage au sort

Nous vous avons dit que si l'ordinateur doit choisir entre des valeurs identiques, ce dernier devrait en choisir une « au hasard ». Cependant, un ordinateur est en vérité incapable d'effectuer cette tâche (c'est pour cela que nous employons des guillemets). Pour y remédier, il existe des algorithmes qui permettent de produire des suites de nombres dits *pseudo-aléatoires*, c'est-à-dire qui s'approchent de suites aléatoires sur le plan statistique.

La plupart de ceux-ci fonctionnent à partir d'une *graine*, c'est-à-dire un nombre de départ qui va déterminer tout le reste de la suite. C'est ce système qui a été choisi par la bibliothèque standard du C. Deux fonctions vous sont dès lors proposées : `srand()` et `rand()` (elles sont déclarées dans l'en-tête `<stdlib.h>`).

```
1 void srand(unsigned int seed);
2 int rand(void);
```

La fonction `srand()` est utilisée pour initialiser la génération de nombres à l'aide de la graine fournie en argument (un entier non signé en l'espèce). Le plus souvent vous ne l'appellerez qu'une seule fois au début de votre programme sauf si vous souhaitez réinitialiser la génération de nombres.

La fonction `rand()` retourne un nombre pseudo-aléatoire compris entre zéro et `RAND_MAX` (qui est une constante également définie dans l'en-tête `<stdlib.h>`) suivant la graine fournie à la fonction `srand()`.

Toutefois, il y a un petit bémol : étant donné que l'algorithme de génération se base sur la graine fournie pour construire la suite de nombres, une même graine donnera *toujours la même suite* ! Dès lors, comment faire pour que les nombres générés ne soient pas identiques entre deux exécutions du programme ? Pour que cela soit possible, nous devrions fournir un nombre différent à `srand()` à chaque fois, mais comment obtenir un tel nombre ?

C'est ici que la fonction `time()` (déclarée dans l'en-tête `<time.h>`) entre en jeu.

```
1 time_t time(time_t *t);
```

La fonction `time()` retourne la date actuelle sous forme d'une valeur de type `time_t` (qui est un nombre entier ou flottant). Le plus souvent, il s'agit du nombre de secondes écoulé depuis une date fixée arbitrairement appelée [Epoch](#) [↗](#) en anglais. Cette valeur peut également être stockée dans une variable de type `time_t` dont l'adresse lui est fournie en argument (un pointeur nul peut lui être envoyé si cela n'est pas nécessaire). En cas d'erreur, la fonction retourne la valeur `-1` convertie vers le type `time_t`.

Pour obtenir des nombres différents à chaque exécution, nous pouvons donc appeler `srand()` avec le retour de la fonction `time()` converti en entier non signé. L'exemple suivant génère donc une suite de trois nombres pseudo-aléatoires différents à chaque exécution (en vérité à chaque exécution espacée d'une seconde).

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5
6 int main(void)
7 {
8     time_t t;
9
10    if (time(&t) == (time_t)-1)
11    {
12        fprintf(stderr, "Impossible d'obtenir la date courante\n");
13        return EXIT_FAILURE;
14    }
15
16    srand((unsigned)t);
17    printf("1 : %d\n", rand());
18    printf("2 : %d\n", rand());
19    printf("3 : %d\n", rand());
20    return 0;
21 }
```

IV.10.3.2.1. Tirer un nombre dans un intervalle donné

Il est possible de générer des nombres dans un intervalle précis en procédant en deux temps. Tout d'abord, il nous est nécessaire d'obtenir un nombre compris entre zéro inclus et un exclus. Pour ce faire, nous pouvons diviser le nombre pseudo-aléatoire obtenu par la plus grande valeur qui peut être retournée par la fonction `rand()` augmentée de un. Celle-ci nous est fournie via la macroconstante `RAND_MAX` qui est définie dans l'en-tête `<stdlib.h>`.

```
1 double nb_aleatoire(void)
2 {
3     return rand() / (RAND_MAX + 1.);
4 }
```

Notez que nous avons bien indiqué que la constante `1.` est un nombre flottant afin que le résultat de l'opération soit de type `double`.

Ensuite, il nous suffit de multiplier le nombre obtenu par la différence entre le maximum et le minimum augmenté de un et d'y ajouter le minimum.

```
1 int nb_aleatoire_entre(int min, int max)
2 {
3     return nb_aleatoire() * (max - min + 1) + min;
4 }
```

Afin d'illustrer ce qui vient d'être dit, le code suivant affiche trois nombres pseudo-aléatoires compris entre zéro et dix inclus.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5
6 static double nb_aleatoire(void)
7 {
8     return rand() / (RAND_MAX + 1.);
9 }
10
11
12 static int nb_aleatoire_entre(int min, int max)
13 {
14     return nb_aleatoire() * (max - min + 1) + min;
15 }
16
17
18 int main(void)
19 {
20     time_t t;
21
22     if (time(&t) == (time_t)-1)
23     {
24         fprintf(stderr, "Impossible d'obtenir la date courante\n");
25         return EXIT_FAILURE;
26     }
27
28     srand((unsigned)t);
29     printf("1 : %d\n", nb_aleatoire_entre(0, 10));
30     printf("2 : %d\n", nb_aleatoire_entre(0, 10));
31     printf("3 : %d\n", nb_aleatoire_entre(0, 10));
32     return 0;
33 }
```

À présent, c'est à vous ! 🍊

IV.10.4. Correction

👁 Contenu masqué n°47

Le programme demande désormais combien de joueurs sont présents. Dans le cas où ils sont deux, le programme se comporte de la même manière que précédemment. S'il n'y en a qu'un seul, la fonction `ia()` est appelée lors du tour du deuxième joueur. Cette fonction retourne un numéro de colonne après analyse de la grille. Notez également que les fonctions `time()` et `srand()` sont appelées au début afin d'initialiser la génération de nombre pseudo-aléatoires.

La fonction `ia()` agit comme suit :

- les numéros des colonnes ayant les plus grandes valeurs calculées sont stockés dans le tableau `meilleurs_col` ;
- si la colonne est complète, celle-ci est passée ;
- si la colonne est incomplète, la fonction `calcule_position_depuis()` est appelée afin de déterminer combien de pièces seraient adjacentes si un jeton était posé à cet endroit. S'il s'avère que l'ordinateur peut gagner, la fonction retourne immédiatement le numéro de la colonne courante ;
- un des numéros de colonne présent dans le tableau `meilleurs_col` est tiré « au hasard » et est retourné.

IV.10.5. Troisième et dernière étape: un système de sauvegarde/restauration

Pour terminer, nous allons ajouter un système de sauvegarde/restauration à notre programme.

Durant une partie, un utilisateur doit pouvoir demander à sauvegarder le jeu courant ou de charger une ancienne partie en lieu et place d'entrer un numéro de colonne.

À vous de voir comment organiser les données au sein d'un fichier et comment les récupérer depuis votre programme. 🍏

IV.10.6. Correction

👁 Contenu masqué n°48

À présent, durant la partie, un joueur peut entrer la lettre « s » (en minuscule ou en majuscule) afin d'effectuer une sauvegarde de la partie courante au sein d'un fichier qu'il doit spécifier. Les données sont sauvegardées sous forme de texte avec, dans l'ordre : le joueur courant, le nombre de joueurs et le contenu de la grille. Une fois la sauvegarde effectuée, le jeu est stoppé.

Il peut également entrer la lettre « c » (de nouveau en minuscule ou en majuscule) pour charger une partie précédemment sauvegardée.

Contenu masqué

Contenu masqué n°46

```
1  #include <ctype.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  #define P4_COLONNES (7)
6  #define P4_LIGNES (6)
7
8  #define J1_JETON ('O')
9  #define J2_JETON ('X')
10
11 #define ACT_ERR (0)
12 #define ACT_JOUER (1)
13 #define ACT_NOUVELLE_SAISIE (2)
14 #define ACT_QUITTER (3)
15
16 #define STATUT_OK (0)
17 #define STATUT_GAGNE (1)
18 #define STATUT_EGALITE (2)
19
20 struct position
21 {
22     int colonne;
23     int ligne;
24 };
25
26 static void affiche_grille(void);
27 static void calcule_position(int, struct position *);
28 static unsigned calcule_nb_jetons_depuis_vers(struct position *,
29     int, int, char);
30 static unsigned calcule_nb_jetons_depuis(struct position *, char);
31 static int coup_valide(int);
32 static int demande_action(int *);
33 static int grille_complete(void);
34 static void initialise_grille(void);
35 static int position_valide(struct position *);
36 static int statut_jeu(struct position *pos, char);
37 static unsigned umax(unsigned, unsigned);
38 static int vider_tampon(FILE *);
39
40 static char grille[P4_COLONNES][P4_LIGNES];
41
42 static void affiche_grille(void)
43 {
```

```

44     /*
45     * Affiche la grille pour le ou les joueurs.
46     */
47
48     int col;
49     int lgn;
50
51     putchar('\n');
52
53     for (col = 1; col <= P4_COLONNES; ++col)
54         printf(" %d ", col);
55
56     putchar('\n');
57     putchar('+');
58
59     for (col = 1; col <= P4_COLONNES; ++col)
60         printf("----+");
61
62     putchar('\n');
63
64     for (lgn = 0; lgn < P4_LIGNES; ++lgn)
65     {
66         putchar('|');
67
68         for (col = 0; col < P4_COLONNES; ++col)
69             if (isalpha(grille[col][lgn]))
70                 printf(" %c |", grille[col][lgn]);
71             else
72                 printf(" %c |", ' ');
73
74         putchar('\n');
75         putchar('+');
76
77         for (col = 1; col <= P4_COLONNES; ++col)
78             printf("----+");
79
80         putchar('\n');
81     }
82
83     for (col = 1; col <= P4_COLONNES; ++col)
84         printf(" %d ", col);
85
86     putchar('\n');
87 }
88
89
90 static void calcule_position(int coup, struct position *pos)
91 {
92     /*
93     * Traduit le coup joué en un numéro de colonne et de ligne.

```

```

94     */
95
96     int lgn;
97
98     pos->colonne = coup;
99
100    for (lgn = P4_LIGNES - 1; lgn >= 0; --lgn)
101        if (grille[pos->colonne][lgn] == ' ')
102            {
103                pos->ligne = lgn;
104                break;
105            }
106    }
107
108
109    static unsigned calcule_nb_jetons_depuis_vers(struct position
110    *pos, int dpl_hrz, int dpl_vrt, char jeton)
111    {
112        /*
113         * Calcule le nombre de jetons adjacents identiques depuis une
114         * position donnée en se
115         * déplaçant de `dpl_hrz` horizontalement et `dpl_vrt`
116         * verticalement.
117         * La fonction s'arrête si un jeton différent ou une case vide
118         * est rencontrée ou si
119         * les limites de la grille sont atteintes.
120         */
121
122        struct position tmp;
123        unsigned nb = 1;
124
125        tmp.colonne = pos->colonne + dpl_hrz;
126        tmp.ligne = pos->ligne + dpl_vrt;
127
128        while (position_valide(&tmp))
129        {
130            if (grille[tmp.colonne][tmp.ligne] == jeton)
131                ++nb;
132            else
133                break;
134
135            tmp.colonne += dpl_hrz;
136            tmp.ligne += dpl_vrt;
137        }
138
139        return nb;
140    }

```



```

139 static unsigned calcule_nb_jetons_depuis(struct position *pos,
140     char jeton)
141 {
142     /*
143      * Calcule le nombre de jetons adjacents en vérifiant la
144      * colonne courante,
145      * de la ligne courante et des deux obliques courantes.
146      * Pour ce faire, la fonction calcule_nb_jeton_depuis_vers()
147      * est appelé à
148      * plusieurs reprises afin de parcourir la grille suivant la
149      * vérification
150      * à effectuer.
151      */
152     unsigned max;
153
154     max = calcule_nb_jetons_depuis_vers(pos, 0, 1, jeton);
155     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 0,
156         jeton) + \
157         calcule_nb_jetons_depuis_vers(pos, -1, 0, jeton) - 1);
158     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 1,
159         jeton) + \
160         calcule_nb_jetons_depuis_vers(pos, -1, -1, jeton) - 1);
161     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, -1,
162         jeton) + \
163         calcule_nb_jetons_depuis_vers(pos, -1, 1, jeton) - 1);
164
165     return max;
166 }
167
168 static int coup_valide(int col)
169 {
170     /*
171      * Si la colonne renseignée est inférieure ou égal à zéro
172      * ou que celle-ci est supérieure à la longueur du tableau
173      * ou que la colonne indiquée est saturée
174      * alors le coup est invalide.
175      */
176     if (col <= 0 || col > P4_COLONNES || grille[col - 1][0] != ' ')
177         return 0;
178
179     return 1;
180 }
181
182 static int demande_action(int *coup)
183 {
184     /*

```

```

182     * Demande l'action à effectuer au joueur courant.
183     * S'il entre un chiffre, c'est qu'il souhaite jouer.
184     * S'il entre la lettre « Q » ou « q », c'est qu'il souhaite
      quitter.
185     * S'il entre autre chose, une nouvelle saisie sera demandée.
186     */
187
188     char c;
189     int ret = ACT_ERR;
190
191     if (scanf("%d", coup) != 1)
192     {
193         if (scanf("%c", &c) != 1)
194         {
195             fprintf(stderr, "Erreur lors de la saisie\n");
196             return ret;
197         }
198
199         switch (c)
200         {
201             case 'Q':
202             case 'q':
203                 ret = ACT_QUITTER;
204                 break;
205             default:
206                 ret = ACT_NOUVELLE_SAISIE;
207                 break;
208         }
209     }
210     else
211         ret = ACT_JOUER;
212
213     if (!vider_tampon(stdin))
214     {
215         fprintf(stderr, "Erreur lors de la vidange du tampon.\n");
216         ret = ACT_ERR;
217     }
218
219     return ret;
220 }
221
222
223 static int grille_complete(void)
224 {
225     /*
226     * Détermine si la grille de jeu est complète.
227     */
228
229     unsigned col;
230     unsigned lgn;

```

```

231
232     for (col = 0; col < P4_COLONNES; ++col)
233         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
234             if (grille[col][lgn] == ' ')
235                 return 0;
236
237     return 1;
238 }
239
240
241 static void initialise_grille(void)
242 {
243     /*
244      * Initialise les caractères de la grille.
245      */
246
247     unsigned col;
248     unsigned lgn;
249
250     for (col = 0; col < P4_COLONNES; ++col)
251         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
252             grille[col][lgn] = ' ';
253 }
254
255
256 static int position_valide(struct position *pos)
257 {
258     /*
259      * Vérifie que la position fournie est bien comprise dans la
260      * grille.
261      */
262
263     int ret = 1;
264
265     if (pos->colonne >= P4_COLONNES || pos->colonne < 0)
266         ret = 0;
267     else if (pos->ligne >= P4_LIGNES || pos->ligne < 0)
268         ret = 0;
269
270     return ret;
271 }
272
273 static int statut_jeu(struct position *pos, char jeton)
274 {
275     /*
276      * Détermine s'il y a lieu de continuer le jeu ou s'il doit
277      * être
278      * arrêté parce qu'un joueur a gagné ou que la grille est
279      * complète.

```

```

278     */
279
280     if (grille_complete())
281         return STATUT_EGALITE;
282     else if (calcule_nb_jetons_depuis(pos, jeton) >= 4)
283         return STATUT_GAGNE;
284
285     return STATUT_OK;
286 }
287
288
289 static unsigned umax(unsigned a, unsigned b)
290 {
291     /*
292      * Retourne le plus grand des deux arguments.
293      */
294
295     return (a > b) ? a : b;
296 }
297
298
299 static int vider_tampon(FILE *fp)
300 {
301     /*
302      * Vide les données en attente de lecture du flux spécifié.
303      */
304
305     int c;
306
307     do
308         c = fgetc(fp);
309     while (c != '\n' && c != EOF);
310
311     return ferror(fp) ? 0 : 1;
312 }
313
314
315 int main(void)
316 {
317     int statut;
318     char jeton = J1_JETON;
319
320     initialise_grille();
321     affiche_grille();
322
323     while (1)
324     {
325         struct position pos;
326         int action;
327         int coup;

```

```
328
329     printf("Joueur %d : ", (jeton == J1_JETON) ? 1 : 2);
330
331     action = demande_action(&coup);
332
333     if (action == ACT_ERR)
334         return EXIT_FAILURE;
335     else if (action == ACT_QUITTER)
336         return 0;
337     else if (action == ACT_NOUVELLE_SAISIE ||
338              !coup_valide(coup))
339     {
340         fprintf(stderr,
341                 "Vous ne pouvez pas jouer à cet endroit\n");
342         continue;
343     }
344
345     calcule_position(coup - 1, &pos);
346     grille[pos.colonne][pos.ligne] = jeton;
347     affiche_grille();
348     statut = statut_jeu(&pos, jeton);
349
350     if (statut != STATUT_OK)
351         break;
352
353     jeton = (jeton == J1_JETON) ? J2_JETON : J1_JETON;
354 }
355
356 if (statut == STATUT_GAGNE)
357     printf("Le joueur %d a gagné\n", (jeton == J1_JETON) ? 1 :
358         2);
359 else if (statut == STATUT_EGALITE)
360     printf("Égalité\n");
361
362 return 0;
363 }
```

[Retourner au texte.](#)

Contenu masqué n°47

```
1 #include <ctype.h>
2 #include <stddef.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define P4_COLONNES (7)
```

```

7  #define P4_LIGNES (6)
8
9  #define J1_JETON ('O')
10 #define J2_JETON ('X')
11
12 #define ACT_ERR (0)
13 #define ACT_JOUER (1)
14 #define ACT_NOUVELLE_SAISIE (2)
15 #define ACT_QUITTER (3)
16
17 #define STATUT_OK (0)
18 #define STATUT_GAGNE (1)
19 #define STATUT_EGALITE (2)
20
21 struct position
22 {
23     int colonne;
24     int ligne;
25 };
26
27 static void affiche_grille(void);
28 static void calcule_position(int, struct position *);
29 static unsigned calcule_nb_jetons_depuis_vers(struct position *,
30     int, int, char);
31 static unsigned calcule_nb_jetons_depuis(struct position *, char);
32 static int coup_valide(int);
33 static int demande_action(int *);
34 static int demande_nb_joueur(void);
35 static int grille_complete(void);
36 static int ia(void);
37 static void initialise_grille(void);
38 double nb_aleatoire(void);
39 int nb_aleatoire_entre(int, int);
40 static int position_valide(struct position *);
41 static int statut_jeu(struct position *pos, char);
42 static unsigned umax(unsigned, unsigned);
43 static int vider_tampon(FILE *);
44
45 static char grille[P4_COLONNES][P4_LIGNES];
46
47 static void affiche_grille(void)
48 {
49     /*
50      * Affiche la grille pour le ou les joueurs.
51      */
52
53     int col;
54     int lgn;
55

```

```

56     putchar('\n');
57
58     for (col = 1; col <= P4_COLONNES; ++col)
59         printf(" %d ", col);
60
61     putchar('\n');
62     putchar('+');
63
64     for (col = 1; col <= P4_COLONNES; ++col)
65         printf("----+");
66
67     putchar('\n');
68
69     for (lgn = 0; lgn < P4_LIGNES; ++lgn)
70     {
71         putchar('|');
72
73         for (col = 0; col < P4_COLONNES; ++col)
74             if (isalpha(grille[col][lgn]))
75                 printf(" %c |", grille[col][lgn]);
76             else
77                 printf(" %c |", ' ');
78
79         putchar('\n');
80         putchar('+');
81
82         for (col = 1; col <= P4_COLONNES; ++col)
83             printf("----+");
84
85         putchar('\n');
86     }
87
88     for (col = 1; col <= P4_COLONNES; ++col)
89         printf(" %d ", col);
90
91     putchar('\n');
92 }
93
94
95 static void calcule_position(int coup, struct position *pos)
96 {
97     /*
98      * Traduit le coup joué en un numéro de colonne et de ligne.
99      */
100
101     int lgn;
102
103     pos->colonne = coup;
104
105     for (lgn = P4_LIGNES - 1; lgn >= 0; --lgn)

```

```

106     if (grille[pos->colonne][lgn] == ' ')
107     {
108         pos->ligne = lgn;
109         break;
110     }
111 }
112
113
114 static unsigned calcule_nb_jetons_depuis_vers(struct position
115     *pos, int dpl_hrz, int dpl_vrt, char jeton)
116 {
117     /*
118     * Calcule le nombre de jetons adjacents identiques depuis une
119     * position donnée en se
120     * déplaçant de `dpl_hrz` horizontalement et `dpl_vrt`
121     * verticalement.
122     * La fonction s'arrête si un jeton différent ou une case vide
123     * est rencontrée ou si
124     * les limites de la grille sont atteintes.
125     */
126
127     struct position tmp;
128     unsigned nb = 1;
129
130     tmp.colonne = pos->colonne + dpl_hrz;
131     tmp.ligne = pos->ligne + dpl_vrt;
132
133     while (position_valide(&tmp))
134     {
135         if (grille[tmp.colonne][tmp.ligne] == jeton)
136             ++nb;
137         else
138             break;
139
140         tmp.colonne += dpl_hrz;
141         tmp.ligne += dpl_vrt;
142     }
143
144     return nb;
145 }
146
147 static unsigned calcule_nb_jetons_depuis(struct position *pos,
148     char jeton)
149 {
150     /*
151     * Calcule le nombre de jetons adjacents en vérifiant la
152     * colonne courante,
153     * de la ligne courante et des deux obliques courantes.

```



```

149     * Pour ce faire, la fonction calcule_nb_jeton_depuis_vers()
        est appelé à
150     * plusieurs reprises afin de parcourir la grille suivant la
        vérification
151     * à effectuer.
152     */
153
154     unsigned max;
155
156     max = calcule_nb_jetons_depuis_vers(pos, 0, 1, jeton);
157     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 0,
        jeton) + \
158     calcule_nb_jetons_depuis_vers(pos, -1, 0, jeton) - 1);
159     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 1,
        jeton) + \
160     calcule_nb_jetons_depuis_vers(pos, -1, -1, jeton) - 1);
161     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, -1,
        jeton) + \
162     calcule_nb_jetons_depuis_vers(pos, -1, 1, jeton) - 1);
163
164     return max;
165 }
166
167
168 static int coup_valide(int col)
169 {
170     /*
171     * Si la colonne renseignée est inférieure ou égale à zéro
172     * ou que celle-ci est supérieure à la longueur du tableau
173     * ou que la colonne indiquée est saturée
174     * alors le coup est invalide.
175     */
176
177     if (col <= 0 || col > P4_COLONNES || grille[col - 1][0] != ' ')
178         return 0;
179
180     return 1;
181 }
182
183
184 static int demande_action(int *coup)
185 {
186     /*
187     * Demande l'action à effectuer au joueur courant.
188     * S'il entre un chiffre, c'est qu'il souhaite jouer.
189     * S'il entre la lettre « Q » ou « q », c'est qu'il souhaite
        quitter.
190     * S'il entre autre chose, une nouvelle saisie sera demandée.
191     */
192

```

```

193     char c;
194     int ret = ACT_ERR;
195
196     if (scanf("%d", coup) != 1)
197     {
198         if (scanf("%c", &c) != 1)
199         {
200             fprintf(stderr, "Erreur lors de la saisie\n");
201             return ret;
202         }
203
204         switch (c)
205         {
206             case 'Q':
207             case 'q':
208                 ret = ACT_QUITTER;
209                 break;
210             default:
211                 ret = ACT_NOUVELLE_SAISIE;
212                 break;
213         }
214     }
215     else
216         ret = ACT_JOUER;
217
218     if (!vider_tampon(stdin))
219     {
220         fprintf(stderr, "Erreur lors de la vidange du tampon.\n");
221         ret = ACT_ERR;
222     }
223
224     return ret;
225 }
226
227
228 static int demande_nb_joueur(void)
229 {
230     /*
231      * Demande et récupère le nombre de joueurs.
232      */
233
234     int njeueur = 0;
235
236     while (1)
237     {
238         printf(
239             "Combien de joueurs prennent part à cette partie ? ");
240
241         if (scanf("%d", &njeueur) != 1 && ferror(stdin))
242         {

```

```

242         fprintf(stderr, "Erreur lors de la saisie\n");
243         return 0;
244     }
245     else if (njoueur != 1 && joueur != 2)
246         fprintf(stderr, "Plait-il ?\n");
247     else
248         break;
249
250     if (!vider_tampon(stdin))
251     {
252         fprintf(stderr,
253             "Erreur lors de la vidange du tampon.\n");
254         return 0;
255     }
256
257     return joueur;
258 }
259
260
261 static int grille_compleete(void)
262 {
263     /*
264      * Détermine si la grille de jeu est complète.
265      */
266
267     unsigned col;
268     unsigned lgn;
269
270     for (col = 0; col < P4_COLONNES; ++col)
271         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
272             if (grille[col][lgn] == ' ')
273                 return 0;
274
275     return 1;
276 }
277
278
279 static int ia(void)
280 {
281     /*
282      * Fonction mettant en œuvre l'IA présentée.
283      * Assigne une valeur pour chaque colonne libre et retourne
284      * ensuite le numéro de
285      * colonne ayant la plus haute valeur. Dans le cas où
286      * plusieurs valeurs égales sont
287      * générées, un numéro de colonne est « choisi au hasard »
288      * parmi celles-ci.
289      */

```

```

288     unsigned meilleurs_col[P4_COLONNES];
289     unsigned nb_meilleurs_col = 0;
290     unsigned max = 0;
291     unsigned col;
292
293     for (col = 0; col < P4_COLONNES; ++col)
294     {
295         struct position pos;
296         unsigned longueur;
297
298         if (grille[col][0] != ' ')
299             continue;
300
301         calcule_position(col, &pos);
302         longueur = calcule_nb_jetons_depuis(&pos, J2_JETON);
303
304         if (longueur >= 4)
305             return col;
306
307         longueur = umax(longueur, calcule_nb_jetons_depuis(&pos,
308             J1_JETON));
309
310         if (longueur >= max)
311         {
312             if (longueur > max)
313             {
314                 nb_meilleurs_col = 0;
315                 max = longueur;
316             }
317             meilleurs_col[nb_meilleurs_col++] = col;
318         }
319     }
320
321     return meilleurs_col[nb_aleatoire_entre(0, nb_meilleurs_col -
322         1)];
323
324
325 static void initialise_grille(void)
326 {
327     /*
328     * Initialise les caractères de la grille.
329     */
330
331     unsigned col;
332     unsigned lgn;
333
334     for (col = 0; col < P4_COLONNES; ++col)
335         for (lgn = 0; lgn < P4_LIGNES; ++lgn)

```

```

336         grille[col][lgn] = ' ';
337     }
338
339
340     static unsigned umax(unsigned a, unsigned b)
341     {
342         /*
343          * Retourne le plus grand des deux arguments.
344          */
345
346         return (a > b) ? a : b;
347     }
348
349
350     double nb_aleatoire(void)
351     {
352         /*
353          * Retourne un nombre pseudo-aléatoire compris entre zéro
354             inclus et un exclus.
355          */
356
357         return rand() / ((double)RAND_MAX + 1.);
358     }
359
360     int nb_aleatoire_entre(int min, int max)
361     {
362         /*
363          * Retourne un nombre pseudo-aléatoire entre `min` et `max`
364             inclus.
365          */
366
367         return nb_aleatoire() * (max - min + 1) + min;
368     }
369
370     static int position_valide(struct position *pos)
371     {
372         /*
373          * Vérifie que la position fournie est bien comprise dans la
374             grille.
375          */
376
377         int ret = 1;
378
379         if (pos->colonne >= P4_COLONNES || pos->colonne < 0)
380             ret = 0;
381         else if (pos->ligne >= P4_LIGNES || pos->ligne < 0)
382             ret = 0;

```

```

383     return ret;
384 }
385
386
387 static int statut_jeu(struct position *pos, char jeton)
388 {
389     /*
390      * Détermine s'il y a lieu de continuer le jeu ou s'il doit
391      * être
392      * arrêté parce qu'un joueur a gagné ou que la grille est
393      * complète.
394      */
395     if (grille_complete())
396         return STATUT_EGALITE;
397     else if (calcule_nb_jetons_depuis(pos, jeton) >= 4)
398         return STATUT_GAGNE;
399     return STATUT_OK;
400 }
401
402
403 static int vider_tampon(FILE *fp)
404 {
405     /*
406      * Vide les données en attente de lecture du flux spécifié.
407      */
408
409     int c;
410
411     do
412         c = fgetc(fp);
413     while (c != '\n' && c != EOF);
414
415     return ferror(fp) ? 0 : 1;
416 }
417
418
419 int main(void)
420 {
421     int statut;
422     char jeton = J1_JETON;
423     int njeueur;
424
425     initialise_grille();
426     affiche_grille();
427     njeueur = demande_nb_joueur();
428
429     if (!njeueur)
430         return EXIT_FAILURE;

```

```

431
432     while (1)
433     {
434         struct position pos;
435         int action;
436         int coup;
437
438         if (njoueur == 1 && jeton == J2_JETON)
439         {
440             coup = ia();
441             printf("Joueur 2 : %d\n", coup + 1);
442             calcule_position(coup, &pos);
443         }
444         else
445         {
446             printf("Joueur %d : ", (jeton == J1_JETON) ? 1 : 2);
447             action = demande_action(&coup);
448
449             if (action == ACT_ERR)
450                 return EXIT_FAILURE;
451             else if (action == ACT_QUITTER)
452                 return 0;
453             else if (action == ACT_NOUVELLE_SAISIE ||
454                      !coup_valide(coup))
455             {
456                 fprintf(stderr,
457                     "Vous ne pouvez pas jouer à cet endroit\n");
458                 continue;
459             }
460
461             calcule_position(coup - 1, &pos);
462
463             grille[pos.colonne][pos.ligne] = jeton;
464             affiche_grille();
465             statut = statut_jeu(&pos, jeton);
466
467             if (statut != STATUT_OK)
468                 break;
469
470             jeton = (jeton == J1_JETON) ? J2_JETON : J1_JETON;
471         }
472
473         if (statut == STATUT_GAGNE)
474             printf("Le joueur %d a gagné\n", (jeton == J1_JETON) ? 1 :
475                 2);
476         else if (statut == STATUT_EGALITE)
477             printf("Égalité\n");
478
479         return 0;

```

478 }

[Retourner au texte.](#)

Contenu masqué n°48

```

1  #include <ctype.h>
2  #include <stddef.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  #define P4_COLONNES (7)
8  #define P4_LIGNES (6)
9
10 #define J1_JETON ('O')
11 #define J2_JETON ('X')
12
13 #define ACT_ERR (0)
14 #define ACT_JOUER (1)
15 #define ACT_NOUVELLE_SAISIE (2)
16 #define ACT_SAUVEGARDER (3)
17 #define ACT_CHARGER (4)
18 #define ACT_QUITTER (5)
19
20 #define STATUT_OK (0)
21 #define STATUT_GAGNE (1)
22 #define STATUT_EGALITE (2)
23
24 #define MAX_NOM (255)
25
26 struct position
27 {
28     int colonne;
29     int ligne;
30 };
31
32 static void affiche_grille(void);
33 static void calcule_position(int, struct position *);
34 static unsigned calcule_nb_jetons_depuis_vers(struct position *,
35     int, int, char);
36 static unsigned calcule_nb_jetons_depuis(struct position *, char);
37 static int charger_jeu(char *nom, char *, int *);
38 static int coup_valide(int);
39 static int demande_action(int *);
40 static int demande_fichier(char *, size_t);
41 static int demande_nb_joueur(void);

```



```

41 static int grille_complete(void);
42 static int ia(void);
43 static void initialise_grille(void);
44 double nb_aleatoire(void);
45 int nb_aleatoire_entre(int, int);
46 static int position_valide(struct position *);
47 static int sauvegarder_jeu(char *nom, char, int);
48 static int statut_jeu(struct position *pos, char);
49 static unsigned umax(unsigned, unsigned);
50 static int vider_tampon(FILE *);
51
52 static char grille[P4_COLONNES][P4_LIGNES];
53
54
55 static void affiche_grille(void)
56 {
57     /*
58      * Affiche la grille pour le ou les joueurs.
59      */
60
61     int col;
62     int lgn;
63
64     putchar('\n');
65
66     for (col = 1; col <= P4_COLONNES; ++col)
67         printf(" %d ", col);
68
69     putchar('\n');
70     putchar('+');
71
72     for (col = 1; col <= P4_COLONNES; ++col)
73         printf("----+");
74
75     putchar('\n');
76
77     for (lgn = 0; lgn < P4_LIGNES; ++lgn)
78     {
79         putchar('|');
80
81         for (col = 0; col < P4_COLONNES; ++col)
82             if (isalpha(grille[col][lgn]))
83                 printf(" %c |", grille[col][lgn]);
84             else
85                 printf(" %c |", ' ');
86
87         putchar('\n');
88         putchar('+');
89
90         for (col = 1; col <= P4_COLONNES; ++col)

```

```

91         printf("---+");
92
93         putchar('\n');
94     }
95
96     for (col = 1; col <= P4_COLONNES; ++col)
97         printf(" %d ", col);
98
99     putchar('\n');
100 }
101
102
103 static void calcule_position(int coup, struct position *pos)
104 {
105     /*
106      * Traduit le coup joué en un numéro de colonne et de ligne.
107      */
108
109     int lgn;
110
111     pos->colonne = coup;
112
113     for (lgn = P4_LIGNES - 1; lgn >= 0; --lgn)
114         if (grille[pos->colonne][lgn] == ' ')
115             {
116                 pos->ligne = lgn;
117                 break;
118             }
119 }
120
121
122 static unsigned calcule_nb_jetons_depuis_vers(struct position
123 *pos, int dpl_hrz, int dpl_vrt, char jeton)
124 {
125     /*
126      * Calcule le nombre de jetons adjacents identiques depuis une
127      * position donnée en se
128      * déplaçant de `dpl_hrz` horizontalement et `dpl_vrt`
129      * verticalement.
130      * La fonction s'arrête si un jeton différent ou une case vide
131      * est rencontrée ou si
132      * les limites de la grille sont atteintes.
133      */
134
135     struct position tmp;
136     unsigned nb = 1;
137
138     tmp.colonne = pos->colonne + dpl_hrz;
139     tmp.ligne = pos->ligne + dpl_vrt;

```

```

137     while (position_valide(&tmp))
138     {
139         if (grille[tmp.colonne][tmp.ligne] == jeton)
140             ++nb;
141         else
142             break;
143
144         tmp.colonne += dpl_hrz;
145         tmp.ligne += dpl_vrt;
146     }
147
148     return nb;
149 }
150
151
152 static unsigned calcule_nb_jetons_depuis(struct position *pos,
153     char jeton)
154 {
155     /*
156      * Calcule le nombre de jetons adjacents en vérifiant la
157      * colonne courante,
158      * de la ligne courante et des deux obliques courantes.
159      * Pour ce faire, la fonction calcule_nb_jeton_depuis_vers()
160      * est appelé à
161      * plusieurs reprises afin de parcourir la grille suivant la
162      * vérification
163      * à effectuer.
164      */
165
166     unsigned max;
167
168     max = calcule_nb_jetons_depuis_vers(pos, 0, 1, jeton);
169     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 0,
170         jeton) + \
171         calcule_nb_jetons_depuis_vers(pos, -1, 0, jeton) - 1);
172     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, 1,
173         jeton) + \
174         calcule_nb_jetons_depuis_vers(pos, -1, -1, jeton) - 1);
175     max = umax(max, calcule_nb_jetons_depuis_vers(pos, 1, -1,
176         jeton) + \
177         calcule_nb_jetons_depuis_vers(pos, -1, 1, jeton) - 1);
178
179     return max;
180 }
181
182
183 static int charger_jeu(char *nom, char *jeton, int *njourer)
184 {
185     /*
186      * Charge une partie existante depuis le fichier spécifié.

```

```

180     */
181
182     FILE *fp;
183     unsigned col;
184     unsigned lgn;
185
186     fp = fopen(nom, "r");
187
188     if (fp == NULL)
189     {
190         fprintf(stderr, "Impossible d'ouvrir le fichier %s\n",
191             nom);
192         return 0;
193     }
194     else if (fscanf(fp, "%c%d", jeton, njeueur) != 2)
195     {
196         fprintf(stderr,
197             "Impossible de récupérer le joueur et le nombre de joueurs\n");
198         return 0;
199     }
200
201     for (col = 0; col < P4_COLONNES; ++col)
202     {
203         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
204         {
205             if (fscanf(fp, "|%c", &grille[col][lgn]) != 1)
206             {
207                 fprintf(stderr,
208                     "Impossible de récupérer la grille\n");
209                 return 0;
210             }
211         }
212     }
213
214     if (fclose(fp) != 0)
215     {
216         fprintf(stderr, "Impossible de fermer le fichier %s\n",
217             nom);
218         return 0;
219     }
220
221     return 1;
222 }
223
224 static int coup_valide(int col)
225 {
226     /*
227     * Si la colonne renseignée est inférieure ou égale à zéro
228     * ou que celle-ci est supérieure à la longueur du tableau
229     * ou que la colonne indiquée est saturée

```

```

225     * alors le coup est invalide.
226     */
227
228     if (col <= 0 || col > P4_COLONNES || grille[col - 1][0] != ' ')
229         return 0;
230
231     return 1;
232 }
233
234
235 static int demande_action(int *coup)
236 {
237     /*
238     * Demande l'action à effectuer au joueur courant.
239     * S'il entre un chiffre, c'est qu'il souhaite jouer.
240     * S'il entre la lettre « Q » ou « q », c'est qu'il souhaite
241     * quitter.
242     * S'il entre autre chose, une nouvelle saisie sera demandée.
243     */
244
245     char c;
246     int ret = ACT_ERR;
247
248     if (scanf("%d", coup) != 1)
249     {
250         if (scanf("%c", &c) != 1)
251         {
252             fprintf(stderr, "Erreur lors de la saisie\n");
253             return ret;
254         }
255
256         switch (c)
257         {
258             case 'Q':
259             case 'q':
260                 ret = ACT_QUITTER;
261                 break;
262
263             case 'C':
264             case 'c':
265                 ret = ACT_CHARGER;
266                 break;
267
268             case 'S':
269             case 's':
270                 ret = ACT_SAUVEGARDER;
271                 break;
272
273             default:
274                 ret = ACT_NOUVELLE_SAISIE;

```

```

274         break;
275     }
276 }
277 else
278     ret = ACT_JOUER;
279
280 if (!vider_tampon(stdin))
281 {
282     fprintf(stderr, "Erreur lors de la vidange du tampon.\n");
283     ret = ACT_ERR;
284 }
285
286 return ret;
287 }
288
289
290 static int demande_fichier(char *nom, size_t max)
291 {
292     /*
293      * Demande et récupère un nom de fichier.
294      */
295
296     char *nl;
297
298     printf("Veuillez entrer un nom de fichier : ");
299
300     if (fgets(nom, max, stdin) == NULL)
301     {
302         fprintf(stderr, "Erreur lors de la saisie\n");
303         return 0;
304     }
305
306     nl = strchr(nom, '\n');
307
308     if (nl == NULL && !vider_tampon(stdin))
309     {
310         fprintf(stderr, "Erreur lors de la vidange du tampon.\n");
311         return 0;
312     }
313
314     if (nl != NULL)
315         *nl = '\0';
316
317     return 1;
318 }
319
320
321 static int demande_nb_joueur(void)
322 {
323     /*

```

```

324     * Demande et récupère le nombre de joueurs.
325     */
326
327     int njeueur = 0;
328
329     while (1)
330     {
331         printf(
332             "Combien de joueurs prennent part à cette partie ? ");
333
334         if (scanf("%d", &njeueur) != 1 && ferror(stdin))
335         {
336             fprintf(stderr, "Erreur lors de la saisie\n");
337             return 0;
338         }
339         else if (!vider_tampon(stdin))
340         {
341             fprintf(stderr,
342                 "Erreur lors de la vidange du tampon.\n");
343             return 0;
344         }
345         if (njeueur != 1 && njeueur != 2)
346             fprintf(stderr, "Plait-il ?\n");
347         else
348             break;
349     }
350     return njeueur;
351 }
352
353
354 static int grille_complete(void)
355 {
356     /*
357     * Détermine si la grille de jeu est complète.
358     */
359
360     unsigned col;
361     unsigned lgn;
362
363     for (col = 0; col < P4_COLONNES; ++col)
364         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
365             if (grille[col][lgn] == ' ')
366                 return 0;
367
368     return 1;
369 }
370
371

```

```

372 static int ia(void)
373 {
374     /*
375      * Fonction mettant en œuvre l'IA présentée.
376      * Assigne une valeur pour chaque colonne libre et retourne
377      * ensuite le numéro de
378      * colonne ayant la plus haute valeur. Dans le cas où
379      * plusieurs valeurs égales sont
380      * générées, un numéro de colonne est « choisi au hasard »
381      * parmi celles-ci.
382      */
383     unsigned meilleurs_col[P4_COLONNES];
384     unsigned nb_meilleurs_col = 0;
385     unsigned max = 0;
386     unsigned col;
387
388     for (col = 0; col < P4_COLONNES; ++col)
389     {
390         struct position pos;
391         unsigned longueur;
392
393         if (grille[col][0] != ' ')
394             continue;
395
396         calcule_position(col, &pos);
397         longueur = calcule_nb_jetons_depuis(&pos, J2_JETON);
398
399         if (longueur >= 4)
400             return col;
401
402         longueur = umax(longueur, calcule_nb_jetons_depuis(&pos,
403             J1_JETON));
404
405         if (longueur >= max)
406         {
407             if (longueur > max)
408             {
409                 nb_meilleurs_col = 0;
410                 max = longueur;
411             }
412
413             meilleurs_col[nb_meilleurs_col++] = col;
414         }
415     }
416
417     return meilleurs_col[nb_aleatoire_entre(0, nb_meilleurs_col -
418         1)];
419 }

```



```

417
418 static void initialise_grille(void)
419 {
420     /*
421      * Initialise les caractères de la grille.
422      */
423
424     unsigned col;
425     unsigned lgn;
426
427     for (col = 0; col < P4_COLONNES; ++col)
428         for (lgn = 0; lgn < P4_LIGNES; ++lgn)
429             grille[col][lgn] = ' ';
430 }
431
432
433 static unsigned umax(unsigned a, unsigned b)
434 {
435     /*
436      * Retourne le plus grand des deux arguments.
437      */
438
439     return (a > b) ? a : b;
440 }
441
442
443 double nb_aleatoire(void)
444 {
445     /*
446      * Retourne un nombre pseudo-aléatoire compris entre zéro
447      * inclus et un exclus.
448      */
449
450     return rand() / ((double)RAND_MAX + 1.);
451 }
452
453 int nb_aleatoire_entre(int min, int max)
454 {
455     /*
456      * Retourne un nombre pseudo-aléatoire entre `min` et `max`
457      * inclus.
458      */
459
460     return nb_aleatoire() * (max - min + 1) + min;
461 }
462
463 static int position_valide(struct position *pos)
464 {

```

```

465     /*
466     * Vérifie que la position fournie est bien comprise dans la
467     * grille.
468     */
469     int ret = 1;
470
471     if (pos->colonne >= P4_COLONNES || pos->colonne < 0)
472         ret = 0;
473     else if (pos->ligne >= P4_LIGNES || pos->ligne < 0)
474         ret = 0;
475
476     return ret;
477 }
478
479 static int sauvegarder_jeu(char *nom, char jeton, int njeueur)
480 {
481     /*
482     * Sauvegarde la partie courant dans le fichier indiqué.
483     */
484
485     FILE *fp;
486     unsigned col;
487     unsigned lgn;
488
489     fp = fopen(nom, "w");
490
491     if (fp == NULL)
492     {
493         fprintf(stderr, "Impossible d'ouvrir le fichier %s\n",
494             nom);
495         return 0;
496     }
497     else if (fprintf(fp, "%c%d", jeton, njeueur) < 0)
498     {
499         fprintf(stderr,
500             "Impossible de sauvegarder le joueur courant\n");
501         return 0;
502     }
503
504     if (fputc('|', fp) == EOF)
505     {
506         fprintf(stderr, "Impossible de sauvegarder la grille\n");
507         return 0;
508     }
509
510     for (col = 0; col < P4_COLONNES; ++col)
511     {
512         for (lgn = 0; lgn < P4_LIGNES; ++lgn)

```

```

512     {
513         if (fprintf(fp, "%c|", grille[col][lgn]) < 0)
514         {
515             fprintf(stderr,
516                 "Impossible de sauvegarder la grille\n");
517             return 0;
518         }
519     }
520
521     if (fputc('\n', fp) == EOF)
522     {
523         fprintf(stderr, "Impossible de sauvegarder la grille\n");
524         return 0;
525     }
526
527     if (fclose(fp) != 0)
528     {
529         fprintf(stderr, "Impossible de fermer le fichier %s\n",
530             nom);
531         return 0;
532     }
533     printf("La partie a bien été sauvegardée dans le fichier %s\n",
534         nom);
535     return 1;
536 }
537
538 static int statut_jeu(struct position *pos, char jeton)
539 {
540     /*
541      * Détermine s'il y a lieu de continuer le jeu ou s'il doit
542      * être
543      * arrêté parce qu'un joueur a gagné ou que la grille est
544      * complète.
545      */
546     if (grille_complete())
547         return STATUT_EGALITE;
548     else if (calcule_nb_jetons_depuis(pos, jeton) >= 4)
549         return STATUT_GAGNE;
550     return STATUT_OK;
551 }
552
553
554 static int vider_tampon(FILE *fp)
555 {

```

```

556     /*
557     * Vide les données en attente de lecture du flux spécifié.
558     */
559
560     int c;
561
562     do
563         c = fgetc(fp);
564     while (c != '\n' && c != EOF);
565
566     return ferror(fp) ? 0 : 1;
567 }
568
569
570 int main(void)
571 {
572     static char nom[MAX_NOM];
573     int statut;
574     char jeton = J1_JETON;
575     int njeueur;
576
577     initialise_grille();
578     affiche_grille();
579     njeueur = demande_nb_joueur();
580
581     if (!njeueur)
582         return EXIT_FAILURE;
583
584     while (1)
585     {
586         struct position pos;
587         int action;
588         int coup;
589
590         if (njeueur == 1 && jeton == J2_JETON)
591         {
592             coup = ia();
593             printf("Joueur 2 : %d\n", coup + 1);
594             calcule_position(coup, &pos);
595         }
596         else
597         {
598             printf("Joueur %d : ", (jeton == J1_JETON) ? 1 : 2);
599             action = demande_action(&coup);
600
601             if (action == ACT_ERR)
602                 return EXIT_FAILURE;
603             else if (action == ACT_QUITTER)
604                 return 0;
605             else if (action == ACT_SAUVEGARDER)

```

```

606         {
607             if (!demande_fichier(nom, sizeof nom) ||
608                 !sauvegarder_jeu(nom, jeton, njeueur))
609                 return EXIT_FAILURE;
610             else
611                 return 0;
612         }
613     else if (action == ACT_CHARGER)
614     {
615         if (!demande_fichier(nom, sizeof nom) ||
616             !charger_jeu(nom, &jeton, &njeueur))
617             return EXIT_FAILURE;
618         else
619         {
620             affiche_grille();
621             continue;
622         }
623     }
624     else if (action == ACT_NOUVELLE_SAISIE ||
625             !coup_valide(coup))
626     {
627         fprintf(stderr,
628             "Vous ne pouvez pas jouer à cet endroit\n");
629         continue;
630     }
631     }
632     calcule_position(coup - 1, &pos);
633     grille[pos.colonne][pos.ligne] = jeton;
634     affiche_grille();
635     statut = statut_jeu(&pos, jeton);
636     if (statut != STATUT_OK)
637         break;
638     jeton = (jeton == J1_JETON) ? J2_JETON : J1_JETON;
639 }
640
641 if (statut == STATUT_GAGNE)
642     printf("Le joueur %d a gagné\n", (jeton == J1_JETON) ? 1 :
643         2);
644 else if (statut == STATUT_EGALITE)
645     printf("Égalité\n");
646
647 return 0;
648 }

```

[Retourner au texte.](#)

IV.11. La gestion d'erreurs (2)

Introduction

Jusqu'à présent, nous vous avons présenté les bases de la gestion d'erreurs en vous parlant des retours de fonctions et de la variable globale `errno`. Dans ce chapitre, nous allons approfondir cette notion afin de réaliser des programmes plus robustes.

IV.11.1. Gestion de ressources

IV.11.1.1. Allocation de ressources

Dans cette deuxième partie, nous avons découvert l'allocation dynamique de mémoire et la gestion de fichiers. Ces deux sujets ont un point en commun : il est nécessaire de passer par une fonction d'allocation et par une fonction de libération lors de leur utilisation. Il s'agit d'un processus commun en programmation appelé la **gestion de ressources**. Or, celle-ci pose un problème particulier dans le cas de la gestion d'erreurs. En effet, regardez de plus près ce code simplifié permettant l'allocation dynamique d'un tableau multidimensionnel de trois fois trois `int`.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      int **p;
8      unsigned i;
9
10     p = malloc(3 * sizeof(int *));
11
12     if (p == NULL)
13     {
14         fprintf(stderr, "Échec de l'allocation\n");
15         return EXIT_FAILURE;
16     }
17
18     for (i = 0; i < 3; ++i)
19     {
```

```

20     p[i] = malloc(3 * sizeof(int));
21
22     if (p[i] == NULL)
23     {
24         fprintf(stderr, "Échec de l'allocation\n");
25         return EXIT_FAILURE;
26     }
27 }
28
29 /* ... */
30 return 0;
31 }

```

Vous ne voyez rien d'anormal dans le cas de la seconde boucle ? Celle-ci quitte le programme en cas d'échec de la fonction `malloc()`. Oui, mais nous avons *déjà* fait appel à la fonction `malloc()` auparavant, ce qui signifie que si nous quittons le programme à ce stade, nous le ferons *sans avoir libéré certaines ressources* (ici de la mémoire).

Seulement voilà, comment faire cela sans rendre le programme bien plus compliqué et bien moins lisible ? C'est ici que l'utilisation de l'instruction `goto` devient pertinente et d'une aide précieuse. La technique consiste à placer la libération de ressources en fin de fonction et de se rendre au bon endroit de cette zone de libération à l'aide de l'instruction `goto`. Autrement dit, voici ce que cela donne avec le code précédent.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      int **p;
8      unsigned i;
9      int status = EXIT_FAILURE;
10
11     p = malloc(3 * sizeof(int *));
12
13     if (p == NULL)
14     {
15         fprintf(stderr, "Échec de l'allocation\n");
16         goto fin;
17     }
18
19     for (i = 0; i < 3; ++i)
20     {
21         p[i] = malloc(3 * sizeof(int));
22
23         if (p[i] == NULL)
24         {

```

```

25         fprintf(stderr, "Échec de l'allocation\n");
26         goto liberer_p;
27     }
28 }
29
30 status = 0;
31
32 /* Zone de libération */
33 liberer_p:
34 while (i > 0)
35 {
36     --i;
37     free(p[i]);
38 }
39
40 free(p);
41 fin:
42 return status;
43 }

```

Comme vous le voyez, nous avons placé deux étiquettes : une référençant l'instruction `return` et une autre désignant la première instruction de la zone de libération. Ainsi, nous quittons la fonction en cas d'échec de la première allocation, mais nous libérons les ressources auparavant dans le cas des allocations suivantes. Notez que nous avons ajouté une variable `status` afin de pouvoir retourner la bonne valeur suivant qu'une erreur est survenue ou non.

IV.11.1.2. Utilisation de ressources

Si le problème se pose dans le cas de l'allocation de ressources, il se pose également dans le cas de leur utilisation.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      FILE *fp;
8
9      fp = fopen("texte.txt", "w");
10
11     if (fp == NULL)
12     {
13         fprintf(stderr,
14             "Le fichier texte.txt n'a pas pu être ouvert\n");
15         return EXIT_FAILURE;
16     }

```



```

16     if (fputs("Au revoir !\n", fp) == EOF)
17     {
18         fprintf(stderr, "Erreur lors de l'écriture d'une ligne\n");
19         return EXIT_FAILURE;
20     }
21     if (fclose(fp) == EOF)
22     {
23         fprintf(stderr, "Erreur lors de la fermeture du flux\n");
24         return EXIT_FAILURE;
25     }
26
27     return 0;
28 }

```

Dans le code ci-dessus, l'exécution du programme est stoppée en cas d'erreur de la fonction `fputs()`. Cependant, l'arrêt s'effectue alors qu'une ressource n'a pas été libérée (en l'occurrence le flux `fp`). Aussi, il est nécessaire d'appliquer la solution présentée juste avant pour rendre ce code correct.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  int main(void)
6  {
7      FILE *fp;
8      int status = EXIT_FAILURE;
9
10     fp = fopen("texte.txt", "w");
11
12     if (fp == NULL)
13     {
14         fprintf(stderr,
15             "Le fichier texte.txt n'a pas pu être ouvert\n");
16         goto fin;
17     }
18     if (fputs("Au revoir !\n", fp) == EOF)
19     {
20         fprintf(stderr, "Erreur lors de l'écriture d'une ligne\n");
21         goto fermer_flux;
22     }
23     if (fclose(fp) == EOF)
24     {
25         fprintf(stderr, "Erreur lors de la fermeture du flux\n");
26         goto fin;
27     }
28
29     status = 0;

```

```
29
30 fermer_flux:
31     fclose(fp);
32 fin:
33     return status;
34 }
```

Nous attirons votre attention sur deux choses :

- En cas d'échec de la fonction `fclose()`, le programme est arrêté immédiatement. Étant donné que c'est la fonction `fclose()` qui pose problème, il n'y a pas lieu de la rappeler (notez que cela n'est toutefois pas une erreur de l'appeler une seconde fois).
- Le retour de la fonction `fclose()` n'est pas vérifié en cas d'échec de la fonction `fputs()` étant donné que nous sommes *déjà* dans un cas d'erreur.

IV.11.2. Fin d'un programme

Vous le savez, l'exécution de votre programme se termine lorsque celui-ci quitte la fonction `main()`. Toutefois, que se passe-t-il exactement lorsque celui-ci s'arrête ? En fait, un petit bout de code appelé **épilogue** est exécuté afin de réaliser quelques tâches. Parmi celles-ci figure la vidange et la fermeture de tous les flux encore ouverts. Une fois celui-ci exécuté, la main est rendue au système d'exploitation qui, le plus souvent, se chargera de libérer toutes les ressources qui étaient encore allouées.

?

Mais ?! Vous venez de nous dire qu'il était nécessaire de libérer les ressources avant l'arrêt du programme. Du coup, pourquoi s'amuser à appeler les fonctions `fclose()` ou `free()` alors que l'épilogue ou le système d'exploitation s'en charge ?

Pour quatre raisons principales :

1. Afin de libérer des ressources pour les autres programmes. En effet, si vous ne libérez les ressources allouées qu'à la fin de votre programme alors que celui-ci n'en a plus besoin, vous gaspillez des ressources qui pourraient être utilisées par d'autres programmes.
2. Pour être à même de détecter des erreurs, de prévenir l'utilisateur en conséquence et d'éventuellement lui proposer des solutions.
3. En vue de rendre votre code plus facilement modifiable par après et d'éviter les oublis.
4. Parce qu'il n'est pas garanti que les ressources seront effectivement libérées par le système d'exploitation.

!

Aussi, de manière générale :

- considérez que l'objectif de l'épilogue est de fermer *uniquement* les flux `stdin`, `stdout` et `stderr` ;
- *ne comptez pas* sur votre système d'exploitation pour la libération de quelques ressources que ce soit (et notamment la mémoire) ;



— libérez vos ressources le plus tôt possible, autrement dit dès que votre programme n'en a plus l'utilité.

IV.11.2.1. Terminaison normale

Le passage par l'épilogue est appelée la **terminaison normale** du programme. Il est possible de s'y rendre directement en appelant la fonction `exit()` qui est déclarée dans l'en-tête `<stdlib.h>`.

```
1 void exit(int status);
```

Appeler cette fonction revient au même que de quitter la fonction `main()` à l'aide de l'instruction `return`. L'argument attendu est une expression entière identique à celle fournie comme opérande de l'instruction `return`. Ainsi, il vous est possible de mettre fin directement à l'exécution de votre programme *sans* retourner jusqu'à la fonction `main()`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 void fin(void)
6 {
7     printf("C'est la fin du programme\n");
8     exit(0);
9 }
10
11
12 int main(void)
13 {
14     fin();
15     printf("Retour à la fonction main\n");
16     return 0;
17 }
```

Résultat

```
1 C'est la fin du programme
```

Comme vous le voyez l'exécution du programme se termine une fois que la fonction `exit()` est appelée. La fin de la fonction `main()` n'est donc *jamaïs* exécutée.

IV.11.2.2. Terminaison anormale

Il est possible de terminer l'exécution d'un programme *sans passer par l'épilogue* à l'aide de la fonction `abort()` (déclarée également dans l'en-tête `<stdlib.h>`). Dans un tel cas, il s'agit d'une **terminaison anormale** du programme.

```
1 void abort(void);
```

Une terminaison anormale signifie qu'une condition *non prévue* par le programmeur est survenue. Elle est donc à distinguer d'une terminaison normale survenue suite à une erreur qui, elle, était prévue (comme une erreur lors d'une saisie de l'utilisateur). De manière générale, la terminaison anormale est utilisée lors de la phase de développement d'un logiciel afin de faciliter la détection d'erreurs de programmation, celle-ci entraînant le plus souvent la production d'une **image mémoire** (*core dump* en anglais) qui pourra être analysée à l'aide d'un **débogueur** (*debugger* en anglais).



Une image mémoire est en fait un fichier contenant l'état des registres et de la mémoire d'un programme lors de la survenance d'un problème..

IV.11.3. Les assertions

IV.11.3.1. La macrofonction assert

Une des utilisations majeure de la fonction `abort()` est réalisée via la macrofonction `assert()` (définie dans l'en-tête `<assert.h>`). Cette dernière est utilisée pour placer des tests à certains points d'un programme. Dans le cas où un de ces tests s'avère faux, un message d'erreur est affiché (ce dernier comprend la condition dont l'évaluation est fausse, le nom du fichier et la ligne courante) après quoi la fonction `abort()` est appelée afin de produire une image mémoire.

Cette technique est très utile pour détecter rapidement des erreurs au sein d'un programme lors de sa phase de développement. Généralement, les assertions sont placées en début de fonction afin de vérifier un certain nombre de conditions. Par exemple, si nous reprenons la fonction `long_colonne()` du TP sur le Puissance 4.

```
1 static unsigned long_colonne(unsigned joueur, unsigned col,  
    unsigned ligne, unsigned char (*grille)[6])  
2 {  
3     unsigned i;  
4     unsigned n = 1;  
5  
6     for (i = ligne + 1; i < 6; ++i)  
7     {  
8         if (grille[col][i] == joueur)
```

```

9         ++n;
10        else
11            break;
12    }
13
14    return n;
15 }
```

Nous pourrions ajouter quatre assertions vérifiant si :

- le numéro du joueur est un ou deux ;
- le numéro de la colonne est compris entre zéro et six inclus ;
- le numéro de la ligne est compris entre zéro et cinq ;
- le pointeur `grille` n'est pas nul.

Ce qui nous donne le code suivant.

```

1  static unsigned long_colonne(unsigned joueur, unsigned col,
2      unsigned ligne, unsigned char (*grille)[6])
3  {
4      unsigned i;
5      unsigned n = 1;
6
7      assert(joueur == 1 || joueur == 2);
8      assert(col < 7);
9      assert(ligne < 6);
10     assert(grille != NULL);
11
12     for (i = ligne + 1; i < 6; ++i)
13     {
14         if (grille[col][i] == joueur)
15             ++n;
16         else
17             break;
18     }
19
20     return n;
21 }
```

Ainsi, si nous ne respectons pas une de ces conditions pour une raison ou pour une autre, l'exécution du programme s'arrêtera et nous aurons droit à un message du type (dans le cas où la première assertion échoue).

```

1  a.out: main.c:71: long_colonne: Assertion `joueur == 1 || joueur
   == 2' failed.
2  Aborted
```

Comme vous le voyez, le nom du programme est indiqué, suivi du nom du fichier, du numéro de ligne, du nom de la fonction et de la condition qui a échoué. Intéressant, non ? 🍊

IV.11.3.2. Suppression des assertions

Une fois votre programme développé et dûment testé, les assertions ne vous sont plus vraiment utiles étant donné que celui-ci fonctionne. Les conserver alourdirait l'exécution de votre programme en ajoutant des vérifications. Toutefois, heureusement, il est possible de supprimer ces assertions *sans modifier votre code* en ajoutant simplement l'option `-DNDEBUG` lors de la compilation.

```
1 $ gcc -DNDEBUG main.c
```

IV.11.4. Les fonctions `strerror` et `perror`

Vous le savez, certaines (beaucoup) de fonctions standards peuvent échouer. Toutefois, en plus de signaler à l'utilisateur qu'une de ces fonctions a échoué, cela serait bien de lui spécifier *pourquoi*. C'est ici qu'entrent en jeu les fonctions `strerror()` et `perror()`.

```
1 char *strerror(int num);
```

La fonction `strerror()` (déclarée dans l'en-tête `<string.h>`) retourne une chaîne de caractères correspondant à la valeur entière fournie en argument. Cette valeur sera en fait *toujours* celle de la variable `errno`. Ainsi, il vous est possible d'obtenir plus de détails quand une fonction standard rencontre un problème. L'exemple suivant illustre l'utilisation de cette fonction en faisant appel à la fonction `fopen()` afin d'ouvrir un fichier qui n'existe pas (ce qui provoque une erreur).

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5
6
7 int main(void)
8 {
9     FILE *fp;
10
11     fp = fopen("nawak.txt", "r");
12
13     if (fp == NULL)
```

```

14     {
15         fprintf(stderr, "fopen: %s\n", strerror(errno));
16         return EXIT_FAILURE;
17     }
18
19     fclose(fp);
20     return 0;
21 }

```

Résultat

```

1 fopen: No such file or directory

```

Comme vous le voyez, nous avons obtenu des informations supplémentaires : la fonction a échoué parce que le fichier « `nawak.txt` » n'existe pas.



Notez que les messages d'erreur retournés par la fonction `strerror()` sont le plus souvent en anglais.

```

1 void perror(char *message);

```

La fonction `perror()` (déclarée dans l'en-tête `<stdio.h>`) écrit sur le flux d'erreur standard (`stderr`, donc) la chaîne de caractères fournie en argument, suivie du caractère `:`, d'un espace et du retour de la fonction `strerror()` avec comme argument la valeur de la variable `errno`. Autrement dit, cette fonction revient au même que l'appel suivant.

```

1 fprintf(stderr, "message: %s\n", strerror(errno));

```

L'exemple précédent peut donc également s'écrire comme suit.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     FILE *fp;
8
9     fp = fopen("nawak.txt", "r");

```

```
10
11     if (fp == NULL)
12     {
13         perror("fopen");
14         return EXIT_FAILURE;
15     }
16
17     fclose(fp);
18     return 0;
19 }
```

Résultat

```
1 fopen: No such file or directory
```

Conclusion

En résumé

- La gestion des ressources implique une gestion des erreurs ;
- Le mieux est de libérer les ressources inutilisées le plus tôt possible, pour ne pas les monopoliser pour rien ;
- Les ressources doivent être libérées correctement et non laissées aux soins du système d'exploitation ;
- La fonction `abort()` termine brutalement le programme en cas d'erreur ;
- Les assertions placent des tests à certains endroits du code et sont utiles pour détecter rapidement des erreurs de la phase de développement d'un programme ;
- Elles sont désactivées en utilisant l'option de compilation `-DNDEBUG` ;
- La fonction `strerror()` retourne une chaîne de caractère qui correspond au code d'erreur que contient `errno` ;
- La fonction `perror()` fait la même chose, en affichant en plus, sur `stderr`, un message passé en argument suivi de `:` et d'un espace.

Cinquième partie

Notions avancées

V.1. La représentation des types

Introduction

Dans le chapitre sur l'allocation dynamique, nous avons effleuré la notion de représentation en parlant de celle d'objet. Pour rappel, la représentation d'un type correspond à la manière dont les données sont réparties en mémoire, plus précisément comment les multiplètes et les *bits* les composant sont agencés et utilisés.

Dans ce chapitre, nous allons plonger au cœur de ce concept et vous exposer la représentation des types.

V.1.1. La représentation des entiers

V.1.1.1. Les entiers non signés

La représentation des entiers non signés étant la plus simple, nous allons commencer par celle-ci.



Dans un entier non signé, les différents *bits* correspondent à une puissance de deux. Plus précisément, le premier correspond à 2^0 , le second à 2^1 , le troisième à 2^2 et ainsi de suite jusqu'au dernier *bit* composant le type utilisé. Pour calculer la valeur d'un entier non signé, il suffit donc d'additionner les puissances de deux correspondant aux *bits* à 1.

Pour illustrer notre propos, voici un tableau comprenant la représentation de plusieurs nombres au sein d'un objet de type `unsigned char` (ici sur un octet).

	Re-pré- Nombres ta- tion							
	0	0	0	0	0	0	0	1
1	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	2^0							
	=							
	1							

	0	0	0	0	0	0	1	1
3	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	2^0							
	+							
	2^1							
	=							
	3							
	0	0	1	0	1	0	1	0
42	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	2^1							
	+							
	2^3							
	+							
	2^5							
	=							
	42							
	1	1	1	1	1	1	1	1
255	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	2^0							
	+							
	2^1							
	+							
	2^2							
	+							
	2^3							
	+							
	2^4							
	+							
	2^5							
	+							
	2^6							
	+							
	2^7							
	=							
	255							

i

Notez que nous écrivons ici les nombres binaires *par puissance de deux décroissantes*. Nous suivons en fait la même logique que pour les nombres en base dix. En effet, 195 par exemple se décompose en une suite décroissante de puissances de 10 : $1 \times 10^2 + 9 \times 10^1 + 5 \times 10^0$.

V.1.1.2. Les entiers signés

Les choses se compliquent un peu avec les entiers signés. En effet, il nous faut représenter une information supplémentaire : le signe de la valeur.

V.1.1.2.1. La représentation en signe et magnitude

La première solution qui vous est peut-être venue à l'esprit serait de réserver un *bit*, par exemple le dernier, pour représenter le signe. Ainsi, la représentation est identique à celle des entiers non signés si ce n'est que le dernier *bit* est réservé pour le signe (ce qui diminue donc en conséquence la valeur maximale représentable). Cette méthode est appelée **représentation en signe et magnitude**.

Re-pré-sen-ta-tion	
Nombre	
signe et ma-gni-tude	
1	0 0 0 0 0 0 0 1
	+ 2 ⁶ 2 ⁵ 2 ⁴ 2 ³ 2 ² 2 ¹ 2 ⁰
	2 ⁰
	= 1
-	1 0 0 0 0 0 0 1
	- 2 ⁶ 2 ⁵ 2 ⁴ 2 ³ 2 ² 2 ¹ 2 ⁰
	2 ⁰
	= - 1

Cependant, si la représentation en signe et magnitude paraît simple, elle n'est en vérité pas facile à mettre en œuvre au sein d'un processeur car elle implique plusieurs vérifications (notamment au niveau du signe) qui se traduisent par des circuits supplémentaires et un surcoût en calcul. De plus, elle laisse deux représentations possibles pour le zéro (0000 0000 et 1000 0000), ce qui est gênant pour l'évaluation des conditions.

V.1.1.2.2. La représentation en complément à un

Dès lors, comment pourrions-nous faire pour simplifier nos calculs en évitant des vérifications liées au signe ? Eh bien, sachant que le maximum représentable dans notre exemple est 255

(soit **1111 1111**), il nous est possible de représenter chaque nombre négatif comme la différence entre le maximum et sa valeur absolue. Par exemple **-1** sera représenté par **255 - 1**, soit **254 (1111 1110)**. Cette représentation est appelée **représentation en complément à un**.

Ainsi, si nous additionnons **1** et **-1**, nous n'avons pas de vérifications à faire et obtenons le maximum, **255**. Toutefois, ceci implique, comme pour la représentation en signe et magnitude, qu'il existe deux représentations pour le zéro : **0000 0000** et **1111 1111**.

	Re- pré- sen- ta- tion
Nombre	com- plé- ment à un
1	0 0 0 0 0 0 0 1
+	$2^6 2^5 2^4 2^3 2^2 2^1 2^0$
	2^0
=	
1	
-	1 1 1 1 1 1 1 0
1	$2^6 2^5 2^4 2^3 2^2 2^1 2^0$
	(255
-	
	2^7
-	
	2^6
-	
	2^5
-	
	2^4
-	
	2^3
-	
	2^2
-	
	2^1)
=	
-	
	1



Notez qu'en fait, cette représentation revient à inverser tous les *bits* d'un nombre positif pour obtenir son équivalent négatif. Par exemple, si nous inversons **1** (**0000 0001**), nous obtenons bien **-1** (**1111 1110**) comme ci-dessus.

Par ailleurs, il subsiste un second problème : dans le cas où deux nombres négatifs sont additionnés, le résultat obtenu n'est pas valide. En effet, $-1 + -1$ nous donne **1111 1110** + **1111 1110** soit **1 1111 1100** ; autrement dit, comme nous travaillons sur huit *bits* pour nos exemples, **-3**... Mince !

Pour résoudre ce problème, il nous faut reporter la dernière retenue (soit ici le dernier *bit* que nous avons ignoré) au début (ce qui revient à ajouter un) ce qui nous permet d'obtenir **1111 1101**, soit **-2**.

V.1.1.2.3. La représentation en complément à deux

Ainsi est apparue une troisième représentation : celle en **complément à deux**. Celle-ci conserve les qualités de la représentation en complément à un, mais lui corrige certains défauts. En fait, elle représente les nombres négatifs de la même manière que la représentation en complément à un, si ce n'est que la soustraction s'effectue entre le maximum augmenté de un (soit **256** dans notre cas) et non le maximum.

Ainsi, par exemple, la représentation en complément à deux de **-1** est **256 - 1**, soit **255** (**1111 1111**).

Re- pré- sen- ta- tion		Nombre com- plé- ment à deux							
1		0	0	0	0	0	0	0	1
	+	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
		2^0							
	=	1							
1		1	1	1	1	1	1	1	1
	-	2^6	2^5	2^4	2^3	2^2	2^1	2^0	

-	(256
-	
-	2^7
-	
-	2^6
-	
-	2^5
-	
-	2^4
-	
-	2^3
-	
-	2^2
-	
-	2^1
-	
-	2^0)
=	
-	
-	1

i

Remarquez que cette représentation revient finalement à inverser tous les *bits* d'un nombre positif et à augmenter le résultat de un en vue d'obtenir son équivalent négatif. Par exemple, si nous inversons les *bits* du nombre 1 nous obtenons 1111 1110 et si nous lui ajoutons un, nous avons bien 1111 1111.

Cette fois ci, l'objectif recherché est atteint ! 🍏

En effet, si nous additionnons 1 et -1 (soit 0000 0001 et 1111 1111) et que nous ignorons la retenue, nous obtenons bien zéro. De plus, il n'y a plus de cas particulier de retenue à déplacer comme en complément à un, puisque, par exemple, la somme -1 + -2, soit 1111 1111 + 1111 1110 donne 1 1111 1101, autrement dit -3 sans la retenue. Enfin, nous n'avons plus qu'une seule représentation pour le zéro.

i

Point de norme

La norme du langage C ne précise pas quelle représentation doit être utilisée pour les entiers signés¹. Elle impose uniquement qu'il s'agisse d'une de ces trois. Cependant, dans les faits, il s'agit presque toujours de la représentation en complément à deux.

✗

Point de norme

Notez que chacune de ces représentations dispose toutefois d'une suite de *bits* qui est susceptible de ne pas représenter une valeur et de produire une erreur en cas d'utilisation².
— Pour les représentations en signe et magnitude et en complément à deux, il s'agit



- de la suite où tous les *bits* sont à zéro et le *bit* de signe à un : `1000 0000` ;
- Pour la représentation en complément à un, il s'agit de la suite où tous les *bits* sont à un, y compris le *bit* de signe : `1111 1111`.

Dans le cas des représentations en signe et magnitude et en complément à un, il s'agit des représentations possibles pour le « zéro négatif ». Pour la représentation en complément à deux, cette suite est le plus souvent utilisée pour représenter un nombre négatif supplémentaire (dans le cas de `1000 0000`, il s'agira de `-128`). Toutefois, même si ces représentations peuvent être utilisées pour représenter une valeur valide, ce n'est pas forcément le cas. Néanmoins, rassurez-vous, ces valeurs ne peuvent être produites dans le cas de calculs ordinaires, sauf survenance d'une condition exceptionnelle comme un dépassement de capacité (nous en parlerons bientôt). Vous n'avez donc pas de vérifications supplémentaires à effectuer à leur égard. Évitez simplement d'en produire une délibérément, par exemple en l'affectant directement à une variable.

V.1.1.3. Les bits de bourrage

Il est important de préciser que tous les *bits* composant un type entier ne sont pas forcément utilisés pour représenter la valeur qui y est stockée. Nous l'avons vu avec le *bit* de signe, qui ne correspond pas à une valeur. La norme prévoit également qu'il peut exister des *bits* de bourrage, et ce *aussi bien pour les entiers signés que pour les entiers non signés* à l'exception du type `char`. Ceux-ci peuvent par exemple être employés pour maintenir d'autres informations (par exemple : le type de la donnée stockée, ou encore un *bit de parité*  pour vérifier l'intégrité de celle-ci).



Par conséquent, il n'y a pas forcément une corrélation parfaite entre le nombre de *bits* composant un type entier et la valeur maximale qui peut y être stockée.



Sachez toutefois que les *bits* de bourrage au sein des types entiers sont assez rares, les architectures les plus courantes n'en emploient pas.

V.1.1.4. Les entiers de taille fixe

Face à ces incertitudes quant à la représentation des entiers, la norme C99 a introduit de nouveaux types entiers : les entiers de taille fixe. À la différence des autres types entiers, leur représentation est plus stricte :

1. Ces types ne comprennent pas de *bits* de bourrage ;
2. Comme leur nom l'indique, leur taille est un nombre de *bits* fixe ;
3. La représentation en complément à deux est utilisée pour les nombres négatifs.

Ces types sont définis dans l'en-tête `<stdint.h>` et sont les suivants.

Type	Minimum	Maximum
<code>int8_t</code>	-128	127
	INT8_MIN	INT8_MAX
<code>uint8_t</code>	0	255
		UINT8_MAX
<code>int16_t</code>	-32768	32767
	INT16_MIN	INT16_MAX
<code>uint16_t</code>	0	65535
		UINT16_MAX
<code>int32_t</code>	-2147483648	2147483647
	INT32_MIN	INT32_MAX
<code>uint32_t</code>	0	4294.967.295
		UINT32_MAX
<code>int64_t</code>	-9.223.372.036.854.775.807	9.223.372.036.854.775.807
	INT64_MIN	INT64_MAX
<code>uint64_t</code>	0	18.446.744.073.709.551.615
		UINT64_MAX

TABLE V.1.6. – Notez que les limites sont ici exactes, le nombre de *bits* étant fixé et l'absence de *bits* de bourrage garantie.

Pour le reste, ces types s'utilisent comme les autres types entiers, si ce n'est qu'ils n'ont pas d'indicateur de conversion spécifique. À la place, il est nécessaire de recourir à des macroconstantes qui sont remplacées par une chaîne de caractères comportant le bon indicateur de conversion. Ces dernières sont définies dans l'en-tête `<inttypes.h>` et sont les suivantes.

Type	Macroconstantes
<code>int8_t</code>	PRId8 (ou PRIi8) et SCNd8 (ou SCNi8)
<code>uint8_t</code>	PRId8 (ou PRIo8, PRIx8, PRIx8) et SCNu8 (ou SCNo8, SCNx8)

<code>int16_t</code>	<code>PRId16</code> (ou <code>PRi16</code>) et <code>SCNd16</code> (ou <code>SCNi16</code>)
<code>uint16_t</code>	<code>PRId16</code> (ou <code>PRi16</code>) et <code>SCNd16</code> (ou <code>SCNi16</code>)
<code>int32_t</code>	<code>PRId32</code> (ou <code>PRi32</code>) et <code>SCNd32</code> (ou <code>SCNi32</code>)
<code>uint32_t</code>	<code>PRId32</code> (ou <code>PRi32</code>) et <code>SCNd32</code> (ou <code>SCNi32</code>)
<code>int64_t</code>	<code>PRId64</code> (ou <code>PRi64</code>) et <code>SCNd64</code> (ou <code>SCNi64</code>)
<code>uint64_t</code>	<code>PRId64</code> (ou <code>PRi64</code>) et <code>SCNd64</code> (ou <code>SCNi64</code>)

```

1 #include <inttypes.h>
2 #include <stdint.h>
3 #include <stdio.h>
4
5
6 int
7 main(void)
8 {
9     int16_t n = INT16_MIN;
10    uint64_t m = UINT64_MAX;
11
12    printf("n = %" PRId16 ", m = %" PRIu64 "\n", n, m);
13    return 0;
14 }
```

Résultat

```
1 n = -32768, m = 18446744073709551615
```



Le support de ces types est *facultatif*, il est donc possible qu'ils ne soient pas disponibles, bien que cela soit assez peu probable.

1. ISO/IEC 9899:201x, doc. N1570, § 6.2.6.2, Integer types, p. 45.
2. ISO/IEC 9899:201x, doc. N1570, § 6.2.6.1, General, al. 5, p. 44.

V.1.2. La représentations des flottants

La représentation des types flottants amène deux difficultés supplémentaires :

- la gestion de nombres réels, c'est-à-dire potentiellement composés d'une partie entière et d'une suite de décimales ;
- la possibilité de stocker des nombres de différents ordres de grandeur, entre 10^{-37} et 10^{37} .

V.1.2.1. Première approche

V.1.2.1.1. Avec des entiers

Une première solution consisterait à garantir une précision à 10^{-37} en utilisant deux nombres entiers : un, signé, pour la partie entière et un, non signé, pour stocker la suite de décimales.

Cependant, si cette approche a le mérite d'être simple, elle a le désavantage d'utiliser *beaucoup* de mémoire. En effet, pour stocker un entier de l'ordre de 10^{37} , il serait nécessaire d'utiliser un peu moins de 128 *bits*, soit 16 octets (et donc environ 32 octets pour la représentation globale). Un tel coût est inconcevable, même à l'époque actuelle.

Une autre limite provient de la difficulté d'effectuer des calculs sur les nombres flottants avec une telle représentation : il faudrait tenir compte du passage des décimales vers la partie entière et inversement.

V.1.2.1.2. Avec des puissances de deux négatives

Une autre représentation possible consiste à attribuer des puissances de deux *néglatives* à une partie des *bits*. Les calculs sur les nombres flottants obtenus de cette manière sont similaires à ceux sur les nombres entiers. Le tableau ci-dessous illustre ce concept en divisant un octet en deux : quatre *bits* pour la partie entière et quatre *bits* pour la partie fractionnaire.

	Re-pré- Nomb ta- tion							
1	0	0	0	1	0	0	0	0
	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	2^0 = 1							
1,5	0	0	0	1	1	0	0	0
	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}
	2^{-1} + 2^0 = 1,5							
0,875	0	0	0	0	1	1	1	0
	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}

	2^{-3}
+	
2^{-2}	
+	
2^{-1}	
=	
0,875	

Toutefois, notre problème de capacité persiste : il nous faudra toujours une grande quantité de mémoire pour pouvoir stocker des nombres d'ordre de grandeur aussi différents.

V.1.2.2. Une histoire de virgule flottante

Par conséquent, la solution qui a été retenue historiquement consiste à réserver quelques *bits* qui contiendront la valeur d'un exposant. Celui-ci sera ensuite utilisé pour déterminer à quelles puissances de deux correspondent les *bits* restants. Ainsi, la virgule séparant la partie entière de sa suite décimales est dite « flottante », car sa position est ajustée par l'exposant. Toutefois, comme nous allons le voir, ce gain en mémoire ne se réalise pas sans sacrifice : la précision des calculs va en pâtir.

Le tableau suivant utilise deux octets : un pour stocker un exposant sur sept *bits* avec un *bit* de signe ; un autre pour stocker la valeur du nombre. L'exposant est lui aussi signé et est représenté en signe et magnitude (par souci de facilité). Par ailleurs, cet exposant est attribué au premier *bit* du deuxième octet ; les autres correspondent à une puissance de deux chaque fois inférieure d'une unité.

		Ex- posant							Bits du nombre						
		0	0	0	0	0	0	0	1	0	0	0	0	0	0
1	+	2^5	2^4	2^3	2^2	2^1	2^0	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
	+	0							2^0						
									=						
									1						
-	1	1	0	0	0	0	0	0	1	0	0	0	0	0	0
	-	2^5	2^4	2^3	2^2	2^1	2^0	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
	-	0							2^0						
									=						
									-						
									1						
0,5	+	0	0	0	0	0	0	0	0	1	0	0	0	0	0
	+	2^5	2^4	2^3	2^2	2^1	2^0	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
	+	0							2^{-1}						
									=						
									0,5						
10	+	0	0	0	0	0	0	1	1	1	0	1	0	0	0
	+	2^5	2^4	2^3	2^2	2^1	2^0	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}

[illegible]

Les quatre premiers exemples n'amènent normalement pas de commentaires particuliers ; néanmoins, les deux derniers illustrent deux problèmes posés par les nombres flottants.

V.1.2.2.1. Les approximations

Les nombres réels mettent en évidence la difficulté (voir l'impossibilité) de représenter une suite de décimales à l'aide d'une somme de puissances de deux. En effet, le plus souvent, les valeurs ne peuvent être qu'approchées et obtenues suite à des arrondis. Dans le cas de 0,00001, la somme $2^{-24} + 2^{-23} + 2^{-22} + 2^{-20} + 2^{-19} + 2^{-17}$ donne en vérité 0,000010907649993896484375 qui, une fois arrondie, donnera bien 0,00001.

V.1.2.2.2. Les pertes de précision

Dans le cas où la valeur à représenter ne peut l'être entièrement avec le nombre de *bits* disponibles (autrement dit, le nombre de puissances de deux disponibles), la partie la moins significative

sera abandonnée et, corrélativement, de la précision.

Ainsi, dans notre exemple, si nous souhaitons représenter le nombre 32769, nous avons besoin d'un exposant de 15 afin d'obtenir 32768. Toutefois, vu que nous ne possédons que de 8 *bits* pour représenter notre nombre, il nous est impossible d'attribuer l'exposant 0 à un des *bits*. Cette information est donc perdue et seule la valeur la plus significative (ici 32768) est conservée.

V.1.2.3. Formalisation

L'exemple simplifié que nous vous avons montré illustre dans les grandes lignes la représentation des nombres flottants. Cependant, vous vous en doutez, la réalité est un peu plus complexe. De manière plus générale, un nombre flottant est représenté à l'aide : d'un signe, d'un exposant et d'une **mantisse** qui est en fait la suite de puissances qui sera additionnée.

Par ailleurs, nous avons utilisé une suite de puissances de deux, mais il est parfaitement possible d'utiliser une autre base. Beaucoup de calculatrices utilisent par exemple des suites de puissances de dix et non de deux.



IEEE 754

À l'heure actuelle, les nombres flottants sont presque toujours représentés suivant la norme IEEE 754 [↗](#) qui utilise une représentation en base deux. Si vous souhaitez en apprendre davantage, nous vous invitons à lire le tutoriel [Introduction à l'arithmétique flottante ↗](#) d'Aabu

V.1.3. La représentation des pointeurs

Cet extrait sera relativement court : la représentation des pointeurs est indéterminée. Sur la plupart des architectures, il s'agit en vérité d'entiers non signés, mais il n'y a absolument aucune garantie à ce sujet.

V.1.4. Ordre des multipliets et des bits

Jusqu'à présent, nous vous avons montré les représentations binaires en ordonnant les *bits* et les multipliets par puissance de deux décroissantes (de la plus grande puissance de deux à la plus petite). Cependant, s'il s'agit d'une représentation possible, elle n'est pas la seule. En vérité, l'ordre des multipliets et des *bits* peut varier d'une machine à l'autre.

V.1.4.1. Ordre des multipliets

Dans le cas où un type est composé de plus d'un multipliet (ce qui, à l'exception du type **char**, est pour ainsi dire toujours le cas), ceux-ci peuvent être agencés de différentes manières. L'ordre des multipliets d'une machine est appelé son **boutisme** (*endianness* en anglais).

Il en existe principalement deux : le gros-boutisme et le petit-boutisme.

V.1.4.1.1. Le gros-boutisme

Sur une architecture gros-boutiste, les multipliets sont ordonnés par **poids** décroissant.

Le poids d'un *bit* se détermine suivant la puissance de deux qu'il représente : plus elle est élevée, plus le *bit* a un poids important. Le *bit* représentant la puissance de deux la plus basse est appelé de *bit* de **poids faible** et celui correspondant à la puissance la plus grande est nommé *bit* de **poids fort**. Pour les multiplets, le raisonnement est identique : son poids est déterminé par celui des *bits* le composant.

Autrement dit, une machine gros-boutiste place les multiplets comprenant les puissances de deux les plus élevées en premier (nos exemples précédents utilisaient donc cette représentation).

V.1.4.1.2. Le petit-boutisme

Une architecture petit-boutiste fonctionne de manière inverse : les multiplets sont ordonnés par poids croissant.

Ainsi, si nous souhaitons stocker le nombre **1** dans une variable de type **unsigned short** (qui sera, pour notre exemple, d'une taille de deux octets), nous aurons les deux résultats suivants, selon le boutisme de la machine.

Re- pré- sen- tion gros- boutiste	Nombre															
	Oc- tet de poids fort								Oc- tet de poids faible							
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	2^0															
1	=															
	1															

Re- pré- sen- tion petit- boutiste	Nombre															
	Oc- tet de poids faible								Oc- tet de poids fort							
	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8

	2^0
	=
	1

i

Le boutisme est relativement transparent du point de vue du programmeur, puisqu'il s'agit d'une propriété de la mémoire. En pratique, de tels problèmes ne se posent que lorsqu'on accède à la mémoire à travers des types différents de celui de l'objet stocké initialement (par exemple via un pointeur sur `char`). En particulier, la communication entre plusieurs machines doit les prendre en compte.

V.1.4.2. Ordre des bits

Cependant, ce serait trop simple si le problème en restait là... 🍊

Malheureusement, l'ordre des *bits* varie également d'une machine à l'autre et ce, *indépendamment de l'ordre des multipléts*. Comme pour les multipléts, il existe différentes possibilités, mais les plus courantes sont le gros-boutisme et le petit-boutisme. Ainsi, il est par exemple possible que la représentation des multipléts soit gros-boutiste, mais que celle des *bits* soit petit-boutiste (et inversement).

Re- pré- sen- ta- tion gros- boutiste pour les mul- ti- pléts et petit- boutiste pour les <i>bits</i>	
Oc- tet de poids fort	Oc- tet de poids faible
0 0 0 0 0 0 0 0	1 0 0 0 0 0 0 0
2^8 2^9 2^{10} 2^{11} 2^{12} 2^{13} 2^{14} 2^{15}	2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7
2^0	
1 =	
1	

Re- pré- sen- ta- tion petit- boutiste pour les multiplets et petit- boutiste pour les <i>bits</i>	
Octet de poids faible	Octet de poids fort
1 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0
2^0 2^1 2^2 2^3 2^4 2^5 2^6 2^7	2^8 2^9 2^{10} 2^{11} 2^{12} 2^{13} 2^{14} 2^{15}
2^0	
1 =	
1	



Notez toutefois que, le plus souvent, une représentation petit-boutiste ou gros-boutiste s'applique aussi bien aux octets qu'aux *bits*.

Par ailleurs, sachez qu'à quelques exceptions près, l'ordre du stockage des *bits* n'apparaît pas en C. En particulier, les opérateurs de manipulation de *bits*, vus au chapitre suivant, n'en dépendent pas.

V.1.4.3. Applications

V.1.4.3.1. Connaître le boutisme d'une machine

Le plus souvent, il est possible de connaître le boutisme employé pour les *multiplètes* à l'aide du code suivant. Ce dernier stocke la valeur 1 dans une variable de type `unsigned short`, et accède à son premier octet à l'aide d'un pointeur sur `unsigned char`. Si celui-ci est nul, c'est que la machine est gros-boutiste, sinon, c'est qu'elle est petit-boutiste.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     unsigned short n = 1;
7     unsigned char *p = (unsigned char *)&n;
8
9     if (*p != 0)
10        printf("Je suis petit-boutiste.\n");
11    else
12        printf("Je suis gros-boutiste.\n");
13
14    return 0;
15 }

```



Notez bien que cette technique n'est pas entièrement fiable. En effet, rappelez-vous : d'une part, les deux boutismes présentés, s'ils sont les plus fréquents, ne sont pas les seuls et, d'autre part, la présence de *bits* de bourrage au sein du type entier, bien que rare, pourrait fausser le résultat.

V.1.4.3.2. Le boutisme des constantes octales et hexadécimales

Suivant ce qui vient de vous être présenté, peut-être vous êtes vous posé la question suivante : si les boutismes varient d'une machine à l'autre, alors que vaut finalement la constante `0x01` ? En effet, suivant les boutismes employés, celle-ci peut valoir 1 ou 16.

Heureusement, cette question est réglée par la norme¹ : le boutisme ne change *rien* à l'écriture des constantes octales et hexadécimales, elle est toujours réalisée suivant la convention classique des chiffres de poids fort en premier. Ainsi, la constante `0x01` vaut 1 et la constante `0x8000` vaut 32768 (en non signé).

De même, les indicateurs de conversion `x`, `X` et `o` affichent toujours leurs résultats suivant cette écriture.

```
1 printf("%02x\n", 1);
```

Résultat

1	01
---	----

1. ISO/IEC 9899:201x, doc. N1570, § 6.4.4.1, Integer constants, al. 4, p. 63.

V.1.5. Les fonctions `memset`, `memcpy`, `memmove` et `memcmp`

Pour terminer ce chapitre, il nous reste à vous présenter quatre fonctions que nous avons passées sous silence lors de notre présentation de l'en-tête `<string.h>` : `memset()`, `memcpy()`, `memmove()` et `memcmp()`. Bien qu'elles soient définies dans cet en-tête, ces fonctions ne sont pas véritablement liées aux chaînes de caractères et opèrent en vérité sur les multipliets composant les objets. De telles modifications impliquant la représentation des types, nous avons attendu ce chapitre pour vous en parler.

V.1.5.1. La fonction `memset`

```
1 void *memset(void *obj, int val, size_t taille);
```

La fonction `memset()` initialise les `taille` premiers multipliets de l'objet référencé par `obj` avec la valeur `val` (qui est convertie vers le type `unsigned char`). Cette fonction est très utile pour (ré)initialiser les différents éléments d'un agrégat sans devoir recourir à une boucle.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     int tab[] = { 10, 20, 30, 40, 50 };
8
9     memset(tab, 0, sizeof tab);
10
11     for (size_t i = 0; i < sizeof tab / sizeof tab[0]; ++i)
12         printf("%zu : %d\n", i, tab[i]);
13
14     return 0;
15 }
```

Résultat

```
1 0 : 0
2 1 : 0
3 2 : 0
4 3 : 0
5 4 : 0
```



Faites attention ! Manipuler ainsi les objets *byte* par *byte* nécessite de connaître leur représentation. Dans un souci de portabilité, on ne devrait donc pas utiliser `memset` sur des pointeurs ou des flottants. De même, pour éviter d'obtenir les représentations problématiques dans le cas d'entiers signés, il est conseillé de n'employer cette fonction que pour mettre ces derniers à zéro.

V.1.5.2. Les fonctions `memcpy` et `memmove`

```
1 void *memcpy(void *destination, void *source, size_t taille);
2 void *memmove(void *destination, void *source, size_t taille);
```

Les fonctions `memcpy()` et `memmove()` copient toutes deux les `taille` premiers multiplètes de l'objet pointé par `source` vers les premiers multiplètes de l'objet référencé par `destination`. La différence entre ces deux fonctions est que `memcpy()` ne doit être utilisée qu'avec deux objets qui ne se chevauchent pas (autrement dit, les deux pointeurs ne doivent pas accéder ou modifier une même zone mémoire ou une partie d'une même zone mémoire) alors que `memmove()` ne souffre pas de cette restriction.

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     int n = 10;
8     int m;
9
10    memcpy(&m , &n, sizeof n);
11    printf("m = n = %d\n", m);
12
13    memmove(&n, ((unsigned char *)&n) + 1, sizeof n - 1);
14    printf("n = %d\n", n);
15    return 0;
16 }
```

Résultat

```
1 m = n = 10
2 n = 0
```

L'utilisation de `memmove()` amène quelques précisions : nous copions ici les trois derniers

multiplets de l'objet `n` vers ses trois premiers multiplets. Étant donné que notre machine est petit-boutiste, les trois derniers multiplets sont à zéro et la variable `n` a donc pour valeur finale zéro.

V.1.5.3. La fonction `memcmp`

```
1 int memcmp(void *obj1, void *obj2, size_t taille);
```

La fonction `memcmp()` compare les multiplets des objets référencés par `obj1` et `obj2`. Si tous leurs multiplets sont égaux, elle retourne zéro. Dans le cas contraire, elle s'arrête à la première paire de multiplets qui diffère et retourne un nombre négatif ou positif suivant que la valeur du multiplot comparé de `obj1` est inférieure ou supérieure à celle du multiplot comparé de `obj2` (pour la comparaison, les deux valeurs sont converties en `unsigned char`).

```
1 #include <stdio.h>
2 #include <string.h>
3
4
5 int main(void)
6 {
7     int n = 10;
8     int m = 10;
9     int o = 20;
10
11     if (memcmp(&n, &m, sizeof n) == 0)
12         printf("n et m sont égaux\n");
13
14     if (memcmp(&n, &o, sizeof n) != 0)
15         printf("n et o ne sont pas égaux\n");
16
17     return 0;
18 }
```

Résultat

```
1 n et m sont égaux
2 n et o ne sont pas égaux
```



Dit autrement, la fonction `memcmp()` retourne zéro si les deux portions comparées ont la même représentation.



Étant donné que `memcmp()` compare deux objets multiplète par multiplète, ceci inclut leurs éventuels *bits* ou multiplètes de bourrages. Il est donc parfaitement possible, par exemple, que `memcmp()` considère deux structures ou deux entiers comme différents simplement parce que leurs *bits* ou multiplètes de bourrages sont différents.

Conclusion

En résumé

1. les entiers non signés sont représentés sous forme d'une somme de puissance de deux ;
2. il existe trois représentations possibles pour les entiers signés, mais la plus fréquente est celle en complément à deux ;
3. les types entiers peuvent contenir des *bits* de bourrage *sauf* le type `char` ;
4. les types flottants sont représentés sous forme d'un signe, d'un exposant et d'une mantisse, mais sont susceptibles d'engendrer des approximations et des pertes de précision ;
5. la représentation des pointeurs est indéterminée ;
6. le boutisme utilisé dépend de la machine cible ;
7. les constantes sont toujours écrites suivant la représentation gros-boutiste.

Liens externes Si vous souhaitez quelques exemples d'architectures exotiques, nous vous invitons à lire l'article « [C portability](#) » de Joe Nelson.

V.2. Les limites des types

Introduction

Nous avons abordé les limites des types à plusieurs reprises dans les chapitres précédents, mais nous ne nous sommes pas encore véritablement arrêté sur ce point. Il est à présent temps pour nous de vous présenter les implications de ces limites.

V.2.1. Les limites des types

Au début de ce cours, nous vous avons précisé que tous les types avaient des bornes et qu'en conséquence, ils ne pouvaient stocker qu'un intervalle défini de valeurs. Nous vous avons également indiqué que la norme du langage imposait des minimas pour ces intervalles qui, pour rappel, sont les suivants.

Type	Minimum	Maximum
<code>_Bool</code>	0	1
<code>signed char</code>	-127	127
<code>unsigned char</code>	0	255
<code>short</code>	-32 767	32 767
<code>unsigned short</code>	0	65 535
<code>int</code>	-32 767	32 767
<code>unsigned int</code>	0	65 535
<code>long</code>	-2 147 483 647	2 147 483 647
<code>unsigned long</code>	0	4 294 967 295
<code>long long</code>	-9 223 372 036 854 775 807	9 223 372 036 854 775 807
<code>unsigned long long</code>	0	18 446 744 073 709 551 615
<code>float</code>	-1×10^{37}	1×10^{37}
<code>double</code>	-1×10^{37}	1×10^{37}
<code>long double</code>	-1×10^{37}	1×10^{37}

Toutefois, mis à part pour les types `_Bool` et `char`, les bornes des types vont le plus souvent au-delà de ces minimums garantis. Cependant, pour pouvoir manipuler nos types dans leur limites, encore faut-il les connaître. Heureusement pour nous, la norme nous fournit deux

en-têtes définissant des macroconstantes correspondant aux limites des types entiers et flottants : `<limits.h>` et `<float.h>`.

V.2.1.1. L'en-tête `<limits.h>`

Type	Minimum	Maximum
signed char	SCHAR_MIN	SCHAR_MAX
unsigned char	0	UCHAR_MAX
short	SHRT_MIN	SHRT_MAX
unsigned short	0	USHRT_MAX
int	INT_MIN	INT_MAX
unsigned int	0	UINT_MAX
long	LONG_MIN	LONG_MAX
unsigned long	0	ULONG_MAX
long long	LLONG_MIN	LLONG_MAX
unsigned long long	0	ULLONG_MAX



Remarquez qu'il n'y a pas de macroconstantes pour le minimum des types entiers non signés puisque celui-ci est *toujours* zéro.

À ces macroconstantes, il faut également ajouter `CHAR_BIT` qui vous indique le nombre *bits* composant un multiplét. Celle-ci a pour valeur minimale huit, c'est-à-dire qu'en C, un multiplét ne peut avoir moins de huit *bits*.

L'exemple suivant utilise ces macroconstantes et affiche leur valeur.

```

1  #include <limits.h>
2  #include <stdio.h>
3
4
5  int
6  main(void)
7  {
8      printf("Un multiplét se compose de %d bits.\n", CHAR_BIT);
9      printf("signed char : min = %d ; max = %d.\n", SCHAR_MIN,
10             SCHAR_MAX);
11     printf("unsigned char : min = 0 ; max = %u.\n", UCHAR_MAX);
12     printf("short : min = %d ; max = %d.\n", SHRT_MIN,
13            SHRT_MAX);

```



```

12     printf("unsigned short : min = 0 ; max = %u.\n",
           USHRT_MAX);
13     printf("int : min = %d ; max = %d.\n", INT_MIN, INT_MAX);
14     printf("unsigned int : min = 0 ; max = %u.\n", UINT_MAX);
15     printf("long : min = %ld ; max = %ld.\n", LONG_MIN,
           LONG_MAX);
16     printf("unsigned long : min = 0 ; max = %lu.\n",
           ULONG_MAX);
17     printf("long long : min = %lld ; max = %lld.\n",
           LLONG_MIN, LLONG_MAX);
18     printf("unsigned long long : min = 0 ; max = %llu.\n",
           ULLONG_MAX);
19     return 0;
20 }

```

Résultat

```

1  Un multiplét se compose de 8 bits.
2  signed char : min = -128 ; max = 127.
3  unsigned char : min = 0 ; max = 255.
4  short : min = -32768 ; max = 32767.
5  unsigned short : min = 0 ; max = 65535.
6  int : min = -2147483648 ; max = 2147483647.
7  unsigned int : min = 0 ; max = 4294967295.
8  long : min = -9223372036854775808 ; max = 9223372036854775807.
9  unsigned long : min = 0 ; max = 18446744073709551615.
10 long long : min = -9223372036854775808 ; max =
    9223372036854775807.
11 unsigned long long : min = 0 ; max = 18446744073709551615.

```



Bien entendu, les valeurs obtenues dépendent de votre machine, celles-ci sont simplement données à titre d'illustration.

V.2.1.2. L'en-tête <float.h>

Les types flottants amènent une subtilité supplémentaire : ceux-ci disposent en vérité de *quatre* bornes : le plus grand nombre représentable, le plus petit nombre représentable (qui est l'opposé du précédent), la plus petite partie fractionnaire représentable (autrement dit, le nombre le plus proche de zéro représentable) et son opposé. Dit autrement, les bornes des types flottants peuvent être schématisées comme suit.

1	-MAX	-MIN	0	MIN	MAX
2	+-----+		+	+-----+	

Où **MAX** représente le plus grand nombre représentable et **MIN** le plus petit nombre représentable avant zéro. Étant donné que deux des bornes ne sont finalement que l'opposé des autres, deux macroconstantes sont suffisantes pour chaque type flottant.

Type	Maximum	Minimum de la partie fractionnaire
float	FLT_MAX	FLT_MIN
double	DBL_MAX	DBL_MIN
long double	LDBL_MAX	LDBL_MIN

```

1 #include <float.h>
2 #include <stdio.h>
3
4
5 int
6 main(void)
7 {
8     printf("float : min = %e ; max = %e.\n", FLT_MIN, FLT_MAX);
9     printf("double : min = %e ; max = %e.\n", DBL_MIN,
10           DBL_MAX);
11     printf("long double : min = %Le ; max = %Le.\n", LDBL_MIN,
12           LDBL_MAX);
13     return 0;
14 }
```

Résultat

```

1 float : min = 1.175494e-38 ; max = 3.402823e+38.
2 double : min = 2.225074e-308 ; max = 1.797693e+308.
3 long double : min = 3.362103e-4932 ; max = 1.189731e+4932.
```

V.2.2. Les dépassements de capacité

Nous savons à présent comment obtenir les limites des différents types de base, c'est bien. Toutefois, nous ignorons toujours *quand* la limite d'un type est dépassée et ce qu'il se passe dans un tel cas.

Le franchissement de la limite d'un type est appelé un **dépassement de capacité** (*overflow* ou *underflow* en anglais, suivant que la limite maximale ou minimale est outrepassée). Un

dépassement survient lorsqu'une opération produit un résultat dont la valeur n'est pas représentable par le type de l'expression ou lorsqu'une valeur est convertie vers un type qui ne peut la représenter.

Ainsi, dans l'exemple ci-dessous, l'expression `INT_MAX + 1` provoque un dépassement de capacité, de même que la conversion (implicite) de la valeur `INT_MAX` vers le type `signed char`.

```
1 signed char c;
2
3 if (INT_MAX + 1) /* Dépassement. */
4     c = INT_MAX; /* Dépassement. */
5
6 c = SCHAR_MAX; /* Ok. */
```



Gardez en mémoire que des conversions implicites ont lieu lors des opérations. Revoyez le chapitre sur les opérations mathématiques si cela vous paraît lointain.



D'accord, mais que se passe-t-il dans un tel cas ?

C'est ici que le bât blesse : ce n'est pas précisé. 🍊

Ou, plus précisément, cela dépend des situations.

V.2.2.1. Dépassement lors d'une opération

V.2.2.1.1. Les types entiers

V.2.2.1.1.1. Les entiers signés Si une opération travaillant avec des entiers signés dépasse la capacité du type du résultat, le comportement est indéfini. Dans la pratique, il y a trois possibilités :

1. La valeur boucle, c'est-à-dire qu'une fois une limite atteinte, la valeur revient à la seconde. Cette solution est la plus fréquente et s'explique par le résultat de l'arithmétique en complément à deux. En effet, à supposer qu'un `int` soit représenté sur 32 *bits* :
 - `INT_MAX + 1` se traduit par `0111 1111 1111 1111 1111 1111 1111 1111 + 0000 0000 0000 0000 0000 0000 0001`, ce qui donne `1000 0000 0000 0000 0000 0000 0000`, soit `INT_MIN` ; et
 - `INT_MIN - 1` se traduit par `1000 0000 0000 0000 0000 0000 0000 0000 + 1111 1111 1111 1111 1111 1111 1111` ce qui donne `0111 1111 1111 1111 1111 1111 1111`, soit `INT_MAX`.
2. La valeur sature, c'est-à-dire qu'une fois une limite atteinte, la valeur reste bloquée à celle-ci.
3. Le processeur considère l'opération comme invalide et stoppe l'exécution du programme.

V.2.2.1.1.2. Les entiers non signés Si une opération travaillant avec des entiers non signés dépasse la capacité du type du résultat, alors il n'y a à proprement parler *pas* de dépassement et la valeur boucle. Autrement dit, l'expression `UINT_MAX + 1` donne par exemple *toujours* zéro.

V.2.2.1.2. Les types flottants

Si une opération travaillant avec des flottants dépasse la capacité du type du résultat, alors, comme pour les entiers signés, le comportement est indéfini. Toutefois, les flottants étant généralement représentés suivant la norme IEEE 754, le résultat sera souvent le suivant :

1. En cas de dépassement de la borne maximale ou minimale, le résultat sera égal à l'infini (positif ou négatif).
2. En cas de dépassement de la limite de la partie fractionnaire, le résultat sera arrondi à zéro.

Néanmoins, gardez à l'esprit que ceci *n'est pas garanti* par la norme. Il est donc parfaitement possible que votre programme soit tout simplement arrêté.

V.2.2.2. Dépassement lors d'une conversion

V.2.2.2.1. Valeur entière vers entier signé

Si la conversion d'une valeur entière vers un type signé produit un dépassement, le résultat n'est pas déterminé.

V.2.2.2.2. Valeur entière vers entier non signé

Dans le cas où la conversion d'une valeur entière vers un type non signé, la valeur boucle, comme précisé dans le cas d'une opération impliquant des entiers non signés.

V.2.2.2.3. Valeur flottante vers entier

Lors d'une conversion d'une valeur flottante vers un type entier (signé ou non signé), la partie fractionnaire est ignorée. Si la valeur restante n'est pas représentable, le résultat est indéfini (*y compris* pour les types non signés, donc !).

V.2.2.2.4. Valeur entière vers flottant

Si la conversion d'une valeur entière vers un type flottant implique un dépassement, le comportement est indéfini.

i

Notez que si le résultat n'est pas *exactement* représentable (rappelez-vous des pertes de précision) la valeur sera approchée, mais cela ne constitue pas un dépassement.

V.2.2.2.5. Valeur flottante vers flottant

Enfin, si la conversion d'une valeur flottante vers un autre type flottant produit un dépassement, le comportement est indéfini.

i

Comme pour le cas d'une conversion d'un type entier vers un type flottant, si le résultat n'est pas *exactement* représentable, la valeur sera approchée.

V.2.3. Gérer les dépassements entiers

Nous savons à présent ce qu'est un dépassement, quand ils se produisent et dans quel cas ils sont problématiques (en vérité, potentiellement tous, puisque même si la valeur boucle, ce n'est pas forcément ce que nous souhaitons). Reste à présent pour nous à les gérer, c'est-à-dire les détecter et les éviter. Malheureusement pour nous, la bibliothèque standard ne nous fournit aucun outil à cet effet, mise à part les macroconstantes qui vous ont été présentées. Aussi il va nous être nécessaire de réaliser nos propres outils.

V.2.3.1. Préparation

Avant de nous lancer dans la construction de ces fonctions, il va nous être nécessaire de déterminer leur forme. En effet, si, intuitivement, il vous vient sans doute à l'esprit de créer des fonctions prenant deux paramètres et fournissant un résultat, reste le problème de la gestion d'erreur.

```
1 int safe_add(int a, int b);
```

En effet, comment précise-t-on à l'utilisateur qu'un dépassement a été détecté, toutes les valeurs du type `int` pouvant être retournées ? Eh bien, pour résoudre ce souci, nous allons recourir dans les exemples qui suivent à la variable `errno`, à laquelle nous affecterons la valeur `ERANGE` en cas de dépassement. Ceci implique qu'elle soit mise à zéro avant tout appel à ces fonctions (revoyez le premier chapitre sur la gestion d'erreur si cela ne vous dit rien).

Par ailleurs, afin que ces fonctions puissent éventuellement être utilisées dans des suites de calculs, nous allons faire en sorte qu'elles retournent la borne maximale ou minimale en cas de dépassement.

Ceci étant posé, passons à la réalisation de ces fonctions. 🍊

V.2.3.2. L'addition

Afin d'empêcher un dépassement, il va nous falloir le détecter et ceci, bien entendu, à l'aide de conditions qui ne doivent pas elles-mêmes produire de dépassement. Ainsi, ce genre de condition est à proscrire : `if (a + b > INT_MAX)`. Pour le cas d'une addition deux cas de figure se présentent :

1. `b` est positif et il nous faut alors vérifier que la somme ne dépasse pas le maximum.
2. `b` est négatif et il nous faut dans ce cas déterminer si la somme dépasse ou non le minimum.

Ceci peut être contrôlé en vérifiant que `a` n'est pas supérieur ou inférieur à, respectivement, `MAX - b` et `MIN - b`. Ainsi, il nous est possible de rédiger une fonction comme suit.

```
1 int safe_add(int a, int b)
2 {
3     int err = 1;
4
5     if (b > 0 && a > INT_MAX - b)
6         goto overflow;
```

```

7         else if (b < 0 && a < INT_MIN - b)
8             goto underflow;
9
10        return a + b;
11    underflow:
12        err = -1;
13    overflow:
14        errno = ERANGE;
15        return err > 0 ? INT_MAX : INT_MIN;
16    }

```

Contrôlons à présent son fonctionnement.

```

1  #include <errno.h>
2  #include <limits.h>
3  #include <stdio.h>
4  #include <string.h>
5
6  /* safe_add() */
7
8
9  static void test_add(int a, int b)
10 {
11     errno = 0;
12     printf("%d + %d = %d", a, b, safe_add(a, b));
13     printf(" (%s).\n", strerror(errno));
14 }
15
16
17 int main(void)
18 {
19     test_add(INT_MAX, 1);
20     test_add(INT_MIN, -1);
21     test_add(INT_MIN, 1);
22     test_add(INT_MAX, -1);
23     return 0;
24 }

```

```

1  2147483647 + 1 = 2147483647 (Numerical result out of range).
2  -2147483648 + -1 = -2147483648 (Numerical result out of range).
3  -2147483648 + 1 = -2147483647 (Success).
4  2147483647 + -1 = 2147483646 (Success).

```

V.2.3.3. La soustraction

Pour la soustraction, le principe est le même que pour l'addition si ce n'est que les tests doivent être quelque peu modifiés. En guise d'exercice, essayez de trouver la solution par vous-même.



👁 Contenu masqué n°49



Mais ?! Pourquoi ne pas simplement faire `safe_add(a, -b)` ?

Bonne question ! 🍊

Cela a de quoi surprendre de prime à bord, mais l'opérateur de négation `-` peut produire un dépassement. En effet, souvenez-vous : la représentation en complément à deux ne dispose que d'une seule représentation pour zéro, du coup, il reste une représentation qui est possiblement invalide où seul le *bit* de signe est à un. Cependant, le plus souvent, cette représentation est utilisée pour fournir un nombre négatif supplémentaire. Or, si c'est le cas, il y a une asymétrie entre la borne inférieure et supérieure et la négation de la limite inférieure produira un dépassement. Il nous est donc nécessaire de prendre ce cas en considération.

V.2.3.4. La négation

À ce sujet, pour réaliser cette opération, deux solutions s'offrent à nous :

1. Vérifier si l'opposé de la borne supérieure est plus grand que la borne inférieure (autrement dit que `-INT_MAX > INT_MIN`) et, si c'est le cas, vérifier que le paramètre n'est pas la borne inférieure ; ou
2. Vérifier `sub(0, a)` où `a` est l'opérande qui doit subir la négation.

Notre préférence allant à la seconde solution, notre fonction sera donc la suivante.

```
1 int safe_neg(int a)
2 {
3     return safe_sub(0, a);
4 }
```

V.2.3.5. La multiplication

Pour la multiplication, cela se corse un peu, notamment au vu de ce qui a été dit précédemment au sujet de la représentation en complément à deux. Procédons par ordre :

1. Tout d'abord, si l'un des deux opérandes est positif, nous devons vérifier que le minimum n'est pas atteint (puisque le résultat sera négatif) et, si les deux sont positifs, que le maximum n'est pas dépassé. Toutefois, vu que nous allons recourir à la division pour le déterminer, nous devons en plus vérifier que le diviseur n'est pas nul.

```

1  if (b > 0)
2  {
3      if (a > 0 && a > INT_MAX / b)
4          goto overflow;
5      else if (a < 0 && a < INT_MIN / b)
6          goto underflow;
7  }

```

Ensuite, si l'un des deux opérandes est négatif, nous devons effectuer la même vérification que précédemment et, si les deux sont négatifs, vérifier que le résultat ne dépasse pas le maximum. Toutefois, nous devons également faire attention à ce que les opérandes ne soient pas nuls ainsi qu'au cas de dépassement possible par l'expression `INT_MIN / -1` (en cas de représentation en complément à deux).

```

1  else if (b < 0)
2  {
3      if (a < 0 && a < INT_MAX / b)
4          goto overflow;
5      else if ((-INT_MAX > INT_MIN && b < -1) && a > INT_MIN / b)
6          goto underflow;
7  }

```

Ce qui nous donne finalement la fonction suivante.

```

1  int safe_mul(int a, int b)
2  {
3      int err = 1;
4
5      if (b > 0)
6      {
7          if (a > 0 && a > INT_MAX / b)
8              goto overflow;
9          else if (a < 0 && a < INT_MIN / b)
10             goto underflow;
11     }
12     else if (b < 0)
13     {
14         if (a < 0 && a < INT_MAX / b)
15             goto overflow;
16         else if ((-INT_MAX > INT_MIN && b < -1) && a >
17                 INT_MIN / b)
18             goto underflow;
19     }
20     return a * b;

```



```

21 underflow:
22     err = -1;
23 overflow:
24     errno = ERANGE;
25     return err > 0 ? INT_MAX : INT_MIN;
26 }

```

V.2.3.6. La division

Rassurez-vous, la division est nettement moins pénible à vérifier. 🍊

En fait, il y a un seul cas problématique : si les deux opérandes sont `INT_MIN` et `-1` et que l'opposé du maximum est inférieur à la borne minimale.

```

1 int safe_div(int a, int b)
2 {
3     if (-INT_MAX > INT_MIN && b == -1 && a == INT_MIN)
4     {
5         errno = ERANGE;
6         return INT_MIN;
7     }
8
9     return a / b;
10 }

```

V.2.3.7. Le modulo

Enfin, pour le modulo, les mêmes règles que celles de la division s'appliquent.

```

1 int safe_mod(int a, int b)
2 {
3     if (-INT_MAX > INT_MIN && b == -1 && a == INT_MIN)
4     {
5         errno = ERANGE;
6         return INT_MIN;
7     }
8
9     return a % b;
10 }

```

V.2.4. Gérer les dépassements flottants

La tâche se complique un peu avec les flottants, puisqu'en plus de tester les limites supérieures (cas d'*overflow*), il faudra aussi s'intéresser à celles de la partie fractionnaire (cas d'*underflow*).

V.2.4.1. L'addition

Comme toujours, il faut prendre garde à ce que les codes de vérification ne provoquent pas eux-mêmes des dépassements. Pour implémenter l'opération d'addition, on commence donc par se ramener au cas plus simple où on connaît le signe des opérandes **a** et **b**.

- Si **a** et **b** sont de signes opposés (par exemple : **a** $\geq$ 0 et **b** $\geq$ 0, alors il ne peut pas se produire d'*overflow*.
- Si **a** et **b** sont de mêmes signes, alors il ne peut pas se produire d'*underflow*.

Dans le premier cas, tester l'*underflow* revient à tester si **a + b** est censé (dans l'arithmétique usuelle) être compris entre **-DBL_MIN** et **DBL_MIN**. Il faut cependant exclure 0 de cet intervalle, de manière à ce que **1.0 + (-1.0)**, par exemple, ne soit pas considéré comme un *underflow*. Si par exemple **a** $\geq$ 0 (et dans ce cas **b** $\leq$ 0), il suffit alors de tester si **-DBL_MIN - a < b** et **a < DBL_MIN - b**.

Dans le deuxième cas, il reste à vérifier l'absence d'*overflow*. Si par exemple **a** et **b** sont tous deux positifs, il s'agit de vérifier que **a** $\leq$ **DBL_MAX - b**.

```

1  /* Retourne a+b. */
2  /* En cas d'overflow, +-DBL_MAX est retourné (suivant le signe de
   a+b) */
3  /* En cas d'underflow, +- DBL_MIN est retourné */
4  /* errno est fixé en conséquence */
5  double safe_addf(double a, double b)
6  {
7      int positive_result = a > -b;
8      double ret_value = positive_result ? DBL_MAX : -DBL_MAX;
9
10     if (a == -b)
11         return 0;
12     else if ((a < 0) != (b < 0))
13     {
14         /* On se ramène au cas où a <= 0 et b >= 0 */
15         if (a <= 0)
16         {
17             double t = a;
18             a = b;
19             b = t;
20         }
21         if (-DBL_MIN - a < b && a < DBL_MIN - b)
22             goto underflow;
23     }
24     else
25     {
26         int negative = 0;
27
28         /* On se ramène au cas où a et b sont positifs
           */
29         if (a < 0)
30         {
31             a = -a;

```

```

32         b = -b;
33         negative = 1;
34     }
35
36     if (a > DBL_MAX - b)
37         goto overflow;
38
39     if (negative)
40     {
41         a = -a;
42         b = -b;
43     }
44 }
45
46 return a + b;
47
48 underflow:
49     ret_value = positive_result ? DBL_MIN :
50         -DBL_MIN;
51 overflow:
52     errno = ERANGE;
53     return ret_value;

```

Quelques tests :

```

1 static void test_addf(double a, double b)
2 {
3     errno = 0;
4     printf("%e + %e = %e", a, b, safe_addf(a, b));
5     printf(" (%s)\n", strerror(errno));
6 }
7
8 int main(void)
9 {
10     test_addf(1, 1);
11     test_addf(5, -6);
12     test_addf(DBL_MIN, -3./2*DBL_MIN); /* underflow */
13     test_addf(DBL_MAX, 1);
14     test_addf(DBL_MAX, -DBL_MAX);
15     test_addf(DBL_MAX, DBL_MAX); /* overflow */
16     test_addf(-DBL_MAX, -1./2*DBL_MAX); /* overflow */
17     return 0;
18 }

```

Résultat

```

1 1.000000e+00 + 1.000000e+00 = 2.000000e+00 (Success)
2 5.000000e+00 + -6.000000e+00 = -1.000000e+00 (Success)
3 2.225074e-308 + -3.337611e-308 = -2.225074e-308 (Numerical
   result out of range)
4 1.797693e+308 + 1.000000e+00 = 1.797693e+308 (Success)
5 1.797693e+308 + -1.797693e+308 = 0.000000e+00 (Success)
6 1.797693e+308 + 1.797693e+308 = 1.797693e+308 (Numerical
   result out of range)
7 -1.797693e+308 + -8.988466e+307 = -1.797693e+308 (Numerical
   result out of range)

```

On peut faire deux remarques sur la fonction `safe_addf` ci-dessus :

- `addf(DBL_MAX, 1)` peut ne pas être considéré comme un *overflow* du fait des pertes de précision.
- à l'inverse, si un *underflow* se déclare, cela ne veut pas forcément dire qu'il y a une erreur irrécupérable dans l'algorithme. Il est possible que les pertes de précision dans les représentations des flottants les causent elles-mêmes.



Pour être tout à fait rigoureux, il faut remarquer que des expressions telles que `-DBL_MIN - a` et `DBL_MAX - b` (avec `a >= 0` et `b >= 0`) peuvent théoriquement générer respectivement un *overflow* et un *underflow*. Cela ne peut cependant se produire que sur des implémentations marginales des flottants, puisque cela signifierait que la partie fractionnaire (la mantisse) est représentée sur plus de bits que la différence entre les valeurs extrémales de l'exposant. Nous avons donc omis ces vérifications pour des raisons de concision.

V.2.4.2. La soustraction

Pour notre plus grand bonheur, étant donné qu'il n'y a pas d'asymétrie entre les bornes, la soustraction revient à faire `safe_addf(a, -b)`. 🍊

V.2.4.3. La multiplication

Heureusement, la multiplication est un peu plus simple que l'addition ! Comme tout à l'heure, on va essayer de contrôler la valeur des arguments de manière à savoir dans quel cas on se situe potentiellement (*underflow* ou *overflow*).

Ici, ce n'est plus le signe de `a` et de `b` qui va nous intéresser : au contraire, on va même le gommer en utilisant la valeur absolue. Cependant, la position de `a` et de `b` par rapport à `+1` et `-1` est essentielle :

- si `fabs(b) <= 1`, `a * b` peut produire un *underflow* mais pas d'*overflow*.
- si `fabs(b) >= 1`, `a * b` peut produire un *overflow* mais pas d'*underflow*.



La fonction `fabs()` est définie dans l'en-tête `<math.h>` et, comme son nom l'indique, retourne la valeur absolue de son argument. N'oubliez pas à cet égard de préciser l'option `-lm` lors de la compilation.

Ainsi, dans le premier cas, on vérifiera si `fabs(a) < DBL_MIN / fabs(b)`. L'expression `DBL_MIN / fabs(b)` ne peut ni produire d'*underflow* (puisque `fabs(b) <= 1`) ni d'*overflow* (puisque `DBL_MIN <= fabs(b)`). De même, dans le deuxième cas, la condition pourra s'écrire `fabs(a) > DBL_MAX / fabs(b)`.

```

1 double safe_mulf(double a, double b)
2 {
3     int different_signs = (a < 0) != (b < 0);
4     double ret_value = different_signs ? -DBL_MAX : DBL_MAX;
5
6     if (fabs(b) < 1)
7     {
8         if (fabs(a) < DBL_MIN / fabs(b))
9             goto underflow;
10    }
11    else
12    {
13        if (fabs(a) > DBL_MAX / fabs(b))
14            goto overflow;
15    }
16
17    return a * b;
18
19 underflow:
20     ret_value = different_signs ? -DBL_MIN : DBL_MIN;
21 overflow:
22     errno = ERANGE;
23     return ret_value;
24 }
```

Quelques tests :

```

1 static void test_add(double a, double b)
2 {
3     errno = 0;
4     printf("%e + %e = %e", a, b, safe_addf(a, b));
5     printf(" (%s)\n", strerror(errno));
6 }
7
8 int main(void)
9 {
```

```

10     test_add(1, 1);
11     test_add(5, -6);
12     test_add(DBL_MIN, -3./2*DBL_MIN); /* underflow */
13     test_add(DBL_MAX, 1);
14     test_add(DBL_MAX, -DBL_MAX);
15     test_add(DBL_MAX, DBL_MAX); /* overflow */
16     test_add(-DBL_MAX, -1./2*DBL_MAX); /* overflow */
17     return 0;
18 }

```

Résultat

```

1 -1.000000e+00 * -1.000000e+00 = 1.000000e+00 (Success)
2 2.000000e+00 * 3.000000e+00 = 6.000000e+00 (Success)
3 1.797693e+308 * 2.000000e+00 = 1.797693e+308 (Numerical result
  out of range)
4 2.225074e-308 * 5.000000e-01 = 2.225074e-308 (Numerical result
  out of range)
5 -3.337611e-308 * 6.666667e-01 = -2.225074e-308 (Success)
6 -8.988466e+307 * 1.500000e+00 = -1.348270e+308 (Success)
7 -8.988466e+307 * 3.000000e+00 = -1.797693e+308 (Numerical
  result out of range)

```

V.2.4.4. La division

La division ressemble sensiblement à la multiplication. On traite cependant en plus le cas où le dénominateur est nul.

```

1 double safe_divf(double a, double b)
2 {
3     int different_signs = (a < 0) != (b < 0);
4     double ret_value = different_signs ? -DBL_MAX : DBL_MAX;
5
6     if (b == 0)
7     {
8         errno = EDOM;
9         return ret_value;
10    }
11    else if (fabs(b) < 1)
12    {
13        if (fabs(a) > DBL_MAX * fabs(b))
14            goto overflow;
15    }
16    else

```

```

17     {
18         if (fabs(a) < DBL_MIN * fabs(b))
19             goto underflow;
20     }
21
22     return a / b;
23
24 underflow:
25     ret_value = different_signs ? -DBL_MIN : DBL_MIN;
26 overflow:
27     errno = ERANGE;
28     return ret_value;
29 }

```

Conclusion

Ce chapitre aura été fastidieux, n'hésitez pas à vous poser après sa lecture et à tester les différents codes fournis afin de correctement appréhender les différentes limites.

En résumé

1. Les limites des différents types sont fournies par des macroconstantes définies dans les en-têtes `<limits.h>` et `<float.h>` ;
2. Les types flottants disposent de quatre limites et non de deux ;
3. Un dépassement de capacité se produit soit quand une opération produit un résultat hors des limites du type de son expression ou lors de la conversion d'une valeur vers un type qui ne peut représenter celle-ci ;
4. En cas de dépassement, le comportement est indéfini *sauf* pour les calculs impliquant des entiers non signés et pour les conversions d'un type entier vers un type entier non signé, auquel cas la valeur boucle.
5. La bibliothèque standard ne fournit aucun outil permettant de détecter et/ou de gérer ces dépassements.

Contenu masqué

Contenu masqué n°49

```

1 int safe_sub(int a, int b)
2 {
3     int err = 1;
4
5     if (b > 0 && a < INT_MIN + b)
6         goto underflow;
7     else if (b < 0 && a > INT_MAX + b)
8         goto overflow;

```

```
9
10     return a - b;
11 underflow:
12     err = -1;
13 overflow:
14     errno = ERANGE;
15     return err > 0 ? INT_MAX : INT_MIN;
16 }
```

[Retourner au texte.](#)

V.3. Manipulation des bits

Introduction

Nous avons vu dans le chapitre précédent la représentation des différents types et notamment celle des types entiers. Ce chapitre est en quelque sorte le prolongement de celui-ci puisque nous allons à présent voir comment manipuler directement les *bits* à l'aide des opérateurs de manipulation des *bits* et des champs de *bits*.

V.3.1. Les opérateurs de manipulation des bits

V.3.1.1. Présentation

Le langage C définit six opérateurs permettant de manipuler les *bits* :

- l'opérateur « et » : `&` ;
- l'opérateur « ou inclusif » : `|` ;
- l'opérateur « ou exclusif » : `^` ;
- l'opérateur de négation ou de complément : `~` ;
- l'opérateur de décalage à droite : `>` ;
- l'opérateur de décalage à gauche : `<`.



Veillez à ne pas confondre les opérateurs de manipulation des *bits* « et » (`&`) et « ou inclusif » (`|`) avec leurs homologues « et » (`&&`) et « ou » (`||`) logiques. Il s'agit d'opérateurs totalement différents au même titre que les opérateurs d'affectation (`=`) et d'égalité (`==`). De même, l'opérateur de manipulation des *bits* « et » (`&`) n'a pas de rapport avec l'opérateur d'adressage (`&`), ce dernier n'utilisant qu'un opérande.



Notez que tous ces opérateurs travaillent uniquement sur des *entiers*. Si néanmoins vous souhaitez modifier la représentation d'un type flottant (ce que nous vous déconseillons), vous pouvez accéder à ses *bits* à l'aide d'un pointeur sur `unsigned char` (revoyez [le chapitre sur l'allocation dynamique](#) [↗](#) si cela ne vous dit rien).

V.3.1.2. Les opérateurs « et », « ou inclusif » et « ou exclusif »

Chacun de ces trois opérateurs attend deux opérandes entiers et produit un nouvel entier en appliquant une table de vérité à chaque paire de *bits* formée à partir des *bits* des deux nombres de départs. Plus précisément :

- L'opérateur « et » (`&`) donnera 1 si les deux *bits* de la paire sont à 1 ;
- L'opérateur « ou inclusif » (`|`) donnera 1 si *au moins* un des deux *bits* de la paire est à 1 ;

— L'opérateur « ou exclusif » (^) donnera 1 si *un seul* des deux *bits* de la paire est à 1. Ceci est résumé dans le tableau ci-dessous, donnant le résultat des trois opérateurs pour chaque paire de *bits* possible.

Bit 1	Bit 2	Opérateur « et »	Opérateur « ou inclusif »	Opérateur « ou exclusif »
0	0	0	0	0
1	0	0	1	1
0	1	0	1	1
1	1	1	1	0

L'exemple ci-dessous utilise ces trois opérateurs. Comme vous le voyez, les *bits* des deux opérandes sont pris un à un pour former des paires et chacune d'elle se voit appliquer la table de vérité correspondante afin de produire un nouveau nombre entier.

```

1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     int a = 0x63; /* 0x63 == 99 == 0110 0011 */
7     int b = 0x2A; /* 0x2A == 42 == 0010 1010 */
8
9     /* 0110 0011 & 0010 1010 == 0010 0010 == 0x22 == 34 */
10    printf("%2X\n", a & b);
11    /* 0110 0011 | 0010 1010 == 0110 1011 == 0x6B == 107 */
12    printf("%2X\n", a | b);
13    /* 0110 0011 ^ 0010 1010 == 0100 1001 == 0x49 == 73 */
14    printf("%2X\n", a ^ b);
15    return 0;
16 }
```

Résultat

```

1 22
2 6B
3 49
```



Le chiffre « 2 » devant l'indicateur de conversion X demande à `printf()` de toujours afficher deux chiffres hexadécimaux. Si la valeur à afficher est inférieure à 10 (en hexadécimal), un zéro sera placé devant.

V.3.1.3. L'opérateur de négation ou de complément

L'opérateur de négation a un fonctionnement assez simple : il inverse les *bits* de son opérande (les uns deviennent des zéros et les zéros des uns).

```

1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      unsigned a = 0x7F; /* 0111 1111 */
8
9      /* ~0111 1111 == 1000 0000 */
10     printf("%2X\n", ~a);
11     return 0;
12 }
```

Résultat

1	FFFFFF80
---	----------



Notez que *tous* les *bits* ont été inversés, d'où le nombre élevé que nous obtenons puisque les *bits* de poids forts ont été mis à un. Ceci nous permet par ailleurs de constater que, sur notre machine, il y a visiblement quatre octets qui sont utilisés pour représenter la valeur d'un objet de type `unsigned int`.



N'oubliez pas que les représentations entières *signées* ont chacune une représentation qui est susceptible d'être invalide. Les opérateurs de manipulation des *bits* vous permettant de modifier directement la représentation, vous devez éviter d'obtenir ces dernières.

Ainsi, les exemples suivants sont susceptibles de produire des valeurs incorrectes (à supposer que la taille du type `int` soit de quatre octets sans *bits* de bourrages).

```

1  /* Invalide en cas de représentation en complément à deux ou signe
   et magnitude */
2  int a = ~0x7FFFFFFF;
3  /* Idem */
4  int b = 0x00000000 | 0x80000000;
5  /* Invalide en cas de représentation en complément à un */
```

```

6  int c = ~0;
7  /* Idem */
8  int d = 0x11110000 ^ 0x00001111;

```



Notez toutefois que les entiers *non signés*, eux, ne subissent pas ces restrictions.

V.3.1.4. Les opérateurs de décalage

Les opérateurs de décalage, comme leur nom l'indique, décalent la valeur des *bits* d'un objet d'une certaine quantité, soit vers la gauche (c'est-à-dire vers le *bit* de poids fort), soit vers la droite (autrement dit, vers le *bit* de poids faible). Ils attendent deux opérandes : le nombre dont les *bits* doivent être décalés et la grandeur du décalage.



Un décalage ne peut être négatif ni être supérieur *ou égal* au nombre de *bits* composant l'objet décalé. Ainsi, si le type `int` utilise 32 *bits* (sans *bits* de bourrage), le décalage ne peut être plus grand que 31.

V.3.1.4.1. L'opérateur de décalage à gauche

L'opérateur de décalage à gauche translate la valeur des *bits* vers le *bit* de poids fort. Les *bits* de poids faibles perdant leur valeur durant l'opération sont mis à zéro. Techniquement, $a < y$ revient à calculer la valeur de l'expression $a \times 2^y$.

```

1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      /* 0000 0001 << 2 == 0000 0100 */
8      int a = 1 << 2;
9      /* 0010 1010 << 2 == 1010 1000 */
10     int b = 42 << 2;
11
12     printf("a = %d, b = %d\n", a, b);
13     return 0;
14 }

```

Résultat

1	a = 4, b = 168
---	----------------



Le premier opérande ne peut être un nombre négatif.



L'opération de décalage à gauche revenant à effectuer une multiplication, celle-ci est soumise au risque de dépassement de capacité.

V.3.1.4.2. L'opérateur de décalage à droite

L'opérateur de décalage à droite translate la valeur des *bits* vers le *bit* de poids faible. Dans le cas où le premier opérande est un entier *non signé* ou un entier signé *positif*, les *bits* de poids forts perdant leur valeur durant l'opération sont mis à zéro. Si en revanche il s'agit d'un nombre signé *négatif*, les *bits* perdant leur valeur se voient mis à zéro ou un suivant la machine cible. Techniquement, $a >> y$ revient à calculer la valeur de l'expression $a / 2^y$.

```

1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      /* 0001 0000 >> 2 == 0000 0100 */
8      int a = 16 >> 2;
9      /* 0010 1010 >> 2 == 0000 1010 */
10     int b = 42 >> 2;
11
12     printf("a = %d, b = %d\n", a, b);
13     return 0;
14 }
```

Résultat

1	a = 4, b = 10
---	---------------

i

Dans le cas où un *bit* est translaté au-delà du *bit* de poids faible, il est tout simplement perdu.

V.3.1.5. Opérateurs combinés

Enfin, sachez que, comme pour les opérations arithmétiques usuelles, les opérateurs de manipulation des *bits* disposent d'opérateurs combinés réalisant une affectation et une opération.

Opé- ra- teur com- biné	Équi- valent à
-------------------------------------	----------------------

```

va
riable
=
va
riable
&
nombre
va
va
riable
riable
|=
va
nombre
va
riable
va
riable
|=
va
nombre
va
riable
^=
va
riable
nombre
va
nombre
riable
va
<<
riable
nombre
=
va
va
riable
riable
>>
<
nombre
va
riable
=
va
riable
>
nombre

```

V.3.2. Masques et champs de bits

V.3.2.1. Les masques

Une des utilisations fréquentes des opérateurs de manipulation des *bits* est l'emploi de **masques**. Un masque est un ensemble de *bits* appliqué à un second ensemble *de même taille* lors d'une opération de manipulation des *bits* (plus précisément, uniquement les opérations `&`, `|` et `^`) en vue soit de sélectionner un sous-ensemble, soit de modifier un sous-ensemble.

V.3.2.1.1. Modifier la valeur d'un bit

V.3.2.1.1.1. Mise à zéro Cette définition doit probablement vous paraître quelque peu abstraite, aussi, prenons un exemple.

```
1 unsigned short n;
```

Nous disposons d'une variable `n` de type `unsigned short` (que nous supposons composées de deux octets pour nos exemples) et souhaiterions mettre le *bit* de poids fort à zéro.

Une solution consiste à appliquer les opérateurs de manipulation des *bits* afin d'obtenir la valeur voulue. Étant donné que nous désirons mettre un *bit* à zéro, nous pouvons déjà abandonner l'opérateur `|` au vu de sa table de vérité. Également, l'opérateur `^` ne convient pas tout à fait puisqu'il inverserait la valeur du *bit* au lieu de la mettre à zéro. Il nous reste donc l'opérateur `&`.

Avec cet opérateur, il nous est possible d'utiliser une valeur qui nous donnera le bon résultat. Cette valeur, de même taille que celle de la variable `n`, est précisément un masque qui va « cacher », « masquer » une partie de la valeur.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     unsigned short n;
8
9     if (scanf("%hx", &n) != 1)
10    {
11        perror("scanf");
12        return EXIT_FAILURE;
13    }
14
15    printf("%X\n", n & 0x7FFF);
16    return 0;
17 }
```

Résultat

```
1 8FFF
2 FFF
3
4 7F
5 7F
```


Comme vous le voyez, l'opérateur `&` peut être utilisé pour sélectionner une partie de la valeur de `n` en mettant à un les *bits* que nous souhaitons garder (en l'occurrence tous sauf le *bit* de poids fort) et les autres à zéro.

V.3.2.1.1.2. Mise à un À l'inverse, les opérateurs de manipulation des *bits* peuvent être employés pour mettre un ou plusieurs *bits* à un. Dans ce cas, c'est l'opérateur `&` qui ne convient pas au vu de sa table de vérité.

Si nous reprenons notre exemple précédent et que nous souhaitons modifier la valeur de la variable `n` de sorte de mettre à un le *bit* de signe, nous pouvons procéder comme suit.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     unsigned short n;
8
9     if (scanf("%hx", &n) != 1)
10    {
11        perror("scanf");
12        return EXIT_FAILURE;
13    }
14
15    printf("%X\n", n | 0x8000);
16    return 0;
17 }
```

Résultat

1	FFF
2	8FFF
3	
4	7F
5	807F

Comme vous le voyez, l'opérateur `|` peut être utilisé de la même manière dans ce cas ci à l'aide d'un masque dont les *bits* qui doivent être mis à un sont... à un.

V.3.2.2. Les champs de bits

V.3.2.2.1. Mise en situation

Une autre utilisation des opérateurs de manipulation des *bits* est le compactage de données entières.

Imaginons que nous souhaitions stocker la date courante sous la forme de trois entiers : l'année, le mois et le jour. La première solution qui vous viendra à l'esprit sera probablement de recourir à une structure, par exemple comme celle ci-dessous, ce qui est un bon réflexe.

```
1 struct date {
2     unsigned char jour;
3     unsigned char mois;
4     unsigned short annee;
5 };
```

Toutefois, nous gaspillons finalement de la mémoire avec ce système. En effet, techniquement, nous aurions besoin de 12 *bits* pour stocker l'année (afin d'avoir un peu de marge jusque l'an 4095 🍊), 4 pour le mois et 5 pour le jour, ce qui nous fait un total de 21 *bits* contre 32 pour notre structure (à supposer que le type `short` fasse deux octets et le type `char` un octet), sans compter les multipliants de bourrage (revoyez le chapitre sur les structures si cela ne vous dit rien 🍊).

Ceci n'est pas gênant dans la plupart des cas, mais cela peut le devenir si la mémoire disponible vient à manquer ou si cette structure est amenée à être créée un grand nombre de fois.

Avec les opérateurs de manipulation des *bits*, il nous est possible de stocker ces trois champs dans un tableau de trois `unsigned char` afin d'économiser de la place.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 static void modifie_jour(unsigned char *date, unsigned jour)
6 {
7     /* Nous stockons le jour (cinq bits). */
8     date[0] |= jour;
9 }
10
11
12 static void modifie_mois(unsigned char *date, unsigned mois)
13 {
14     /* Nous ajoutons les trois premiers bits du mois après
15        ceux du jour. */
16     date[0] |= (mois & 0x07) << 5;
17     /* Puis le bit restant dans le second octet. */
18     date[1] |= (mois >> 3);
19 }
20
21 static void modifie_annee(unsigned char *date, unsigned annee)
22 {
23     /* Nous ajoutons sept bits de l'année après le dernier bit
24        du mois. */
```

```

24     date[1] |= (annee & 0x7F) << 1;
25     /* Et ensuite les cinq restants. */
26     date[2] |= (annee) >> 7;
27 }
28
29
30 static unsigned calcule_jour(unsigned char *date)
31 {
32     return date[0] & 0x1F;
33 }
34
35
36 static unsigned calcule_mois(unsigned char *date)
37 {
38     return (date[0] >> 5) | ((date[1] & 0x1) << 3);
39 }
40
41
42 static unsigned calcule_annee(unsigned char *date)
43 {
44     return (date[1] >> 1) | (date[2] << 7);
45 }
46
47
48 int
49 main(void)
50 {
51     unsigned char date[3] = { 0 }; /* Initialisation à zéro. */
52     unsigned jour, mois, annee;
53
54     printf("Entrez une date au format jj/mm/aaaa : ");
55
56     if (scanf("%u/%u/%u", &jour, &mois, &annee) != 3) {
57         perror("fscanf");
58         return EXIT_FAILURE;
59     }
60
61     modifie_jour(date, jour);
62     modifie_mois(date, mois);
63     modifie_annee(date, annee);
64     printf("Le %u/%u/%u\n", calcule_jour(date),
65           calcule_mois(date), calcule_annee(date));
66     return 0;
67 }

```

Résultat

```
1 Entrez une date au format jj/mm/aaaa : 31/12/2042
2 Le 31/12/2042
```

Cet exemple amène quelques explications. 🍊

Une fois les trois valeurs récupérées, il nous faut les compacter dans le tableau d'`unsigned char` :

1. Pour le jour, c'est assez simple, nous incorporons ses cinq *bits* à l'aide de l'opérateur `|` (les trois éléments du tableau étant à zéro au début, cela ne pose pas de problème).
2. Pour le mois, seuls trois *bits* étant encore disponibles, il va nous falloir répartir ceux-ci sur deux éléments du tableau. Tout d'abord, nous sélectionnons les trois premiers *bits* à l'aide du masque `0x07`, nous les décalons ensuite de cinq *bits* vers la gauche (afin de ne pas écraser les cinq *bits* du jour) et, enfin, nous les ajoutons à l'aide de l'opérateur `|`. Le dernier *bit* est lui stocké dans le second élément et est sélectionné à l'aide d'un décalage vers la droite afin d'éliminer les trois premiers *bits* (qui ont déjà été traité).
3. Pour l'année, nous utilisons la même technique que pour le mois : nous sélectionnons les sept premiers *bits* à l'aide du masque `0x7F`, les décalons d'un *bit* vers la gauche en vue de ne pas écraser le *bit* du mois et les intégrons avec l'opérateur `|`. Les cinq *bits* restants sont ensuite insérés en recourant préalablement à un décalage de sept *bit* vers la droite.

V.3.2.2.2. Présentation

Comme vous le voyez, si nous gagnons effectivement de la place en mémoire, nous y perdons en temps de calcul et, plus important, notre code est nettement plus complexe. C'est la raison pour laquelle cette méthode n'est employée que dans le cas de contraintes particulières.

Bien entendu, nous pourrions recourir à des fonctions ou à des macrofonctions pour simplifier la lecture du code, toutefois, nous ne ferions que reporter la difficulté de compréhension sur ces dernières. Heureusement, en vue de simplifier ce type d'écritures, le langage C met à notre disposition les **champs de *bits***.

Un champ de *bits* est un membre de type `_Bool`, `int` ou `unsigned int` dont la taille en *bits* est précisée. Cette taille *ne peut être* supérieure à la taille en *bits* du type utilisé. L'exemple ci-dessous définit une structure composée de trois champs de *bits*, `a`, `b` et `c` de respectivement un, deux et trois *bits*.

```
1 struct champ_de_bits
2 {
3     unsigned a : 1;
4     unsigned b : 2;
5     unsigned c : 3;
6 };
```

Ainsi, notre exemple précédent peut être réécrit comme ceci.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  struct date {
6      unsigned jour : 5;
7      unsigned mois : 4;
8      unsigned annee : 12;
9  };
10
11
12  int
13  main(void)
14  {
15      unsigned jour, mois, annee;
16
17      printf("Entrez une date au format jj/mm/aaaa : ");
18
19      if (scanf("%u/%u/%u", &jour, &mois, &annee) != 3) {
20          perror("fscanf");
21          return EXIT_FAILURE;
22      }
23
24      struct date date = { .jour = jour, .mois = mois, .annee =
25                          annee };
26
27      printf("Le %u/%u/%u\n", date.jour, date.mois, date.annee);
28      return 0;
29  }

```



Les champs de *bits* ne disposent pas d'une adresse et ne peuvent en conséquence se voir appliquer l'opérateur d'adressage. Par ailleurs, nous vous conseillons de ne les employer que pour stocker des nombres non signés, le support des nombres signés n'étant pas garanti par la norme.

Si vous avez poussé la curiosité jusqu'à vérifier la taille de cette structure, il y a de fortes chances pour que celle-ci équivaille à celle du type `int`. En fait, il s'agit de la méthode la plus courante pour conserver les champs de *bits* : ils sont stockés dans des suites de `int`. Dès lors, si vous souhaitez économiser de la place, faites en sorte que les données à stocker coïncident le plus possible avec la taille d'un ou plusieurs objets de type `int`.



Gardez à l'esprit que le compactage de données et les champs de *bits* répondent à des besoins particuliers et complexifient votre code. Dès lors, ne les utilisez que lorsque cela semble réellement se justifier.

V.3.3. Les drapeaux

Une autre utilisation régulière des opérateurs de manipulation des *bits* consiste en la gestion des **drapeaux**. Un drapeau correspond en fait à un *bit* qui est soit « levé », soit « baissé » dans l'objectif d'indiquer si une situation est vraie ou fausse.

Supposons que nous souhaitions fournir à une fonction un nombre et plusieurs de ses propriétés, par exemple s'il est pair, s'il s'agit d'une puissance de deux et s'il est premier. Nous pourrions bien entendu lui fournir quatre paramètres, mais cela fait finalement beaucoup pour simplement fournir un nombre et, foncièrement, trois *bits*.

À la place, il nous est possible d'employer un entier dont trois *bits* seront utilisés pour représenter chaque condition. Par exemple, le premier indiquera si le nombre est pair, le second s'il s'agit d'une puissance de deux et le troisième s'il est premier.

```

1 void traitement(int nombre, unsigned char drapeaux)
2 {
3     if (drapeaux & 0x01) /* Si le nombre est pair */
4     {
5         /* ... */
6     }
7     if (drapeaux & 0x02) /* Si le nombre est une puissance de
8         deux */
9     {
10        /*... */
11    }
12    if (drapeaux & 0x04) /* Si le nombre est premier */
13    {
14        /*... */
15    }
16 }
17
18 int main(void)
19 {
20     int nombre = 2;
21     unsigned char drapeaux = 0x01 | 0x02; /* 0000 0011 */
22
23     traitement(nombre, drapeaux);
24     nombre = 17;
25     drapeaux = 0x04; /* 0000 0100 */
26     traitement(nombre, drapeaux);
27     return 0;
28 }

```

Comme vous le voyez, nous utilisons l'opérateur `|` pour combiner plusieurs drapeaux et l'opérateur `&` pour déterminer si un drapeau est levé ou non.



Notez que, chaque drapeau représentant un *bit*, ceux-ci correspondent toujours à des puissances de deux.

Voilà qui est plus efficace, mais en somme assez peu lisible... En effet, il serait bon de préciser dans notre code à quoi correspond chaque drapeau. Pour ce faire, nous pouvons recourir au préprocesseur afin de clarifier un peu tout cela.

```

1 #define PAIR                (1 << 0)
2 #define PUISSANCE          (1 << 1)
3 #define PREMIER             (1 << 2)
4
5
6 void traitement(int nombre, unsigned char drapeaux)
7 {
8     if (drapeaux & PAIR) /* Si le nombre est pair */
9     {
10         /* ... */
11     }
12     if (drapeaux & PUISSANCE) /* Si le nombre est une
13         puissance de deux */
14     {
15         /*... */
16     }
17     if (drapeaux & PREMIER) /* Si le nombre est premier */
18     {
19         /*... */
20     }
21 }
22
23 int main(void)
24 {
25     int nombre = 2;
26     unsigned char drapeaux = PAIR | PUISSANCE; /* 0000 0011 */
27
28     traitement(nombre, drapeaux);
29     nombre = 17;
30     drapeaux = PREMIER; /* 0000 0100 */
31     traitement(nombre, drapeaux);
32     return 0;
33 }
```

Voici qui est mieux. 🍊

Pour terminer, remarquez qu'il s'agit d'un bon cas d'utilisation des champs de *bits*, chacun d'entre eux représentant alors un drapeau.

```

1 struct propriete
2 {
3     unsigned pair : 1;
4     unsigned puissance : 1;
5     unsigned premier : 1;
6 };
7
8
9 void traitement(int nombre, struct propriete prop)
10 {
11     if (prop.pair) /* Si le nombre est pair */
12     {
13         /* ... */
14     }
15     if (prop.puissance) /* Si le nombre est une puissance de
16         deux */
17     {
18         /*... */
19     }
20     if (prop.premier) /* Si le nombre est premier */
21     {
22         /*... */
23     }
24 }
25
26 int main(void)
27 {
28     int nombre = 2;
29     struct propriete prop = { .pair = 1, .puissance = 1 };
30
31     traitement(nombre, prop);
32     return 0;
33 }

```

V.3.4. Exercices

V.3.4.1. Obtenir la valeur maximale d'un type non signé

Maintenant que nous connaissons la représentation des nombres non signés ainsi que les opérateurs de manipulation des *bits*, vous devriez pouvoir trouver comment obtenir la plus grande valeur représentable par le type `unsigned int`.

V.3.4.1.1. Indice

👁 Contenu masqué n°50

V.3.4.1.2. Solution

👁 Contenu masqué n°51

V.3.4.2. Afficher la représentation en base deux d'un entier

Vous le savez, il n'existe pas de format de la fonction `printf()` qui permet d'afficher la représentation binaire d'un nombre. Pourtant, cela pourrait s'avérer bien pratique dans certains cas, même si la représentation hexadécimale est disponible.

Dans ce second exercice, votre tâche sera de réaliser une fonction capable d'afficher la représentation binaire d'un `unsigned int` *en gros-boutiste*.

V.3.4.2.1. Indice

👁 Contenu masqué n°52

V.3.4.2.2. Solution

👁 Contenu masqué n°53

V.3.4.3. Déterminer si un nombre est une puissance de deux

Vous le savez : les puissances de deux ont la particularité de n'avoir qu'un seul bit à un, tous les autres étant à zéro. Toutefois, elles ont une autre propriété : si l'on soustrait un à une puissance de deux n , tous les *bits* précédents celui de la puissance seront mis à un (par exemple $0000\ 1000 - 1 == 0000\ 0111$). En particulier, on remarque que n et $n - 1$ n'ont aucun bit à 1 en commun. Réciproquement, si n n'est pas une puissance de 2, alors le bit à 1 le plus fort est aussi à 1 dans $n - 1$. par exemple $0000\ 1010 - 1 == 0000\ 1001$).

Sachant cela, il nous est possible de créer une fonction très simple déterminant si un nombre est une puissance de 2 ou non.

👁 Contenu masqué n°54

Conclusion

En résumé

1. Le C fournit six opérateurs de manipulation des *bits* ;
2. Ces derniers travaillant directement sur la représentation des entiers, il est *impératif* d'éviter d'obtenir des représentations potentiellement invalide dans le cas des entiers signés ;
3. L'utilisation de masques permet de sélectionner ou modifier une portion d'un nombre entier ;
4. Les champs de *bits* permettent de stocker des entiers de taille arbitraire, mais doivent *toujours* avoir une taille inférieure à celle du type `int`. Par ailleurs, ils n'ont pas d'adresse et ne supportent pas forcément le stockage d'entiers signés ;
5. Les drapeaux constituent une solution élégante pour stocker des états binaires (« vrai » ou « faux »).

Contenu masqué

Contenu masqué n°50

Rappelez-vous : dans la représentation des entiers non signés, chaque *bit* représente une puissance de deux.

[Retourner au texte.](#)

Contenu masqué n°51

```
1 #include <stdio.h>
2
3
4 int main(void)
5 {
6     printf("%u\n", ~0U);
7     return 0;
8 }
```



Notez bien que nous avons utilisé le suffixe `U` afin que le type de la constante `0` soit `unsigned int` et non `int` (n'hésitez pas à revoir le chapitre relatif aux opérations mathématiques si cela ne vous dit rien).



Cette technique n'est valable que pour les entiers *non signés*, la représentation où tous les *bits* sont à un étant potentiellement invalide dans le cas des entiers signés (représentation en complément à un).

[Retourner au texte.](#)

Contenu masqué n°52

Pour afficher la représentation gros-boutiste, il va vous falloir commencer par afficher le *bit* de poids de fort suivit des autres. Pour ce faire, vous allez avoir besoin d'un masque dont seul ce *bit* sera à un. Pour ce faire, vous pouvez vous aider de l'exercice précédent. [Retourner au texte.](#)

Contenu masqué n°53

```

1  #include <stdio.h>
2
3
4  void affiche_bin(unsigned n)
5  {
6      unsigned mask = ~(~0U >> 1);
7      unsigned i = 0;
8
9      while (mask > 0)
10     {
11         if (i != 0 && i % 4 == 0)
12             putchar(' ');
13
14         putchar((n & mask) ? '1' : '0');
15         mask >>= 1;
16         ++i;
17     }
18
19     putchar('\n');
20 }
21
22
23 int main(void)
24 {
25     affiche_bin(1);
26     affiche_bin(42);
27     return 0;
28 }

```

Résultat

1	0000 0000 0000 0000 0000 0000 0000 0001
2	0000 0000 0000 0000 0000 0000 0010 1010

L'expression `~(~0U > 1)` nous permet d'obtenir un masque où seul le *bit* de poids fort est à un. Nous pouvons ensuite l'employer successivement en décalant le *bit* à un de sorte d'obtenir la représentation binaire d'un entier *non signé*.



À nouveau, faites bien attention que ceci n'est valable que pour les entiers *non signés*, une représentation dont tous les *bits* sont à un ou dont seul le *bit* de poids fort est à un étant possiblement incorrecte dans le cas des entiers signés.

[Retourner au texte.](#)

Contenu masqué n°54

```
1 #include <stdio.h>
2
3
4 int puissance_de_deux(unsigned int n)
5 {
6     return n != 0 && (n & (n - 1)) == 0;
7 }
8
9
10 int main(void)
11 {
12     if (puissance_de_deux(256))
13         printf("256 est une puissance de deux\n");
14     else
15         printf("256 n'est pas une puissance de deux\n");
16
17     if (puissance_de_deux(48))
18         printf("48 est une puissance de deux\n");
19     else
20         printf("48 n'est pas une puissance de deux\n");
21
22     return 0;
23 }
```

Résultat

```
1 256 est une puissance de deux
2 48 n'est pas une puissance de deux
```

[Retourner au texte.](#)

V.4. Internationalisation et localisation

Introduction

Cela est resté finalement assez discret jusqu'ici, mais en y regardant de plus près, les programmes que nous avons réalisés sont en fait destinés à un environnement anglophone. En effet, prenez par exemple les entrées : si nous souhaitons fournir un nombre flottant à notre programme, nous devons utiliser le point comme séparateur entre la partie entière et décimale. Or, dans certains pays, on pourrait vouloir utiliser la virgule à la place. Cela nous paraît moins étrange étant donné que les constantes flottantes sont écrites de cette manière en C, mais il n'en va pas de même pour nos utilisateurs.

Ce qu'il faudrait finalement, c'est que nos programmes puissent s'adapter aux usages, coutumes et langues de notre utilisateur et c'est que nous allons voir (partiellement) dans ce chapitre.



V.4.1. Définitions

Avant toute chose, il nous est nécessaire de définir deux concepts afin de bien cerner de quoi nous allons parler.

V.4.1.1. L'internationalisation

L'**internationalisation** (parfois abrégée « i18n ») est un procédé par lequel un programme est rendu capable de s'adapter aux préférences linguistiques et régionales d'un utilisateur.

V.4.1.2. La localisation

La **localisation** (parfois abrégée « l10n ») est une opération par laquelle un programme internationalisé se voit fournir les informations nécessaires pour s'adapter aux préférences linguistiques et régionales d'un utilisateur.

V.4.2. La fonction `setlocale`

De manière générale, les programmes que nous avons conçus jusqu'ici étaient déjà partiellement internationalisés, car la bibliothèque standard du langage C l'est dans une certaine mesure. Toutefois, nous n'avons jamais recouru à un processus de localisation pour que ceux-ci s'adaptent à nos usages. Nous vous le donnons en mille : la localisation en C s'effectue à l'aide de... la fonction `setlocale()`.

```
1 char *setlocale(int categorie, char *localisation);
```

Cette fonction attend deux arguments : une catégorie et la localisation qui doit être employée pour cette catégorie. Elle retourne la localisation demandée si elle a pu être appliquée, un pointeur nul sinon.

V.4.2.0.1. Les catégories

La bibliothèque standard du C divise la localisation en plusieurs catégories, plus précisément cinq :

1. La catégorie `LC_COLLATE` qui modifie le comportement des fonctions `strcoll()` et `strxfrm()` ;
2. La catégorie `LC_CTYPE` qui adapte le comportement des fonctions de traduction entre chaîne de caractères et chaînes de caractères larges (nous y viendrons bientôt), ainsi que les fonctions de l'en-tête `<ctype.h>` ;
3. La catégorie `LC_MONETARY` qui influence le comportement de la fonction `localeconv()` ;
4. La catégorie `LC_NUMERIC` qui altère le comportement des fonctions `*printf()` et `*scanf()` ainsi que des fonctions de conversions de chaînes de caractères en ce qui concerne les nombres flottants ;
5. La catégorie `LC_TIME` qui change le comportement de la fonction `strftime()`.

Enfin, la catégorie `LC_ALL` (qui n'en est pas vraiment une) représente toutes les catégories en même temps.



Nous ne nous attarderons que sur les catégories `LC_NUMERIC` et `LC_TIME` dans la suite de ce chapitre.

V.4.2.0.2. Les localisations

La bibliothèque standard prévoit deux localisations possibles :

1. La localisation `"C"` qui correspond à celle par défaut. Celle-ci utilise les usages anglophones ;
2. La localisation `""` (une chaîne vide) qui correspond à celle utilisée par votre système.

Il est également possible de fournir un pointeur nul comme localisation, auquel cas la localisation actuelle est retournée.

```
1 #include <locale.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int main(void)
7 {
8     char *s;
9
10    s = setlocale(LC_ALL, NULL);
11    puts(s);
```

```

12
13     if (setlocale(LC_ALL, "") == NULL)
14     {
15         perror("setlocale");
16         return EXIT_FAILURE;
17     }
18
19     s = setlocale(LC_ALL, NULL);
20     puts(s);
21     return 0;
22 }

```

Résultat

```

1 C
2 fr_BE.UTF-8

```

Comme vous le voyez, la localisation de départ est bien `C`.

i

La forme que prend la localisation dépend de votre système. Sous unixoïdes et dans notre exemple, elle prend la forme de la langue en minuscule (au format [ISO 639](#) [↗](#)) suivie d'un tiret bas et du pays en majuscule (au format [ISO 3166-1](#) [↗](#)) et, éventuellement, terminée par un point et par l'encodage utilisé.

V.4.3. La catégorie `LC_NUMERIC`

La catégorie `LC_NUMERIC` permet de modifier le comportement des fonctions `scanf()` et `printf()` afin qu'elles adaptent leur gestion et leur affichage des nombres flottants.

i

La catégorie `LC_NUMERIC` affecte également les fonctions `atoi()`, `atol()`, `atoll()`, `strtol()`, `strtoll()`, `strtoul()`, `strtoull()`, `strtoimax()`, `strtoumax()`, `atof()`, `strtof()`, `strtod()` et `strtold()` qui forment une suite de fonctions dédiées à la conversion de chaînes de caractères vers des nombres. Toutefois, nous ne nous étendrons pas sur ces dernières.

```

1 #include <locale.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5

```

```

6 int main(void)
7 {
8     double f;
9
10    if (setlocale(LC_NUMERIC, "") == NULL)
11    {
12        perror("setlocale");
13        return EXIT_FAILURE;
14    }
15
16    printf("Veuillez entrer un nombre flottant : ");
17
18    if (scanf("%lf", &f) != 1)
19    {
20        perror("scanf");
21        return EXIT_FAILURE;
22    }
23
24    printf("Vous avez entré : %f.\n", f);
25    return 0;
26 }

```

Résultat (localisation francophone)

```

1  Veuillez entrer un nombre flottant : 45,5
2  Vous avez entré : 45,500000.
3
4  Veuillez entrer un nombre flottant : 45.5
5  Vous avez entré : 45,000000.

```

Comme vous le voyez, avec une locale francophone et après l'appel à `setlocale()`, seule la virgule est considérée comme séparateur de la partie entière et de la partie décimale.

V.4.4. La catégorie LC_TIME

La catégorie `LC_TIME` modifie le comportement de la fonction `strftime()`.

```

1 size_t strftime(char *chaine, size_t taille, char *format, struct
   tm *date);

```

Cette fonction, déclarée dans l'en-tête `<time.h>`, écrit dans la chaîne de caractères `chaine` différents éléments décrivant la date `date` en suivant la chaîne de format `format` (à l'image de la fonction `snprintf()`). S'il y a assez de place dans la chaîne `chaine` pour écrire l'entièreté des données, la fonction retourne le nombre de caractères écrits, sinon elle retourne zéro.

Structure `tm`

La date doit être sous la forme d'une structure `tm` (déclarée dans l'en-tête `<time.h>`), structure qui peut être obtenue via la fonction `localtime()`.

```
1 struct tm *localtime(time_t *date);
```

Cette fonction attend simplement l'adresse d'un objet de type `time_t` et retourne l'adresse d'une structure `tm`.

La fonction `strftime()` supporte un grand nombre d'indicateurs de conversion pour construire la chaîne de format. En voici une liste non exhaustive.

Indicateur de conversion	Signification
A	Nom du jour de la semaine
B	Nom du mois de l'année
d	Jour du mois
Y	Année sur quatre chiffres

```
1 #include <locale.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <time.h>
5
6
7 int
8 main(void)
9 {
10     char buf[255];
11     time_t t;
12
13     if (setlocale(LC_TIME, "") == NULL)
14     {
15         perror("setlocale");
16         return EXIT_FAILURE;
17     }
18     if (time(&t) == (time_t)-1)
19     {
20         perror("time");
21         return EXIT_FAILURE;
22     }
23
24     struct tm *tm = localtime(&t);
25
```

```
26     if (tm == NULL)
27     {
28         perror("localtime");
29         return EXIT_FAILURE;
30     }
31     if (!strftime(buf, sizeof buf, "%A %d %B %Y", tm))
32     {
33         perror("strftime");
34         return EXIT_FAILURE;
35     }
36
37     printf("%s\n", buf);
38     return 0;
39 }
```

Résultat

1	Wednesday 07 November 2018
2	mercredi 07 novembre 2018

À nouveau, comme vous pouvez le voir, suivant la locale utilisée la chaîne produite n'est pas la même.

Conclusion

En résumé

1. L'internationalisation permet de rendre un programme sensible aux préférences régionales d'un utilisateur ;
2. La localisation permet à un programme d'appliquer les préférences régionales d'un utilisateur ;
3. La fonction `setlocale()` permet de modifier la localisation de certaines fonctions de la bibliothèque standard ;
4. La localisation par défaut (notée `C`) correspond aux usages anglophones.

V.5. La représentation des chaînes de caractères

Introduction

Nous nous sommes attardés précédemment sur la représentation des types entiers et flottants. Dans ce chapitre, nous allons nous intéresser à la représentation des chaînes de caractères. Nous l'avons vu, une chaîne de caractères est un tableau de `char` fini par un caractère nul. Toutefois, il reste à savoir comment ces caractères sont stockés au sein de ces tableaux de `char`, ce qui est l'objet de ce chapitre. 🍌

V.5.1. Les séquences d'échappement

Avant de nous pencher sur la représentation des chaînes de caractères, un petit aparté sur les séquences d'échappement s'impose. Vous avez vu tout au long de ce tutoriel la séquence `\0`, utilisée pour représenter le caractère nul terminant une chaîne de caractères, sans qu'elle ne vous soit véritablement expliquée.

Cette séquence est en fait tout simplement remplacée par la valeur *octale* indiquée après le symbole `\`, ici zéro. Ainsi les trois définitions ci-dessous sont équivalentes.

```
1 char s1[] = { 0, 0 };
2 char s2[] = { '\0', '\0' };
3 char s3[] = "\0"; /* N'oubliez pas qu'un '\0' est ajouté
    automatiquement s'agissant d'une chaîne de caractères */
```

Ces séquences peuvent toutefois être utilisées pour produire n'importe quelle valeur, du moment qu'elle ne dépasse pas la capacité du type `char`. Ainsi, la séquence `\42` a pour valeur 42 (en octal, soit 34 en décimal).

L'octal n'étant cependant pas la base la plus pratique à utiliser, il est également possible d'employer la base hexadécimale en faisant suivre le symbole `\` de la lettre « x ». Ainsi, la séquence `\x0` est équivalente à la séquence `\0`.

V.5.2. Les tables de correspondances



Cette section ne constitue qu'une introduction posant les bases nécessaires pour la suite. Si vous souhaitez en apprendre davantage à ce sujet, nous vous invitons à lire le [cours de Maëlan sur les encodages](#) [↗](#) sur lequel nous nous sommes appuyés pour écrire cette section.

V.5.2.1. La table ASCII

Vous le savez, un ordinateur ne manipule jamais que des nombres et, de ce fait, toute donnée non numérique à la base doit être numérisée pour qu'il puisse la traiter. Les caractères ne font dès lors pas exception à cette règle et il a donc été nécessaire de trouver une solution pour transformer les caractères en nombres.

La solution retenue dans leur cas a été d'utiliser des **tables de correspondance** (aussi appelées *code page* en anglais) entre nombres et caractères. Chaque caractère se voit alors attribuer un nombre entier qui est ensuite stocké dans un `char`.



Vocabulaire

Le terme de « **jeu de caractères** » (*character set* en anglais, ou *charset* en abrégé) est également souvent employé. Ce dernier désigne tout simplement un ensemble de caractères. Ainsi, l'ASCII est également un jeu de caractères qui comprend les caractères composant sa table de correspondance.

Une des tables les plus connues est la [table ASCII](#) [↗](#). Cette table sert quasi toujours de base à toutes les autres tables existantes, ce que vous pouvez d'ailleurs vérifier aisément.

```

1  #include <stdio.h>
2
3
4  int
5  main(void)
6  {
7      char ascii[] = "ASCII";
8
9      for (unsigned i = 0; ascii[i] != '\0'; ++i)
10         printf("%c' = %d\n", ascii[i], ascii[i]);
11
12     return 0;
13 }
```

Résultat

1	'A' = 65
2	'S' = 83
3	'C' = 67
4	'I' = 73
5	'I' = 73

Si vous jetez un coup d'œil à la table vous verrez qu'au caractère **A** correspond bien le nombre 65 (en décimal), au caractère **S** le nombre 83, au caractère **C** le nombre 67 et au caractère **I** le nombre 73. Le compilateur a donc remplacé pour nous chaque caractère par le nombre entier correspondant dans la table ASCII et a ensuite stocké chacun de ces nombres dans un **char** de la chaîne de caractères **ascii**.

Toutefois, si vous regardez plus attentivement la table ASCII, vous remarquerez qu'il n'y a pas beaucoup de caractères. En fait, il y a de quoi écrire en anglais et... en latin. 🍊

La raison est historique : l'informatique s'étant principalement développée aux États-Unis, la table choisie ne nécessitait que de quoi écrire l'anglais (l'acronyme ASCII signifie d'ailleurs *American Standard Code for Information Interchange*). Par ailleurs, la capacité des ordinateurs de l'époque étant limitée, il était le bienvenu que tous ces nombres puissent tenir sur un seul multiplet (il n'était pas rare à l'époque d'avoir des machines dont les multiplets comptaient 7 *bits*).

V.5.2.2. La multiplication des tables

L'informatique s'exportant, il devenait nécessaire de pouvoir employer d'autres caractères. C'est ainsi que d'autres tables ont commencé à faire leur apparition, quasiment toutes basées sur la table ASCII et l'étendant comme le [latin-1 ou ISO 8859-1](#) pour les pays francophones et plus généralement « latins », l'[ISO 8859-5](#) pour l'alphabet cyrillique ou l'[ISO 8859-6](#) pour l'alphabet arabe.

Ainsi, il devenait possible d'employer d'autres caractères que ceux de la table ASCII sans modifier la représentation des chaînes de caractères. En effet, en créant grosso modo une table par alphabet, il était possible de maintenir la taille de ces tables dans les limites du type **char**.

Cependant, cette multiplicité de tables de correspondance amène deux problèmes :

1. Il est impossible de mélanger des caractères de différents alphabets puisqu'une seule table de correspondance peut être utilisée.
2. L'échange de données peut vite devenir problématique si le destinataire et la destinataire n'utilisent pas la même table de correspondance.

Par ailleurs, si cette solution fonctionne pour la plupart des alphabets, ce n'est par exemple pas le cas pour les langues asiatiques qui comportent un grand nombre de caractères.

V.5.2.3. La table Unicode

Afin de résoudre ces problèmes, il a été décidé de construire une table unique, universelle qui contiendrait l'ensemble des alphabets : la table [Unicode](#). Ainsi, finis les problèmes de dissonance entre tables : à chaque caractère correspond un seul et unique nombre. Par ailleurs,

comme *tous* les caractères sont présents dans cette table, il devient possible d'employer plusieurs alphabets. Il s'agit aujourd'hui de la table la plus utilisée.

Sauf qu'avec ses 137.374 caractères (à l'heure où ces lignes sont écrites), il est impossible de représenter les chaînes de caractères simplement en assignant la valeur entière correspondant à chaque caractère dans un `char`. Il a donc fallu changer de tactique avec l'Unicode en introduisant des méthodes pour représenter ces valeurs sur plusieurs `char` au lieu d'un seul. Ces représentations sont appelées des **encodages**.

Il existe trois encodages majeures pour l'Unicode : l'UTF-32, l'UTF-16 et l'UTF-8.

V.5.2.3.1. L'UTF-32

L'UTF-32 stocke la valeur correspondant à chaque caractère sur 4 `char` en utilisant la même représentation que les entiers non signés. Autrement dit, les 4 `char` sont employés comme s'ils constituaient les 4 multipliants d'un entier non signé. Ainsi, le caractère `é` qui a pour valeur 233 (soit E9 en hexadécimal) donnera la suite `\xE9\x0\x0\x0` (ou l'inverse suivant le boutisme utilisé) en UTF-32.

Oui, vous avez bien lu, vu qu'il emploie la représentation des entiers non signés, cet encodage induit l'ajout de pas mal de `char` nuls. Or, ces derniers étant utilisés en C pour représenter la fin d'une chaîne de caractères, il est assez peu adapté... De plus, cette représentation implique de devoir gérer les questions de boutisme.

V.5.2.3.2. L'UTF-16

L'UTF-16 est fort semblable à l'UTF-32 mis à part qu'il stocke la valeur correspondant à chaque caractère sur 2 `char` au lieu de 4. Avec une différence notable toutefois : les valeurs supérieures à 65535 (qui sont donc trop grandes pour être stockées sur deux `char`) sont, elles, stockées sur des paires de 2 `char`.

Toutefois, dans notre cas, l'UTF-16 pose les mêmes problèmes que l'UTF-32 : il induit la présence de `char` nuls au sein des chaînes de caractères et nécessite de s'occuper des questions de boutisme.

V.5.2.3.3. L'UTF-8

L'UTF-8 utilise un nombre variable de `char` pour stocker la valeur d'un caractère. Plus précisément, les caractères ASCII sont stockés sur un `char`, comme si la table ASCII était utilisée, mais tous les autres caractères sont stockés sur des séquences de plusieurs `char`.

À la différence de l'UTF-32 ou de l'UTF-16, cet encodage ne pose pas la question du boutisme (s'agissant de séquences d'un ou plusieurs `char`) et n'induit pas l'ajout de caractères nuls (la méthode utilisée pour traduire la valeur d'un caractère en une suite de `char` a été pensée à cet effet).

Par exemple, en UTF-8, le caractère `é` sera traduit par la suite de `char` `\xc3\xa9`. Vous pouvez vous en convaincre en exécutant le code suivant.

```
1 #include <stdio.h>
2
3
4 int
```

```

5 main(void)
6 {
7     char utf8[] = u8"é";
8
9     for (unsigned i = 0; utf8[i] != '\0'; ++i)
10         printf("%02x\n", (unsigned char)utf8[i]);
11
12     return 0;
13 }

```

Résultat

1	c3
2	a9



Notez que nous avons fait précéder la chaîne de caractères "é" de la suite `u8` afin que celle-ci soit encodée en UTF-8.

Comme vous le voyez, notre chaîne est bien composée de deux `char` ayant pour valeur `\xc3` et `\xa9`. Toutefois, si l'UTF-8 permet de stocker n'importe quel caractère Unicode au sein d'une chaîne de caractères, il amène un autre problème : *il n'y a plus de correspondance systématique entre un `char` et un caractère.*

En effet, si vous prenez l'exemple présenté juste au-dessus, nous avons une chaîne composée d'un seul caractère, é, mais dans les faits composée de deux `char` valant `\xc3` et `\xa9`. Or, ni `\xc3`, ni `\xa9` ne représentent un caractère (rappelez-vous : seuls les caractères ASCII tiennent sur un `char` en UTF-8), mais uniquement leur succession.

Ceci amène plusieurs soucis, notamment au niveau de la manipulation des chaînes de caractères, puisque les fonctions de la bibliothèque standard partent du principe qu'à un `char` correspond un caractère (pensez à `strlen()`, `strchr()` ou `strpbrk()`).

V.5.3. Des chaînes, des encodages

Bien, nous savons à présent comment les chaînes de caractère sont représentées. Toutefois, il reste une question en suspend : vu qu'il existe pléthore de tables de correspondance, quelle est celle qui est utilisée en C ? Autrement dit : quelle table est utilisée en C pour représenter les chaînes de caractères ? Eh bien, cela dépend de *quelles* chaînes de caractères vous parlez... 🍊

Pour faire bref :

1. La table de correspondance et l'encodage des chaînes littérales dépendent de votre compilateur ;
2. La table de correspondance et l'encodage des chaînes reçues depuis le terminal dépendent de votre système d'exploitation ;

3. La table de correspondance et l'encodage des fichiers dépendent de ceux utilisés lors de leur écriture (typiquement celui utilisé par un éditeur de texte).

Et le plus amusant, c'est qu'ils peuvent *tous être différents* ! 🍌

L'exemple suivant illustre tout l'enfer qui peut se cacher derrière cette possibilité.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4
5  static void chomp(char *s)
6  {
7      while (*s != '\n' && *s != '\0')
8          ++s;
9
10     *s = '\0';
11 }
12
13
14 int
15 main(void)
16 {
17     char ligne[255];
18
19     printf("Cette ligne est encodée en UTF-8\n");
20
21     if (fgets(ligne, sizeof ligne, stdin) == NULL)
22     {
23         perror("scanf");
24         return EXIT_FAILURE;
25     }
26
27     chomp(ligne);
28     printf("Cette chaîne en ISO 8859-1 : %s\n", ligne);
29     FILE *fp = fopen("fichier.txt", "r");
30
31     if (fp == NULL)
32     {
33         perror("fopen");
34         return EXIT_FAILURE;
35     }
36     if (fgets(ligne, sizeof ligne, fp) == NULL)
37     {
38         perror("fscanf");
39         return EXIT_FAILURE;
40     }
41
42     chomp(ligne);
43     printf("Et celle-là en IBM 850 : %s\n", ligne);
44     fclose(fp);
```



```

45     return 0;
46 }
```

Résultat

```

1 Cette ligne est encodÃ©e en UTF-8
2 L'ISO 8859-1 c'est tellement plus élégant !
3 Cette chaîne en ISO 8859-1 : L'ISO 8859-1 c'est tellement
  plus élégant !
4 Et celle-là en IBM 850 : Moi aussi je suis lgant !
```

Cet exemple illustre l’affichage d’un programme dont les chaînes littérales sont encodées en UTF-8, les entrées du terminal en ISO 8859-1 et le texte d’un fichier (dont le contenu est « Moi aussi je suis élégant ! ») en [IBM 850](#) [↗](#).

Que se passe-t-il, donc ?

1. Tout d’abord, la chaîne « Cette ligne est encodée en UTF-8 » est affichée sur le terminal. Comme celui-ci utilise l’ISO 8859-1 et non l’UTF-8, la suite `\xc3\xa9` (le « é » en UTF-8) est interprétée comme deux caractères distincts (l’ISO 8859-1 encode les caractères sur un seul multiplet) : « Ã » et « © » (dont les valeurs correspondent à `\xc3` et `\xa9` dans [la table ISO 8859-1](#) [↗](#)).
2. Ensuite, la chaîne « L’ISO 8859-1 c’est tellement plus élégant ! » est récupérée en entrée et est affichée. Étant donné qu’elle est lue depuis le terminal, elle est encodée en ISO 8859-1 et est donc correctement affichée. Notez que comme « Cette chaîne en ISO 8859-1 : » est une chaîne littérale, elle est encodée en UTF-8, d’où la suite de caractères « Ã© » au lieu du « î » (`\xc3\xae` en UTF-8).
3. Enfin, la chaîne « Moi aussi je suis élégant ! » est lue depuis le fichier `fichier.txt`. Celle-ci est encodée en IBM 850 qui associe au caractère « é » le nombre 130 (`\x82`). Cependant, ce nombre ne correspond à rien dans la table ISO 8859-1, du coup, nous obtenons « Moi aussi je suis lgant ! ». En passant, remarquez à nouveau que la chaîne littérale « Et celle-là en IBM 850 : » est elle encodée en UTF-8, ce qui fait que la suite correspondant au caractère « à » (`\xc3\xa0`) est affichée comme deux caractères : « Ã » et un espace insécable.

V.5.3.1. Vers une utilisation globale de l’UTF-8

Toutefois, si ce genre de situations peut encore se présenter, de nos jours, elles sont rares car la plupart des compilateurs (c’est le cas de Clang et GCC par exemple) et systèmes d’exploitation (à l’exception notable de Windows, mais vous évitez ce problème en utilisant MSys2) utilisent l’Unicode comme table de correspondance et l’UTF-8 comme encodage par défaut. Cependant, *rien n’est imposé par la norme de ce côté*, il est donc nécessaire de vous en référer à votre compilateur et/ou à votre système d’exploitation sur ce point.

V.5.3.2. Choisir la table et l'encodage utilisés par le compilateur

Cela étant dit, dans le cas du compilateur et plus précisément de GCC, il est possible d'imposer l'emploi d'un encodage spécifique.

V.5.3.2.1. Les fichiers source

Par défaut, GCC considère que l'encodage utilisé pour écrire vos fichiers sources est l'UTF-8. Vous pouvez toutefois préciser l'encodage que vous utilisez à l'aide de l'option `-finput-charset=<encodage>`.

V.5.3.2.2. Les chaînes littérales

Comme nous l'avons déjà vu, il est possible d'imposer l'UTF-8 comme encodage en faisant précéder vos chaînes de caractères par la suite `u8`. Toutefois, GCC vous permet également de spécifier l'encodage que vous souhaitez utiliser à l'aide de l'option `--fexec-charset=<encodage>`.

```

1 #include <stdio.h>
2
3
4 int
5 main(void)
6 {
7     char s[] = "Élégant";
8
9
10    for (unsigned i = 0; s[i] != '\0'; ++i)
11        printf("%02x ", (unsigned char)s[i]);
12
13    printf("\n");
14    return 0;
15 }
```

Résultat

```

1 $ gcc main.c --fexec-charset=ISO8859-1 -o x
2 $ ./x
3 c9 6c e9 67 61 6e 74
4
5 $ gcc main.c -o x
6 $ ./x
7 c3 89 6c c3 a9 67 61 6e 74
```

Comme vous le voyez, nous avons demandé à GCC d'utiliser l'encodage ISO 8859-1, puis nous n'avons rien spécifié et c'est alors l'UTF-8 qui a été utilisé.

Conclusion

En résumé

1. Les séquences d'échappement `\n` et `\xn` sont remplacées, respectivement, par la valeur octale ou hexadécimale précisée après `\` ou `\x`.
2. Les caractères sont numérisés à l'aide de tables de correspondance comme la table ASCII ou la table ISO 8859-1 ;
3. L'Unicode a pour vocation de construire une table de correspondance universelle pour tous les caractères ;
4. Une chaîne de caractères étant une suite de `char` finie par un multipliet nul, il est nécessaire de répartir les valeurs supérieures à la capacité d'un `char` sur plusieurs `char` suivant un encodage comme l'UTF-8 ;
5. La table de correspondance et l'encodage utilisés pour représenter les chaînes de caractères littérales dépendent du compilateur ;
6. La table de correspondance et l'encodage utilisés pour représenter les chaînes de caractères lues depuis le terminal dépendent de votre système ;
7. La table de correspondance et l'encodage des fichiers dépendent de ceux utilisés lors de leur écriture ;
8. Il est possible d'imposer l'UTF-8 comme encodage pour une chaîne de caractères en la faisant précéder de la suite `u8` ;
9. L'option `--finput-charset` de GCC permet de préciser l'encodage utilisé dans vos fichiers source ;
10. L'option `--fexec-charset` de GCC permet de préciser l'encodage désiré pour représenter les chaînes de caractères.

V.6. Les caractères larges

Introduction

Dans le chapitre précédent, nous avons vu que la représentation des chaînes de caractères reposait le plus souvent sur l'encodage UTF-8, ce qui avait pour conséquence de potentiellement supprimer la correspondance entre un `char` et un caractère. Dans ce chapitre, nous allons voir le concept de **caractère large** (et, conséquemment, de chaîne de caractères larges) qui a été introduit pour résoudre ce problème.

V.6.1. Introduction

Si la non correspondance entre un `char` et un caractère n'est pas forcément gênante (elle n'empêche pas de lire ou d'écrire du texte par exemple), elle peut l'être dans d'autres cas, par exemple lorsque l'on souhaite compter le nombre de caractères composant une chaîne de caractères.

Pour ces derniers cas, il faudrait revenir à une correspondance une à une. Or, vous le savez, pour cela il faudrait un autre type que le type `char`, ce dernier n'ayant pas une taille suffisante (d'où le recours aux encodages comme l'UTF-8).

Dans cette optique, le type `wchar_t` (pour *wide character*, littéralement « caractère large »), défini dans l'en-tête `<stddef.h>`, a été introduit. Celui-ci n'est rien d'autre qu'un type entier (signé ou non signé) capable de représenter la valeur la plus élevée d'une table de correspondance donnée. Ainsi, il devient par exemple possible de stocker toutes les valeurs de la table Unicode sur un système utilisant celle-ci.

Toutefois, disposer d'un type dédié n'est pas suffisant. En effet, il nous faut également « décoder » nos chaînes de caractères afin de récupérer les valeurs correspondantes à chaque caractère (qui, pour rappel, sont pour l'instant potentiellement réparties sur plusieurs `char`). Cependant, pour effectuer ce décodage, il est nécessaire de connaître l'encodage utilisé pour représenter les chaînes de caractères. Or, vous le savez, cet encodage est susceptible de varier suivant qu'il s'agit d'une chaîne littérale, d'une entrée récupérée depuis le terminal ou de données provenant d'un fichier.

Dès lors, assez logiquement, la méthode pour convertir une chaîne de caractères en une chaîne de caractères larges varie suivant le même critère.

V.6.2. Traduction en chaîne de caractères larges et vice versa

V.6.2.1. Les chaînes de caractères littérales

Le cas des chaînes de caractères littérales est le plus simple : étant donné que la table et l'encodage sont déterminés par le compilateur, c'est lui qui se charge également de leur conversion en chaînes de caractères larges.

V. Notions avancées

Pour demander au compilateur de convertir une chaîne de caractères littérale en une chaîne de caractères larges littérale, il suffit de précéder la définition d'une chaîne de caractères littérale de la lettre `L`.

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5
6 int main(void)
7 {
8     wchar_t ws[] = L"é";
9
10    printf("%u\n", (unsigned)ws[0]);
11    return 0;
12 }
```

Résultat

1	233
---	-----

Comme vous le voyez, nous obtenons 233, qui est bien la valeur du caractère `é` dans la table Unicode. Techniquement, le compilateur a lu les deux `char` composant le caractère `é` et en a recalculé la valeur : 233, qu'il a ensuite assigné au premier élément du tableau `ws`.

Notez qu'il est également possible de faire de même pour un caractère littéral seul, en le précédant également de la lettre `L`. Le code suivant produit à nouveau le même résultat.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4
5 int main(void)
6 {
7     printf("%u\n", (unsigned)L'é');
8     return 0;
9 }
```

Résultat

1	233
---	-----

V.6.2.2. Les entrées récupérées depuis le terminal

Dans le cas des chaînes de caractères lues depuis le terminal, la conversion doit être effectuée par nos soins. Pour ce faire, la bibliothèque standard nous fournit deux fonctions déclarées dans l'en-tête `<stdlib.h>` : `mbstowcs()` et `wcstombs()`.

V.6.2.2.1. La fonction `mbstowcs`

```
1 size_t mbstowcs(wchar_t *destination, char *source, size_t max);
```

La fonction `mbstowcs()` traduit la chaîne de caractères `source` en une chaîne caractères larges d'au maximum `max` caractères qui sera stockée dans `destination`. Elle retourne le nombre de caractères larges écrits, sans compter le caractère nul final. En cas d'erreur, la fonction renvoie `(size_t)-1` (soit la valeur maximale du type `size_t`).

Le code ci-dessous récupère une ligne depuis le terminal, la converti en une chaîne de caractères larges et affiche ensuite la valeur de chaque caractère la composant.

```
1 #include <locale.h>
2 #include <stddef.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6
7 int main(void)
8 {
9     char ligne[255];
10
11     if (fgets(ligne, sizeof ligne, stdin) == NULL)
12     {
13         perror("fgets");
14         return EXIT_FAILURE;
15     }
16     if (setlocale(LC_CTYPE, "") == NULL)
17     {
18         perror("setlocale");
19         return EXIT_FAILURE;
20     }
21
22     wchar_t ws[sizeof ligne];
23
24     if (mbstowcs(ws, ligne, sizeof ligne) == (size_t)-1)
25     {
26         perror("mbstowcs");
27         return EXIT_FAILURE;
28     }
29
30     for (unsigned i = 0; ws[i] != L'\n' && ws[i] != L'\0'; ++i)
```

```

31         printf("%d ", ws[i]);
32
33     printf("\n");
34     return 0;
35 }

```

Résultat

```

1  Élégant
2  201 108 233 103 97 110 116

```

Comme vous le voyez, nous obtenons 201 et 233 pour « É » et « é », qui correspondent bien aux valeurs que leur attribue la table Unicode.

V.6.2.2.2. La catégorie LC_CTYPE

Vous l'aurez sûrement remarqué : la fonction `setlocale()` est employée dans l'exemple précédent pour modifier la localisation de la catégorie `LC_CTYPE`. Cette catégorie affecte le comportement des fonctions de traduction entre chaînes de caractères et chaînes de caractères larges.

En fait, l'appel à `setlocale()` permet aux fonctions de traductions de connaître la table et l'encodage utilisés par notre système et ainsi de traduire correctement les entrées récupérées depuis le terminal.



Localisation par défaut

Avec la localisation par défaut (C), les fonctions de traductions se contentent d'affecter la valeur de chaque `char` à un `wchar_t` ou inversement.

V.6.2.2.3. La fonction `wcstombs`

```

1  size_t wcstombs(char *destination, wchar_t *source, size_t max);

```

La fonction `wcstombs()` effectue l'opération inverse de la fonction `mbstowcs()` : elle traduit la chaîne de caractères larges `source` en une chaîne de caractères d'au maximum `max` caractères qui sera stockée dans `destination`.

La fonction retourne le nombre de *multiplets* écrits ou `(size_t)-1` en cas d'erreur.

```

1  #include <locale.h>
2  #include <stddef.h>
3  #include <stdio.h>

```

```

4  #include <stdlib.h>
5
6
7  int main(void)
8  {
9      char ligne[255];
10
11     if (fgets(ligne, sizeof ligne, stdin) == NULL)
12     {
13         perror("fgets");
14         return EXIT_FAILURE;
15     }
16     if (setlocale(LC_CTYPE, "") == NULL)
17     {
18         perror("setlocale");
19         return EXIT_FAILURE;
20     }
21
22     wchar_t ws[sizeof ligne];
23
24     if (mbstowcs(ws, ligne, sizeof ligne) == (size_t)-1)
25     {
26         perror("mbstowcs");
27         return EXIT_FAILURE;
28     }
29
30     size_t n = wcstombs(ligne, ws, sizeof ligne);
31
32     if (n == (size_t)-1)
33     {
34         perror("wcstombs");
35         return EXIT_FAILURE;
36     }
37
38     printf("%zu multiplet(s) écrit(s) : %s\n", n, ligne);
39     return 0;
40 }

```

Résultat

```

1  Élégant
2  10 multiplet(s) écrit(s) : Élégant

```


V.6.2.3. Les données provenant d'un fichier

Dans le cas des fichiers, malheureusement, il n'y a pas de solution pour déterminer la table de correspondance ou l'encodage utilisés lors de l'écriture de ses données. La seule solution est de s'assurer d'une manière ou d'une autre que ces derniers sont les mêmes que ceux employés par le système.



Des chaînes, des encodages

Ceci étant posé, nous nous permettons d'attirer votre attention sur un point : étant donné que la table et l'encodage utilisé pour représenter les chaînes de caractères littérales et les entrées du terminal sont susceptibles d'être différents, il est parfaitement possible d'obtenir des chaînes de caractères larges différentes.

Ainsi, si le code suivant donnera le plus souvent le résultat attendu étant donné que l'emploi de l'UTF-8 se généralise, rien ne vous le garanti de manière générale. En effet, si, par exemple, les chaînes littérales sont encodées en UTF-8 et les entrées du terminal en IBM 850, alors la fonction `nombre_de_e()` ne retournera pas le bon résultat.



```
1 #include <locale.h>
2 #include <stddef.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6
7 static void chomp(char *s)
8 {
9     while (*s != '\n' && *s != '\0')
10         ++s;
11
12     *s = '\0';
13 }
14
15
16 static unsigned nombre_de_e(wchar_t *ws)
17 {
18     unsigned n = 0;
19
20     while (*ws != L'\n' && *ws != L'\0')
21     {
22         switch(*ws)
23         {
24             case L'é':
25             case L'ê':
26             case L'è':
27             case L'e':
28             case L'É':
29             case L'Ê':
30             case L'È':
31             case L'E':
32                 ++n;
33                 break;
34         }
35
36         ++ws;
37     }
38
39     return n;
40 }
41
42
43 int main(void)
44 {
45     char ligne[255];
46
47     if (fgets(ligne, sizeof ligne, stdin) == NULL)
48     {
49         perror("fgets");
50         return EXIT_FAILURE;
51     }
52     if (setlocale(LC_CTYPE, "") == NULL)
53     {
54         472
55         perror("setlocale");
56         return EXIT_FAILURE;
57     }
```



Résultat

```
1 Élégamment trouvé !
2 Il y a 4 "e" dans la phrase : "Élégamment trouvé !"
```

V.6.3. L'en-tête <wchar.h>

Nous venons de le voir, il nous est possible de convertir une chaîne de caractères en chaîne de caractères larges et vice versa à l'aide des fonctions `mbstowcs()` et `wcstombs()`. Toutefois, c'est d'une part assez fastidieux et, d'autre part, nous ne disposons d'aucune fonction pour manipuler ces chaînes de caractères larges (pensez à `strlen()`, `strcmp()` ou `strchr()`).

Heureusement pour nous, la bibliothèque standard nous fournit un nouvel en-tête : `<wchar.h>` pour nous aider. 🍊

Ce dernier fournit à peu de chose près un équivalent gérant les caractères larges de chaque fonction de lecture/écriture et de manipulation des chaînes de caractères.

V.6.4. <wchar.h> : les fonctions de lecture/écriture

Le tableau ci-dessous liste les fonctions de lecture/écriture classique et leur équivalent gérant les caractères larges.

<stdio.h>	<wchar.h>
<code>printf</code>	<code>wprintf</code>
<code>fprintf</code>	<code>fwprintf</code>
<code>scanf</code>	<code>wscanf</code>
<code>fscanf</code>	<code>fwscanf</code>
<code>getc</code>	<code>getwc</code>
<code>fgetc</code>	<code>fgetwc</code>
<code>getchar</code>	<code>getwchar</code>
<code>fgets</code>	<code>fgetws</code>
<code>putc</code>	<code>putwc</code>
<code>fputc</code>	<code>fputwc</code>
<code>putchar</code>	<code>putwchar</code>
<code>fputs</code>	<code>fputws</code>
<code>ungetc</code>	<code>ungetwc</code>

Les fonctions de lecture/écriture « larges » s'utilisent de la même manière que les autres, la seule différence est qu'elles effectuent les conversions nécessaires pour nous. Ainsi, l'exemple suivant est identique à celui de la section précédente, si ce n'est que la fonction `fgetws()` se charge d'effectuer la conversion en chaîne de caractères larges pour nous.

```
1 #include <locale.h>
2 #include <stddef.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <wchar.h>
6
7
8 int main(void)
9 {
10     wchar_t ligne[255];
11
12     if (setlocale(LC_CTYPE, "") == NULL)
13     {
14         perror("setlocale");
15         return EXIT_FAILURE;
16     }
17     if (fgetws(ligne, sizeof ligne / sizeof ligne[0], stdin)
18         == NULL)
19     {
20         perror("fgetws");
21         return EXIT_FAILURE;
22     }
23     for (unsigned i = 0; ligne[i] != L'\n' && ligne[i] !=
24         L'\0'; ++i)
25         printf("%d ", ligne[i]);
26     printf("\n");
27     return 0;
28 }
```

Résultat

```
1 Élégant
2 201 108 233 103 97 110 116
```



N'oubliez pas de faire appel à la fonction `setlocale()` *avant* toute utilisation des fonctions de lecture/écriture larges !

V.6.4.1. L'orientation des flux

Toutefois, les fonctions de lecture/écriture larges amènent une subtilité supplémentaire. En effet, lorsque nous vous avons présenté les flux et les fonctions de lecture/écriture, nous avons omis de vous parler d'une caractéristique des flux : leur **orientation**.

L'orientation d'un flux détermine si ce dernier manipule des caractères larges ou des caractères simples. Lors de sa création, un flux n'a pas d'orientation, c'est la première fonction de lecture/écriture utilisée sur ce dernier qui la déterminera. Si c'est une fonction de lecture/écriture classique, le flux sera dit « orienté multipléts », si c'est une fonction de lecture/écriture large, le flux sera dit « orienté caractères larges ».

Cette orientation a son importance et amène quatre restrictions supplémentaires quant à l'usage des flux.

1. Une fois l'orientation d'un flux établie, elle ne peut plus être changée ;
2. Un flux orienté multipléts ne peut pas être utilisé par des fonctions de lecture/écriture larges et, inversement, un flux orienté caractères larges ne peut pas être utilisé par des fonctions de lecture/écriture classiques ;

```
1 fgets(s, sizeof s, stdin);
2
3 /* Faux, car stdin est déjà un flux orienté multipléts. */
4 fgetws(ws, sizeof ligne / sizeof ligne[0], stdin);
```

1. Un flux binaire orienté caractères larges voit ses déplacements possibles via la fonction `fseek()` davantage limités. Ainsi, seul le repère `SEEK_SET` avec une valeur nulle ou précédemment retournée par la fonction `ftell` et le repère `SEEK_CUR` avec une valeur nulle peuvent être utilisés ;
2. Si la position au sein d'un flux orienté caractères larges est modifiée et est suivie d'une opération d'écriture, alors le contenu qui suit les données écrites devient indéterminé, sauf si le déplacement a lieu à la fin du fichier (c'est assez logique, puisque dans ce cas il n'y a rien après).

```
1 #include <locale.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <wchar.h>
5
6
7 int main(void)
8 {
9     wchar_t ligne[255];
```

```

10     FILE *fp = fopen("fichier.txt", "w+");
11
12     if (setlocale(LC_CTYPE, "") == NULL)
13     {
14         perror("setlocale");
15         return EXIT_FAILURE;
16     }
17     if (fputws(L"Une ligne\n", fp) < 0)
18     {
19         perror("fputws");
20         return EXIT_FAILURE;
21     }
22     if (fputws(L"Une autre ligne\n", fp) < 0)
23     {
24         perror("fputws");
25         return EXIT_FAILURE;
26     }
27
28     rewind(fp);
29
30     /*
31     * On réécrit par dessus « Une ligne ».
32     */
33     if (fputws(L"Une ligne\n", fp) < 0)
34     {
35         perror("fputws");
36         return EXIT_FAILURE;
37     }
38
39     /*
40     * Obligatoire entre une opération d'écriture et une
41     * opération de lecture, rappelez-vous.
42     */
43     if (fseek(fp, 0L, SEEK_CUR) < 0)
44     {
45         perror("fseek");
46         return EXIT_FAILURE;
47     }
48
49     /*
50     * Rien ne garantit que l'on va lire « Une autre ligne »...
51     */
52     if (fgetws(ligne, sizeof ligne / sizeof ligne[0], fp) ==
53         NULL)
54     {
55         perror("fgetws");
56         return EXIT_FAILURE;
57     }
58
59     fclose(fp);

```

```

58     return 0;
59 }

```

i

Notez que dans cet exemple nous passons des chaînes de caractères larges littérales comme arguments à la fonction `fputws()`. Si cela ne pose le plus souvent pas de problèmes, n'oubliez pas que la table et l'encodage des chaînes de caractères larges littérales sont susceptibles d'être différents de ceux employés par votre système.

V.6.4.2. La fonction `fwide`

```

1 int fwide(FILE *flux, int mode);

```

La fonction `fwide()` permet de connaître l'orientation du flux `flux` ou d'en attribuer une s'il n'en a pas encore. Dans le cas où `mode` est un entier positif, la fonction essaye de fixer une orientation « caractères larges », si c'est un nombre négatif, une orientation « multipléts ». Enfin, si `mode` est nul, alors la fonction détermine simplement l'orientation du flux. Elle retourne un nombre positif si le flux est orienté caractères larges, un nombre négatif s'il est orienté multipléts et zéro s'il n'a pas encore d'orientation.

```

1 #include <locale.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <wchar.h>
5
6
7 static void affiche_orientation(FILE *fp, char *nom)
8 {
9     int orientation = fwide(fp, 0);
10
11     fprintf(stderr, "Le flux %s ", nom);
12
13     if (orientation < 0)
14         fprintf(stderr, "est orienté multipléts\n");
15     else if (orientation > 0)
16         fprintf(stderr, "est orienté caractères larges\n");
17     else
18         fprintf(stderr, "n'a pas encore d'orientation\n");
19 }
20
21
22 int main(void)
23 {
24     if (setlocale(LC_CTYPE, "") == NULL)

```

```

25     {
26         perror("setlocale");
27         return EXIT_FAILURE;
28     }
29
30     affiche_orientation(stdout, "stdout");
31     printf("Une ligne\n");
32     affiche_orientation(stdout, "stdout");
33     return 0;
34 }

```

Résultat

```

1  Le flux stdout n'a pas encore d'orientation
2  Une ligne
3  Le flux stdout est orienté multipléts

```

V.6.4.3. L'indicateur de conversion ls

Enfin, sachez que les fonctions `printf()` et `scanf()` disposent d'un indicateur de conversion permettant d'afficher ou de récupérer des chaînes de caractères larges : l'indicateur `ls`. Toutefois, il y a une subtilité : la conversion en caractères larges est effectuée avant l'écriture ou après la lecture, dès lors, le flux utilisé *doit être orienté multipléts*.

```

1  #include <locale.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <wchar.h>
5
6
7  static void affiche_orientation(FILE *fp, char *nom)
8  {
9      int orientation = fwide(fp, 0);
10
11      fprintf(stderr, "Le flux %s ", nom);
12
13      if (orientation < 0)
14          fprintf(stderr, "est orienté multipléts\n");
15      else if (orientation > 0)
16          fprintf(stderr, "est orienté caractères larges\n");
17      else
18          fprintf(stderr, "n'a pas encore d'orientation\n");
19  }
20

```



```

21
22 int main(void)
23 {
24     wchar_t ws[255];
25
26     affiche_orientation(stdout, "stdout");
27
28     if (setlocale(LC_CTYPE, "") == NULL)
29     {
30         perror("setlocale");
31         return EXIT_FAILURE;
32     }
33     if (scanf("%254ls", ws) != 1)
34     {
35         perror("scanf");
36         return EXIT_FAILURE;
37     }
38
39     printf("%ls\n", ws);
40     affiche_orientation(stdout, "stdout");
41     return 0;
42 }

```

Résultat

```

1  Le flux stdout n'a pas encore d'orientation
2  Élégant
3  Élégant
4  Le flux stdout est orienté multipléts

```

Comme vous le voyez, nous avons lu et écrit une chaîne de caractères larges, pourtant le flux `stdout` est orienté multipléts. Cela tient au fait que `scanf()` lit des caractères et effectue seulement ensuite leur conversions en caractères larges. Inversement `printf()` converti les caractères larges en caractères avant de les écrire.

Notez que l'inverse est possible à l'aide des fonctions `wscanf()` et `wprintf()`.

```

1  #include <locale.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <wchar.h>
5
6
7  static void affiche_orientation(FILE *fp, char *nom)
8  {

```

```

 9      int orientation = fwide(fp, 0);
10
11      fprintf(stderr, "Le flux %s ", nom);
12
13      if (orientation < 0)
14          fprintf(stderr, "est orienté multiplets\n");
15      else if (orientation > 0)
16          fprintf(stderr, "est orienté caractères larges\n");
17      else
18          fprintf(stderr, "n'a pas encore d'orientation\n");
19  }
20
21
22  int main(void)
23  {
24      char s[255];
25
26      affiche_orientation(stdout, "stdout");
27
28      if (setlocale(LC_CTYPE, "") == NULL)
29      {
30          perror("setlocale");
31          return EXIT_FAILURE;
32      }
33      if (wscanf(L"%254s", s) != 1)
34      {
35          perror("wscanf");
36          return EXIT_FAILURE;
37      }
38
39      wprintf(L"%s\n", s);
40      affiche_orientation(stdout, "stdout");
41      return 0;
42  }

```

Résultat

1	Le flux stdout n'a pas encore d'orientation
2	Élégant
3	Élégant
4	Le flux stdout est orienté caractères larges

V.6.5. <wchar.h>: les fonctions de manipulation des chaînes de caractères

Le tableau ci-dessous liste quelques fonctions et leur équivalent déclaré dans l'en-tête <wchar.h>.

<string.h>	<wchar.h>
strlen	wcslen
strcpy	wcscpy
strcat	wcscat
strcmp	wcscmp
strchr	wcschr
strpbrk	wcspbrk
strctr	wcsstr
strtok	wcstok

Ces fonctions se comportent de manière identique à leur équivalent, la seule différence est qu'elles manipulent des caractères larges ou des chaînes de caractères larges au lieu de caractères ou de chaînes de caractères.

```

1  #include <locale.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <wchar.h>
6
7
8  int
9  main(void)
10 {
11     char ligne[255];
12
13     if (fgets(ligne, sizeof ligne, stdin) == NULL)
14     {
15         perror("fgets");
16         return EXIT_FAILURE;
17     }
18     if (setlocale(LC_CTYPE, "") == NULL)
19     {
20         perror("setlocale");
21         return EXIT_FAILURE;
22     }
23 
```

```

24     wchar_t ws[sizeof ligne];
25
26     if (mbstowcs(ws, ligne, sizeof ligne) == (size_t)-1)
27     {
28         perror("mbstowcs");
29         return EXIT_FAILURE;
30     }
31
32     printf("strlen : %zu\n", strlen(ligne));
33     printf("wcslen : %zu\n", wcslen(ws));
34     return 0;
35 }

```

Résultat

```

1  Élégant
2  strlen : 10
3  wcslen : 8

```

V.6.6. L'en-tête <wctype.h>

Pour terminer, sachez qu'il existe l'en-tête <wctype.h> qui fournit les mêmes fonctions de classification que l'en-tête <ctype.h>, mais pour les caractères larges.

<ctype.h>	<wctype.h>
isupper	iswupper
islower	iswlower
isdigit	iswdigit
isxdigit	iswxdigit
isblank	iswblank
isspace	iswspace
isctrl	iswctrl
ispunct	iswpunct
isalpha	iswalpha
isalnum	iswalnum
isgraph	iswgraph

isprint	iswprint
---------	----------

Conclusion

En résumé

1. Le type entier `wchar_t` permet de stocker la valeur la plus élevée d'une table de correspondance donnée ;
2. La conversion des chaînes de caractères littérales en chaînes de caractères littérales larges est assurée par le compilateur ;
3. Pour les autres chaînes, les fonctions de conversions `mbstowcs()` et `wcstombs()` doivent être utilisées ;
4. La table de correspondance et l'encodage utilisé par le système sont récupérés grâce à la fonction `setlocale()` ;
5. Il n'y a pas de solution pour déterminer la table de correspondance et/ou l'encodage utilisés pour écrire les données d'un fichier ;
6. Etant donné que la table de correspondance et l'encodage utilisés pour représenter les chaînes de caractères littérales et les entrées du terminal sont susceptibles d'être différents, il est parfaitement possible d'obtenir des chaînes de caractères larges différentes ;
7. Les flux ont une orientation qui détermine s'ils manipulent des caractères ou des caractères larges. Une fois cette orientation fixée, elle ne peut plus être changée ;
8. Un flux orienté multiplète ne peut pas être utilisé par des fonctions de lecture/écriture larges et, inversement, un flux orienté caractères larges ne peut pas être utilisé par des fonctions de lecture/écriture classiques ;
9. La fonction `fwide()` permet de connaître et/ou fixer l'orientation d'un flux ;
10. L'en-tête `<wchar.h>` fournit à peu de chose près un équivalent gérant les caractères larges de chaque fonction de lecture/écriture et de manipulation des chaînes de caractères ;
11. L'en-tête `<wctype.h>` fournit les mêmes fonctions de classification que l'en-tête `<ctype.h>`, mais pour les caractères larges.

V.7. Les énumérations

Introduction

Jusqu'à présent, nous avons toujours employé le préprocesseur pour définir des constantes au sein de nos codes. Toutefois, une solution un peu plus commode existe pour les constantes entières : les **énumérations**.

V.7.1. Définition

Une énumération se définit à l'aide du mot-clé `enum` suivi du nom de l'énumération et de ses membres.

```
1 enum naturel { ZERO, UN, DEUX, TROIS, QUATRE, CINQ };
```

La particularité de cette définition est qu'elle crée en vérité deux choses : un type dit « énuméré » `enum naturel` et des constantes dites « énumérées » `ZERO`, `UN`, `DEUX`, etc. Le type énuméré ainsi produit peut être utilisé de la même manière que n'importe quel autre type. Quant aux constantes énumérées, il s'agit de constantes entières.

Certes me direz-vous, mais que valent ces constantes ? *Eh* bien, à défaut de préciser leur valeur, chaque constante énumérée se voit attribuer la valeur de celle qui la précède augmentée de un, sachant que la première constante est mise à zéro. Dans notre cas donc, la constante `ZERO` vaut zéro, la constante `UN` un et ainsi de suite jusque cinq.

L'exemple suivant illustre ce qui vient d'être dit.

```
1 #include <stdio.h>
2
3 enum naturel { ZERO, UN, DEUX, TROIS, QUATRE, CINQ };
4
5
6 int main(void)
7 {
8     enum naturel n = ZERO;
9
10    printf("n = %d.\n", (int)n);
11    printf("UN = %d.\n", UN);
12    return 0;
13 }
```

Résultat

```
1 n = 0.
2 UN = 1.
```



Notez qu'il n'est pas obligatoire de préciser un nom lors de la définition d'une énumération. Dans un tel cas, seules les constantes énumérées sont produites.

```
1 enum { ZERO, UN, DEUX, TROIS, QUATRE, CINQ };
```

Toutefois, il est possible de préciser la valeur de certaines constantes (voire de toutes les constantes) à l'aide d'une affectation.

```
1 enum naturel { DIX = 10, ONZE, DOUZE, TREIZE, QUATORZE, QUINZE };
```

Dans un tel cas, la règle habituelle s'applique : les constantes sans valeur se voient attribuer celle de la constante précédente augmentée de un et celle dont la valeur est spécifiée sont initialisées avec celle-ci. Dans le cas ci-dessus, la constante `DIX` vaut donc dix, la constante `ONZE` onze et ainsi de suite jusque quinze. Notez que le code ci-dessous est parfaitement équivalent.

```
1 enum naturel { DIX = 10, ONZE = 11, DOUZE = 12, TREIZE = 13,
    QUATORZE = 14, QUINZE = 15 };
```

V.7.1.0.1. Types entiers sous-jacents

Vous aurez sans doute remarqué que, dans notre exemple, nous avons converti la variable `n` vers le type `int`. Cela tient au fait qu'un type énuméré est un type entier (ce qui est logique puisqu'il est censé stocker des constantes entières), mais que le type sous-jacent n'est pas déterminé (cela peut donc être `_Bool`, `char`, `short`, `int`, `long` ou `long long`) et dépend entre autres des valeurs devant être contenues. Ainsi, une conversion s'impose afin de pouvoir utiliser un format d'affichage correct.

Pour ce qui est des constantes énumérées, c'est plus simple : elles sont toujours de type `int`.

V.7.2. Utilisation

Dans la pratique, les énumérations servent essentiellement à fournir des informations supplémentaires via le typage, par exemple pour les retours d'erreurs. En effet, le plus souvent, les fonctions retournent un entier pour préciser si leur exécution s'est bien déroulée. Toutefois,

indiquer un retour de type `int` ne fournit pas énormément d'information. Un type énuméré prend alors tout son sens.

La fonction `vider_tampon()` du dernier TP s'y prêterait par exemple bien.

```
1 enum erreur { E_OK, E_ERR };
2
3
4 static enum erreur vider_tampon(FILE *fp)
5 {
6     int c;
7
8     do
9         c = fgetc(fp);
10    while (c != '\n' && c != EOF);
11
12    return ferror(fp) ? E_ERR : E_OK;
13 }
```

De cette manière, il est plus clair à la lecture que la fonction retourne le statut de son exécution.

Dans la même idée, il est possible d'utiliser un type énuméré pour la fonction `statut_jeu()` (également employée dans la correction du dernier TP) afin de décrire plus amplement son type de retour.

```
1 enum statut { STATUT_OK, STATUT_GAGNE, STATUT_EGALITE };
2
3
4 static enum statut statut_jeu(struct position *pos, char jeton)
5 {
6     if (grille_complete())
7         return STATUT_EGALITE;
8     else if (calcule_nb_jetons_depuis(pos, jeton) >= 4)
9         return STATUT_GAGNE;
10
11     return STATUT_OK;
12 }
```

Dans un autre registre, un type énuméré peut être utilisé pour contenir des drapeaux. Par exemple, la fonction `traitement()` présentée dans le chapitre relatif aux opérateurs de manipulation des *bits* peut être réécrite comme suit.

```
1 enum drapeau {
2     PAIR = 0x01,
3     PUISSANCE = 0x02,
4     PREMIER = 0x04
5 };
```



```
6
7
8 void traitement(int nombre, enum drapeau drapeaux)
9 {
10     if (drapeaux & PAIR) /* Si le nombre est pair */
11     {
12         /* ... */
13     }
14     if (drapeaux & PUISSANCE) /* Si le nombre est une puissance de
15         deux */
16     {
17         /* ... */
18     }
19     if (drapeaux & PREMIER) /* Si le nombre est premier */
20     {
21         /* ... */
22     }
23 }
```

Conclusion

En résumé

1. Sauf si le nom de l'énumération n'est pas renseigné, une définition d'énumération crée un type énuméré et des constantes énumérées ;
2. Sauf si une valeur leur est attribuée, la valeur de chaque constante énumérée est celle de la précédente augmentée de un et celle de la première est zéro.
3. Le type entier sous-jacent à un type énuméré est indéterminé ; les constantes énumérées sont de type `int`.

V.8. Les unions

Introduction

Dans la deuxième partie de ce cours nous vous avons présenté la notion d'agrégat qui recouvrait les tableaux et les structures. Toutefois, nous avons passé sous silence un dernier agrégat plus discret moins utilisé : les **unions**.

V.8.1. Définition

Une union est, à l'image d'une structure, un regroupement d'objet de type *différents*. La nuance, et elle est de taille, est qu'une union est un agrégat qui ne peut contenir qu'*un seul* de ses membres à la fois. Autrement dit, une union peut accueillir la valeur de n'importe lequel de ses membres, mais un seul à la fois.

Concernant la définition, elle est identique à celle d'une structure ci ce n'est que le mot-clé **union** est employé en lieu et place de **struct**.

```
1 union type
2 {
3     int entier;
4     double flottant;
5     void *pointeur;
6     char lettre;
7 };
```

Le code ci-dessus définit une union **type** pouvant contenir un objet de type **int** ou de type **double** ou de type pointeur sur **void** ou de type **char**. Cette possibilité de ne stocker qu'un objet à la fois est traduite par le résultat de l'opérateur **sizeof**.

```
1 #include <stdio.h>
2
3 union type
4 {
5     int entier;
6     double flottant;
7     void *pointeur;
8     char lettre;
9 };
10
11
```

```

12 int main(void)
13 {
14     printf("%u.\n", sizeof (union type));
15     return 0;
16 }

```

Résultat

1	8.
---	----

Dans notre cas, la taille de l'union correspond à la taille du plus grand type stocké à savoir les types `void *` et `double` qui font huit octets. Ceci traduit bien l'impossibilité de stocker plusieurs objets à la fois.



Notez que, comme les structures, les unions peuvent contenir des *bits* de bourrage, mais uniquement à leur fin.

Pour le surplus, une union s'utilise de la même manière qu'une structure et l'accès aux membres s'effectue à l'aide des opérateurs `.` et `->`.

V.8.2. Utilisation

Étant donné leur singularité, les unions sont rarement employées. Leur principal intérêt est de réduire l'espace mémoire utilisé là où une structure ne le permet pas.

Par exemple, imaginez que vous souhaitiez construire une structure pouvant accueillir plusieurs types possibles, par exemple des entiers et des flottants. Vous aurez besoin de trois champs : un indiquant quel type est stocké dans la structure et deux permettant de stocker soit un entier soit un flottant.

```

1 struct nombre
2 {
3     unsigned entier : 1;
4     unsigned flottant : 1;
5     int e;
6     double f;
7 };

```

Toutefois, vous gaspillez ici de la mémoire puisque seul un des deux objets sera stocké. Une union est ici la bienvenue afin d'économiser de la mémoire.

```

1 struct nombre
2 {
3     unsigned entier : 1;
4     unsigned flottant : 1;
5     union
6     {
7         int e;
8         double f;
9     } u;
10 };

```

Le code suivant illustre l'utilisation de cette construction.

```

1 #include <stdio.h>
2
3 struct nombre
4 {
5     unsigned entier : 1;
6     unsigned flottant : 1;
7     union
8     {
9         int e;
10        double f;
11    } u;
12 };
13
14
15 static void affiche_nombre(struct nombre n)
16 {
17     if (n.entier)
18         printf("%d\n", n.u.e);
19     else if (n.flottant)
20         printf("%f\n", n.u.f);
21 }
22
23
24 int main(void)
25 {
26     struct nombre a = { 0 };
27     struct nombre b = { 0 };
28
29     a.entier = 1;
30     a.u.e = 10;
31     b.flottant = 1;
32     b.u.f = 10.56;
33
34     affiche_nombre(a);

```

```

35     affiche_nombre(b);
36     return 0;
37 }

```

Résultat

```

1  10
2  10.560000

```

La syntaxe est toutefois un peu pénible puisqu'il est nécessaire d'employer deux fois l'opérateur `.` : une fois pour accéder aux membres de la structure et une seconde fois pour accéder aux membres de l'union. Par ailleurs, la nécessité d'intégrer l'union comme un champ de la structure, et donc de lui donner un nom, est également ennuyeux.

Heureusement pour nous, il est possible de rendre l'union « anonyme », c'est-à-dire de l'inclure comme champ de la structure, mais sans lui donner un nom. Dans un tel cas, les champs de l'union sont traités comme s'ils étaient des champs de la structure et sont donc accessibles comme tel.

```

1  #include <stdio.h>
2
3  struct nombre
4  {
5      unsigned entier : 1;
6      unsigned flottant : 1;
7      union
8      {
9          int e;
10         double f;
11     };
12 };
13
14
15 static void affiche_nombre(struct nombre n)
16 {
17     if (n.entier)
18         printf("%d\n", n.e);
19     else if (n.flottant)
20         printf("%f\n", n.f);
21 }
22
23
24 int main(void)
25 {
26     struct nombre a = { 0 };

```

```
27     struct nombre b = { 0 };
28
29     a.entier = 1;
30     a.e = 10;
31     b.flottant = 1;
32     b.f = 10.56;
33
34     affiche_nombre(a);
35     affiche_nombre(b);
36     return 0;
37 }
```

Notez que cette possibilité ne se limite pas aux unions, il est possible de faire de même avec des structures.

Conclusion

En résumé

1. Une union est un agrégat regroupant des objets de types différents, mais ne pouvant en stocker qu'un seul à la fois ;
2. Comme les structures, les unions peuvent comprendre des *bits* de bourrage, mais uniquement à leur fin ;
3. Il est possible de simplifier l'accès aux champs d'une union lorsque celle-ci est elle-même un champ d'une autre structure ou d'une autre union en la rendant « anonyme ».

V.9. Les définitions de type

Introduction

Ce chapitre sera relativement court et pour cause, nous allons aborder un petit point du langage C, mais qui a toute son importance : les **définitions de type**.

V.9.1. Définition et utilisation

Une définition de type permet, comme son nom l'indique, de définir un type, c'est-à-dire d'en produire un nouveau ou, plus précisément, de créer un *alias* (un synonyme si vous préférez) d'un type existant. Une définition de type est identique à une déclaration de variable, si ce n'est que celle-ci doit être précédée du mot-clé **typedef** (pour *type definition*) et que l'identificateur ainsi choisi désignera un type et non une variable.

Ainsi, le code ci-dessous définit un nouveau type **entier** qui sera un *alias* pour le type **int**.

```
1 typedef int entier;
```

Le synonyme ainsi créé peut être utilisé au même endroit que n'importe quel autre type.

```
1 #include <stdio.h>
2
3 typedef int entier;
4
5
6 entier main(void)
7 {
8     entier a = 10;
9
10    printf("%d.\n", a);
11    return 0;
12 }
```



D'accord, mais cela me sert à quoi de créer un synonyme pour un type existant ? 🍌

Les définitions de type permettent en premier lieu de raccourcir certaines écritures, notamment afin de s'affranchir des mots-clés **struct**, **union** et **enum** (bien que, ceci constitue une perte d'information aux yeux de certains).

```

1  #include <stdio.h>
2
3  struct position
4  {
5      int lgn;
6      int col;
7  };
8
9  typedef struct position position;
10
11
12 int main(void)
13 {
14     position pos = { .lgn = 1, .col = 1 };
15
16     printf("%d, %d.\n", pos.lgn, pos.col);
17     return 0;
18 }

```

Notez qu’une définition de type étant une déclaration, il est parfaitement possible de combiner la définition de la structure et la définition de type (comme pour une variable, finalement).

```

1  typedef struct position
2  {
3      int lgn;
4      int col;
5  } position;

```

De même, comme pour une déclaration de variable, vous pouvez déclarer plusieurs *alias* en une déclaration. L’exemple ci-dessous déclare le type `position` (qui est une structure `position`) et le type `ptr_position` (qui est un pointeur sur une structure `position`).

```

1  typedef struct position
2  {
3      int lgn;
4      int col;
5  } position, *ptr_position;

```

Également, dans le même sens que ce qui a été dit au sujet des énumérations, une définition de type peut être employée pour donner plus d’informations via le typage. C’est une manière de désigner plus finement le contenu d’un type en ne se contentant pas d’une information plus générale comme « un entier » ou « un flottant ».

Par exemple, nous aurions pu définir un type `ligne` et un type `colonne` afin de donner plus d’informations sur le contenu de nos variables.


```
1 typedef short ligne;  
2 typedef short colonne;  
3  
4 struct position  
5 {  
6     ligne lgn;  
7     colonne col;  
8 };
```

De même, les définitions de type sont couramment utilisées afin de créer des abstractions. Nous en avons vu un exemple avec l'en-tête `<time.h>` qui définit le type `time_t`. Celui-ci permet de ne pas devoir modifier la fonction `time()` et son utilisation suivant le type qui est sous-jacent. Peu importe que `time_t` soit un entier ou un flottant, la fonction `time()` s'utilise de la même manière.

Enfin, les définitions de type permettent de résoudre quelques points de syntaxe tordus.

Conclusion

En résumé

1. Une définition de type permet de créer un synonyme d'un type existant.

V.10. Les pointeurs de fonction

Introduction

À ce stade, vous pensiez sans doute avoir fait le tour des pointeurs, que ces derniers n'avaient plus de secrets pour vous et que vous maîtrisiez enfin tous leurs aspects ainsi que leur syntaxe parfois déroutante ? *Eh bien, pas encore ! 🍌*

Il reste un dernier type de pointeur (et non des moindres) que nous avons vu jusqu'ici : les **pointeurs de fonction**.

V.10.1. Déclaration et initialisation

Jusqu'à maintenant, nous avons manipulé des pointeurs sur objet, c'est-à-dire des adresses vers des zones mémoires contenant des *données* (des entiers, des flottants, des structures, etc.). Toutefois, il est également possible de référencer des *instructions* et ceci est réalisé en C à l'aide des pointeurs de fonction.

Un pointeur de fonction se définit à l'aide d'une syntaxe mélangeant celle des pointeurs sur tableau et celle des prototypes de fonction. Sans plus attendre, voici ci-dessous la définition d'un pointeur sur une fonction retournant un `int` et attendant un `int` comme argument.

```
1 int (*pf)(int);
```

Comme vous le voyez, il est nécessaire, tout comme les pointeurs sur tableau, d'entourer le symbole `*` et l'identificateur de parenthèses, ici afin d'éviter que cette déclaration ne soit vue comme un prototype et non comme un pointeur de fonction. Autre particularité : le type de retour, le nombre d'arguments et leur type doivent également être spécifiés.

V.10.1.1. Initialisation

?

Ok... Et je lui affecte comment l'adresse d'une fonction, moi, à ce machin ? D'ailleurs, elles ont une adresse, les fonctions ? 🍌

Oui et comme d'habitude, cela est réalisé à l'aide de l'opérateur `&`. 🍌

En fait, dans le cas des fonctions, il n'est pas obligatoire de recourir à cet opérateur, ainsi, les deux syntaxes suivantes sont correctes.

```

1 int (*pf)(int);
2
3 pf = &fonction;
4 pf = fonction;

```

Ceci est dû, à l'image des tableaux, à une conversion implicite : sauf s'il est l'opérande de l'opérateur `&`, un identificateur de fonction est converti en un pointeur sur cette fonction. L'utilisation de l'opérateur `&` est donc facultative, mais elle a le mérite de clarifier un peu les choses. Pour cette raison, nous utiliserons cette syntaxe dans la suite de ce cours.

V.10.2. Utilisation

V.10.2.1. Déréférencement

Un pointeur de fonction s'emploie de la même manière qu'un pointeur classique, si ce n'est que l'opérateur `*` et l'identificateur doivent à nouveau être entre parenthèses. Pour le reste, la liste des arguments suit l'expression déréférencée, comme pour un appel de fonction classique.

```

1 #include <stdio.h>
2
3
4 static int triple(int a)
5 {
6     return a * 3;
7 }
8
9
10 int main(void)
11 {
12     int (*pt)(int) = &triple;
13
14     printf("%d.\n", (*pt)(3));
15     return 0;
16 }

```

Résultat

```
1 9.
```

Toutefois, particularité des fonctions oblige, sachez que le déréférencement *n'est pas nécessaire*. Ceci à cause de la conversion implicite expliquée précédemment : un identificateur de fonction est, sauf s'il est l'opérande de l'opérateur `&`, converti en un pointeur sur cette fonction. L'appel `triple(3)` cache donc en fait un pointeur de fonction qui, comme vous le voyez, n'est *pas*

déréférencé.

?

Heu... Mais pourquoi l'expression `(*pt)(3)` ne provoque-t-elle pas une erreur si c'est un pointeur de fonction qui est nécessaire lors d'un appel ?

Parce que la conversion implicite aura lieu juste après le déréférencement. 🍊

Eh oui, déréférencer un pointeur de fonction, c'est un peu reculer pour mieux sauter : nous obtenons une expression équivalente à un identificateur de fonction qui sera ensuite convertie en un pointeur de fonction. Les deux expressions suivantes sont donc équivalentes.

```
1 triple(3);
2 (*pt)(3);
```

L'intérêt d'employer le déréférencement est purement syntaxique : cela permet de distinguer des appels effectués via des pointeurs des appels de fonction classiques.

V.10.2.2. Passage en argument

Comme n'importe quel pointeur, un pointeur de fonction peut être passé en argument d'une autre fonction (c'est d'ailleurs tout l'intérêt de ceux-ci, comme nous le verrons bientôt). Pour ce faire, il vous suffit d'employer la même syntaxe que pour une déclaration.

```
1 #include <stdio.h>
2
3
4 static int triple(int a)
5 {
6     return a * 3;
7 }
8
9
10 static int quadruple(int a)
11 {
12     return a * 4;
13 }
14
15
16 static void affiche(int a, int (*pf)(int))
17 {
18     printf("%d.\n", (*pf)(a));
19 }
20
21
22 int main(void)
23 {
```

```

24     affiche(3, &triple);
25     affiche(3, &quadruple);
26     return 0;
27 }

```

Résultat

```

1  9.
2  12.

```



Dans l'exemple ci-dessus, les fonctions `triple()` et `quadruple()` sont utilisées comme des **fonctions de rappel** (*callback function* en anglais), c'est-à-dire que leur adresse est passée à la fonction `affichage()` de sorte que cette dernière puisse les appeler par la suite.

V.10.2.3. Retour de fonction

Dans l'autre sens, il est possible de retourner un pointeur de fonction à l'aide d'une syntaxe... un peu lourde. 🍊

```

1  #include <stddef.h>
2  #include <stdio.h>
3
4
5  static void affiche_pair(int a)
6  {
7      printf("%d est pair.\n", a);
8  }
9
10
11 static void affiche_impair(int a)
12 {
13     printf("%d est impair.\n", a);
14 }
15
16
17 static void (*affiche(int a))(int)
18 {
19     if (a % 2 == 0)
20         return &affiche_pair;
21     else
22         return &affiche_impair;

```

```

23 }
24
25
26
27 int main(void)
28 {
29     void (*pf)(int);
30     int a = 2;
31
32     pf = affiche(a);
33     (*pf)(a);
34     return 0;
35 }

```

Résultat

```

1 2 est pair.

```

Comme pour une variable de type pointeur de fonction, le symbole `*` doit être entouré de parenthèses ainsi que l'identificateur qui le suit. Toutefois, lorsqu'il s'agit du type de retour d'une fonction, la liste des arguments doit également être placée entre ces parenthèses.

Dans cet exemple, la fonction `affiche()` attend un `int` et retourne un pointeur sur une fonction ne retournant rien et utilisant un argument de type `int`. Suivant si `a` est pair ou impair, la fonction `affiche()` retourne l'adresse de la fonction `affichage_pair()` ou `affichage_impair()` qui est recueillie par le pointeur `pf` de la fonction `main()`.

V.10.3. Pointeurs de fonction et pointeurs génériques

Vous le savez, le type `void` est employé en C pour produire un pointeur générique qui peut se voir assigner n'importe quel type de pointeur et être converti vers n'importe quel type de pointeurs. Cette définition est toutefois incomplète car il doit en fait être précisé que cela ne fonctionne que pour des pointeurs sur des *objets*. Le code ci-dessous est donc incorrect.

```

1 int (*pf)(int);
2 void *p;
3
4 pf = p; /* Faux. */
5 p = pf; /* Faux également. */

```

Pareillement, une conversion explicite d'un pointeur sur un objet vers ou depuis un pointeur sur fonction est interdite (ou, plus précisément, son résultat est indéterminé). Ceci exclut donc l'utilisation de l'indicateur `p` de la fonction `printf()` pour afficher un pointeur de fonction.

```
1 printf("%p.\n", (void *)pf); /* Faux. */
```

Toutefois, les pointeurs de fonction disposent de leur propre pointeur « générique ». Nous utilisons ici les guillemets car il ne l'est pas tout à fait puisqu'il peut notamment être utilisé à l'inverse d'un pointeur sur `void`. Un pointeur « générique » de fonction se déclare comme un pointeur de fonction, mais en ne spécifiant que le type de retour.

```
1 int (*pf)();
```

Un tel pointeur peut se voir assigner n'importe quel pointeur sur fonction du moment que le type de retour de celui-ci est identique au sien. Inversement, ce pointeur « générique » peut être affecté à un autre pointeur sous la même condition. Dans notre cas, le type de retour doit donc être `int`.

```
1 int (*f)(int, int);
2 int (*g)(char, char, double);
3 void (*h)(void);
4 int (*pf)();
5
6 pf = f; /* Ok. */
7 pf = g; /* Ok. */
8 pf = h; /* Faux car le type de retour de `h` est `void`. */
9 f = pf; /* Ok. */
```



Il existe cependant une exception supplémentaire : une fonction à nombre variable d'arguments ne peut pas être affectée à un tel pointeur, même si le type de retour est identique. Nous verrons bientôt de quoi il s'agit, mais pour l'heure, sachez que les fonctions de la famille de `printf()` et de `scanf()` sont concernées par cette règle.

V.10.3.1. La promotion des arguments



Hé là, minute papillon ! Il se passe quoi si j'utilise le pointeur `pf` avec une fonction qui attend normalement des arguments ?

Excellente question ! 🍊

Vous vous en doutez : les arguments peuvent toujours être envoyés à la fonction référencée. Cependant, il y a une subtilité. Étant donné qu'un pointeur « générique » de fonction ne fournit aucune information quant aux arguments, le compilateur ne peut pas convertir ceux-ci vers le type attendu par la fonction. Ainsi, si vous fournissez un `int` et que la fonction attend un `char`, le `int` ne sera pas converti vers le type `char` par le compilateur.

Toutefois, dans un tel cas, plusieurs conversions implicites sont appliquées afin de limiter les types possibles (on parle de « promotion des arguments ») :

1. Un argument de type entier de capacité inférieure ou égale à celui du type `int` (soit `char` et `short` le plus souvent) est converti vers le type `int` (ou `unsigned int` si le type `int` ne peut pas représenter toutes les valeurs du type d'origine).
2. Un argument de type `float` est converti vers le type `double`.



Ceci signifie qu'une fonction appelée à l'aide d'un pointeur « générique » de fonction ne pourra *jamaïs* recevoir des paramètres de type `char`, `short` ou `float`.

Illustration.

```

1  #include <stdio.h>
2
3
4  static float triple(float f)
5  {
6      return 3.F * f;
7  }
8
9
10 static short quadruple(short n)
11 {
12     return 4 * n;
13 }
14
15
16 int main(void)
17 {
18     float (*pt)() = &triple;
19     short (*pq)() = &quadruple;
20
21     printf("triple = %f.\n", (*pt)(3.F)); /* Faux. */
22     printf("quadruple = %d.\n", (*pq)(2)); /* Faux. */
23     return 0;
24 }
```

V.10.3.2. Les pointeurs nuls

L'absence de conversions par le compilateur dans le cas où aucune information n'est fournie par rapport aux arguments pose un problème particulier dans le cas des pointeurs nuls et tout spécialement lors de l'usage de la macroconstante `NULL`.

Rappelez-vous : un pointeur nul est construit en convertissant, soit explicitement, soit implicitement, zéro (entier) vers un type pointeur. Or, étant donné que le compilateur n'effectuera aucune conversion implicite dans notre cas, nous ne pouvons compter que sur les conversions explicites.

Et c'est ici que le bât blesse : la macroconstante `NULL` a deux valeurs possibles : `(void *)0` ou `0`, le choix étant laissé aux différents systèmes. La première ne pose pas de problème, mais la seconde en pose un plutôt gênant : c'est un `int` qui sera passé comme argument et non un pointeur nul.

Dès lors, lorsque vous employez un pointeur « générique » de fonction, vous *devez* recourir à une conversion explicite si vous souhaitez produire un pointeur nul.

```
1 static void affiche(char *chaine)
2 {
3     if (chaine != NULL)
4         puts(chaine);
5 }
6
7 /* ... */
8
9 void (*pf)() = &affiche;
10
11 (*pf)(NULL); /* Faux. */
12 (*pf)(0); /* Faux. */
13 (*pf)((char*)0); /* Ok. */
```

Conclusion

En résumé

1. Un pointeur de fonction permet de stocker une référence vers une fonction ;
2. Il n'est pas nécessaire d'employer l'opérateur `&` pour obtenir l'adresse d'une fonction ;
3. Le déréférencement n'est pas obligatoire lors de l'utilisation d'un pointeur de fonction ;
4. Une fonction dont l'adresse est passée en argument d'une autre fonction est appelée une fonction de rappel (*callback function* en anglais) ;
5. Il est possible d'utiliser un pointeur « générique » de fonction en ne fournissant aucune information quant aux arguments lors de sa définition ;
6. Un pointeur « générique » de fonction peut être converti vers ou depuis n'importe quel autre type de pointeur de fonction du moment que le type de retour reste identique ;
7. Lors de l'utilisation d'un pointeur « générique » de fonction, les arguments transmis sont promus, mais aucune conversion implicite n'est réalisée par le compilateur ;
8. Lors de l'utilisation d'un pointeur « générique » de fonction, un pointeur nul ne peut être fourni qu'à l'aide d'une conversion explicite.

V.11. Les fonctions et macrofonctions à nombre variable d'arguments

Introduction

Dans le chapitre précédent, il a été question des pointeurs de fonction et, notamment, des pointeurs « génériques » de fonction. Néanmoins, ces derniers ne permettent pas d'expliquer le fonctionnement des fonctions `printf()` et `scanf()`. C'est ce que nous allons voir dans ce chapitre en étudiant les **fonctions à nombre variable d'arguments**. Nous en profiterons pour vous présenter par la suite les **macrofonctions à nombre variable d'arguments**.

V.11.1. Présentation

Une fonction à nombre variable d'arguments est, comme son nom l'indique, une fonction capable de recevoir et de manipuler un nombre variable d'arguments, mais en plus, et cela son nom ne le spécifie pas, potentiellement de types différents.

Une telle fonction se définit en employant comme *dernier* paramètre une suite de trois points appelée une ellipse indiquant que des arguments supplémentaires peuvent être transmis.

```
1 void affiche_suite(int n, ...);
```

Le prototype ci-dessus déclare une fonction recevant un `int` et, éventuellement, un nombre indéterminé d'autres arguments. La partie précédant l'ellipse décrit donc les paramètres attendus par la fonction et qui, comme pour n'importe quelle autre fonction, *doivent* lui être transmis.



Une fonction à nombre variable d'arguments doit *toujours* avoir au moins un paramètre non optionnel.



L'ellipse ne peut être placée qu'à la fin de la liste des paramètres.

Une fois définie, la fonction peut être appelée en lui fournissant zéro ou plusieurs arguments supplémentaires.

```
1 affiche_suite(10, 20);  
2 affiche_suite(10, 20, 30, 40, 50, 60, 70, 80, 90, 100);  
3 affiche_suite(10);
```



Comme pour les pointeurs génériques de fonction, le nombre et le types des arguments est inconnu du compilateur (c'est d'ailleurs bien le but de la manœuvre). Dès lors, les mêmes règles de promotion s'appliquent ainsi que les problèmes qui y sont inhérents, notamment la problématique des pointeurs nuls. N'employer donc pas la macroconstante `NULL` pour fournir un pointeur nul comme argument optionnel.

V.11.2. L'en-tête `<stdarg.h>`

Une fois le prototype de la fonction déterminé, encore faut-il pouvoir manipuler ces arguments supplémentaires au sein de sa définition. Pour ce faire, la bibliothèque standard nous fournit trois macrofonctions : `va_start()`, `va_arg()` et `va_end()` définies dans l'en-tête `<stdarg.h>`. Ces trois macrofonctions attendent comme premier argument une variable de type `va_list` défini dans le même en-tête.

Afin d'illustrer leur fonctionnement, nous vous proposons directement un exemple mettant en œuvre une fonction `affiche_suite()` qui reçoit comme premier paramètre le nombre d'entiers qui vont lui être transmis et les affiche ensuite tous, un par ligne.

```

1  #include <stdarg.h>
2  #include <stdio.h>
3
4
5  void affiche_suite(int nb, ...)
6  {
7      va_list ap;
8
9      va_start(ap, nb);
10
11     while (nb > 0)
12     {
13         int n;
14
15         n = va_arg(ap, int);
16         printf("%d.\n", n);
17         --nb;
18     }
19
20     va_end(ap);
21 }
22
23
24 int main(void)
25 {
26     affiche_suite(10, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100);
27     return 0;
28 }
```

Résultat

1	10.
2	20.
3	30.
4	40.
5	50.
6	60.
7	70.
8	80.
9	90.
10	100.

La macrofonction `va_start` initialise le parcours des paramètres optionnels. Elle attend deux arguments : une variable de type `va_list` et le nom du dernier paramètre obligatoire de la fonction courante (dans notre cas `nb`). Il est impératif de l'appeler avant toute opération sur les paramètres optionnels.

La macrofonction `va_arg()` retourne le paramètre optionnel suivant. Elle attend deux arguments : une variable de type `va_list` précédemment initialisée par la macrofonction `va_start()` et le type du paramètre optionnel suivant.

Enfin, la fonction `va_end()` met fin au parcours des arguments optionnels. Elle doit *toujours* être appelée après un appel à la macrofonction `va_start()`.



La macrofonction `va_arg()` n'effectue *aucune* vérification ! Cela signifie que si vous renseignez un type qui ne correspond pas au paramètre optionnel suivant ou si vous tentez de récupérer un paramètre optionnel qui n'existe pas, vous allez au-devant de comportements indéfinis.

Étant donné cet état de fait, il est impératif de pouvoir déterminer le nombre d'arguments optionnels envoyé à la fonction. Dans notre cas, nous avons opté pour l'emploi d'un paramètre obligatoire indiquant le nombre d'arguments optionnels envoyés.



Mais ?! Et si l'utilisateur de la fonction se plante en ne précisant pas le bon nombre ou le bon type d'arguments ?

Eh bien... C'est foutu. 🍊

Il s'agit là d'une lacune similaire à celle des pointeurs « génériques » de fonction : étant donné que le type et le nombre des arguments optionnels est inconnu, le compilateur ne peut effectuer aucune vérification ou conversion.

V.11.3. Méthodes pour déterminer le nombre et le type des arguments

De manière générale, il existe trois grandes méthodes pour gérer le nombre et le type des arguments optionnels.

V.11.3.1. Les chaînes de formats

Cette méthode, vous la connaissez déjà, il s'agit de celle employée par les fonctions `printf()`, `scanf()` et consœurs. Elle consiste à décrire le nombre et le type des arguments à l'aide d'une chaîne de caractères comprenant des indicateurs.

V.11.3.2. Les suites d'arguments de même type

La seconde solution ne peut être utilisée que si tous les arguments optionnels sont de même type. Elle consiste soit à indiquer le nombre d'arguments transmis, soit à utiliser un délimiteur. La fonction `affiche_suite()` recourt par exemple à un paramètre pour déterminer le nombre de paramètres optionnels qu'elle a reçu.

Un délimiteur pourrait par exemple être un pointeur nul dans le cas d'une fonction similaire affichant une suite de chaîne de caractères.

```
1  #include <stdarg.h>
2  #include <stddef.h>
3  #include <stdio.h>
4
5
6  static void affiche_suite(char *chaine, ...)
7  {
8      va_list ap;
9
10     va_start(ap, chaine);
11
12     do
13     {
14         puts(chaine);
15         chaine = va_arg(ap, char *);
16     } while(chaine != NULL);
17
18     va_end(ap);
19 }
20
21
22 int main(void)
23 {
24     affiche_suite("un", "deux", "trois", (char *)0);
25     return 0;
26 }
```

Résultat

1	un
2	deux
3	trois

V.11.3.3. Emploi d'un pivot

La dernière pratique consiste à recourir à un paramètre « pivot » qui fera varier le parcours des paramètres optionnels en fonction de sa valeur. Notez qu'un type énuméré se prête très bien pour ce genre de paramètre. Par exemple, la fonction suivante affiche un `int` ou un `double` suivant la valeur du paramètre `type`.

```

1  #include <stdarg.h>
2  #include <stdio.h>
3
4  enum type { TYPE_INT, TYPE_DOUBLE };
5
6
7  static void affiche(enum type type, ...)
8  {
9      va_list ap;
10
11     va_start(ap, type);
12
13     switch (type)
14     {
15     case TYPE_INT:
16         printf("Un entier : %d.\n", va_arg(ap, int));
17         break;
18
19     case TYPE_DOUBLE:
20         printf("Un flottant : %f.\n", va_arg(ap, double));
21         break;
22     }
23
24     va_end(ap);
25 }
26
27
28 int main(void)
29 {
30     affiche(TYPE_INT, 10);
31     affiche(TYPE_DOUBLE, 3.14);
32     return 0;

```

```
33 }
```

Résultat

```
1 Un entier : 10.
2 Un flottant : 3.140000.
```

V.11.4. Les macrofonctions à nombre variable d'arguments

Vous vous en doutez en lisant le titre de cette section, il est également possible, à l'image des fonctions, de construire des macrofonctions à nombre variable d'arguments. Comme pour les fonctions, une macrofonction à nombre variable d'arguments se définit en indiquant une suite de trois points comme dernier argument.

Toutefois, à l'inverse des fonctions, la suite d'arguments variables ne peut être utilisée que d'un seul tenant à l'aide de la macroconstante `__VA_ARGS__`.

Ainsi, avec la définition ci-dessous, l'appel `PRINTF("%d, %d, %d\n", 1, 2, 3)` sera remplacé par `printf("%d, %d, %d\n", 1, 2, 3)`.

```
1 #define PRINTF(fmt, ...) printf((fmt), __VA_ARGS__)
```

Notez qu'à l'inverse d'une fonction, une macrofonction ne requiert pas d'argument non optionnel. La définition suivante est donc parfaitement valide.

```
1 #define PRINTF(...) printf(__VA_ARGS__) /* Ok. */
```

Conclusion

En résumé

1. Une fonction à nombre variable d'arguments est capable de recevoir et de manipuler une suite indéterminée d'arguments optionnels de types potentiellement différents.
2. L'ellipse ne peut être placée qu'à la fin de la liste des paramètres et doit impérativement être précédée d'un paramètre non optionnel.
3. La macrofonction `va_arg()` n'effectue *aucune* vérification, il est donc impératif de contrôler le nombre et le types des arguments reçus, par exemple à l'aide d'une chaîne de formats ou d'un paramètre décrivant le nombre d'arguments transmis.
4. Il est possible de construire une macrofonction à nombre variable d'arguments en ajoutant une ellipse à la fin de sa liste d'argument.
5. Les arguments optionnels sont utilisables dans la définition de la macrofonction en employant la macroconstante `__VA_ARGS__`.

6. Une macrofonction ne requiert pas d'argument non optionnel.

V.12. Les sélections génériques

Introduction

Avant de nous atteler au dernier TP de ce cours, voyons ensemble une construction assez particulière ajoutée par la norme C11 : **les sélections génériques**.

V.12.1. Définition et utilisation

Les sélections génériques sont assez singulières en ce qu'elles permettent de construire des expressions en fonction du type d'une autre expression. Syntaxiquement, elles se rapprochent d'un mélange entre l'opérateur conditionnel et le `switch`.

```
1  _Generic(expression, type1: expression1, type2: expression2,  
           default: expression3)
```

Dans cet exemple, le type de l'expression `expression` est évalué et, suivant celui-ci, la sélection générique est « remplacée » (le terme est impropre, mais il illustre bien le résultat obtenu) par `expression1`, `expression2` ou `expression3`. Comme pour les `switch`, il est possible d'employer le mot-clé `default` en lieu et place d'un type afin de signaler que si aucun type ne correspond à celui de l'expression `expression`, alors c'est l'expression désignée par le mot-clé `default` qui doit être choisie.

Un exemple valant mieux qu'un long discours, voici un exemple de sélection générique affichant le type de son expression de contrôle.

```
1  #include <stdio.h>  
2  
3  
4  int  
5  main(void)  
6  {  
7      int x;  
8  
9      printf("type de x : %s\n", _Generic(x, char: "char", short:  
10         "short", int: "int", double: "double", default: "?"));  
11     return 0;  
12 }
```

Résultat

```
1 type de x : int
```

Comme vous le voyez, la sélection générique sera « remplacée » par une des chaînes de caractères de la sélection suivant le type de la variable `x`, dans ce cas ci `"int"`. Ce genre de sélections est toutefois plus utile au sein d'une macrofonction afin de pouvoir être réutilisée.

```
1 #include <stdio.h>
2
3 #define TYPEOF(x) _Generic((x), \
4     char: "char", \
5     short: "short", \
6     int: "int", \
7     long: "long", \
8     long long: "long long", \
9     float: "float", \
10    double: "double", \
11    long double: "long double")
12
13
14 int
15 main(void)
16 {
17     int x;
18
19     printf("type de x : %s\n", TYPEOF(x));
20     return 0;
21 }
```

Néanmoins, une des grandes forces et particularités des sélections génériques tient au fait qu'elles peuvent être employées comme n'importe quel élément d'une expression (du moment que le résultat final est valide, bien entendu). Ainsi, une sélection générique peut finalement produire une constante (comme nous l'avons démontré ci-dessus), mais également un identificateur de variable ou de fonction.

```
1 #include <errno.h>
2 #include <limits.h>
3 #include <stdio.h>
4
5 #define add(x, y) _Generic((x) + (y), int: safe_add, long:
6     safe_addl)((x), (y))
7
```

```

8 int safe_add(int a, int b)
9 {
10     int err = 1;
11
12     if (b >= 0 && a > INT_MAX - b)
13         goto overflow;
14     else if (b < 0 && a < INT_MIN - b)
15         goto underflow;
16
17     return a + b;
18 underflow:
19     err = -1;
20 overflow:
21     errno = ERANGE;
22     return err > 0 ? INT_MAX : INT_MIN;
23 }
24
25
26 long safe_addl(long a, long b)
27 {
28     int err = 1;
29
30     if (b >= 0 && a > LONG_MAX - b)
31         goto overflow;
32     else if (b < 0 && a < LONG_MIN - b)
33         goto underflow;
34
35     return a + b;
36 underflow:
37     err = -1;
38 overflow:
39     errno = ERANGE;
40     return err > 0 ? LONG_MAX : LONG_MIN;
41 }
42
43
44 int
45 main(void)
46 {
47     printf("%d\n", add(INT_MAX, 45));
48     printf("%ld\n", add(LONG_MAX, 1));
49     return 0;
50 }

```

Résultat

1	2147483647
2	9223372036854775807

Dans l'exemple ci-dessus, nous avons repris la fonction `safe_add()` du chapitre sur les limites des types et avons ajouté la fonction `safe_addl()` qui est la même, mais pour le type `long`. Nous utilisons ensuite une sélection générique pour appeler la bonne fonction suivant le type de l'expression `x + y`. Vous remarquerez que dans ce cas ci, la sélection générique est « remplacée » par un identificateur de fonction (`safe_add` ou `safe_addl`) et que ce dernier peut être utilisé comme n'importe quel autre identificateur de fonction, par exemple pour appeler une fonction en lui précisant des arguments.



Évaluation

L'expression de contrôle d'une sélection générique n'est *pas* évaluée. Autrement dit, dans l'exemple précédent, l'addition n'est pas effectuée, seul son type est déterminé.



Si aucune entrée de la sélection générique ne correspond au type de l'expression de contrôle et qu'il n'y a pas d'entrée `default`, alors la compilation échouera.

Conclusion

En résumé

1. Une sélection générique permet de construire une expression en fonction du type d'une autre expression.
2. Il est possible d'utiliser le mot-clé `default` pour définir une expression à utiliser en cas de non correspondance des autres types.
3. Une sélection générique peut être utilisée comme n'importe quel élément d'une expression.
4. Dans le cas où il n'y a pas de correspondance entre le type de l'expression de contrôle et les entrées de la sélection et que le mot-clé `default` n'a pas été utilisé, la compilation échoue.

V.13. T.P.: un allocateur statique de mémoire

Introduction

Dans ce chapitre, nous allons mettre en œuvre une partie des notions présentées précédemment afin de réaliser un allocateur statique de mémoire.

V.13.1. Objectif

Votre objectif sera de parvenir à réaliser un petit allocateur statique de mémoire en mettant en œuvre deux fonctions : `static_malloc()` et `static_free()` comparables aux fonctions `malloc()` et `free()` de la bibliothèque standard. Toutefois, afin d'éviter d'entrer dans les méandres de l'allocation dynamique de mémoire, ces fonctions fourniront de la mémoire préalablement allouée statiquement.

V.13.2. Première étape : allouer de la mémoire

Pour commencer, vous allez devoir construire une fonction `static_malloc()` dont le prototype sera le suivant.

```
1 void *static_malloc(size_t taille);
```

À la lumière de la fonction `malloc()`, celle-ci reçoit une taille en multiplète et retourne un pointeur vers un bloc de mémoire d'au moins la taille demandée ou un pointeur nul s'il n'y a plus de mémoire disponible.

V.13.2.1. Mémoire statique

Afin de réaliser ses allocations, la fonction `static_malloc()` ira piocher dans un bloc de mémoire statique. Celui-ci consistera simplement en un tableau de `char` de classe de stockage statique d'une taille prédéterminée. Pour cet exercice, nous partirons avec un bloc de un [mébimultiplets](#) [↗](#), soit 1.048.576 multiplets (1024×1024).

```
1 static char mem[1048576UL];
```

V.13.2.1.1. Alignement

Toutefois, retourner un bloc de `char` de la taille demandée ne suffit pas. En effet, si nous allouons par exemple treize multiplets, disons pour une chaîne de caractères, puis quatre multiplets pour

un `int`, nous allons retourner une adresse qui ne respecte potentiellement pas l'alignement requis par le type `int`. Étant donné que la fonction `static_malloc()` ne connaît pas les contraintes d'alignements que doit respecter l'objet qui sera stocké dans le bloc qu'elle fournit, elle doit retourner un bloc respectant les contraintes les plus strictes. Dit autrement, la taille de chaque bloc devra être un multiple de l'alignement le plus rigoureux.

Cet alignement peut être connu à l'aide de l'opérateur `_Alignof` et du type spécial `max_align_t` (défini dans l'en-tête `<stddef.h>`).

```

1 #include <stddef.h>
2 #include <stdio.h>
3
4
5 int
6 main(void)
7 {
8     printf("L'alignement le plus strict est : %zu\n",
9           _Alignof(max_align_t));
10    return 0;
11 }
```

Résultat

```
1 L'alignement le plus strict est : 16
```

Mais... ce n'est pas tout ! Le tableau alloué statiquement doit lui aussi être aligné suivant l'alignement le plus sévère. En effet, si celui-ci commence à une adresse non multiple de cet alignement, notre stratégie précédente tombe à l'eau. Pour ce faire, nous pouvons utiliser le spécificateur `_Alignas` qui permet de modifier les contraintes d'alignement d'un type¹.

```
1 static _Alignas(max_align_t) char mem[1048576UL];
```

Ici nous indiquons donc que nous souhaitons que le tableau `mem` ait le même alignement que le type `max_align_t`, soit l'alignement le plus sévère.

Avec ceci, vous devriez pouvoir réaliser la fonction `static_malloc()` sans encombre.

Hop hop ! Au travail ! 🍊

1. Le spécificateur `_Alignas` a été introduit par la norme C11. Auparavant, pour modifier l'alignement d'un type, il était nécessaire d'employer une union comprenant un champ de ce type et un champ du type dont on souhaite copier l'alignement. Pour obtenir l'alignement le plus strict, la même technique était employée, mais en utilisant les types avec les alignements les plus sévères (comme `long double`, `long long` et `void *`).

V.13.3. Correction

👁️ Contenu masqué n°55

La fonction `calcule_multiple_align_max()` détermine le plus petit multiple de l'alignement maximal égal ou supérieur à `a`. Celle-ci nous permet d'« arrondir » la taille demandée de sorte que les blocs alloués soit toujours correctement alignés.

La fonction `static_malloc()` utilise une variable statique : `alloue` qui représente la quantité de mémoire déjà allouée. Dans le cas où la taille demandée (arrondie au multiple de l'alignement qui lui est égal ou supérieur) n'est pas disponible, un pointeur nul est retourné. Sinon, la variable `alloue` est mise à jour et un pointeur vers la position courante du tableau `mem` est retourné.

Notez que nous avons utilisé une assertion afin d'éviter qu'une taille nulle ne soit fournie.

V.13.4. Deuxième étape : libérer de la mémoire

Pouvoir allouer de la mémoire, c'est bien, mais pouvoir en libérer, ce serait mieux. Parce que pour le moment, dès que l'on arrive à la fin du tableau, que la mémoire précédemment allouée soit encore utilisée ou non, c'est grillé. 🍌

Toutefois, pour mettre en place un système de libération de la mémoire, il va nous falloir changer un peu de tactique.

V.13.4.1. Ajout d'un en-tête

Tout d'abord, si nous voulons libérer des blocs puis les réutiliser, il nous faut conserver d'une manière ou d'une autre leur taille. En effet, sans cette information, il nous sera impossible de les réaffecter.

Pour ce faire, il nous est possible d'ajouter un en-tête lors de l'allocation d'un bloc, autrement dit un objet (par exemple un entier de type `size_t`) qui précédera le bloc alloué et contiendra la taille du bloc.

```

1  +-----+-----+
2  | En-tête |  Bloc alloué  |
3  +-----+-----+
```

Ainsi, lors de l'allocation, il nous suffit de transmettre à l'utilisateur le bloc alloué sans son en-tête à l'aide d'une expression du type `(char *)ptr + sizeof (size_t)` et, à l'inverse, lorsque l'utilisateur souhaite libérer un bloc, de le récupérer à l'aide d'une expression comme `(char *)ptr - sizeof (size_t)`. Notez que cette technique nous coûte un peu plus de mémoire puisqu'il est nécessaire d'allouer le bloc *et* l'en-tête.

V.13.4.2. Les listes chaînées

Mais, ce n'est pas tout. Pour pouvoir retrouver un bloc libéré, il nous faut également un moyen pour les conserver et en parcourir la liste. Afin d'atteindre cet objectif, nous allons employer une structure de donnée appelée une **liste chaînée**. Celle-ci consiste simplement en une structure comprenant des données ainsi qu'un pointeur vers une structure du même type.

```

1 struct list
2 {
3     struct list *suivante;
4 };

```

Ainsi, il est possible de créer des *chaines* de structure, chaque maillon de la chaîne référençant le suivant. L'idée pour notre allocateur va être de conserver une liste des blocs libérés, liste qu'il parcourera en vue de rechercher un bloc de taille suffisante *avant* d'allouer de la mémoire provenant de la réserve. De cette manière, il nous sera possible de réutiliser de la mémoire précédemment allouée avant d'aller puiser dans la réserve.

Nous aurons donc a priori besoin d'une liste comprenant une référence vers le bloc libéré et une référence vers le bloc suivant (il ne nous est pas nécessaire d'employer un membre pour la taille du bloc puisque l'en-tête la contient).

```

1 struct bloc
2 {
3     void *p;
4     struct bloc *suivant;
5 };

```

V.13.4.2.1. Le serpent qui se mange la queue

Cependant, avec une telle technique, nous risquons d'entrer dans un cercle vicieux. En effet, imaginons qu'un utilisateur libère un bloc de 64 multiplète. Pour l'ajouter à la liste des blocs libérés, nous avons besoin d'espace pour stocker une structure **bloc**, nous allons donc allouer un peu de mémoire. De plus, si par la suite ce bloc est réutilisé, la structure **bloc** ne nous est plus utile, sauf que si nous la libérons, nous allons devoir allouer une autre structure **bloc** pour référencer... une structure **bloc** (à moins que la structure ne s'autoréférence). Voilà qui n'est pas très optimal.

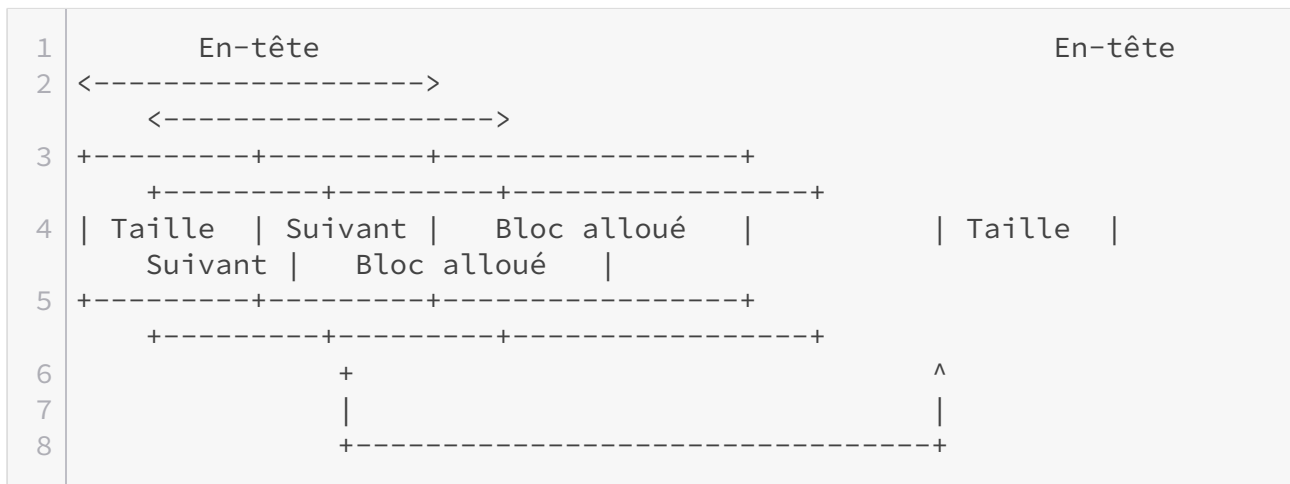
À la place, il nous est possible de recourir à une autre stratégie : inclure la structure **bloc** dans l'en-tête ou, plus précisément, son champ **suivant**, la référence vers le bloc n'étant plus nécessaire dans ce cas. Autrement dit, notre en-tête sera le suivant.

```

1 struct bloc
2 {
3     size_t taille;
4     struct bloc *suivant;
5 };

```

Lors de la libération du bloc, il nous suffit d'employer le champ **suivant** de l'en-tête pour ajouter le bloc à la liste des blocs libres.



Étant donné que l'en-tête précède le bloc alloué, il est impératif que sa taille soit « arrondie » de sorte que les données qui seront stockées dans le bloc respectent l'alignement le plus strict.

Avec ceci, vous devriez pouvoir réaliser une fonction `static_free()` et modifier la fonction `static_malloc()` en conséquence. 🍊

V.13.5. Correction

👁 Contenu masqué n°56

Désormais, la fonction `static_malloc()` fait d'abord appel à la fonction `recherche_bloc_libre()` avant d'allouer de la mémoire de la réserve. Comme son nom l'indique, cette fonction parcourt la liste des blocs libres à la recherche d'un bloc d'une taille égale ou supérieure à celle demandée. Cette liste est matérialisée par la variable globale `libre` qui est un pointeur sur une structure de type `bloc`. Cette variable correspond au premier maillon de la liste et est employée pour initialiser le parcours.

Si aucun bloc libre n'est trouvé, alors la fonction `static_malloc()` pioche dans la réserve un bloc de la taille demandée augmentée de la taille de l'en-tête. Ces deux tailles sont « arrondies » au multiple égal ou directement supérieur de l'alignement le plus contraignant. Étant donné que la taille de l'en-tête est fixe, nous avons représenté sa taille « arrondie » à l'aide de la macroconstante `TAILLE_EN_TETE` et de la macrofonction `MULTIPLE_ALIGN_MAX` (qui réalise le même calcul que la fonction `calcule_multiple_align_max()`). Une fois le tout alloué, l'en-tête est « retiré » en ajoutant la taille de l'en-tête au pointeur référençant la mémoire allouée et le bloc demandé est retourné.

La fonction `static_free()` se rend à la fin de la liste et y ajoute le bloc qui lui est fourni en argument. Pour ce faire, elle « récupère » l'en-tête en soustrayant la taille de l'en-tête au pointeur fourni en argument. Notez également que cette fonction n'effectue aucune opération dans le cas où un pointeur nul lui est fourni (tout comme la fonction `free()` standard), ce qui peut être intéressant pour simplifier la gestion d'erreur dans certains cas.

V.13.6. Troisième étape : fragmentation et défragmentation

Bien, notre allocateur recycle à présent les blocs qu'il a précédemment alloués, c'est une bonne chose. Toutefois, un problème subsiste : la fragmentation de la mémoire allouée, autrement dit sa division en une multitude de petits blocs.

S'il s'agit d'un effet partiellement voulu (nous allouons par petits blocs pour préserver la réserve), il peut avoir des conséquences fâcheuses non désirées. En effet, imaginez que nous ayons alloué toute la mémoire sous forme de blocs de 16, 32 et 64 multiplets, si même tous ces blocs sont libres, notre allocateur retournera un pointeur nul dans le cas d'une demande de par exemple 80 multiplets... Voilà qui est plutôt gênant.

Une solution consiste à défragmenter la liste des blocs libres, c'est-à-dire fusionner plusieurs blocs pour en reconstruire d'autre avec une taille plus importante. Dans notre cas, nous allons mettre en œuvre ce système lors de la recherche d'un bloc libre : désormais, nous allons regarder si un bloc est d'une taille suffisante *ou* si *plusieurs* blocs, une fois fusionnés, seront de taille suffisante.

V.13.6.1. Fusion de blocs

Toutefois, une fusion de blocs n'est possible que si ceux-ci sont adjacents, c'est-à-dire s'ils se suivent en mémoire. Plus précisément, l'adresse suivant le premier bloc à fusionner doit être celle de début du second (autrement dit `(char *)ptr_bloc1 + taille_bloc1 == (char *)ptr_bloc2`).

Néanmoins, il ne nous est pas possible de vérifier cela facilement si notre liste de blocs libres n'est pas un minimum triée. En effet, sans tri, il nous serait nécessaire de parcourir toute la liste à la recherche d'éventuels blocs adjacents au premier, puis, de faire de même pour le deuxième et ainsi de suite, ce qui n'est pas particulièrement efficace.

À la place, il nous est possible de trier notre liste par adresses croissantes (ou décroissantes, le résultat sera le même) de sorte que si un bloc n'est pas adjacent au suivant, la recherche peut être immédiatement arrêtée pour ce bloc ainsi que tous ceux qui lui étaient adjacents. Ce tri peut être réalisé simplement lors de l'insertion d'un nouveau bloc libre en plaçant celui-ci correctement dans la liste à l'aide de comparaisons : s'il a une adresse inférieure à celle d'un élément de la liste, il est placé avant cet élément, sinon, le parcours continue.

En effet, deux pointeurs peuvent tout à fait être comparés du moment que ceux-ci référencent un même objet ou un même agrégat (c'est notre cas ici puisqu'ils référenceront tous des éléments du tableau `mem`) et qu'ils sont du même type (une conversion explicite vers le type pointeur sur `char` sera donc nécessaire comme explicité auparavant).



N'oubliez pas que si deux ou plusieurs blocs sont fusionnés, il n'y a plus besoin que d'un seul en-tête, les autres peuvent donc être comptés comme de la mémoire utilisable.

V.13.7. Correction

© Contenu masqué n°57

La fonction `static_free()` a été aménagée afin que les adresses des différents blocs soient

triées par ordre croissant. Si le nouveau bloc libéré a une adresse inférieure à un autre bloc, il est placé avant lui, sinon il est placé à la fin de la liste.

Quant à la fonction `recherche_bloc_libre()`, elle emploie désormais une structure `suite` qui comprend un pointeur vers le bloc précédent le début de la suite (champ `precedent`), un pointeur vers le premier bloc de la suite (champ `premier`), un pointeur vers le dernier bloc de la suite (champ `dernier`) et la taille totale de la suite (champ `taille`).

Dans le cas où cette structure n'a pas encore été modifiée ou que le dernier bloc de la suite n'est pas adjacent à celui qui le suit, le premier et le dernier blocs de la suite deviennent le bloc courant, la taille est réinitialisée à la taille du bloc courant et le bloc précédent la suite est... celui qui précède le bloc courant (ou un pointeur nul s'il n'y en a pas).

Si la suite atteint une taille suffisante, alors les blocs la composant sont fusionnés et le nouveau bloc ainsi construit est retourné.

Contenu masqué

Contenu masqué n°55

```
1  #include <assert.h>
2  #include <stddef.h>
3  #include <stdio.h>
4
5  #define MEM_MAX (1024UL * 1024UL)
6  #define ALIGN_MAX (_Alignof(max_align_t))
7
8  static _Alignas(max_align_t) char mem[MEM_MAX];
9
10 static size_t calcule_multiple_align_max(size_t);
11 static void *static_malloc(size_t);
12
13
14 static size_t calcule_multiple_align_max(size_t a)
15 {
16     /*
17      * Calcule le plus petit multiple de l'alignement maximal égal
18      * ou supérieur à `a`.
19      */
20     return (a % ALIGN_MAX == 0) ? a : a + ALIGN_MAX - (a %
21         ALIGN_MAX);
22 }
23
24 static void *static_malloc(size_t taille)
25 {
26     /*
27      * Alloue un bloc de mémoire au moins de taille `taille`.
28      */
```

```

29
30     static size_t alloue = 0UL;
31     void *ret;
32
33     assert(taille > 0);
34     taille = calcule_multiple_align_max(taille);
35
36     if (MEM_MAX - alloue < taille)
37     {
38         fprintf(stderr,
39             "Il n'y a plus assez de mémoire disponible.\n");
40         return NULL;
41     }
42     ret = &mem[alloue];
43     alloue += taille;
44     return ret;
45 }

```

[Retourner au texte.](#)

Contenu masqué n°56

```

1  #include <assert.h>
2  #include <stddef.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  #define MEM_MAX (1024UL * 1024UL)
7  #define ALIGN_MAX (_Alignof(max_align_t))
8  #define MULTIPLE_ALIGN_MAX(a) (size_t)((((a) % ALIGN_MAX) == 0) ?
9      (a) : (a) + ALIGN_MAX - (a) % ALIGN_MAX)
10 #define TAILLE_EN_TETE MULTIPLE_ALIGN_MAX(sizeof (struct bloc))
11
12 struct bloc
13 {
14     size_t taille;
15     struct bloc *suivant;
16 };
17
18 static _Alignas(max_align_t) char mem[MEM_MAX];
19 static struct bloc *libre;
20
21 static void bloc_init(struct bloc *, size_t);
22 static size_t calcule_multiple_align_max(size_t);
23 static struct bloc *recherche_bloc_libre(size_t);
24 static void static_free(void *);

```

```

25 static void *static_malloc(size_t);
26
27
28 static void bloc_init(struct bloc *b, size_t taille)
29 {
30     /*
31      * Initialise les membres d'une structure `bloc`.
32      */
33
34     b->taille = taille;
35     b->suivant = NULL;
36 }
37
38
39 static size_t calcule_multiple_align_max(size_t a)
40 {
41     /*
42      * Calcule le plus petit multiple de l'alignement maximal égal
43      * ou supérieur à `a`.
44      */
45     return (a % ALIGN_MAX == 0) ? a : a + ALIGN_MAX - (a %
46         ALIGN_MAX);
47 }
48
49 static struct bloc *recherche_bloc_libre(size_t taille)
50 {
51     /*
52      * Recherche un bloc libre ou une suite de bloc libres dont la
53      * taille
54      * est au moins égale à `taille`.
55      */
56
57     struct bloc *bloc = libre;
58     struct bloc *precedent = NULL;
59     struct bloc *ret = NULL;
60
61     while (bloc != NULL)
62     {
63         if (bloc->taille >= taille)
64         {
65             if (precedent != NULL)
66                 precedent->suivant = bloc->suivant;
67             else
68                 libre = bloc->suivant;
69
70             ret = bloc;
71             break;
72         }
73     }

```

```

72
73     precedent = bloc;
74     bloc = bloc->suivant;
75 }
76
77     return ret;
78 }
79
80
81 static void *static_malloc(size_t taille)
82 {
83     /*
84      * Alloue un bloc de mémoire au moins de taille `taille`.
85      */
86
87     static size_t alloue = 0UL;
88     void *ret;
89
90     assert(taille > 0);
91     taille = calcule_multiple_align_max(taille);
92     ret = recherche_bloc_libre(taille);
93
94     if (ret == NULL)
95     {
96         if (MEM_MAX - alloue < TAILLE_EN_TETE + taille)
97         {
98             fprintf(stderr,
99                 "Il n'y a plus assez de mémoire disponible.\n");
100             return NULL;
101         }
102
103         ret = &mem[alloue];
104         alloue += TAILLE_EN_TETE + taille;
105         bloc_init(ret, taille);
106     }
107
108     return (char *)ret + TAILLE_EN_TETE;
109 }
110
111 static void static_free(void *ptr)
112 {
113     /*
114      * Ajoute le bloc fourni à la liste des blocs libres.
115      */
116
117     struct bloc *bloc = libre;
118     struct bloc *nouveau;
119
120     if (ptr == NULL)

```

```
121         return;
122
123     nouveau = (struct bloc *)((char *)ptr - TAILLE_EN_TETE);
124
125     while (bloc != NULL && bloc->suivant != NULL)
126         bloc = bloc->suivant;
127
128     if (bloc == NULL)
129         libre = nouveau;
130     else
131         bloc->suivant = nouveau;
132 }
```

[Retourner au texte.](#)

Contenu masqué n°57

```
1  #include <assert.h>
2  #include <stddef.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  #define MEM_MAX (1024UL * 1024UL)
7  #define ALIGN_MAX (_Alignof(max_align_t))
8  #define MULTIPLE_ALIGN_MAX(a) (size_t)((((a) % ALIGN_MAX) == 0) ?
9      (a) : (a) + ALIGN_MAX - (a) % ALIGN_MAX)
10 #define TAILLE_EN_TETE MULTIPLE_ALIGN_MAX(sizeof (struct bloc))
11
12 struct bloc
13 {
14     size_t taille;
15     struct bloc *suivant;
16 };
17
18 struct suite
19 {
20     struct bloc *precedent;
21     struct bloc *premier;
22     struct bloc *dernier;
23     size_t taille;
24 };
25
26 static _Alignas(max_align_t) char mem[MEM_MAX];
27 static struct bloc *libre;
28
29 static void bloc_init(struct bloc *, size_t);
30 static size_t calcule_multiple_align_max(size_t);
```

```

31 static struct bloc *recherche_bloc_libre(size_t);
32 static void static_free(void *);
33 static void *static_malloc(size_t);
34
35
36 static void bloc_init(struct bloc *b, size_t taille)
37 {
38     /*
39      * Initialise les membres d'une structure `bloc`.
40      */
41
42     b->taille = taille;
43     b->suivant = NULL;
44 }
45
46
47 static size_t calcule_multiple_align_max(size_t a)
48 {
49     /*
50      * Calcule le plus petit multiple de l'alignement maximal égal
51      * ou supérieur à `a`.
52      */
53     return (a % ALIGN_MAX == 0) ? a : a + ALIGN_MAX - (a %
54         ALIGN_MAX);
55 }
56
57 static struct bloc *recherche_bloc_libre(size_t taille)
58 {
59     /*
60      * Recherche un bloc libre ou une suite de bloc libres dont la
61      * taille
62      * est au moins égale à `taille`.
63      */
64
65     struct suite suite = { 0 };
66     struct bloc *bloc = libre;
67     struct bloc *precedent = NULL;
68     struct bloc *ret = NULL;
69
70     while (bloc != NULL)
71     {
72         if (bloc->taille >= taille)
73         {
74             if (precedent != NULL)
75                 precedent->suivant = bloc->suivant;
76             else
77                 libre = bloc->suivant;

```



```

78         ret = bloc;
79         break;
80     }
81     else if (suite.dernier != NULL && (char *)suite.dernier +
82             TAILLE_EN_TETE \
83 + suite.dernier->taille == (char *)bloc)
84     {
85         suite.dernier = bloc;
86         suite.taille += TAILLE_EN_TETE + bloc->taille;
87     }
88     else
89     {
90         suite.precedent = precedent;
91         suite.premier = bloc;
92         suite.dernier = bloc;
93         suite.taille = bloc->taille;
94     }
95     if (suite.taille >= taille)
96     {
97         if (suite.precedent != NULL)
98             suite.precedent->suivant = suite.dernier->suivant;
99         else
100             libre = suite.dernier->suivant;
101
102         suite.premier->taille = suite.taille;
103         ret = suite.premier;
104         break;
105     }
106
107     precedent = bloc;
108     bloc = bloc->suivant;
109 }
110
111 return ret;
112 }
113
114
115 static void *static_malloc(size_t taille)
116 {
117     /*
118      * Alloue un bloc de mémoire au moins de taille `taille`.
119      */
120
121     static size_t alloue = 0UL;
122     void *ret;
123
124     assert(taille > 0);
125     taille = calcule_multiple_align_max(taille);
126     ret = recherche_bloc_libre(taille);

```

```

127
128     if (ret == NULL)
129     {
130         if (MEM_MAX - alloue < TAILLE_EN_TETE + taille)
131         {
132             fprintf(stderr,
133                 "Il n'y a plus assez de mémoire disponible.\n");
134             return NULL;
135         }
136         ret = &mem[alloue];
137         alloue += TAILLE_EN_TETE + taille;
138         bloc_init(ret, taille);
139     }
140
141     return (char *)ret + TAILLE_EN_TETE;
142 }
143
144
145 static void static_free(void *ptr)
146 {
147     /*
148      * Ajoute le bloc fourni à la liste des blocs libres.
149      */
150
151     struct bloc *bloc = libre;
152     struct bloc *precedent = NULL;
153     struct bloc *nouveau;
154
155     if (ptr == NULL)
156         return;
157
158     nouveau = (struct bloc *)((char *)ptr - TAILLE_EN_TETE);
159
160     while (bloc != NULL)
161     {
162         if (nouveau < bloc)
163         {
164             if (precedent != NULL)
165                 precedent->suivant = nouveau;
166             else
167                 libre = nouveau;
168
169             nouveau->suivant = bloc;
170             break;
171         }
172     }
173
174     precedent = bloc;
175     bloc = bloc->suivant;

```

```
176     }
177
178     if (bloc == NULL)
179         libre = nouveau;
180 }
```

[Retourner au texte.](#)

Sixième partie

Annexes

VI.1. Index

Introduction

Afin de vous aider à retrouver l'une ou l'autre des notions présentées, nous vous proposons ci-dessous un index trié par ordre alphabétique.

VI.1.1. Index

VI.1.1.1. *

- [] (voy. tableau).
- . (voy. structure et union).
- -> (voy. structure et union).
- ++ (voy. opérateurs combinés).
- -- (voy. opérateurs combinés).
- ~ (voy. opérateurs de manipulation des *bits*).
- & (voy. opérateur d'adressage et opérateurs de manipulation des *bits*).
- ! (voy. opérateurs logiques).
- + (voy. opérations mathématiques de base).
- - (voy. opérations mathématiques de base).
- * (voy. opérations mathématiques de base et opérateur d'indirection).
- / (voy. opérations mathématiques de base).
- % (voy. opérations mathématiques de base).
- < (voy. opérateurs de manipulation des *bits*).
- > (voy. opérateurs de manipulation des *bits*).
- == (voy. opérateurs de comparaisons).
- != (voy. opérateurs de comparaisons).
- <= (voy. opérateurs de comparaisons).
- >= (voy. opérateurs de comparaisons).
- < (voy. opérateurs de comparaisons).
- > (voy. opérateurs de comparaisons).
- | (voy. opérateurs de manipulation des *bits*).
- ^ (voy. opérateurs de manipulation des *bits*).
- && (voy. opérateurs logiques).
- || (voy. opérateurs logiques).
- = (voy. affectation).
- += (voy. opérateurs combinés).
- -= (voy. opérateurs combinés).
- /= (voy. opérateurs combinés).
- *= (voy. opérateurs combinés).
- %= (voy. opérateurs combinés).
- ?: (voy. opérateur conditionnel ou opérateur ternaire).

VI.1.1.2. A

- `abort()` [↗](#) .
- Adresse mémoire [↗](#) .
- Affectation [↗](#) .
- `_Alignas` [↗](#)
- Alignement [↗](#) .
- `_Alignof` (voy. alignement).
- Allocation dynamique (de mémoire) [↗](#) .
- Argument(s) (d'une fonction) [↗](#) .
- Assertion (voy. `assert()`).
- `assert()` [↗](#) .
- `auto` [↗](#) (voy. également classe de stockage).

VI.1.1.3. B

- `_Bool` (voy. type).
- B (langage) [↗](#) .
- Binaire [↗](#) (base).
- Bit de bourrage [↗](#) .
- Bit de parité [↗](#) .
- Booléen [↗](#) .
- Boucle infinie [↗](#) .
- Boutisme [↗](#) .
- `break` [↗](#) (voy. également `switch`).

VI.1.1.4. C

- `_Complex` (voy. type, note 4).
- C (locale) voy. classe de caractère, `setlocale()` et caractères larges.
- C89 ; C99 ; C11 (voy. norme).
- Calendrier Grégorien [↗](#) .
- Calendrier Julien [↗](#) .
- `calloc()` [↗](#) .
- Caractères spéciaux [↗](#) .
- Caractères larges [↗](#)
- `char` (voy. type).
- `CHAR_BIT` [↗](#) .
- Chaîne de caractères [↗](#) ; chaîne de caractères littérale [↗](#) , chaîne de caractères larges (voy caractères larges).
- Champ (voy. structure et union).
- Champ de *bits* [↗](#) .
- Chemin d'accès (voy. fichier).
- Classe de caractère [↗](#) .
- Classe de stockage (d'un objet) [↗](#) .
- Compilation conditionnelle (voy. directive conditionnelle).
- `const` [↗](#) .
- Constante (voy. expression) ; constante octale [↗](#) ; constante hexadécimale [↗](#) .
- Constante énumérée [↗](#) .

- Constante de préprocesseur (voy. macroconstante).
- `continue` [↗](#) .
- Conversion (explicite et implicite) [↗](#) .
- Corps d'une fonction [↗](#) .

VI.1.1.5. D

- `DBL_MAX` [↗](#) .
- `DBL_MIN` [↗](#) .
- Déclaration (voy. variable, fonction, structure et union).
- Décrémentation [↗](#) .
- `#define` [↗](#) .
- `defined` (voy. `#if`).
- Définition (voy. variable, fonction, `struct`, `union` et `typedef`).
- Définition de type (voy. `typedef`).
- Dépassement de capacité (d'un type) [↗](#) .
- Directive conditionnelle (voy. `#if`, `#elif`, `#else`, `#ifdef`, `#ifndef` et `#endif`).
- `double` (voy. type).
- `do {} while` [↗](#) .
- Drapeau [↗](#) .
- Durée de vie (d'un objet) [↗](#) .

VI.1.1.6. E

- `EDOM` [↗](#) .
- Effet de bord [↗](#) .
- `EILSEQ` [↗](#) .
- `#elif` [↗](#) .
- `else` [↗](#) .
- `#else` [↗](#) .
- Encodage (voy. table de correspondance).
- `#endif` [↗](#) .
- En-tête [↗](#) .
- `enum` [↗](#) .
- Épilogue [↗](#) .
- `ERANGE` [↗](#) .
- `errno` [↗](#) .
- Étiquette [↗](#) ; étiquette de structure (voy. structure).
- Évaluation en court-circuit [↗](#) .
- `exit()` [↗](#) .
- Expression [↗](#) ; expression-instruction [↗](#) ; type d'une expression [↗](#) .
- `EXIT_FAILURE` [↗](#) .
- `EXIT_SUCCESS` [↗](#) .
- Extension de fichier (voy. fichier).
- `extern` [↗](#) .

VI.1.1.7. F

- `false` (voy. Booléen).
- `fclose()` [↗](#) .
- `feof()` [↗](#) .
- `ferror()` [↗](#) .
- `fflush()` [↗](#) .
- `fgetc()` [↗](#) .
- `fgetwc()` [↗](#) .
- `fgets()` [↗](#) .
- `fgetws()` [↗](#) .
- Fichier [↗](#) .
- `float` (voy. type).
- `FLT_MAX` [↗](#) .
- `FLT_MIN` [↗](#) .
- Flux [↗](#) ; ouverture en lecture et écriture [↗](#) .
- Fonction [↗](#) ; définition de fonction [↗](#) ; prototype [↗](#) ; à nombre variable d'arguments [↗](#) .
- `fopen()` [↗](#) .
- `for` [↗](#) .
- format (voy. `printf()` et `scanf()`).
- `fprintf()` [↗](#) .
- `fwprintf()` [↗](#) .
- `fputc()` [↗](#) .
- `fputwc()` [↗](#) .
- `fputs()` [↗](#) .
- `fputws()` [↗](#) .
- `fread()` [↗](#) .
- `free()` [↗](#) .
- `fscanf()` [↗](#) .
- `fwscanf()` [↗](#) .
- `fseek()` [↗](#) .
- `ftell()` [↗](#) .
- Fuite de mémoire [↗](#) .
- `fwide()` (voy. caractères larges).
- `fwrite()` [↗](#) .

VI.1.1.8. G

- `_Generic` [↗](#)
- `getc()` [↗](#) .
- `getwc()` [↗](#) .
- `getchar()` [↗](#) .
- `getwchar()` [↗](#) .
- Gestion des ressources [↗](#) .
- `goto` [↗](#) .

VI.1.1.9. H

- Hexadécimale [↗](#) (base).

VI.1.1.10. I

- IBM 850 (voy. table de correspondance).
- Identificateur (voy. déclaration de variable).
- `#include` [↗](#) (voy. également en-tête).
- `int` (voy. type).
- `INT_MAX` [↗](#).
- `INT_MIN` [↗](#).
- Internationalisation (voy. `setlocale()`).
- `if` [↗](#).
- `#if` [↗](#).
- `#ifdef` [↗](#).
- `#ifndef` [↗](#).
- Incrémentation [↗](#).
- Indicateur de conversion (voy. `printf()` et `scanf()`).
- Indice (voy. tableau).
- Instruction (voy. `if`, `else`, `switch`, `for`, `do {} while`, `while`, `continue`, `break`, `return`, `goto` et expression-instruction).
- `isalpha()` [↗](#).
- `isalnum()` [↗](#).
- `isblank()` [↗](#).
- `iscntrl()` [↗](#).
- `isdigit()` [↗](#).
- `isgraph()` [↗](#).
- `islower()` [↗](#).
- ISO 639 (voy. `setlocale()`).
- ISO 3166-1 (voy. `setlocale()`).
- ISO 8859-1 (voy. table de correspondance).
- `isprint()` [↗](#).
- `ispunct()` [↗](#).
- `isspace()` [↗](#).
- `isupper()` [↗](#).
- `iswalpha()` [↗](#).
- `iswalnum()` [↗](#).
- `iswblank()` [↗](#).
- `iswcntrl()` [↗](#).
- `iswdigit()` [↗](#).
- `iswgraph()` [↗](#).
- `iswlower()` [↗](#).
- `iswprint()` [↗](#).
- `iswpunct()` [↗](#).
- `iswspace()` [↗](#).
- `iswupper()` [↗](#).
- `iswxdigit()` [↗](#).
- `ixdigit()` [↗](#).

VI.1.1.11. J

- Jeu de caractères (voy. table de correspondance).

VI.1.1.12. L

- `LC_ALL` (voy. `setlocale()`).
- `LC_CTYPE` (voy. `setlocale()` et caractères larges).
- `LC_COLLATE` (voy. `setlocale()`).
- `LC_MONETARY` (voy. `setlocale()`).
- `LC_NUMERIC` (voy. `setlocale()`).
- `LC_TIME` (voy. `setlocale()`).
- `LDBL_MAX` [↗](#) .
- `LDBL_MIN` [↗](#) .
- Localisation (voy. `setlocale()`).
- `long double` (voy. type).
- `long int` (voy. type).
- `long long int` (voy. type).
- `LLONG_MAX` [↗](#) .
- `LLONG_MIN` [↗](#) .
- `LONG_MAX` [↗](#) .
- `LONG_MIN` [↗](#) .
- Limites des types [↗](#) .

VI.1.1.13. M

- Macroconstante [↗](#) .
- Macrofonction [↗](#) ; à nombre variable d'arguments [↗](#) .
- `malloc()` [↗](#) .
- Mantisse (voy. représentation des types).
- Masquage [↗](#) .
- Masque [↗](#) .
- `mbstowcs()` (voy. caractères larges).
- Membre (voy. structure et union).
- `memcmp()` [↗](#) .
- `memcpy()` [↗](#) .
- `memmove()` [↗](#) .
- `memset()` [↗](#) .
- Mémorisation (voy. flux).
- Métadonnées (voy. fichier).
- Mode (voy. `fopen()`).
- Mot [↗](#) .
- `Msys2` [↗](#) .
- Multiplet [↗](#) ; multiplet de bourrage [↗](#) .

VI.1.1.14. N

- `NDEBUG` [↗](#) .
- Norme [↗](#) .
- Notation polonaise inverse [↗](#) .
- `NULL` [↗](#) .

VI.1.1.15. O

- Objet ↗ .
- Octale ↗ (base).
- Octet ↗ .
- `offsetof()` ↗ (note 1).
- Opérateur d'adressage ↗ .
- Opérateurs combinés ↗ .
- Opérateurs de comparaisons ↗ .
- Opérateur conditionnel ↗ .
- Opérateur de déréréférencement (voy. opérateur d'indirection).
- Opérateur d'indirection ↗ .
- Opérateurs logiques ↗ .
- Opérateurs de manipulations des *bits* ↗ .
- Opérations mathématiques de base ↗ .
- Opérateur ternaire (voy. opérateur conditionnel).
- Orientation, d'un flux (voy. caractères larges).

VI.1.1.16. P

- Palindrome ↗ .
- Paramètre(s) (d'une fonction) ↗ .
- `perror()` ↗ .
- Poids (d'un *bit*) voy. boutisme.
- Pointeur ↗ ; pointeur nul ↗ (voy. également promotion des arguments) ; pointeur générique (ou universel) ↗ ; pointeur sur tableau ↗ ; pointeur de fonction ↗ ; pointeur « générique » de fonction ↗ .
- Portée (d'une variable ou d'une fonction) ↗ .
- `pow()` ↗ .
- Préprocesseur ↗ .
- `printf()` ↗ .
- Priorité des opérateurs ↗ .
- Promotion (des arguments) ↗ .
- Prototype (voy. fonction).
- `putc()` ↗ .
- `putwc()` ↗ .
- `putchar()` ↗ .
- `putwchar()` ↗ .
- `puts()` ↗ .

VI.1.1.17. R

- `rand()` ↗ .
- `realloc()` ↗ .
- Répertoire racine (voy. fichier).
- Représentation des types ↗ ; signe et magnitude ↗ ; complément à un ↗ ; complément à deux ↗ ; virgule flottante ↗ ; pointeur ↗ .
- `return` ↗ .
- `rewind()` ↗ .

VI.1.1.18. S

- `scanf()` [↗](#) .
- `SCHAR_MAX` [↗](#) .
- `SCHAR_MIN` [↗](#) .
- `SEEK_CUR` [↗](#) .
- `SEEK_END` [↗](#) .
- `SEEK_SET` [↗](#) .
- Sélection (voy. `if`, `else`, `switch` et opérateur conditionnel).
- Sélection générique (voy. `_Generic`).
- [Séquence d'échappement](#) [↗](#) (voy. également caractères spéciaux).
- `setlocale()` [↗](#) .
- `short` (voy. type).
- `SHRT_MAX` [↗](#) .
- `SHRT_MIN` [↗](#) .
- `signed` (voy. type).
- `sizeof` [↗](#) .
- `snprintf()` [↗](#) .
- `srand()` [↗](#) .
- `sscanf()` [↗](#) .
- `static` (voy. classe de stockage et `extern`).
- `stderr` (voy. flux).
- `stdin` (voy. flux).
- `stdout` (voy. flux).
- `strcat()` [↗](#) .
- `strchr()` [↗](#) .
- `strcmp()` [↗](#) .
- `strcpy()` [↗](#) .
- `strerror()` [↗](#) .
- `strlen()` [↗](#) .
- `strpbrk()` [↗](#) .
- `strstr()` [↗](#) .
- `strtok()` [↗](#) .
- `struct` [↗](#) .
- [Sucre syntaxique](#) [↗](#) .
- [Suite de Fibonnaci](#) [↗](#) .
- `switch` [↗](#) .
- Système de fichier (voy. fichier).

VI.1.1.19. T

- [Tableau](#) [↗](#) ; [unidimensionnels](#) [↗](#) ; [multidimensionnels](#) [↗](#) ; [de longueur variable](#) [↗](#) .
- [Table de correspondance](#) [↗](#) .
- Temporisation (voy. flux, `fflush()` et `ungetc()`).
- `time()` [↗](#) .
- `tolower()` (voy. classe de caractère).
- `toupper()` (voy. classe de caractère).
- [Triangle de Pascal](#) [↗](#) .
- `true` (voy. Booléen).

- Type `☐` ; taille des types `☐` (voy. également `sizeof`) ; type d'une expression `☐` ; limites des types `☐` ; représentation des types `☐` ; type incomplet `☐` .
- `typedef` `☐`

VI.1.1.20. U

- `UCHAR_MAX` `☐` .
- `UINT_MAX` `☐` .
- `ULLONG_MAX` `☐` .
- `ULONG_MAX` `☐` .
- `USHRT_MAX` `☐` .
- `ungetc()` `☐` .
- `ungetwc()` `☐` .
- `#undef` `☐` .
- Unicode (voy. table de correspondance).
- `union` `☐` .
- `unsigned` (voy. type).
- UTF-32 (voy. encodage).
- UTF-16 (voy. encodage).
- UTF-8 (voy. encodage).

VI.1.1.21. V

- `__VA_ARGS__` (voy. macrofonction à nombre variable d'arguments).
- Valeur par défaut `☐` .
- Variable `☐` ; déclaration de variable `☐` ; définition de variable `☐` ; initialisation de variable `☐` ; variable globale `☐` .
- `void` (type) `☐` .

VI.1.1.22. W

- `wchar_t` (voy. caractères larges).
- `wscat()` `☐` .
- `wcschr()` `☐` .
- `wcscmp()` `☐` .
- `wcscpy()` `☐` .
- `wcslen()` `☐` .
- `wcspbrk()` `☐` .
- `wcsstr()` `☐` .
- `wcstok()` `☐` .
- `wcstombs()` (voy. caractères larges).
- `while` `☐` .
- `wprintf()` `☐` .
- `wscanf()` `☐` .

Conclusion

Conclusion

Ainsi s'achève ce cours, mais pas votre parcours dans le monde de la programmation ! En effet, même si vous avez appris certaines choses, vous ne connaissez pas tout : le C est un langage fabuleux qui réserve bien des surprises. Pour continuer votre apprentissage, voici quelques derniers conseils :

- **Soyez curieux** : fouillez sur Internet pour découvrir de nouvelles méthodes, approfondissez celles que vous connaissez, renseignez-vous, testez de nouveaux outils, etc.
- **Entraînez-vous** : c'est le meilleur moyen de progresser. Faites des projets qui vous tiennent à cœur, mettez en œuvre des algorithmes connus, réalisez des exercices, etc.
- **Lisez des codes produits par d'autres personnes** : découvrez comment elles procèdent, apprenez d'elles de nouvelles techniques ou façons de faire et progressez en suivant leurs conseils. Vous pouvez par exemple commencer en visitant les forums de ce site.
- Enfin, le plus important : **amusez-vous** ! 🍊

Liste des abréviations

- ANSI American National Standards Institute. 23
- AT&T American Telephone and Telegraph Company. 23
- GCC GNU Compiler Collection. 27, 30
- GUI Graphical User Interface. 24
- ISO International Organization for Standardization. 23
- RAM Random Access Memory. 40
- SPA Société Protectrice des Animaux. 126